



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE GRADO

Quizzie - Plataforma multidispositivo para la realización y gestión dinámica de tests educativos

Realizado por
Francisco Gago Vázquez

Para la obtención del título de
Grado en Ingeniería Informática - Ingeniería del Software

Dirigido por
Jesús Moreno León

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de octubre, curso 2025/26

*A mis padres y a mi hermana, por estar siempre ahí. Gracias por cuidarme todos estos años,
por vuestra paciencia infinita en las épocas más difíciles y por ser mi apoyo constante en
cada paso del camino. Este trabajo también es vuestro.*

Agradecimientos

En primer lugar, quiero expresar mi más sincero agradecimiento a mi tutor, Jesús Moreno, por su constante guía, cercanía y disponibilidad a lo largo de este proyecto. Sus consejos, revisiones y criterio técnico han sido fundamentales para orientar el trabajo y llevar *Quizzie* a buen puerto.

A todas las amistades y compañeros que me llevo de estos años de carrera. Superar cada curso, las entregas conjuntas y las semanas de exámenes ha sido infinitamente más fácil y enriquecedor gracias a vosotros, por tantas horas compartidas en las aulas y por los buenos momentos fuera de ellas.

Un agradecimiento muy especial para Marta y Paco. Gracias por estar siempre ahí durante los meses de desarrollo de este trabajo, por compartir tantos días en la biblioteca y, sobre todo, por aguantarme, escucharme y apoyarme cuando tocaba desconectar fuera de ella. Teneros cerca ha marcado la diferencia en este proceso.

Por último, a mis padres y a mi hermana. Aunque ya les dedico estas páginas, no puedo cerrar este proyecto sin agradecerles su apoyo incondicional, su paciencia y su confianza.

Resumen

La gamificación en el ámbito educativo se ha consolidado como un enfoque eficaz para fomentar el aprendizaje dinámico y agilizar las sesiones lectivas. No obstante, el análisis del mercado revela una brecha notable: las soluciones comerciales priorizan el entretenimiento a costa del rigor académico, limitan las analíticas exportables y propician el fraude por suplantación digital al permitir que estudiantes ausentes participen de forma remota compartiendo el código PIN de acceso. Por su parte, los cuestionarios de plataformas institucionales garantizan la formalidad evaluativa, pero sacrifican la inmediatez y la fluidez interactiva en el aula.

En este Trabajo de Fin de Grado se presenta *Quizzie*, una plataforma multidispositivo concebida para la creación, realización y gestión dinámica de cuestionarios educativos en tiempo real. La arquitectura adopta un modelo cliente-servidor desacoplado, implementando una interfaz reactiva en React sobre Vite con TypeScript, desplegada en Vercel. El *backend*, alojado en Render, se sustenta sobre un servidor asíncrono desarrollado en Python con FastAPI y gestionado mediante uv, apoyándose en una persistencia relacional ligera con SQLAlchemy y SQLite. La comunicación bidireccional y síncrona de las salas se articula a través del protocolo *WebSocket*. Como elemento innovador y factor diferencial, la plataforma incorpora un mecanismo de verificación de asistencia física basado en la generación de un código QR unívoco en el dispositivo del estudiante al concluir el test, el cual es escaneado por el docente para confirmar de manera inequívoca la presencialidad y registrar la calificación de forma instantánea. Asimismo, el diseño respeta el principio de minimización de datos del RGPD, permitiendo el acceso del alumnado sin registros tradicionales mediante el identificador institucional de la Universidad de Sevilla (*uvus*).

El proyecto se ha abordado siguiendo un ciclo de vida completo de ingeniería del *software*, estructurado mediante metodologías ágiles desde el análisis de requisitos y el modelado arquitectónico hasta la verificación exhaustiva de la solución. El resultado es un producto funcional, robusto y seguro, distribuido bajo licencia de código abierto MIT y concebido como una herramienta viable para su adopción en una amplia variedad de contextos formativos y docentes.

Palabras clave: Gamificación educativa, Tecnología educativa, Evaluación interactiva, Verificación presencial, Prevención de fraude digital, Control de acceso basado en roles (RBAC), Minimización de datos, Arquitectura cliente-servidor desacoplada, Comunicación bidireccional síncrona.

Abstract

Gamification in education has established itself as an effective approach to foster dynamic learning and enhance classroom engagement. However, market analysis reveals a notable gap: commercial solutions often prioritize entertainment at the expense of academic rigor, restrict exportable analytics, and enable academic dishonesty through remote impersonation when absent students join sessions by sharing the game PIN. Conversely, institutional quiz platforms ensure formal assessment integrity but lack real-time fluidity and interactive responsiveness in the classroom.

This Bachelor's Thesis presents *Quizzie*, a cross-device platform designed for the dynamic creation, delivery, and real-time management of educational quizzes. The system adopts a decoupled client-server architecture, featuring a reactive web interface built with React, Vite, and TypeScript, deployed on Vercel. The backend, hosted on Render, is powered by an asynchronous server developed in Python using FastAPI and managed via uv, backed by a lightweight relational persistence layer with SQLAlchemy and SQLite. Synchronous, bidirectional communication in live sessions is orchestrated using the *WebSocket* protocol. As an innovative and differential feature, the platform introduces a physical attendance verification mechanism based on generating a unique QR code on the student's device upon quiz completion; this code is scanned by the instructor to unequivocally confirm in-person presence and instantly record the grade. Furthermore, the system complies with the GDPR data minimization principle by enabling student access via the University of Seville's institutional identifier (*uvus*) without requiring traditional user registration.

The project follows a comprehensive software engineering lifecycle, structured around agile methodologies from requirements elicitation and architectural modeling to thorough system verification. The outcome is a functional, robust, and secure product distributed under the open-source MIT license, conceived as a viable tool for adoption across diverse educational and training environments.

Keywords: Educational gamification, Educational technology, Interactive assessment, On-site attendance verification, Digital impersonation prevention, Role-Based Access Control (RBAC), Data minimization, Decoupled client-server architecture, Synchronous bidirectional communication.

Índice general

1	Introducción	1
1.1.	Contexto y objeto del proyecto	1
1.2.	Motivación	2
1.3.	Objetivos del proyecto	2
1.3.1.	Objetivos académicos	2
1.3.2.	Objetivos técnicos y profesionales	3
1.3.3.	Objetivos de la plataforma	3
1.4.	Resumen del <i>stack</i> tecnológico	4
1.4.1.	Ecosistema del <i>backend</i>	4
1.4.2.	Ecosistema del <i>frontend</i>	5
1.5.	Estructura de la memoria	5
2	Acta de constitución	8
2.1.	Información del proyecto	8
2.2.	Descripción y propósito	8
2.3.	Requerimientos de alto nivel	9
2.4.	Marco legal y normativo	9
2.5.	Riesgos iniciales de alto nivel	10
2.5.1.	Riesgos de alcance	10
2.5.2.	Riesgos técnicos	10
2.5.3.	Riesgos temporales	11
2.6.	Lista de interesados (<i>Stakeholders</i>)	12
3	Gestión del proyecto	13
3.1.	Metodología del trabajo	13
3.2.	Línea base del alcance	14
3.2.1.	Estructura del Desglose del Trabajo (EDT)	14
3.2.2.	Diccionario de la EDT	15
3.3.	Línea base del cronograma	17
3.3.1.	Fases del proyecto	17
3.3.2.	Lista de actividades	17
3.3.3.	Lista de hitos	19
3.3.4.	Diagrama de Gantt	20
3.4.	Línea base de Costes	20
3.4.1.	Análisis de costes	21
3.4.2.	Costes de infraestructura según el número de usuarios	23
3.4.3.	Coste total de propiedad (TCO)	24
3.5.	Plan de gestión de cambios y riesgos	25
4	Estado del arte y fundamentación	26
4.1.	Análisis del mercado	26
4.2.	Factores diferenciadores	27

4.3.	Fundamentación tecnológica	28
4.3.1.	Alternativas consideradas y justificación de las decisiones . .	28
4.3.2.	Limitaciones tecnológicas	32
4.3.3.	Herramientas de soporte al desarrollo y control de calidad . .	32
4.4.	Conclusiones del estudio previo	34
5	Análisis de requisitos	35
5.1.	Historias de usuario	36
5.2.	Reglas de negocio y restricciones	38
5.2.1.	Reglas de negocio	38
5.2.2.	Restricciones	39
5.3.	Requisitos de información	40
5.4.	Requisitos funcionales	40
5.4.1.	Criterios de aceptación y definición de hecho	43
5.5.	Casos de uso	44
5.5.1.	Especificación detallada de Casos de Uso	46
5.6.	Requisitos no funcionales	50
5.7.	Trazabilidad de requisitos	51
6	Diseño y arquitectura	54
6.1.	Patrones arquitectónicos elegidos	54
6.1.1.	Arquitectura cliente-servidor desacoplada	55
6.1.2.	Arquitectura en capas	55
6.1.3.	Arquitectura basada en eventos	56
6.2.	Arquitectura global del sistema	58
6.3.	Justificación y decisiones de arquitectura	59
6.3.1.	Registros de decisiones de arquitectura (ADR)	59
6.3.2.	Trazabilidad entre RNF y tácticas arquitectónicas	60
6.4.	Estructura modular del código	61
6.4.1.	Organización del backend	62
6.4.2.	Organización del frontend	62
6.5.	Modelo de datos	63
6.5.1.	Módulo de sesiones	63
6.5.2.	Módulo de gestión de usuarios	64
6.5.3.	Módulo de contenido	65
6.6.	Diseño de interfaz de usuario	65
6.6.1.	Principios de diseño y guía de estilo	65
6.6.2.	Flujos funcionales y prototipos	66
6.7.	Seguridad	67
6.7.1.	Autenticación síncrona mediante JWT	67
6.7.2.	Gestión de roles y control de acceso (RBAC)	67
6.7.3.	Tokens de verificación para participantes	68
6.8.	Documentación de la API	69
6.8.1.	Integración con OpenAPI y Swagger	69
6.8.2.	Especificación de rutas fundamentales	70

7	Metodología técnica y de desarrollo	72
7.1.	Ecosistema base de desarrollo local	72
7.2.	Políticas del repositorio y flujo de trabajo	73
7.2.1.	Gestión de tareas mediante plantillas de <i>GitHub Issues</i>	73
7.2.2.	Estrategia de ramas	74
7.2.3.	Verificación local mediante <i>pre-commit</i>	75
7.2.4.	Estandarización de commits	76
7.2.5.	Consolidación y despliegue a producción	77
7.3.	Métricas de calidad y análisis estático de código	78
7.3.1.	Justificación y métricas objetivo (<i>Quality Gate</i>)	78
7.3.2.	Configuración y exclusiones técnicas	78
7.4.	Integración y despliegue continuos	79
7.4.1.	Ciclo de vida y arquitectura	79
7.4.2.	Flujo de Integración Continua (CI)	80
7.4.3.	Análisis de calidad con SonarQube	81
7.4.4.	Estrategia de Despliegue Continuo (CD)	81
7.4.5.	Resumen de la estructura de CI/CD	81
7.5.	Gestión de configuración y entorno	83
7.5.1.	Categorización funcional de los secretos	83
7.5.2.	Entorno de desarrollo local	83
7.5.3.	Distribución de secretos por plataforma	84
8	Implementación y pruebas	85
8.1.	Componentes clave e implementación del sistema	85
8.1.1.	Comunicación en tiempo real mediante <i>WebSockets</i>	85
8.1.2.	Gestión de sala, estado, tiempo y aleatoriedad	87
8.1.3.	Seguridad, contexto y autenticación	91
8.2.	Estrategia de <i>testing</i>	93
8.2.1.	Niveles de prueba y selección de herramientas	93
8.2.2.	Organización del plan de pruebas y matriz de trazabilidad	94
8.2.3.	Auditoría y umbrales de cobertura de código	95
8.2.4.	Ejecución y automatización de pruebas	95
8.3.	Casos de prueba y validación	95
8.3.1.	Pruebas de la capa <i>Backend</i>	96
8.3.2.	Pruebas de componentes de la interfaz (<i>Frontend</i>)	98
8.3.3.	Pruebas de extremo a extremo (<i>E2E</i>)	100
8.4.	Pruebas de carga y rendimiento	101
9	Resultados y evaluación	103
9.1.	Cumplimiento de la Línea Base del Alcance	103
9.2.	Tiempo real frente a Línea Base del Cronograma	106
9.3.	Gastos incurridos frente a Línea Base de Costes	107
9.4.	Desviaciones y gestión de cambios	109
9.4.1.	Registro de cambios	109
9.5.	Resultados de las pruebas y validación	113
9.5.1.	Pruebas unitarias y de integración del backend	113
9.5.2.	Pruebas de componentes del frontend	114

9.5.3. Pruebas de extremo a extremo (E2E)	115
9.5.4. Pruebas de carga y rendimiento	115
9.5.5. Resumen de resultados de las pruebas	116
9.6. Análisis estático de código (SonarQube)	117
10 Conclusiones	122
10.1. Grado de cumplimiento de los objetivos	122
10.1.1. Objetivos académicos	122
10.1.2. Objetivos técnicos y profesionales	123
10.1.3. Objetivos de la plataforma	124
10.2. Lecciones aprendidas	125
10.3. Trabajos futuros y líneas de evolución	126
A Prototipos de interfaz de usuario	127
B Manual de usuario	137
C Manual técnico	158
D Registro de pruebas y cobertura	166
E Informe de dedicación temporal	175
Bibliografía	178

Índice de figuras

3.1.	Estructura de Desglose del Trabajo (EDT).	14
3.2.	Diagrama de Gantt inicial del proyecto.	20
3.3.	Costes mensuales de infraestructura	24
3.4.	Evolución de OPEX en función del volumen de usuarios.	24
5.1.	Diagrama general de casos de uso del sistema.	45
6.1.	Estados de la sala de juego en tiempo real.	56
6.2.	Diagrama de secuencia de la transmisión de eventos en una sala. . .	57
6.3.	Arquitectura global del sistema y su implementación concreta. . . .	58
6.4.	Estructura preliminar de módulos y componentes del proyecto. . . .	62
6.5.	Diagrama de clases conceptual y relaciones globales del dominio. . .	63
6.6.	Entidades y atributos del módulo de sesiones.	64
6.7.	Entidades y atributos del módulo de usuarios.	64
6.8.	Entidades y atributos del módulo de contenido.	65
6.9.	Flujo de autenticación del profesor mediante JWT.	67
6.10.	Diagrama del proceso de verificación del participante.	68
6.11.	Flujo de generación de documentación OpenAPI.	69
6.12.	Secuencia del intercambio de eventos en una sala.	71
7.1.	Ciclo de trabajo secuencial implementado en Quizzie.	73
7.2.	Diagrama del flujo de ramas en el repositorio.	75
7.3.	Procedimiento para integrar código en la rama <i>main</i>	77
7.4.	Estructura del <i>pipeline</i> de CI/CD	80
7.5.	Resumen del flujo de CI/CD para la rama <i>main</i>	82
7.6.	Resumen del flujo de CI para la rama <i>develop</i>	82
9.1.	Diagrama de Gantt de seguimiento: cronograma planificado vs. real.	107
9.2.	Evolución del volumen de incidencias en SonarQube.	119
A.1.	Prototipo de inicio de sesión.	127
A.2.	Prototipo de creación de cuenta.	128
A.3.	Prototipo de verificación de cuenta.	128
A.4.	Prototipo de creación de cuestionario.	129
A.5.	Prototipo de listado de cuestionarios.	129
A.6.	Prototipo de configuración de la sala.	130
A.7.	Prototipo de sala de espera recién creada.	130
A.8.	Prototipo de <i>input</i> de código de sala.	131
A.9.	Prototipo de <i>input</i> de UVUS o nickname.	131
A.10.	Prototipo de aviso para registrar nuevo UVUS o nickname.	132
A.11.	Prototipo de sala de espera del alumno recién ingresado.	132
A.12.	Prototipo de sala de espera.	133
A.13.	Prototipo de cuenta atrás.	133
A.14.	Prototipo de tiempo de lectura de pregunta.	134

A.15. Prototipo de pregunta en curso y envío de respuesta.	134
A.16. Prototipo de resultados de la pregunta.	135
A.17. Prototipo de podio de puntuaciones.	135
A.18. Prototipo de visualización global de respuestas.	136
A.19. Prototipo de verificación final.	136
B.1. Punto de entrada a la plataforma.	138
B.2. Formulario de registro para profesores.	138
B.3. Correo electrónico con el código de verificación de cuenta.	139
B.4. Pantalla de introducción del código de verificación de cuenta.	139
B.5. Vista de gestión del perfil.	140
B.6. Correo electrónico para el restablecimiento de contraseña.	141
B.7. Formulario para establecer la nueva contraseña.	141
B.8. Confirmación de eliminación de la cuenta de usuario.	142
B.9. Formulario de inicio de sesión.	143
B.10. Listado general de cuestionarios del profesor.	143
B.11. Vista de edición de un cuestionario.	144
B.12. Historial de salas de un cuestionario.	144
B.13. Resultados de la sala y opción de exportación a CSV.	145
B.14. Diálogo de confirmación para eliminar un cuestionario.	145
B.15. Formulario para la creación de un nuevo cuestionario.	146
B.16. Apartado de cuestionarios nuevos.	146
B.17. Selección de un cuestionario para inicializar una sala.	147
B.18. Configuración de parámetros de la sala.	148
B.19. Sala de espera inicial a la espera de participantes.	148
B.20. Sala de espera con estudiantes incorporados.	149
B.21. Pregunta proyectada en curso y temporizador activo.	149
B.22. Detección y recuperación de sesión activa tras desconexión.	150
B.23. Gráfica de respuestas seleccionadas por el alumnado.	150
B.24. Podio de calificaciones globales al término de las preguntas.	151
B.25. Pantalla de fase de verificación.	151
B.26. Modal de captura y solicitud de permisos de cámara.	152
B.27. Introducción del código de la sala.	153
B.28. Entrada de <i>uvus</i> o apodo.	153
B.29. Confirmación para un nuevo <i>uvus</i> o apodo.	154
B.30. Sala de espera del alumno.	154
B.31. Interfaz de selección y envío de respuesta.	155
B.32. Confirmación inmediata de respuesta correcta.	155
B.33. Indicación de respuesta incorrecta y retroalimentación.	156
B.34. Bloqueo temporal de respuestas por pausa de la sesión.	156
B.35. Bloqueo de opciones al expirar el tiempo reglamentario.	157
B.36. Código QR de verificación del alumno.	157

Índice de tablas

1.1.	Definición de objetivos académicos.	3
1.2.	Definición de objetivos técnicos y profesionales.	3
1.3.	Definición de objetivos de la plataforma <i>Quizzie</i>	4
2.1.	Datos de identificación del proyecto.	8
2.2.	Tabla de riesgos de alcance.	10
2.3.	Tabla de riesgos técnicos.	11
2.4.	Tabla de riesgos temporales.	11
2.5.	Lista de interesados y roles en el proyecto.	12
3.1.	Diccionario de la EDT - 1. Gestión del proyecto	15
3.2.	Diccionario de la EDT - 2. Diseño y arquitectura	15
3.3.	Diccionario de la EDT - 3. Producto software	16
3.4.	Diccionario de la EDT - 4. Documentación final	16
3.5.	Fases temporales del proyecto.	17
3.6.	Actividades organizadas por su fase del cronograma.	18
3.7.	Hitos de control del proyecto.	19
3.8.	Resumen de Gastos de Capital (CAPEX).	21
3.9.	Costes netos laborales agrupados por perfil profesional.	22
3.10.	Resumen de Gastos Operativos mensuales (OPEX).	23
3.11.	Comparativa de OPEX según escenario de escalado.	23
3.12.	Proyección del coste total de propiedad (TCO) a tres años.	25
4.1.	Tabla comparativa del mercado.	27
4.2.	Resumen del <i>stack</i> tecnológico de <i>Quizzie</i>	28
4.3.	Alternativas tecnológicas para la base de datos.	29
4.4.	Alternativas tecnológicas para el ORM y validación.	29
4.5.	Alternativas tecnológicas para el desarrollo del backend.	30
4.6.	Alternativas para el lenguaje de programación frontend.	30
4.7.	Alternativas tecnológicas para la interfaz de usuario.	31
4.8.	Alternativas tecnológicas para el diseño de estilos visuales.	31
4.9.	Alternativas para la infraestructura y el despliegue.	32
5.1.	Objetivos funcionales de <i>Quizzie</i>	35
5.2.	Historias de usuario asociadas al rol de Alumno.	36
5.3.	Historias de usuario asociadas al rol de Profesor.	37
5.4.	Requisitos de información.	40
5.5.	Requisitos funcionales (del RF-01 al RF-19).	41
5.6.	Requisitos funcionales (del RF-20 al RF-37).	42
5.7.	Requisitos funcionales (del RF-38 al RF-47).	43
5.8.	CU-01: Iniciar sesión como docente	46
5.9.	CU-02: Gestionar cuestionarios	46
5.10.	CU-03: Generar preguntas con IA	47

5.11.	CU-04: Administrar sala en vivo	47
5.12.	CU-05: Unirse a sala y realizar test	47
5.13.	CU-06: Visualizar ranking en vivo	48
5.14.	CU-07: Validar calificación mediante QR	48
5.15.	CU-08: Consultar estadísticas	48
5.16.	CU-09: Exportar resultados	49
5.17.	CU-10: Responder pregunta	49
5.18.	CU-11: Gestionar grupos	49
5.19.	Requisitos no funcionales clasificados por dimensión.	50
5.20.	Matriz de trazabilidad RI-OBJ.	51
5.21.	Matriz de trazabilidad RNF-OBJ.	51
5.22.	Matriz de trazabilidad RF-OBJ.	52
5.23.	Matriz de trazabilidad unificada del sistema.	53
6.1.	ADR-01: Adopción de FastAPI y WebSockets	59
6.2.	ADR-02: Arquitectura basada en eventos	59
6.3.	ADR-03: Stack de persistencia SQLite + SQLAlchemy	60
6.4.	ADR-04: Autenticación híbrida JWT + Código QR	60
6.5.	Matriz de trazabilidad entre RNF y tácticas arquitectónicas.	61
7.1.	Clasificación de ramas, nomenclatura y rama de origen.	74
7.2.	Tipos de <i>commits</i> según Conventional Commits.	76
7.3.	Métricas objetivo y umbrales del <i>Quality Gate</i> en SonarQube.	78
7.4.	Variables de entorno y secretos por plataforma.	84
8.1.	Clasificación de niveles de prueba y herramientas del sistema.	94
8.2.	Métricas y umbrales mínimos de cobertura por ámbito de prueba.	95
8.3.	Casos de prueba de la capa <i>backend</i> : Módulo de usuarios.	96
8.4.	Casos de prueba de la capa <i>backend</i> : Módulo de contenido.	96
8.5.	Casos de prueba de la capa <i>backend</i> : Módulo de sesiones.	97
8.6.	Matriz de trazabilidad de pruebas <i>backend</i> con HU y RF.	97
8.7.	Casos de prueba de la capa <i>frontend</i> : Módulo de autenticación y perfil.	98
8.8.	Casos de prueba de la capa <i>frontend</i> : Módulo de gestión de contenidos.	98
8.9.	Casos de prueba de la capa <i>frontend</i> : Módulo de acceso a salas.	98
8.10.	Casos de prueba de la capa <i>frontend</i> : Módulo de sala en directo.	99
8.11.	Matriz de trazabilidad pruebas <i>frontend</i> con RNF.	99
8.12.	Casos de prueba <i>E2E</i>	100
8.13.	Matriz de trazabilidad pruebas <i>E2E</i> con CU.	101
8.14.	Umbrales y criterios de aceptación para las pruebas de rendimiento.	102
9.1.	Objetivos funcionales pospuestos.	103
9.2.	Requisitos funcionales pospuestos asociados a OBJ-09 y OBJ-11.	104
9.3.	Requisitos funcionales pospuestos como puntos de extensión.	104
9.4.	Retrospectiva técnica global de los objetivos funcionales.	105
9.5.	Comparativa de horas previstas y reales por fase del proyecto.	106
9.6.	Comparativa de Gastos de Capital (CAPEX) presupuestados vs. reales.	108

9.7.	Comparativa de dedicación y costes de personal por rol profesional.	108
9.8.	CR-01: Aplazamiento de la defensa a octubre	109
9.9.	CR-02: Aplazamiento del asistente de IA	110
9.10.	CR-03: Postergación de la creación de grupos	110
9.11.	CR-04: Ampliación de la batería de pruebas	111
9.12.	CR-05: Incorporación de Redis	111
9.13.	CR-06: Despliegue de SonarQube y controles locales	112
9.14.	CR-07: Reducción de deuda técnica y congelación de alcance	112
9.15.	Resumen del cumplimiento de umbrales en las baterías de pruebas.	117
9.16.	Comparativa de los umbrales del <i>Quality Gate</i> frente a resultados obtenidos.	118
9.17.	Desglose de historial de <i>issues</i> registradas en SonarQube.	119
10.1.	Evaluación del cumplimiento de objetivos académicos.	122
10.2.	Evaluación del cumplimiento de objetivos técnicos y profesionales. .	123
10.3.	Evaluación del cumplimiento de los objetivos de la plataforma. . . .	124
E.1.	Distribución global del tiempo invertido por fases del proyecto. . . .	175
E.2.	Desglose detallado del cómputo temporal por fase y actividad. . . .	176
E.3.	Horas reales computadas agrupadas por perfil profesional.	177

Índice de extractos de código

8.1. Gestor de conexiones en el <i>backend</i>	85
8.2. Conexión y enrutamiento de eventos WebSocket en React.	86
8.3. Cálculo del tiempo restante sincronizado en el <i>backend</i>	87
8.4. Rutina de pausa automática ante la desconexión del docente.	88
8.5. Algoritmo de barajado determinista anti-copia.	89
8.6. Transición entre preguntas y paso a verificación.	89
8.7. Validación del token y consolidación de calificación.	91
8.8. Persistencia y saneamiento frente a inyecciones XSS.	92
8.9. Extracción y verificación criptográfica de cabeceras JWT.	93

Índice de evidencias

9.1.	Resumen de ejecución y cobertura en backend.	113
9.2.	Resumen de pruebas y cobertura de componentes.	114
9.3.	Resumen de pruebas y cobertura <i>End-to-End</i>	115
9.4.	Informe de evaluación de umbrales de latencia P_{90} y tasa de error. . .	116
9.5.	Estado del <i>Quality Gate</i> y métricas consolidadas en SonarQube. . . .	118
D.1.	Captura de la terminal al ejecutar las pruebas del <i>backend</i>	166
D.2.	Reporte HTML de cobertura del <i>backend</i>	167
D.3.	Captura de la terminal al ejecutar las pruebas de componentes. . . .	168
D.4.	Reporte HTML de cobertura de componentes.	168
D.5.	Captura de la terminal al ejecutar las pruebas E2E.	169
D.6.	Reporte de ejecución de pruebas E2E.	169
D.7.	Informe interactivo de cobertura E2E.	170
D.8.	Salida por terminal al iniciar el panel de control de Locust.	171
D.9.	Vista de la interfaz web de Locust.	171
D.10.	Vista de los resultados de la prueba en la interfaz web de Locust. . .	172
D.11.	Captura de la ejecución de pruebas de carga y rendimiento.	172
D.12.	Registro de excepciones detectadas durante la prueba de carga. . . .	172
D.13.	Historial cronológico de la prueba de carga segundo a segundo. . . .	173
D.14.	Registro de fallos detectados durante la prueba de carga.	174
D.15.	Métricas y percentiles de latencia en la prueba de carga.	174
D.16.	Captura de ejecución de <i>script</i> de verificación de umbrales.	174

1. Introducción

A lo largo de mi trayectoria en la carrera, he asistido a multitud de clases teóricas y prácticas, y si algo he aprendido de ellas es que mantener un dinamismo activo en el aula marca la diferencia en el proceso de aprendizaje. Esta experiencia como estudiante, sumada al reto de diseñar un producto tecnológico completo y autónomo, me motivó a enfocar mi Trabajo de Fin de Grado en el desarrollo de *Quizzie*, una plataforma orientada a transformar las dinámicas de evaluación en el entorno educativo.

El objetivo de este capítulo inicial es asentar las bases del trabajo y ofrecer una hoja de ruta clara. Para ello, en las siguientes secciones se detalla el contexto y objeto del proyecto, las motivaciones personales y profesionales que impulsaron su creación, así como los objetivos académicos y técnicos que se pretenden alcanzar.

1.1. Contexto y objeto del proyecto

A raíz de la creciente digitalización en la enseñanza y la necesidad de mantener el interés de un alumnado, surge la necesidad de buscar soluciones que adapten la forma de evaluar en las aulas a la realidad de los estudiantes. En la primera reunión con el tutor de este proyecto, planteamos el reto de diseñar una herramienta que facilitara el trabajo de los profesores y que, al mismo tiempo, resultara entretenida y motivadora para los alumnos a la hora de comprobar lo aprendido mediante cuestionarios en clase.

La integración de este tipo de cuestionarios rápidos ofrece grandes ventajas en el día a día del aula. Para el alumno, rompe la monotonía de las clases teóricas convencionales introduciendo **aprendizaje dinámico**, elevando la participación y proporcionando *feedback* instantáneo sobre su nivel de comprensión. Por otro lado, para el profesor funciona como una herramienta de **seguimiento en tiempo real**: le permite medir de manera automatizada si el grupo está asimilando los conceptos explicados y, en caso de detectar fallos comunes, aclarar las dudas en ese mismo momento.

Sin embargo, el uso de las plataformas actuales en entornos educativos reales destapa un **problema de control crítico**. Al basarse generalmente en la difusión de un enlace o un código de acceso, es muy sencillo que los estudiantes compartan estas credenciales por grupos de mensajería. Esto provoca que alumnos que no han asistido a clase, o que se encuentran en su casa, puedan responder al cuestionario a distancia. Esta falta de **verificación de la presencia física** falsea las estadísticas de rendimiento que recibe el docente e impide usar estos test como herramientas de **evaluación formal**.

Como respuesta a este escenario, el objeto de este proyecto es el desarrollo de *Quizzie*. La aplicación está diseñada para mantener la sencillez y el dinamismo que

se busca en los cuestionarios en vivo, pero introduciendo una **capa de verificación** para los alumnos. A través de este mecanismo de control, *Quizzie* asegura que la participación se acote estrictamente a los alumnos presentes en el aula, evitando el **fraude a distancia** y consolidándose como un instrumento de evaluación válido y seguro.

1.2. Motivación

La elección de este proyecto nace, en gran medida, de mi propia experiencia como estudiante a lo largo de mi formación. He vivido en primera persona cómo la introducción de **cuestionarios interactivos** en mitad de una clase es capaz de transformar por completo el clima del aula, despertando el interés de los alumnos y fijando los conceptos de una manera mucho más eficaz que los métodos tradicionales. No obstante, también he sido testigo de las limitaciones de las herramientas actuales: el uso de pseudónimos informales, la posibilidad de participar de forma remota sin acudir a clase o la frustración del docente al no poder **validar la autenticidad** de los resultados. Ver cómo una dinámica con tanto potencial perdía rigor por falta de control o por limitaciones de las plataformas utilizadas fue el detonante que me impulsó a buscar esta solución.

El desarrollo de *Quizzie* representa la oportunidad perfecta para demostrar mi capacidad de afrontar un **proyecto de software completo** guiado por las metodologías y principios clásicos de la ingeniería del *software* descritos por autores de referencia como Pressman [1] y Sommerville [2]. Abordar este **ciclo de vida de extremo a extremo** no solo me permite consolidar mis conocimientos técnicos, sino también validar mi criterio al tomar decisiones justificadas ante problemas, convirtiendo este trabajo en el puente definitivo hacia mi madurez como ingeniero de *software*.

1.3. Objetivos del proyecto

Para guiar el desarrollo del proyecto y asegurar el cumplimiento de las expectativas tanto formativas como técnicas y de producto, he definido una serie de metas divididas en tres vertientes: académica, técnico-profesional y operativa de la plataforma.

1.3.1. Objetivos académicos

Metas orientadas a consolidar la formación recibida durante los estudios de grado:

ID	Nombre	Descripción
OA-01	Aplicación de competencias	Integrar los conocimientos del grado en la resolución de los requisitos del sistema, validando la solución mediante una batería de pruebas con una cobertura mínima del 80 % tanto en la capa <i>backend</i> como en <i>frontend</i> .
OA-02	Metodologías ágiles	Aplicar metodologías ágiles en iteraciones estructuradas, controlando el cronograma para completar los hitos temporales fijados con una desviación inferior al 15 %.
OA-03	Buenas prácticas e ingeniería	Aplicar buenas prácticas bajo los paradigmas <i>Clean Code</i> [3] y <i>Clean Architecture</i> [4], asegurando el 100 % de <i>commits</i> semánticos y la superación automatizada de las reglas de lindeo y formato.

Tabla 1.1: Definición de objetivos académicos.

1.3.2. Objetivos técnicos y profesionales

Retos tecnológicos asumidos para el producto final y el desarrollo de competencias laborales:

ID	Nombre	Descripción
OT-01	Dominio tecnológico	Dominar tecnologías y herramientas del ecosistema actual, diseñando una arquitectura cliente-servidor desacoplada que separe la lógica de negocio en el <i>backend</i> asíncrono de la interfaz reactiva.
OT-02	Ciclo de vida completo	Gestionar de forma autónoma el ciclo de vida del software, cubriendo secuencialmente cada fase y justificando el diseño mediante registros formales de decisiones arquitectónicas (ADR).
OT-03	Seguridad y prevención de fraude	Diseñar un mecanismo de validación presencial mediante <i>tokens</i> y códigos QR que garantice un 0 % de fraude en salas cerradas, contrastando su solidez frente a vectores de suplantación y auditorías de seguridad.
OT-04	Estrategia de pruebas y calidad	Desplegar una estrategia de pruebas exhaustiva con cobertura $\geq 80\%$ y análisis estático integrado para evitar deuda técnica crítica y vulnerabilidades.
OT-05	Automatización CI/CD	Configurar un <i>pipeline</i> automatizado bajo los principios de entrega continua [5], validando y desplegando en producción el 100 % de los cambios que superen las pruebas y las metas de calidad establecidas.

Tabla 1.2: Definición de objetivos técnicos y profesionales.

1.3.3. Objetivos de la plataforma

Aspiraciones funcionales y operativas de *Quizzie*:

ID	Nombre	Descripción
OP-01	Evaluación docente simplificada	Proveer al profesorado de una gestión integral de cuestionarios, reduciendo a menos de 5 minutos la puesta en marcha de la sala y automatizando la recolección y corrección de respuestas.
OP-02	Dinamismo e interactividad	Garantizar fluidez bidireccional en tiempo real con una tasa de error de red $< 1,0\%$ bajo simulaciones de concurrencia (≥ 100 usuarios) orientadas al aula docente.
OP-03	Seguimiento continuo	Facilitar herramientas analíticas para monitorizar el rendimiento individual y colectivo mediante estadísticas en tiempo real y volcado histórico estructurado por grupos de clase.
OP-04	Validación presencial física	Asegurar la legitimidad presencial mediante captura de código QR y comprobación criptográfica por cámara en menos de 2 segundos por participante, evitando cuellos de botella.
OP-05	Accesibilidad e incorporación	Optimizar la incorporación del alumnado permitiendo el acceso directo a la sala sin registro previo en menos de 5 segundos mediante código PIN.
OP-06	Compatibilidad multidispositivo	Ofrecer una interfaz completamente responsiva y consistente entre dispositivos y navegadores web estándar, preservando la fidelidad visual y funcional.
OP-07	Privacidad y cumplimiento normativo	Cumplir las directrices del RGPD [6] mediante minimización de datos en el acceso de estudiantes y protección criptográfica robusta en el almacenamiento de credenciales docentes.

Tabla 1.3: Definición de objetivos de la plataforma *Quizzie*.

1.4. Resumen del *stack* tecnológico

Para garantizar el cumplimiento de los objetivos técnicos del proyecto y asegurar un desarrollo robusto, eficiente y escalable, se ha seleccionado un conjunto de herramientas alineadas con los estándares de la industria actual. Esta sección ofrece una visión inicial del *stack* tecnológico empleado para la construcción de *Quizzie*. Las ventajas técnicas de cada opción frente a otras alternativas del mercado se argumentarán en profundidad en el **Estado del arte**.

El núcleo del sistema se divide claramente bajo una **arquitectura cliente-servidor**, donde el *backend* gestiona la lógica de negocio, la seguridad y la comunicación en tiempo real, mientras que el *frontend* ofrece una experiencia interactiva y fluida para profesores y alumnos.

1.4.1. Ecosistema del *backend*

El entorno se ha diseñado priorizando el alto rendimiento, la seguridad y la agilidad en el desarrollo. Las tecnologías y herramientas principales que componen

esta capa son:

- **Lenguaje de programación:** Python 3.12 [7], aprovechando las últimas mejoras de rendimiento y tipado estático del lenguaje.
- **Framework de desarrollo:** FastAPI [8], seleccionado por su alta velocidad de ejecución, soporte nativo de operaciones asíncronas y validación automática de datos.
- **Comunicación en tiempo real:** Protocolo WebSocket [9], implementado para gestionar de manera bidireccional y eficiente el flujo de eventos y la sincronización de las salas en vivo.
- **Persistencia y ORM:** SQLAlchemy [10] en combinación con SQLite [11]. Al no requerir un gestor de base de datos externo complejo en esta fase, esta solución permite interactuar de forma nativa con la base de datos utilizando objetos de Python de manera eficiente.
- **Gestión de dependencias:** uv [12], utilizado como gestor de paquetes ultrarrápido para optimizar los tiempos de instalación y asegurar la reproducibilidad del entorno.

1.4.2. Ecosistema del *frontend*

La interfaz de usuario se ha planteado como una *Single Page Application* (SPA) responsiva para ofrecer la fluidez que requiere una aplicación con dinámicas lúdicas y en vivo:

- **Framework principal:** React [13], para la construcción de una interfaz basada en componentes reutilizables y reactivos.
- **Herramienta de construcción (*Build Tool*):** Vite [14], que proporciona un entorno de desarrollo ágil gracias a su velocidad de compilación y recarga en caliente.
- **Gestión de dependencias:** npm [15], utilizado para la resolución de módulos, control de versiones de paquetes y ejecución de *scripts*.
- **Lenguajes:** TypeScript [16] como eje principal para dotar al código de tipado fuerte y robustez, apoyado puntualmente en JavaScript.
- **Enrutado y consumo de API:** React Router Dom [17] para la navegación interna de la aplicación y Axios [18] para realizar peticiones asíncronas y comunicarse con el *backend*.

1.5. Estructura de la memoria

Para facilitar la lectura y comprensión de este documento, la memoria se ha estructurado en diez capítulos principales y dos manuales adicionales organizados de la siguiente manera:

- **Capítulo 1: Introducción.** Define el punto de partida del proyecto, contextualizando el problema de la evaluación presencial frente al uso de herramientas digitales. Se exponen la motivación, los objetivos del proyecto y una visión general del *stack* tecnológico seleccionado.
- **Capítulo 2: Acta de constitución.** Formaliza el inicio del proyecto, detallando la descripción general del producto, los requerimientos de alto nivel, el marco legal y normativo (incluyendo cumplimiento de RGPD y licencias), los riesgos preliminares y los interesados o *stakeholders*.
- **Capítulo 3: Gestión del proyecto.** Describe la planificación organizativa inicial, incluyendo la metodología de trabajo, la línea base del alcance (a través de la EDT y su diccionario), la línea base del cronograma (con el diagrama de Gantt e hitos), la línea base de costes (estimación de presupuestos, Capex, Opex y TCO), así como los mecanismos para la gestión de riesgos y solicitudes de cambios.
- **Capítulo 4: Estado del arte y fundamentación.** Presenta un análisis comparativo de las soluciones y competidores existentes en el mercado de la gamificación educativa, identificando los factores diferenciales de *Quizzie*. Además, justifica detalladamente las decisiones tecnológicas del *stack* y las alternativas consideradas.
- **Capítulo 5: Análisis de requisitos.** Modela los requisitos del sistema empleando casos de uso, historias de usuario y reglas de negocio. Especifica tanto los requisitos funcionales como los no funcionales, y establece la matriz de trazabilidad que vincula los objetivos iniciales con las pruebas del sistema.
- **Capítulo 6: Diseño y arquitectura.** Detalla la arquitectura de *software* seleccionada (cliente-servidor estructurada en capas y basada en eventos), la modularización del código en el *frontend* y el *backend*, el modelo de datos relacional mediante diagramas de dominio, el diseño de la interfaz de usuario (*mockups* iniciales frente al acabado final), los mecanismos de seguridad (JWT y *tokens* de acceso) y la documentación de la API con OpenAPI/Swagger.
- **Capítulo 7: Metodología técnica (CI/CD).** Detalla los aspectos de ingeniería y DevOps integrados en el repositorio, abarcando las políticas de ramas y *commits* (Commitizen, *hooks* de *pre-commit*), el *pipeline* de integración y despliegue continuo (GitHub Actions), la gestión de perfiles de configuración y secretos, y el control de calidad estática mediante SonarQube.
- **Capítulo 8: Implementación y pruebas.** Recoge las decisiones del desarrollo real mostrando fragmentos de código significativos. Describe la estrategia de *testing* (pruebas unitarias, de integración y extremo a extremo), los umbrales de cobertura y el diseño de las pruebas de carga y rendimiento de la plataforma.
- **Capítulo 9: Resultados y evaluación.** Expone el análisis tras la finalización del desarrollo, comparando los datos de ejecución reales con la planificación inicial (desviaciones en alcance, tiempos y costes). Asimismo, presenta los resultados de los casos de prueba ejecutados y una evaluación técnica integral

como reflejo de la madurez del sistema.

- **Capítulo 10: Conclusiones.** Evalúa el **grado de consecución de los objetivos** generales e individuales planteados. Recoge las lecciones aprendidas durante el ciclo de vida del proyecto, una valoración personal sobre el proceso y las futuras líneas de evolución de la plataforma.

Adicionalmente, se incluyen dos manuales prácticos al final del documento como anexos independientes:

- **Prototipos de interfaz de usuario:** Conjunto de *mockups* y diagramas de flujo que ilustran la experiencia de usuario prevista, mostrando la navegación y las interacciones clave dentro de la plataforma.
- **Manual de usuario:** Guía visual e instructiva orientada a profesores y alumnos para el correcto uso y manejo de la plataforma.
- **Manual técnico:** Guía técnica detallada para el despliegue del sistema tanto en un entorno local de desarrollo como en entornos de producción en la nube.
- **Registro de pruebas y cobertura:** Documentación de los resultados de las pruebas ejecutadas, incluyendo métricas de cobertura de código.
- **Informe de dedicación temporal:** Registro de las horas invertidas en cada fase del proyecto, con detalle de las actividades realizadas y su distribución por roles profesionales.

2. Acta de constitución

El desarrollo de *Quizzie* requiere fijar un marco formal que defina sus reglas y compromisos iniciales antes de comenzar cualquier fase de diseño o desarrollo técnico. El propósito de este capítulo es establecer la línea base del proyecto, delimitando su alcance inicial, sus interesados y las restricciones bajo las que se ejecutará. Este documento define el propósito estratégico de la aplicación, garantizando desde el primer momento la viabilidad técnica, la gestión de riesgos y el estricto cumplimiento del marco legal establecido.

2.1. Información del proyecto

La formalización de *Quizzie* como Trabajo de Fin de Grado requiere establecer, en primer lugar, los datos de identificación administrativa y académica que muestran su autoría y tutorización.

Ficha técnica del proyecto	
Nombre del proyecto	Quizzie
Descripción	Plataforma multidispositivo para la realización y gestión dinámica de tests educativos
Tipo de proyecto	Trabajo de Fin de Grado (TFG)
Titulación	Grado en Ingeniería Informática - Ingeniería del Software
Autor	Francisco Gago Vázquez
Tutor	Jesús Moreno León
Departamento	Lenguajes y Sistemas Informáticos (LSI)

Tabla 2.1: Datos de identificación del proyecto.

2.2. Descripción y propósito

Quizzie nace como una plataforma multidispositivo orientada a transformar la dinámica de evaluación en el entorno educativo. El propósito fundamental de su desarrollo es ofrecer una **herramienta interactiva** que agilice la creación, gestión y ejecución de tests en tiempo real. Frente a soluciones comerciales existentes, la aplicación pone el foco en la inmediatez, la robustez técnica y el control del flujo de evaluación por parte del profesor gracias a la **verificación de la presencialidad de los alumnos**. El desglose de estas capacidades se materializará detalladamente en el capítulo de **análisis de requisitos**.

2.3. Requerimientos de alto nivel

El éxito de la plataforma depende del cumplimiento de una serie de condiciones críticas, atributos de calidad y restricciones de diseño iniciales. Estas directrices sientan las bases del desarrollo y actúan como los criterios de aceptación esenciales para considerar el producto como terminado y listo para su lanzamiento.

Para validar el correcto estado del sistema, este debe cumplir estrictamente con la siguiente lista de verificación:

- **Disponibilidad y despliegue en la nube:** La plataforma completa debe estar desplegada en un entorno de producción real y accesible a través de internet para sus usuarios.
- **Compatibilidad multiplataforma:** La interfaz debe ser completamente *responsive*, garantizando que los alumnos interactúen de forma fluida desde dispositivos móviles y el profesor desde equipos de escritorio.
- **Simplicidad de uso:** El diseño de las pantallas debe priorizar la usabilidad, permitiendo el acceso a las salas de cuestionarios y la creación de test con el menor número de interacciones posible.
- **Seguridad en las comunicaciones:** El intercambio de datos en tiempo real debe realizarse bajo protocolos seguros (HTTPS y WSS), asegurando el cifrado de la información en tránsito.
- **Privacidad de los datos:** El almacenamiento del histórico de calificaciones y datos académicos debe cumplir estrictamente con el marco legal de protección de datos vigente.

2.4. Marco legal y normativo

El desarrollo y uso de *Quizzie* se rige por la **normativa vigente** en materia de **protección de datos, seguridad de la información y propiedad intelectual**. Al tratarse de una herramienta orientada al entorno educativo, el diseño del sistema ha priorizado el **principio de minimización de datos** para mitigar los riesgos asociados al tratamiento de información sensible.

A continuación, se detallan los pilares normativos y legales de la aplicación:

- **Reglamento General de Protección de Datos (RGPD):** La plataforma aplica el principio de privacidad desde el diseño conforme al Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo [6]. Para los estudiantes, el acceso a las salas se realiza mediante un código PIN y su *uvus* (Universidad de Sevilla), prescindiendo de un registro tradicional y limitando el almacenamiento de datos personales a este identificador institucional único. En el caso de los docentes, el registro obligatorio se limita a un correo electrónico y un alias, omitiendo identificadores directos como nombres, apellidos o DNI.

- **Seguridad y persistencia de calificaciones:** El histórico de resultados almacenado en el sistema solo vincula las calificaciones al *uvus* del alumno y cuenta con la previa verificación física del profesor. Esto garantiza la integridad del proceso de evaluación sin exponer la identidad real del estudiantado en la base de datos.
- **Propiedad intelectual y licenciamiento:** El código fuente y los entregables de este Trabajo de Fin de Grado se distribuyen bajo una licencia de código abierto MIT [19]. Esta elección garantiza la total transparencia del desarrollo, permite la auditoría pública del sistema y fomenta su reutilización o extensión por parte de la comunidad académica, eximiendo al autor de cualquier responsabilidad derivada de su uso.

2.5. Riesgos iniciales de alto nivel

El desarrollo de una plataforma orientada a la evaluación en tiempo real conlleva una serie de riesgos de **alcance**, **técnicos** y **temporales**. Su identificación temprana y categorización mediante matrices de probabilidad e impacto sigue las directrices del cuerpo de conocimiento del PMI [20], lo que permite evaluar la gravedad de cada amenaza y definir las estrategias de mitigación asociadas:

2.5.1. Riesgos de alcance

Desviaciones relacionadas con el volumen de funcionalidades implementadas y la correcta definición de los requisitos del sistema:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-01	Corrupción del alcance (<i>Scope Creep</i>) al querer añadir más modos de juego o analíticas complejas.	Definición estricta del Producto Mínimo Viable (MVP). Las mejoras secundarias se derivan a "Trabajos Futuros".	Media	Alto
R-02	Ambigüedad en los requisitos iniciales que obligue a rehacer lógica de negocio ya programada.	Validación previa del modelo de datos e historias de usuario mediante diagramas de flujo antes de codificar.	Baja	Medio

Tabla 2.2: Tabla de riesgos de alcance.

2.5.2. Riesgos técnicos

Incidencias tecnológicas ligadas a la arquitectura del sistema, conectividad de red e infraestructura empleada:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-03	Saturación del servidor o retardos (<i>lag</i>) ante múltiples alumnos conectados a la vez.	Diseñar un modelo de datos ligero para las tramas de WebSockets y realizar pruebas de carga tempranas.	Media	Alto
R-04	Indisponibilidad, caídas o latencia elevada en los servicios en la nube de terceros (<i>Gemini</i> o <i>Render</i>).	Implementación de zonas de control de errores (<i>error boundaries</i>) y reintentos exponenciales en el cliente.	Baja	Alto
R-05	Despliegue y red: fallos de conectividad o puertos bloqueados en la red wifi que impidan la comunicación.	Configurar el despliegue bajo protocolos seguros de producción y prever mecanismos de reconexión rápida.	Baja	Alto
R-06	Incompatibilidades visuales menores o descuadres de la interfaz en ciertos navegadores o pantallas antiguas.	Uso de CSS responsivo estándar mediante <i>Flexbox/Grid</i> y testeo rápido en Google Chrome y Firefox.	Alta	Bajo

Tabla 2.3: Tabla de riesgos técnicos.

2.5.3. Riesgos temporales

Amenazas asociadas directa o indirectamente con el cumplimiento de los plazos y el calendario de entregas de la memoria y el software:

ID	Riesgo identificado	Estrategia de mitigación	Prob.	Impacto
R-07	Retraso en el calendario por la curva de aprendizaje en WebSockets o la API de <i>Gemini Pro</i> .	Reservar las primeras semanas para crear pruebas de concepto aisladas antes de programar el núcleo.	Media	Alto
R-08	Falta de disponibilidad por la carga de trabajo de otras asignaturas simultáneas del curso.	Planificación mediante <i>sprints</i> semanales rígidos con objetivos mínimos viables para mantener el ritmo.	Alta	Medio
R-09	Retrasos menores en la redacción de la memoria final debido al foco exclusivo en el desarrollo de código.	Establecer bloques de tiempo semanales dedicados a documentar los módulos que se acaban de programar.	Media	Bajo

Tabla 2.4: Tabla de riesgos temporales.

2.6. Lista de interesados (*Stakeholders*)

En este apartado se identifica a los actores, grupos e instituciones que interactúan con la plataforma o se ven afectados por su desarrollo, aplicando la metodología de análisis de interesados descrita por Sommerville [2] para definir sus roles, necesidades y expectativas principales.

Interesado	Rol en el proyecto	Intereses / expectativas
Autor	Desarrollar e implementar el sistema completo.	Diseñar una arquitectura robusta, cumplir los plazos de entrega y superar con éxito la defensa del TFG.
Tutor	Supervisar, guiar y evaluar el desarrollo del proyecto.	Garantizar el rigor metodológico, la calidad técnica del software y validar el cumplimiento de los objetivos.
Departamento LSI	Entidad académica de la Universidad responsable de la calificación del TFG.	Asegurar el cumplimiento de la normativa de los Trabajos de Fin de Grado y el correcto proceso de evaluación.
Cuerpo docente	Futuros usuarios (Profesores).	Disponer de una herramienta intuitiva que facilite la evaluación de los alumnos y asegure su presencialidad.
Alumnado	Futuros usuarios (Estudiantes).	Acceder de forma ágil y multidispositivo sin registros complejos para participar en las evaluaciones dinámicas.

Tabla 2.5: Lista de interesados y roles en el proyecto.

3. Gestión del proyecto

En la ingeniería de software, el éxito de un proyecto no depende únicamente de la calidad de su código, sino de una rigurosa gobernanza que garantice la **viabilidad técnica, temporal y económica** del proyecto. Para ello, se adopta un enfoque predictivo en la definición de las líneas base fundamentado en el estándar de gestión del PMBOK [20], el cual se complementa con las prácticas de **desarrollo iterativo e incremental** del Manifiesto Ágil [21] para la gestión del cambio.

3.1. Metodología del trabajo

El desarrollo de *Quizzie* se organiza mediante un flujo de trabajo ágil apoyado en el modelo de ramificación y gestión de versiones *GitHub Flow* [22], el cual centraliza el control de tareas en el repositorio para coordinar las integraciones y el despliegue de las *releases*.

- **GitHub:** Centraliza el almacenamiento del código fuente y la gestión de las tareas mediante las siguientes herramientas:
 - **Issues y Milestones:** Cada funcionalidad o corrección se registra como una *Issue* (tarea). Estas se agrupan en *Milestones* (hitos temporales) para asociar el avance del código con los plazos asignados en el cronograma.
 - **Ramas permanentes:** Se mantiene una separación estricta entre la rama *main* (exclusiva para versiones definitivas y estables) y la rama *develop* (entorno de integración donde se vuelcan las novedades).
 - **Ramas de tareas:** Para cada *issue* se abre una rama específica desde *develop* (ej. *feature/websocket-timer*). Una vez completada y probada la tarea de forma aislada, esta rama se fusiona (*merge*) de vuelta en *develop*.
 - **Releases:** Al completar un *Milestone* y consolidar los cambios en *main*, se genera una *Release* con su etiqueta de versión.
- **Clockify:** Se utiliza exclusivamente para cronometrar el tiempo real dedicado a cada tarea e hito, permitiendo controlar el esfuerzo invertido.
- **Outlook:** Canal formal para comunicaciones con el tutor y envío de entregables o documentación.

El avance del proyecto se valida de forma continua mediante tutorías. La dinámica de estas reuniones consiste en presentar y revisar el software funcionando o el texto redactado hasta esa fecha para recibir el visto bueno del tutor y, acto seguido, definir los siguientes pasos y prioridades a realizar para la próxima tutoría.

3.2. Línea base del alcance

La línea base del alcance define con precisión los **límites del proyecto**, asegurando que todos los esfuerzos se concentren exclusivamente en los **entregables necesarios** para cumplir con los objetivos académicos y técnicos. A continuación, se detallan los supuestos, restricciones y exclusiones que condicionan esta planificación.

- **Supuestos:** Se asume la disponibilidad continua de las infraestructuras en la nube (proveedores de alojamiento o bases de datos) durante las fases de desarrollo y despliegue, así como el acceso ilimitado a la documentación de las API y tecnologías que componen el *stack* tecnológico.
- **Restricciones:** El proyecto está sujeto a una restricción temporal vinculada a los plazos institucionales de entrega y defensa del Trabajo Fin de Grado. Asimismo, el desarrollo debe ceñirse al presupuesto simulado y a la capacidad operativa de un único desarrollador.
- **Exclusiones:** Queda fuera del alcance del proyecto el despliegue comercial en producción y el soporte para navegadores web obsoletos o sin compatibilidad con WebSockets modernos.

3.2.1. Estructura del Desglose del Trabajo (EDT)

Para organizar y estructurar el alcance total del proyecto de forma sistemática, se ha elaborado la siguiente **Estructura de Desglose del Trabajo (EDT)**.

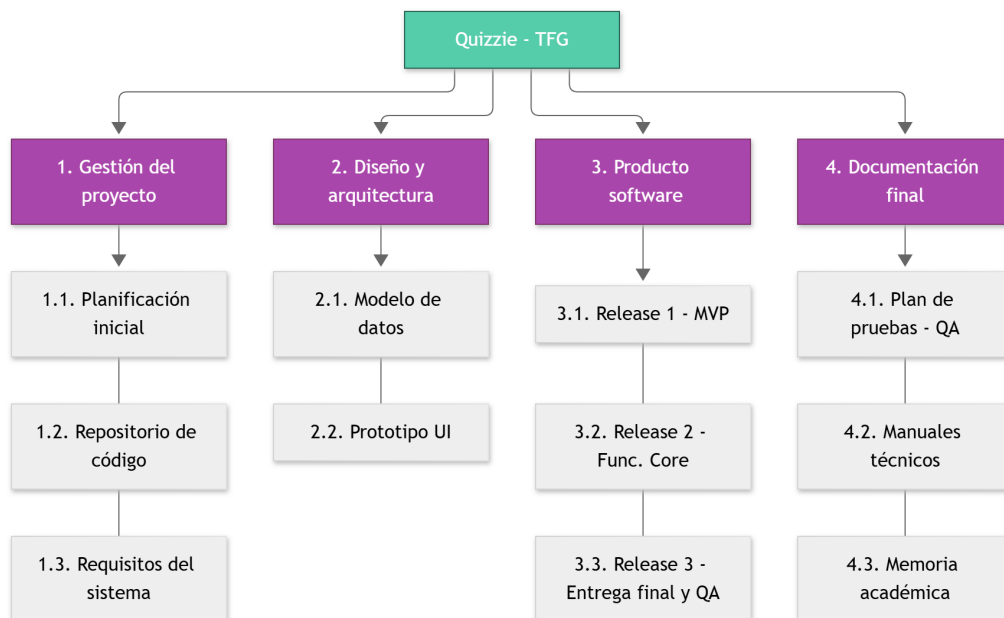


Figura 3.1: Estructura de Desglose del Trabajo (EDT).

Esta representación jerárquica descompone el proyecto en **paquetes de trabajo**

y **entregables específicos** según las directrices de descomposición del estándar ISO/IEC/IEEE 21839 [23], los cuales se detallan en el diccionario de la EDT.

3.2.2. Diccionario de la EDT

El diccionario de la EDT define formalmente el **alcance** de cada entregable y desglosa las actividades necesarias para su consecución, especificando los perfiles profesionales asignados y la estimación temporal.

Paquete 1: Gestión del proyecto

Entregable	Descripción	Actividad	Recurso	Horas
1.1. Planificación inicial	Acta de constitución del proyecto y cronograma base estimado para fijar los hitos académicos.	ACT-01	Project Manager	4h
		ACT-02	Project Manager	4h
1.2. Repositorio de código	Configuración de GitHub con reglas de protección de ramas, flujos de trabajo basados en fusiones (<i>merges</i>) y plantillas estandarizadas para <i>issues</i> .	ACT-13	Técnico	11h
		ACT-14	Técnico	5h
		ACT-15	Técnico	12h
1.3. Requisitos del sistema	Catálogo de requisitos funcionales y no funcionales, junto al modelado de casos de uso y flujos <i>core</i> de la aplicación.	ACT-08	Analista	3h
		ACT-09	Analista	4h

Tabla 3.1: Diccionario de la EDT - 1. Gestión del proyecto

Paquete 2: Diseño y arquitectura

Entregable	Descripción	Actividad	Recurso	Horas
2.1. Modelo de datos	Diseño conceptual, lógico y físico de la base de datos relacional, materializado en el diagrama de entidades del sistema.	ACT-10	Analista	4h
		ACT-11	Analista	4h
		ACT-12	Programador Backend	2h
2.2. Prototipo UI	Diseños visuales y plantillas para las pantallas del sistema.	ACT-03	Diseñador UI	2h
		ACT-04	Diseñador UI	5h
		ACT-05	Diseñador UI	5h

Tabla 3.2: Diccionario de la EDT - 2. Diseño y arquitectura

Paquete 3: Producto software

Entregable	Descripción	Actividad	Recurso	Horas
3.1. Release 1 (MVP)	Habilitación y despliegue del flujo completo base del sistema, permitiendo la creación de cuestionarios por parte del profesor y la realización de los mismos por los alumnos.	ACT-16	Programador Backend	25h
		ACT-17	Programador Frontend	25h
		ACT-18	Programador Backend	15h
3.2. Release 2 (Funcionalidades Core)	Implementación de la capa de comunicación en tiempo real mediante WebSockets y el motor de validación de datos del sistema.	ACT-21	Programador Backend	20h
		ACT-22	Programador Frontend	17h
		ACT-23	Programador Backend	12h
3.3. Release 3 (Entrega final y QA)	Integración de funcionalidades avanzadas de gestión, pulido general de la interfaz y despliegue del sistema.	ACT-24	Programador Backend	5h
		ACT-25	Programador Frontend	5h
		ACT-26	Técnico	3h

Tabla 3.3: Diccionario de la EDT - 3. Producto software

Paquete 4: Documentación final

Entregable	Descripción	Actividad	Recurso	Horas
4.1. Plan de pruebas (QA)	Diseño y ejecución de baterías de test de integración y <i>scripts</i> específicos de pruebas de carga.	ACT-19	Tester	7h
		ACT-20	Tester	8h
4.2. Manuales técnicos	Redacción del manual de usuario y del manual de despliegue.	ACT-06	Analista	2h
		ACT-07	Técnico	1h
4.3. Memoria académica	Redacción técnica, maquetación y consolidación del documento final del TFG junto con el diseño de las diapositivas para la defensa.	ACT-27	Analista	70h
		ACT-28	Diseñador UI	10h
		ACT-29	Project Manager	10h

Tabla 3.4: Diccionario de la EDT - 4. Documentación final

3.3. Línea base del cronograma

A partir de los paquetes de trabajo de la **EDT**, se ha planificado la distribución temporal del proyecto. El flujo de trabajo se estructura dividiendo el ciclo de vida en fases temporales que agrupan las actividades y culminan en hitos de control.

3.3.1. Fases del proyecto

El proyecto se divide en 10 fases temporales consecutivas y paralelas, que se han organizado en 4 bloques de trabajo fundamentales:

ID	Nombre	Descripción	Bloque
F-01	Planificación	Planificación inicial, estimación de tiempos y recursos.	Gestión y planificación
F-02	Documentación	Redacción de manuales y documentación técnica de soporte.	Documentación y cierre
F-03	Inicio	Configuración del entorno base y análisis de requisitos.	Gestión y planificación
F-04	CI/CD	Automatización de flujos de trabajo e integración continua.	Ciclo de vida y calidad
F-05	MVP	Construcción de la primera versión mínima funcional.	Desarrollo de código
F-06	Testing	Diseño y ejecución de la batería de pruebas del sistema.	Ciclo de vida y calidad
F-07	Core	Desarrollo de las características principales (WebSockets).	Desarrollo de código
F-08	QA	Estabilización de código y corrección de errores finales.	Desarrollo de código
F-09	Memoria	Consolidación, revisión final y entrega de la memoria.	Documentación y cierre
F-10	Defensa	Preparación del material de soporte y ensayos.	Documentación y cierre

Tabla 3.5: Fases temporales del proyecto.

3.3.2. Lista de actividades

Cada una de las actividades del cronograma conserva una relación directa con los paquetes de trabajo definidos en el diccionario de la EDT. Para garantizar el seguimiento y la trazabilidad temporal, a continuación se detallan todas las tareas del proyecto:

Fase	ID	Actividad
F-01 – Planificación	ACT-01	Redacción del acta de constitución
	ACT-02	Definición de hitos y estimación del cronograma inicial
	ACT-03	Elección de estilos, paleta de colores y componentes básicos
	ACT-04	Diseño de prototipos de las pantallas de gestión del profesor
	ACT-05	Diseño de la interfaz de respuesta para pantallas de alumnos
F-02 – Documentación	ACT-06	Elaboración del manual funcional como guía para usuarios
	ACT-07	Redacción del manual de despliegue técnico de infraestructura
	ACT-08	Elicitación y redacción de requisitos funcionales y no funcionales
	ACT-09	Modelado de diagramas de casos de uso y flujos esenciales
	ACT-10	Diseño del modelo conceptual y lógico de datos
	ACT-11	Modelado del diagrama de entidades relacional
	ACT-12	Generación del <i>script</i> de inicialización de tablas y poblado
F-03 – Inicio	ACT-13	Inicialización del repositorio en GitHub y estructura base
F-04 – CI/CD	ACT-14	Diseño de <i>templates</i> para <i>issues</i> y automatización base
	ACT-15	Desarrollo de <i>scripts</i> de despliegue e integración continuos
F-05 – MVP	ACT-16	Diseño y desarrollo de la persistencia base para cuestionarios
	ACT-17	Implementación de la interfaz de creación de tests y panel base
	ACT-18	Lógica de control y validación para la ejecución asíncrona
F-06 – Testing	ACT-19	Desarrollo y ejecución de test de integración de servicios
	ACT-20	Pruebas de componentes, de carga y <i>end-to-end</i>
F-07 – Core	ACT-21	Desarrollo de la infraestructura síncrona y canales WebSocket
	ACT-22	Implementación en frontend de la conexión reactiva a salas
	ACT-23	Programación del motor de validación y control de integridad
F-08 – QA	ACT-24	Implementación del módulo de autenticación y panel avanzado
	ACT-25	Desarrollo de vistas de historial y exportación de datos
	ACT-26	Puesta a punto, empaquetado, corrección de errores y despliegue
F-09 – Memoria	ACT-27	Redacción y estructuración técnica de la memoria en LaTeX
F-10 – Defensa	ACT-28	Diseño, desarrollo y maquetación de diapositivas de defensa
	ACT-29	Ensayos de oratoria y preparación de la defensa pública

Tabla 3.6: Actividades organizadas por su fase del cronograma.

3.3.3. Lista de hitos

Los hitos representan los puntos de control clave y logros de duración cero dentro del cronograma del proyecto. Su consecución suponen grandes avances en el desarrollo o la finalización de fases completas, y permiten evaluar el progreso general del proyecto.

Fase	ID	Hito / logro	Límite
F-01 – Planificación	H-01	Acta de constitución del proyecto redactada y firmada	21 Ene
	H-02	Líneas base del proyecto constituidas y aprobadas	28 Ene
F-02 – Documentación	H-03	Catálogo de requisitos, casos de uso y modelos de datos definidos	18 Feb
	H-04	Manuales técnicos de usuario y despliegue finalizados	14 Abr
F-03 – Inicio	H-05	Repositorio inicializado y entorno de desarrollo listo	11 Feb
F-04 – CI/CD	H-06	Flujo de CI/CD automatizado y plantillas de trabajo desplegadas	1 Mar
F-05 – MVP	H-07	Funcionalidades del MVP operativas (creación y respuesta de tests)	8 Mar
F-06 – Testing	H-08	Baterías de pruebas de software ejecutadas y superadas con éxito	26 Abr
F-07 – Core	H-09	Conexión y respuesta a cuestionarios en tiempo real operativos	29 Mar
	H-10	Motor de validación y control de integridad de respuestas completado	12 Abr
F-08 – QA	H-11	Vistas de historial de salas pasadas y exportación de datos funcionales	24 Abr
F-09 – Memoria	H-12	Memoria académica consolidada y entregada	17 May
F-10 – Defensa	H-13	Presentación y material de soporte maquetados para la defensa	5 Jun
	H-14	Defensa pública del proyecto ejecutada ante el tribunal	10 Jun

Tabla 3.7: Hitos de control del proyecto.

3.3.4. Diagrama de Gantt

El siguiente diagrama de Gantt[24] sintetiza de manera visual la distribución cronológica del proyecto, clave para el seguimiento del trabajo realizado y asegurar el cumplimiento de los plazos fijados.

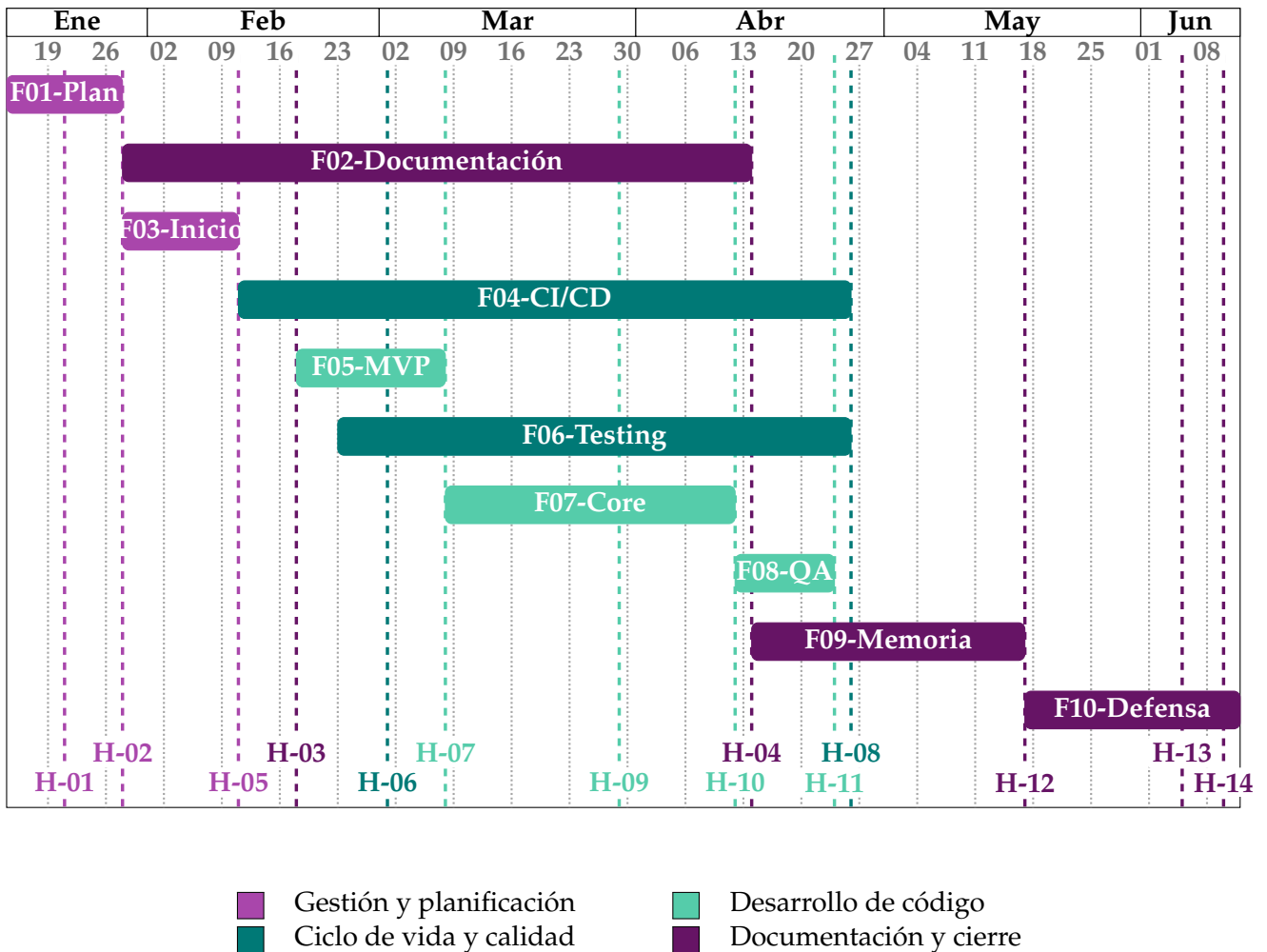


Figura 3.2: Diagrama de Gantt inicial del proyecto.

3.4. Línea base de Costes

El desarrollo de un proyecto como *Quizzie* no solo exige viabilidad técnica, sino también un **modelo financiero sostenible** que justifique su implantación frente a otras soluciones. Para ello, se analizan a continuación los costes operativos, el impacto económico que supone el escalado de la infraestructura y el Coste Total de Propiedad (TCO).

3.4.1. Análisis de costes

El estudio económico de la plataforma se fundamenta en la clasificación financiera de costes de tecnologías de la información [25], dividiéndose en dos componentes esenciales: los Gastos de Capital (*Capital Expenditure* o **CAPEX**), que engloban el coste de desarrollo y los recursos fijos iniciales, y los Gastos Operativos (*Operational Expenditure* o **OPEX**), que representan el coste recurrente de la infraestructura en la nube para la explotación en producción.

Concepto	Coste neto
Costes laborales	11.552,65 €
Equipamiento informático	500,00 €
Licencia comercial anual Google AI Premium (Gemini Pro)	218,07 €
Licencia GitHub Team (6 meses)	21,82 €
Licencia Microsoft Teams Essentials (6 meses)	21,82 €
Reserva de contingencia	12.314,36 €
Total CAPEX	13.545,80 €

Tabla 3.8: Resumen de Gastos de Capital (CAPEX).

Para justificar el coste laboral del CAPEX, se adopta un **modelo de roles mixtos** debido a que el proyecto ha sido ejecutado íntegramente por un único ingeniero. De este modo, cada actividad de la EDT se presupuesta en función del perfil profesional específico requerido para su ejecución técnica.

Con el fin de garantizar el rigor y la objetividad de las tarifas aplicadas, los precios base por hora se han extraído del *Informe Final sobre la Consulta Preliminar del Mercado: “Perfiles Profesionales Ámbito Informático”* de la Junta de Andalucía [26]. Siguiendo las directrices de dicho organismo, se utiliza como referencia la **media acotada**, métrica estadística que descarta valores atípicos del sector mediante el test de Tukey.

Sobre la base salarial neta resultante de 8.886,65 €, se ha aplicado un **recargo del 30 %** en concepto de cotizaciones a la Seguridad Social y costes de contratación directos, consolidando el coste laboral real en **11.552,65 €**. El desglose detallado de horas y perfiles profesionales que sustenta este cálculo se recoge en la siguiente tabla de costes netos laborales.

Recurso (Rol)	Actividad	Precio/hora	Horas	Coste neto
Project Manager	ACT-01	46,09 €	4 h	184,36 €
	ACT-02	46,09 €	4 h	184,36 €
	ACT-29	46,09 €	10 h	460,90 €
	Subtotal		18 h	829,62 €
Analista	ACT-06	38,33 €	2 h	76,66 €
	ACT-08	38,33 €	3 h	114,99 €
	ACT-09	38,33 €	4 h	153,32 €
	ACT-10	38,33 €	4 h	153,32 €
	ACT-11	38,33 €	4 h	153,32 €
	ACT-27	38,33 €	70 h	2.683,10 €
	Subtotal		87 h	3.334,71 €
Diseñador UI	ACT-03	33,43 €	2 h	66,86 €
	ACT-04	33,43 €	5 h	167,15 €
	ACT-05	33,43 €	5 h	167,15 €
	ACT-28	33,43 €	10 h	334,30 €
	Subtotal		22 h	735,46 €
Programador Backend	ACT-12	23,85 €	2 h	47,70 €
	ACT-16	23,85 €	25 h	596,25 €
	ACT-18	23,85 €	15 h	357,75 €
	ACT-21	23,85 €	20 h	477,00 €
	ACT-23	23,85 €	12 h	286,20 €
	ACT-24	23,85 €	5 h	119,25 €
	Subtotal		79 h	1.884,15 €
Programador Frontend	ACT-17	23,85 €	25 h	596,25 €
	ACT-22	23,85 €	17 h	405,45 €
	ACT-25	23,85 €	5 h	119,25 €
	Subtotal		47 h	1.120,95 €
Tester	ACT-19	23,68 €	7 h	165,76 €
	ACT-20	23,68 €	8 h	189,44 €
	Subtotal		15 h	355,20 €
Técnico	ACT-07	19,58 €	1 h	19,58 €
	ACT-13	19,58 €	11 h	215,38 €
	ACT-14	19,58 €	5 h	97,90 €
	ACT-15	19,58 €	12 h	234,96 €
	ACT-26	19,58 €	3 h	58,74 €
	Subtotal		32 h	626,56 €
TOTAL			300 h	8.886,65 €

Tabla 3.9: Costes netos laborales agrupados por perfil profesional.

Una vez cuantificada la base de la mano de obra directa y consolidados los recursos materiales del proyecto, el CAPEX queda cerrado incorporando los costes empresariales y la reserva de contingencia definidos. Frente a esta inversión no recurrente asociada a la fase de construcción, es necesario proyectar los **compromisos económicos continuos** requeridos para mantener la plataforma operativa en **producción**.

A continuación se desglosa la partida correspondiente al OPEX base estimada para el arranque del sistema:

Concepto	Coste neto
Infraestructura web Frontend (Vercel Pro)	18,18 €/mes
Servidor de aplicaciones y Base de Datos (Render Starter)	12,73 €/mes
Consumo de la API de Gemini Pro para el asistente de IA	15,00 €/mes
Total OPEX	45,91 €/mes

Tabla 3.10: Resumen de Gastos Operativos mensuales (OPEX).

Cabe destacar que las cifras reflejadas en el resumen del OPEX **no constituyen un coste estático**, sino que representan una **proyección** basada en una estimación inicial de 500 usuarios activos mensuales, asumiendo un consumo moderado de recursos y una utilización estándar de la API de inteligencia artificial. En escenarios de escalado masivo, donde el volumen de usuarios concurrentes o la demanda de cómputo aumente significativamente, es previsible que estos costes operativos se incrementen de forma proporcional. Por ello, en la siguiente subsección se detalla y aclara matemáticamente el comportamiento financiero de la plataforma ante diferentes escenarios de carga y volumen de usuarios.

3.4.2. Costes de infraestructura según el número de usuarios

Para prever el impacto financiero de la plataforma en entornos reales, se analiza el comportamiento de los costes operativos (OPEX) bajo tres escenarios de demanda: **pesimista**, **conservador** y **optimista**. Esta clasificación permite evaluar la **elasticidad de la infraestructura en la nube** (Render y Vercel) y el **consumo variable** de la API de Gemini según el volumen de uso.

Concepto	Pesimista	Conservador	Optimista
Vercel Pro	0,00 €	18,18 €	18,18 €
Render Starter	0,00 €	12,73 €	27,27 €
Gemini Pro	2,50 €	15,00 €	60,00 €
Total OPEX	2,50 €/mes	45,91 €/mes	105,45 €/mes

Tabla 3.11: Comparativa de OPEX según escenario de escalado.

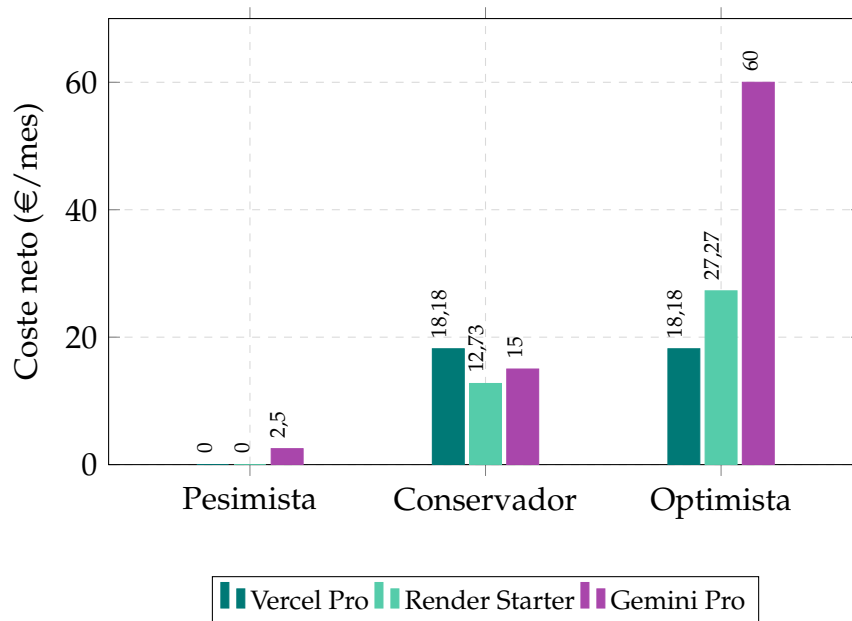


Figura 3.3: Costes mensuales de infraestructura según escenario de escalado.

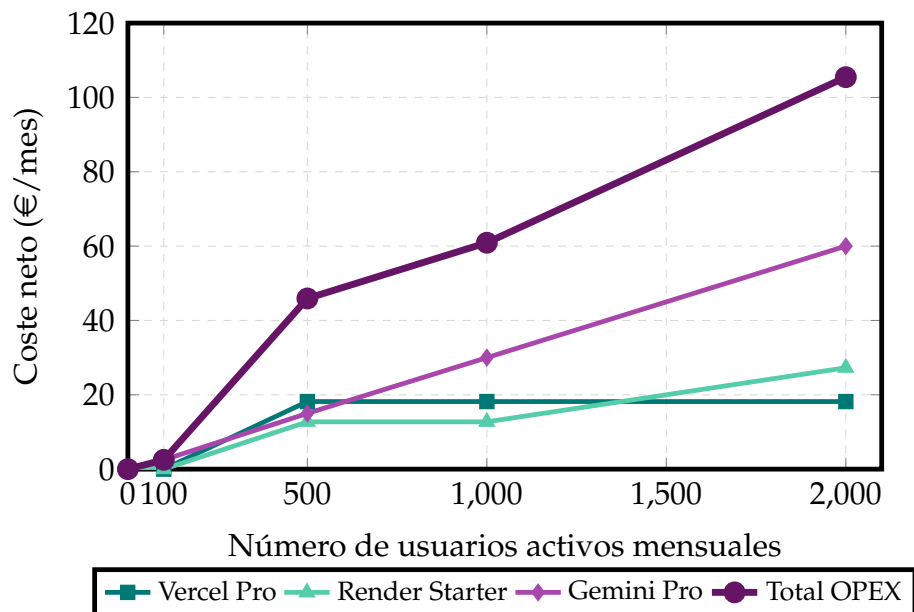


Figura 3.4: Evolución de OPEX en función del volumen de usuarios.

3.4.3. Coste total de propiedad (TCO)

El coste total de propiedad (*Total Cost of Ownership*, **TCO**) permite evaluar el **impacto económico global** de la plataforma a medio plazo mediante la metodología de consolidación financiera de costes directos e indirectos planteada por Ellram [27], integrando la inversión inicial de desarrollo (CAPEX) junto con la proyección de los gastos operativos (OPEX) a tres años.

De este modo, asumiendo los gastos de infraestructura del escenario conservador, la proyección resultante se detalla en la siguiente tabla.

Concepto	Año 1	Año 2	Año 3
Inversión inicial (CAPEX)	13.545,80 €	0,00 €	0,00 €
Infraestructura Frontend (<i>Vercel Pro</i>)	218,16 €	218,16 €	218,16 €
Servidor de aplicaciones y BD (<i>Render Starter</i>)	152,76 €	152,76 €	152,76 €
Consumo de la API de <i>Gemini Pro</i>	180,00 €	180,00 €	180,00 €
Total anual	14.096,72 €	550,92 €	550,92 €
TCO acumulado (3 años)	15.198,56 €		

Tabla 3.12: Proyección del coste total de propiedad (TCO) a tres años.

Nota: No se han imputado costes de personal ni horas de mantenimiento dentro del OPEX debido a que el esfuerzo laboral se ha contabilizado íntegramente como parte del CAPEX, contando con las 300 horas reglamentarias del TFG.

3.5. Plan de gestión de cambios y riesgos

Un desarrollo tecnológico está inherentemente sujeto a imprevistos y desviaciones. Para asegurar el éxito y la entrega en plazo de la plataforma, el procedimiento formal de control de cambios y contingencias se estructura según las directrices de la norma IEEE 828 para la gestión de la configuración en ingeniería de software [28], apoyándose de forma sistemática en las tablas de riesgos (**de alcance, técnicos y temporales**) definidas en la sección de **riesgos iniciales**.

Cualquier propuesta de modificación se gestionará siguiendo este flujo secuencial:

1. **Identificación:** Se detecta la necesidad de cambio y se formaliza mediante una solicitud de cambio (*Change Request*, identificada como **CR-XX**), registrándola en el *backlog* del proyecto y clasificando si afecta a requisitos de software, planificación o entregables documentales.
2. **Análisis de impacto:** Se evalúa el esfuerzo requerido y se cuantifica su repercusión sobre la triple restricción del proyecto: **impacto en alcance, impacto en tiempo** (cronograma e hitos) e **impacto en coste**.
3. **Resolución y aprobación:** Se decide de forma justificada el estado del cambio (aprobado, pospuesto o descartado), determinando las acciones correctivas pertinentes.
4. **Implementación y registro:** Se ejecutan las tareas asociadas, se actualiza el cronograma o el código y se incorpora la ficha correspondiente al **registro de cambios** de la memoria.

4. Estado del arte y fundamentación

El éxito de una plataforma tecnológica no solo radica en su correcto funcionamiento, sino en su capacidad para aportar valor real frente a las soluciones existentes y sustentarse sobre decisiones de ingeniería justificadas. Antes de iniciar el diseño arquitectónico, es clave analizar el **ecosistema competitivo** y evaluar críticamente las **herramientas de desarrollo** disponibles en la industria actual.

A lo largo de las siguientes secciones se realiza un **análisis del mercado** de plataformas de evaluación mediante cuestionarios para identificar sus limitaciones y fijar los **factores diferenciadores** de *Quizzie*. Asimismo, se detalla la **fundamentación tecnológica** del proyecto, examinando las alternativas consideradas y justificando la selección del **stack definitivo**, para concluir con la síntesis estratégica del estudio previo.

4.1. Análisis del mercado

Para fundamentar la necesidad de desarrollar una nueva plataforma de evaluación, es imprescindible analizar el ecosistema de soluciones empleadas actualmente en el ámbito educativo. Aunque el mercado cuenta con soluciones consolidadas, el estudio de sus características revela limitaciones críticas en el control de la presencialidad, la exportación de datos y la asistencia al profesorado. A continuación, se examinan las tres alternativas más representativas:

- **Kahoot!:** Pionera en el ámbito de la gamificación educativa, destaca por su alta capacidad de fidelización y su interfaz visual atractiva. Sin embargo, su enfoque lúdico penaliza la rigurosidad académica: prioriza la velocidad de respuesta sobre la reflexión profunda, apenas ofrece analíticas avanzadas exportables en sus versiones gratuitas y carece de cualquier mecanismo que impida a un alumno participar de forma remota compartiendo el código PIN de la sala.
- **Wooclap:** Orientada a un entorno universitario y corporativo más formal, ofrece una excelente variedad de tipos de preguntas y herramientas de interacción en vivo. Su principal barrera reside en la usabilidad y la agilidad de los flujos de trabajo; el proceso de diseño de cuestionarios extensos resulta monótono y manual para el docente, y no tiene funcionalidades inteligentes de automatización que agilicen la generación de contenido académico.
- **Cuestionarios de Enseñanza Virtual:** Las plataformas institucionales integradas ofrecen una robustez indiscutible a nivel de seguridad, evaluación formal y exportación nativa de calificaciones. No obstante, sacrifican por completo la inmediatez y la experiencia de usuario. Su interfaz rígida, la complejidad técnica para configurar sesiones en tiempo real y la

imposibilidad de mitigar de forma sencilla el fraude por suplantación digital a distancia limitan su efectividad como dinámicas de aula ágiles.

Frente a este escenario, se ha sintetizado un análisis en la siguiente tabla comparativa del mercado, evaluando las características esenciales para la docencia presencial moderna. Las carencias detectadas en las alternativas comerciales e institucionales delimitan el espacio de oportunidad para una solución integral.

Característica	Kahoot!	Wooclap	Enseñanza Virtual	Quizzie
Evaluación en tiempo real	Sí	Sí	Limitado	Sí
Acceso rápido (sin registro de alumnos)	Sí	Sí	No	Sí
Verificación de presencialidad física	No	No	No	Sí
Generación de test asistida por IA	No	Limitado	No	Sí
Creación ágil de cuestionarios	Sí	Limitado	No	Sí
Visualización de estadísticas y analíticas	Sí	Sí	Limitado	Sí
Exportación de resultados	De pago	Limitado	Sí	Sí

Tabla 4.1: Tabla comparativa del mercado.

4.2. Factores diferenciadores

Como respuesta a las debilidades del mercado identificadas en la sección anterior, *Quizzie* se diseña no como una alternativa lúdica adicional, sino como un **entorno equilibrado** que fusiona el **dinamismo interactivo** con la **formalidad académica**. Sus elementos innovadores y ventajas competitivas se estructuran en los siguientes pilares:

- **Validación de presencia física mediante QR:** Para evitar el fraude por suplantación o que los alumnos participen a distancia, la plataforma genera un código QR único en el dispositivo del estudiante justo al finalizar el test, el cual incluye su nombre y la nota obtenida. El docente es quien escanea este código desde su propio dispositivo al terminar la clase, confirmando de manera inequívoca la asistencia física del alumno y registrando su calificación en el acto.
- **Creación inteligente asistida por IA:** Para resolver el flujo de trabajo monótono de diseñar cuestionarios desde cero, *Quizzie* integra un motor de Inteligencia Artificial que asiste al docente generando automáticamente baterías de preguntas coherentes a partir del temario o texto introducido, agilizando drásticamente la preparación de la clase.
- **Exportación directa de calificaciones:** La plataforma simplifica la gestión de las calificaciones permitiendo descargar las notas de los alumnos directamente

en formatos estándar como CSV o PDF, listos para su revisión o publicación en canales oficiales.

- **Accesibilidad y agilidad en el aula:** Los alumnos acceden de forma inmediata sin necesidad de registros previos ni cuentas obligatorias, mientras que el profesor dispone de un panel simplificado para lanzar y controlar el flujo del cuestionario con un solo clic.

4.3. Fundamentación tecnológica

A continuación, se presenta la arquitectura tecnológica que da soporte a *Quizzie*. Como vista general de la infraestructura, la siguiente tabla muestra la distribución definitiva de las herramientas seleccionadas siguiendo un orden ascendente desde la persistencia hasta el despliegue del cliente.

<i>Stack</i> tecnológico seleccionado	
Base de datos	SQLite
ORM / Validación	SQLModel
Backend	FastAPI (Python)
Lenguaje frontend	TypeScript
Frontend (SPA)	React + Vite
Estilos frontend	CSS <i>vanilla</i>
Alojamiento servidor	Render
Alojamiento cliente	Vercel
Flujo de CI/CD	GitHub Actions

Tabla 4.2: Resumen del *stack* tecnológico de *Quizzie*.

4.3.1. Alternativas consideradas y justificación de las decisiones

Para fundamentar la elección de cada componente del sistema, se analizan a continuación las matrices comparativas correspondientes a cada uno de los bloques lógicos de la aplicación.

Persistencia de datos

La gestión del almacenamiento del proyecto requiere evaluar la estructura de los datos frente a la complejidad de la infraestructura en la nube.

Tecnología	Ventajas	Desventajas	Elección
SQLite	Portabilidad en un solo archivo; cero administración de red y coste económico nulo.	Bloqueo del archivo completo durante la concurrencia de escritura masiva.	✓
PostgreSQL	Escalabilidad masiva y gestión avanzada de accesos y concurrencia.	Elevada complejidad de despliegue y administración de accesos en producción.	✗
MySQL	Motor relacional maduro, robusto y con gran soporte de la comunidad.	Costes económicos asociados a instancias en la nube innecesarios en esta fase.	✗
MongoDB	Flexibilidad de esquemas basada en documentos NoSQL y alta velocidad de lectura.	Descartado porque el dominio de datos del proyecto es puramente relacional.	✗

Tabla 4.3: Alternativas tecnológicas para la base de datos.

Mapeo objeto-relacional (ORM)

Se evalúa la forma de interactuar con el motor relacional buscando la mayor limpieza y eficiencia en el código.

Tecnología	Ventajas	Desventajas	Elección
SQLModel	Cumple el principio <i>DRY</i> : fusiona eficientemente esquemas de API y modelos de BD.	Herramienta más reciente en el ecosistema, con menor volumen de hilos de soporte.	✓
SQLAlchemy + Pydantic	Estándar clásico de la industria con máxima madurez y documentación.	Duplicidad de código obligatoria al tener que declarar los modelos por separado.	✗

Tabla 4.4: Alternativas tecnológicas para el ORM y validación.

Backend y servidor de sockets

La lógica de negocio exige un entorno que soporte una alta concurrencia interactiva en tiempo real.

Tecnología	Ventajas	Desventajas	Elección
FastAPI	Rendimiento asíncrono nativo (<i>async/await</i>) según los principios de aplicaciones web modernas de Fox y Patterson [29], ideal para mantener conexiones <i>WebSocket</i> concurrentes.	Estructura menos guiada en comparación con <i>frameworks</i> monolíticos.	✓
Node.js	Estándar tradicional para <i>WebSockets</i> ; unifica el lenguaje del proyecto a JavaScript.	Se descartó para priorizar el uso de Python debido a su ecosistema de IA.	✗
Django	<i>Framework</i> muy robusto, maduro y con arquitectura <i>batteries-included</i> basada en patrones empresariales de Fowler [30].	Estructura fundamentalmente síncrona por defecto y peso excesivo para una API REST.	✗
Flask	<i>Microframework</i> extremadamente simple, minimalista y rápido de configurar.	Carece de herramientas nativas de validación y de soporte asíncrono integrado.	✗

Tabla 4.5: Alternativas tecnológicas para el desarrollo del backend.

Lenguaje del cliente web

Se contrasta la seguridad del tipado frente a la velocidad de desarrollo directa en el frontend.

Tecnología	Ventajas	Desventajas	Elección
TypeScript	Tipado estático estricto; mitiga errores con contratos claros para <i>WebSockets</i> .	Requiere tiempo adicional en el ciclo de desarrollo para configurar interfaces.	✓
JavaScript <i>vanilla</i>	Curva de aprendizaje nula y compatibilidad nativa directa en el navegador.	Falta de seguridad de tipos en la mensajería asíncrona en tiempo real.	✗

Tabla 4.6: Alternativas para el lenguaje de programación frontend.

Frontend (interfaz de usuario)

Se analiza la arquitectura de las interfaces y la velocidad en la compilación del entorno de desarrollo.

Tecnología	Ventajas	Desventajas	Elección
React + Vite	<i>Virtual DOM</i> reactivo siguiendo las pautas de React de Porcello y Banks [31], patrones de interfaz según MacDonald [32] y compilación <i>HMR</i> instantánea.	Curva de aprendizaje inicial superior frente a opciones más básicas.	✓
Angular	<i>Framework</i> corporativo muy sólido, estructurado y altamente escalable.	Curva de aprendizaje muy empinada y verbosidad excesiva para el alcance del TFG.	✗
Vue.js	Alternativa equilibrada con un sistema de reactividad sencillo y progresivo.	Ecosistema y penetración en el mercado laboral inferior en comparación con React.	✗
Svelte	Máxima ligereza al eliminar el <i>virtual DOM</i> y compilar a código nativo.	Comunidad reducida y menor disponibilidad de librerías de terceros maduras.	✗
Webpack	Estándar histórico para el empaquetado de aplicaciones web.	Procesos de construcción muy lentos en desarrollo local comparado con Vite.	✗

Tabla 4.7: Alternativas tecnológicas para la interfaz de usuario.

Estilización visual de la interfaz

Se evalúa el equilibrio entre la velocidad de diseño y la sobrecarga estética en la aplicación del cliente.

Tecnología	Ventajas	Desventajas	Elección
CSS <i>vanilla</i>	Control total de layouts y animaciones sin dependencias ni sobrecarga.	Requiere la escritura manual de todos los estilos desde cero.	✓
Tailwind CSS	Maquetación veloz mediante clases de utilidad atómicas directas.	Saturación excesiva del marcado JSX, perjudicando la legibilidad del código.	✗
Bootstrap	Componentes preconstruidos adaptables y fáciles de implementar.	Estética genérica y obsoleta; complejiza la personalización visual premium.	✗

Tabla 4.8: Alternativas tecnológicas para el diseño de estilos visuales.

Infraestructura y despliegue

Se analizan los entornos de alojamiento priorizando la compatibilidad con canales bidireccionales y la viabilidad económica.

Tecnología	Ventajas	Desventajas	Elección
Vercel + Render	Plan gratuito sin tarjeta, GitOps automatizado y soporte ASGI para <i>WebSockets</i> .	El plan gratuito de Render sufre de retardos iniciales por <i>cold start</i> .	✓
AWS	Control absoluto e ilimitado de la infraestructura de red.	Configuración compleja y riesgo de costes económicos imprevistos.	✗
Heroku	Entorno <i>PaaS</i> históricamente óptimo para servidores de sockets interactivos.	Definición obsoleta por la eliminación por completo de sus planes gratuitos.	✗

Tabla 4.9: Alternativas para la infraestructura y el despliegue.

4.3.2. Limitaciones tecnológicas

A pesar de la idoneidad del stack seleccionado, se han identificado las siguientes limitaciones técnicas inherentes a las decisiones tomadas:

- **Cold start en Render (plan gratuito):** El contenedor del backend se desactiva tras 15 minutos sin recibir peticiones. Esto genera un retardo inicial de entre 30 y 50 segundos en la primera interacción de un usuario mientras el contenedor vuelve a iniciarse.
- **Concurrencia de escritura en SQLite:** SQLite bloquea la base de datos a nivel de archivo completo durante las operaciones de escritura. Aunque es imperceptible para un grupo estándar de alumnos, limitaría el escalado para miles de usuarios simultáneos registrando respuestas al unísono.
- **Falta de SSR (*server-side rendering*) en el frontend:** Al ser una SPA de React pura, el indexado por motores de búsqueda (SEO) es limitado. Sin embargo, al tratarse de una aplicación académica de acceso cerrado mediante credenciales, esta limitación no compromete su viabilidad.

4.3.3. Herramientas de soporte al desarrollo y control de calidad

La robustez del sistema y el cumplimiento de los estándares de calidad se articulan mediante las siguientes herramientas de soporte:

- **Control de versiones y flujo de trabajo:** Se utiliza **Git** como sistema de control local y **GitHub** como repositorio remoto. La gestión de ramas aplica el modelo de ramificación de Driessen [33] (*Git Flow*), manteniendo una rama

estable (main), una de integración (develop) y ramas de características (feat/*). Siguiendo las pautas de organización de Westby [34], cada funcionalidad se desarrolla de forma aislada en su propia rama antes de fusionarse directamente en develop.

- **Gestión de tareas y proyectos:** Para el seguimiento del cronograma se utiliza **GitHub Projects** estructurado como un tablero *Kanban*. Esto garantiza la trazabilidad del trabajo al asociar los *commits* y ramas de desarrollo con su correspondiente tarea (*issue*) e hito (*milestone*).
- **Estandarización de commits:** Para garantizar un historial de Git limpio, semántico y legible, se integra **Commitizen**. Esta herramienta obliga al autor a seguir la convención de *Conventional Commits*, categorizando de forma estricta cada cambio (por ejemplo, *feat*, *fix*, *docs* o *refactor*), lo que facilita la auditoría del código y la futura generación automatizada de diarios de cambios (*changelogs*).
- **Entorno de desarrollo integrado (IDE):** El entorno principal de trabajo es **Visual Studio Code**, seleccionado por su ligereza y su ecosistema de extensiones. Entre ellas, se ha configurado **LaTeX Workshop** para centralizar la composición y compilación local de la memoria académica mediante los motores de **MiKTeX** y **Strawberry Perl**.
- **Gestión de entornos y dependencias:** En el entorno de backend se ha adoptado **uv**, una herramienta de gestión de paquetes y entornos virtuales para Python extremadamente rápida escrita en Rust, que sustituye de forma eficiente a herramientas tradicionales como *pip* y *poetry*. Para el entorno frontend se utiliza **npm** como gestor de paquetes estándar de Node.js.
- **Generación y lectura de códigos QR:** Para dar soporte al sistema de validación de presencia física, en el frontend se integra la librería **qrcode.react** para renderizar de manera eficiente y reactiva el código QR dinámico en la pantalla del alumno al finalizar el test. Por la parte del docente, se utiliza **html5-qrcode** para acceder a la cámara del dispositivo móvil y procesar el escaneo en tiempo real con baja latencia.
- **Linter y formateo:** En el backend se utiliza **Ruff**, *linter* y formateador ultrarrápido programado en Rust que optimiza el cumplimiento de PEP 8 y unifica tareas que antes requerían múltiples librerías independientes (como *Flake8*, *Black* e *isort*). En el frontend se emplea **ESLint** con *flat config* para controlar la calidad de TypeScript junto a **Prettier** para garantizar un formateo de código unificado y consistente.
- **Pre-commit hooks:** Con la librería **pre-commit**, se interceptan los *commits* locales para garantizar que solo se sube código limpio que supere de forma automatizada las reglas de formato, tipado y análisis estático antes de salir del entorno local.
- **Pruebas unitarias y de integración:** Se utiliza **pytest** en el backend junto a **pytest-cov** para el cálculo de cobertura, y **Vitest** con **React Testing Library** en el frontend para evaluar de manera aislada el comportamiento de los

componentes. Para flujos de usuario extremo a extremo (E2E), se integra **Playwright**, permitiendo simular múltiples navegadores concurrentes.

- **Pruebas de carga:** Se emplea **Locust** para verificar de manera empírica la capacidad del servidor y medir la latencia del protocolo *WebSocket* al soportar la avalancha de respuestas concurrentes de los alumnos.
- **SonarQube:** Se ha configurado este auditor externo de análisis estático como parte del flujo de integración y entrega continuas (CI/CD) en GitHub Actions, aplicando los principios de canalizaciones de despliegue automatizadas descritos por Farley [5] para exigir una meta de calidad estricta (*quality gate A*, duplicación < 3 %, cobertura de tests > 80 %).

4.4. Conclusiones del estudio previo

El análisis del estado del arte y el estudio comparativo detallados en este capítulo ponen de manifiesto una brecha crítica en el ámbito de la evaluación gamificada en clase. A pesar de la madurez de las soluciones analizadas, el diagnóstico del mercado revela que ninguna alternativa resuelve con éxito el problema del **fraude por suplantación a distancia** ni ofrece flujos optimizados que mitiguen la carga administrativa del profesorado. La falta de mecanismos de **control de asistencia eficaces** en las herramientas actuales y la rigidez de los entornos virtuales tradicionales justifican plenamente la necesidad de desarrollar una solución integral que logre unificar **inmediatez, control de presencia real e interoperabilidad**.

Como respuesta directa a este escenario de oportunidad, la propuesta de valor de *Quizzie* se consolida a través de los factores diferenciadores definidos. El núcleo innovador del proyecto se materializa en el diseño de un **flujo de validación mediante códigos QR**, devolviendo al docente el control físico del aula sin penalizar el ritmo de la sesión. Asimismo, para contrarrestar la monotonía organizativa identificada en la competencia, la plataforma complementa su arquitectura con herramientas secundarias destinadas a agilizar la gestión diaria: la **generación de cuestionarios asistida por Inteligencia Artificial** y la **exportación directa de calificaciones** en formatos universales (CSV y PDF). Con ello, el proyecto trasciende el enfoque puramente lúdico y se posiciona como una **herramienta formal y eficiente** para el ecosistema educativo.

Para hacer esto posible, se ha elegido a conciencia un stack tecnológico donde cada pieza responde directamente a los retos de rendimiento del proyecto. La combinación de una base de datos relacional y ligera en local junto a un **servidor backend asíncrono** permite gestionar la **conurrencia de los cuestionarios** en el aula con total fluidez, demostrando que es viable construir un sistema interactivo robusto sin depender de infraestructura costosa en la nube. Esta selección técnica se cierra de forma lógica con el **ecosistema de automatización integrado**; al coordinar el formateo, las pruebas de rendimiento y la auditoría continua antes de cada despliegue, se garantiza que *Quizzie* no solo funcione bien en teoría, sino que sea un software **estable, mantenible y listo para su uso real** en clase.

5. Análisis de requisitos

Una vez contrastada la viabilidad de las ideas iniciales y analizadas las oportunidades del mercado, es imperativo **definir de manera precisa el comportamiento del sistema** antes de comenzar la fase de desarrollo técnico. Explicitar las funciones de negocio en este punto es un paso clave para construir un **producto robusto y libre de ambigüedades**. Con el fin de transformar de forma efectiva las aspiraciones generales plasmadas en los **objetivos del proyecto**, la lógica de la aplicación se desglosa inicialmente en doce objetivos funcionales concretos que asientan las bases del sistema.

ID	Nombre	Descripción
OBJ-01	Módulo de usuario y autenticación	Proveer un sistema de autenticación seguro para profesores.
OBJ-02	Gestión de cuestionarios	Permitir la creación, edición y configuración de cuestionarios de preguntas.
OBJ-03	Creación de salas	Habilitar sesiones de test únicas identificadas mediante un código PIN.
OBJ-04	Acceso rápido	Permitir el acceso de alumnos a las salas mediante PIN y <i>nickname</i> sin registro.
OBJ-05	Interacción en tiempo real	Sincronizar el estado de la partida y preguntas entre dispositivos.
OBJ-06	Verificación QR	Generar comprobante digital QR para validación presencial por el profesor.
OBJ-07	Persistencia validada	Almacenar en histórico solo los resultados verificados físicamente.
OBJ-08	Estadísticas y análisis	Visualizar estadísticas de rendimiento basadas en datos validados.
OBJ-09	Asistente IA	Integrar IA para generar preguntas automáticamente según temáticas.
OBJ-10	Escaneo de código QR	Habilitar el uso de la cámara para la lectura ágil de códigos QR.
OBJ-11	Gestión de grupos	Organizar alumnos en clases para un seguimiento histórico.
OBJ-12	Exportación de datos	Permitir la descarga de resultados en formatos CSV/Excel.

Tabla 5.1: Objetivos funcionales de *Quizzie*.

A lo largo de las siguientes secciones se identifican en primer lugar los actores que interactúan con la aplicación para, posteriormente, modelar su comportamiento mediante historias de usuario y casos de uso detallados.

Asimismo, se determinan los requisitos de información, las reglas de negocio y los requisitos funcionales asociados a sus respectivos criterios de aceptación. El capítulo concluye con la definición de las restricciones no funcionales y la matriz de trazabilidad que garantiza la cobertura total del desarrollo propuesto.

5.1. Historias de usuario

Una vez definidos los perfiles que interactúan con la plataforma, el comportamiento del sistema se define mediante historias de usuario siguiendo la metodología de elicitación de requisitos descrita por Sommerville [35]. Estas especificaciones permiten capturar las necesidades reales de cada actor siguiendo la estructura clásica de rol [**como**], acción [**quiero**] y beneficio [**para**]. Para facilitar la trazabilidad y el seguimiento de las historias, se han dividido y numerado de forma independiente según el perfil al que corresponden: el alumno (*HU-AL*) y el profesor (*HU-PR*).

ID	Descripción
HU-AL-01	Como alumno, quiero unirme a una sala solo con un PIN y un apodo para participar de forma inmediata sin necesidad de crear una cuenta.
HU-AL-02	Como alumno, quiero recibir la pregunta y las opciones de respuesta en mi móvil de forma sincronizada para competir en igualdad de condiciones con mis compañeros.
HU-AL-03	Como alumno, quiero saber si mi respuesta ha sido correcta justo después de enviarla para reforzar mi aprendizaje en el momento.
HU-AL-04	Como alumno, quiero que se genere un código QR único al finalizar el test para mostrárselo al profesor y que mi nota sea oficializada.
HU-AL-05	Como alumno, quiero ver mi posición en el ranking después de cada bloque de preguntas para motivarme durante la realización del cuestionario.
HU-AL-06	Como alumno, quiero poder reengancharme a la partida si pierdo la conexión a internet para no perder mi progreso y poder terminar el test.
HU-AL-07	Como alumno, quiero recibir puntos extra por responder rápido o mantener una racha de aciertos para sentirme recompensado por mi agilidad y constancia.

Tabla 5.2: Historias de usuario asociadas al rol de Alumno.

ID	Descripción
HU-PR-01	Como profesor, quiero crear y organizar mis propios cuestionarios para disponer de un repositorio de actividades personalizado por asignatura o tema.
HU-PR-02	Como profesor, quiero poder editar mis cuestionarios existentes para corregir errores o crear variantes rápidas para diferentes grupos.
HU-PR-03	Como profesor, quiero crear y administrar una sala con un PIN único para controlar el acceso de los alumnos y decidir cuándo comienza el test.
HU-PR-04	Como profesor, quiero ver en tiempo real qué están respondiendo los alumnos y quiénes se van conectando para llevar un control del progreso de la clase.
HU-PR-05	Como profesor, quiero poder cerrar la sala una vez iniciada la actividad para que ningún alumno se una tarde o de forma externa.
HU-PR-06	Como profesor, quiero escanear el móvil del alumno para confirmar su presencia física y validar su nota en el sistema evitando respuestas no autorizadas.
HU-PR-07	Como profesor, quiero crear grupos o clases para tener organizados a mis alumnos y facilitar el volcado de notas histórico.
HU-PR-08	Como profesor, quiero visualizar estadísticas de acierto por pregunta al finalizar para detectar conceptos que no han quedado claros y reforzarlos.
HU-PR-09	Como profesor, quiero generar borradores de preguntas mediante IA para reducir el tiempo de preparación de mis clases.
HU-PR-10	Como profesor, quiero exportar los resultados verificados a CSV para integrarlos en mis herramientas de gestión docente externas.
HU-PR-11	Como profesor, quiero configurar si se otorgan puntos extra por rapidez o racha para adaptar el nivel de competitividad de la sesión.
HU-PR-12	Como profesor, quiero decidir si el ranking se muestra tras cada pregunta para gestionar el ritmo y la atención de la clase.
HU-PR-13	Como profesor, quiero añadir etiquetas e imágenes a mis cuestionarios para identificarlos y personalizarlos visualmente.
HU-PR-14	Como profesor, quiero acceder al historial de salas pasadas para revisar resultados y analizar el progreso de mis alumnos en el tiempo.

Tabla 5.3: Historias de usuario asociadas al rol de Profesor.

5.2. Reglas de negocio y restricciones

El correcto funcionamiento de la plataforma *Quizzie* está sujeto a un conjunto de directrices operativas y limitaciones técnicas que garantizan tanto la validez funcional como la seguridad del sistema.

5.2.1. Reglas de negocio

Las reglas de negocio definen las políticas, derechos y leyes lógicas que gobiernan el flujo de trabajo de la aplicación:

- **Ciclo de vida de las salas:** Una sala de evaluación podrá encontrarse en espera, en juego, en fase de verificación o cerrada. Los alumnos carecerán de privilegios para alterar el estado operativo de la sesión.
- **Acceso a salas activas:** Un alumno solo podrá participar en una sala activa si introduce el código PIN correcto y un *nickname* que no esté siendo utilizado por otro participante dentro de la misma sesión.
- **Validación presencial de calificaciones:** Ninguna calificación obtenida en una sesión en vivo se consolidará en el histórico permanente ni será válida para su exportación si no ha sido verificada físicamente en el aula mediante el escaneo del código QR del alumno por parte del dispositivo del profesor.
- **Caducidad y unicidad del token QR:** El código QR generado por el dispositivo del alumno al finalizar el test contendrá un token criptográfico único asociado a su sesión. Este token será de un solo uso; una vez escaneado y marcado como "Verificado", el sistema bloqueará cualquier intento posterior de lectura para evitar su duplicación.
- **Propiedad y gestión de cuestionarios:** Los cuestionarios creados, editados o eliminados lógicamente en la plataforma estarán asociados únicamente al profesor que los diseñó. Un docente no podrá visualizar, modificar ni utilizar para una sala los cuestionarios propiedad de otro usuario del sistema.
- **Generación asistida por IA:** El sistema proporcionará la generación automatizada de preguntas mediante Inteligencia Artificial, actuando como una herramienta de ayuda para el profesor. Todo contenido generado se considerará una propuesta preliminar, siendo responsabilidad exclusiva del profesor su revisión, edición y aprobación final.
- **Almacenamiento del historial de salas:** Una sala de evaluación se trasladará de forma definitiva al historial de sesiones pasadas únicamente tras concluir con éxito la fase de verificación presencial. En este registro permanente solo se consolidarán las calificaciones e identidades de aquellos alumnos que completaron el flujo de validación mediante código QR.
- **Análisis de rendimiento:** El sistema generará métricas analíticas orientadas a facilitar la detección de tendencias de aprendizaje en el aula, identificando

automáticamente los conceptos con mayor índice de error y los patrones de acierto global del alumnado.

- **Exportación de resultados:** El profesorado dispondrá del derecho de portabilidad de las calificaciones oficiales del sistema, permitiendo la generación y descarga local de boletines de calificaciones en formatos estandarizados como PDF o CSV.
- **Seguimiento por grupos:** La plataforma permitirá la estructuración de los alumnos en grupos o clases con el fin de centralizar y evaluar la evolución histórica de un mismo conjunto de estudiantes a lo largo de diferentes sesiones académicas y cuestionarios.

5.2.2. Restricciones

Para que las políticas de negocio descritas anteriormente puedan ejecutarse de manera efectiva, el sistema debe acatar una serie de restricciones técnicas y de diseño, aplicando el principio de arquitectura con autoridad en el servidor expuesto por Gregory [36]:

- **Validación de *uvvus*:** El sistema restringirá el formato del *uvvus* de los alumnos para que coincida estrictamente con el patrón definido por la Universidad de Sevilla. El servidor validará mediante expresiones regulares que cumpla con los formatos oficiales, rechazando cualquier formato de nombre inválido.
- **Dimensionamiento de cuestionarios:** En la creación de contenido se impondrá un límite estructural rígido por motivos de maquetación de la interfaz móvil y rendimiento de la base de datos, restringiendo cada cuestionario a un máximo de 30 preguntas y fijando un límite máximo de 8 opciones de respuesta por cada enunciado.
- **Persistencia ante desconexiones:** Ante una pérdida accidental de conectividad a internet por parte de un alumno en plena partida, el backend retendrá su estado y puntuación. Esto permitirá su reconexión mientras la sala siga activa sin perder las respuestas previas a la desconexión.
- **Protocolos de comunicación síncrona:** Para garantizar la sincronización en tiempo real y asegurar que los eventos de la partida se procesen con la latencia mínima requerida, la comunicación bidireccional entre los dispositivos de la sala (profesor/alumnos) estará restringida exclusivamente al uso del protocolo seguro *WebSockets* (WSS).
- **Cifrado de credenciales:** El sistema operativo del backend prohibirá el almacenamiento de contraseñas en texto plano dentro de la base de datos. Será obligatorio aplicar un algoritmo de *hashing* seguro y unidireccional antes de cualquier operación de persistencia de credenciales de profesores.
- **Seguridad del código QR:** El *token* incrustado en el código QR del alumno deberá generarse mediante criptografía segura y no falsificable desde el servidor.

5.3. Requisitos de información

Los requisitos de información determinan los conjuntos de datos que el sistema debe almacenar de forma persistente para garantizar el correcto funcionamiento de la aplicación y dar soporte a las reglas de negocio e historias de usuario definidas. En esta fase de análisis no se busca detallar la estructura técnica de las tablas, las restricciones de integridad ni sus atributos específicos. Dichos aspectos se detallarán en el capítulo dedicado a **Diseño y arquitectura**.

ID	Nombre	Descripción
RI-01	Profesores y cuentas	Credenciales de acceso de los profesores (correo electrónico y el <i>hash</i> de la contraseña) y su nombre visible.
RI-02	Cuestionarios y preguntas	Datos relativos a los tests diseñados por los profesores. Incluye los campos básicos del cuestionario (título, descripción) y la composición de sus preguntas y opciones asociadas.
RI-03	Configuración y control de salas	Código PIN único de acceso, el estado operativo de la sala (espera, en juego, en fase de verificación o cerrada), el temporizador del servidor para la sincronización mediante <i>WebSockets</i> .
RI-04	Registro temporal de participación	Datos de alumnos conectados identificados mediante su <i>uvus</i> , opciones seleccionadas y puntuaciones obtenidas.
RI-05	Comprobantes de presencialidad y tokens QR	Validación de notas y mitigación del fraude digital. Debe almacenar el token criptográfico seguro y firmado digitalmente que se asocia a cada alumno al terminar un test, su estado de verificación actual.
RI-06	Histórico de calificaciones y respuestas	Salas que han completado con éxito la fase de verificación presencial. Debe registrar los mismos datos de cada alumno que el registro temporal, pero con el estado de verificación completado y persistiendo dichos datos.
RI-07	Gestión de grupos	Etiquetas de grupo o clase, profesor responsable y lista de alumnos asociados.

Tabla 5.4: Requisitos de información.

5.4. Requisitos funcionales

Los requisitos funcionales determinan las acciones específicas, comportamientos y servicios que la plataforma *Quizzie* debe ejecutar para satisfacer las necesidades de los usuarios. Este catálogo traduce las historias de usuario y las políticas lógicas del sistema en directrices operativas de desarrollo. A continuación, se detallan los requisitos funcionales del sistema:

ID	Nombre	Descripción	Prioridad
RF-01	Gestión de registro	Registro de nuevos profesores con validación de correo electrónico y <i>hash</i> de contraseña.	Alta
RF-02	Inicio de sesión	Autenticación de docentes mediante correo electrónico y contraseña cifrada con emisión de tokens JWT.	Alta
RF-03	Baja de usuarios	Eliminación de la cuenta de profesor y todos sus datos asociados de forma permanente del sistema.	Media
RF-04	Recuperación contraseña	Solicitud y tramitación de restablecimiento de credenciales de acceso vía correo electrónico.	Baja
RF-05	Edición perfil	Modificación de los datos de usuario del profesor, incluyendo el nombre visible y la contraseña.	Media
RF-06	Cierre de sesión	Invalidación del token de autenticación activo en el servidor y redirección automática al formulario de acceso.	Alta
RF-07	Crear cuestionario	Creación de nuevas pruebas de evaluación con un límite máximo estructurado de hasta 30 preguntas.	Alta
RF-08	Listar cuestionarios	Visualización del catálogo de pruebas diseñadas por el profesor, ordenadas cronológicamente por fecha.	Alta
RF-09	Mostrar ranking	Presentación de la tabla de clasificación de alumnos después de cada pregunta y al finalizar la sesión.	Media
RF-10	Aleatoriedad	Configuración opcional para alterar de forma automática el orden de las preguntas y sus respectivas respuestas.	Baja
RF-11	Configurar tiempo	Definición personalizada del tiempo límite de respuesta para cada enunciado durante la creación de la sala.	Media
RF-12	Importación	Carga de preguntas y opciones desde archivos externos estructurados exclusivamente en formato .txt o .csv.	Media
RF-13	Edición de cuestionarios	Modificación integral del título, descripción, enunciados y opciones de respuestas de un test existente.	Media
RF-14	Eliminación de datos	Borrado lógico en el sistema de cuestionarios o preguntas individuales sin destruir el histórico pasado.	Media
RF-15	Crear sala	Instanciación en tiempo real de una sesión de evaluación protegida mediante un código PIN único de acceso.	Alta
RF-16	Validar PIN	Verificación inmediata en el servidor de que el código PIN introducido corresponde a una sala en estado de espera.	Alta
RF-17	Validar <i>uvus</i>	Control de acceso que valida el formato alfanumérico institucional de la Universidad de Sevilla y mitiga duplicados.	Alta
RF-18	Sala de espera	Gestión y mantenimiento del flujo de alumnos conectados en tiempo real mediante <i>WebSockets</i> antes del inicio.	Alta
RF-19	Cerrar sala	Finalización manual o automática de la sesión, despliegue de resultados finales y desconexión de los participantes.	Media

Tabla 5.5: Requisitos funcionales (del RF-01 al RF-19).

ID	Nombre	Descripción	Prioridad
RF-20	Comenzar sala	Activación de la cuenta atrás sincronizada en el servidor y apertura de la primera pregunta del test.	Alta
RF-21	Distribución síncrona	Envío simultáneo de la secuencia de enunciados y opciones desde el backend a todos los clientes conectados.	Alta
RF-22	Temporizador servidor	Control centralizado y ejecución estricta del tiempo de respuesta remota en el hilo del backend.	Alta
RF-23	Recepción respuestas	Captura, marca de tiempo y procesamiento de la opción de respuesta seleccionada por el alumno.	Alta
RF-24	Feedback inmediato	Notificación instantánea en el dispositivo del alumno sobre el acierto o fallo sin consolidar la nota permanente.	Alta
RF-25	Registro provisional	Almacenamiento transitorio del resultado del participante bajo el estado de "No Verificado" emisión de un token único.	Alta
RF-26	Generación QR	Renderizado de un código QR dinámico en la pantalla del alumno que encapsula su identidad y su token criptográfico.	Alta
RF-27	Activación cámara	Solicitud de permisos e inicialización del flujo de vídeo de la cámara web en el dispositivo del profesor.	Alta
RF-28	Lectura de QR	Captura óptica, decodificación y extracción de la estructura de datos del alumno desde la interfaz docente.	Alta
RF-29	Validación Token	Comparación criptográfica en el servidor del token escaneado frente al registro transitorio de la sesión activa.	Alta
RF-30	Control de estado	Bloqueo lógico automático de solicitudes de validación de notas si el registro ya consta como "Verificado".	Alta
RF-31	Check verificación	Actualización de la sesión a estado verificado y consolidación oficial de la nota en el histórico definitivo.	Alta
RF-32	Listado en vivo	Actualización en tiempo real del panel docente reflejando la lista de alumnos validados presencialmente.	Media
RF-33	Estadísticas de sala	Cálculo de los porcentajes de acierto y fallo por enunciado para la detección automatizada de conceptos complejos.	Media
RF-34	Generación por IA	Procesamiento automatizado de textos introducidos por el docente para la sugerencia de borradores de preguntas mediante IA.	Media
RF-35	Asistente de ayuda	Resolución de dudas conceptuales y de uso del sistema mediante una interfaz de lenguaje natural integrada.	Baja
RF-36	Navegación por IA	Sistema de soporte inteligente encargado de detectar intenciones de uso y guiar al profesor en el entorno web.	Baja
RF-37	Revisión post IA	Interfaz de edición específica para la inspección manual, corrección y aprobación de las preguntas sugeridas por la IA.	Media

Tabla 5.6: Requisitos funcionales (del RF-20 al RF-37).

ID	Nombre	Descripción	Prioridad
RF-38	Crear clase	Agrupación e indexación de conjuntos estables de alumnos bajo etiquetas de grupo o clase académica.	Baja
RF-39	Exportación resultados	Generación y descarga local de las calificaciones oficiales y verificadas en formatos estandarizados PDF o CSV.	Alta
RF-40	Gestión de conexiones	Manejo de admisiones tardías, retención de estados por caída de red y flujos de reconexión adaptativa.	Media
RF-41	Filtros de búsqueda	Herramientas de filtrado avanzado en el catálogo de cuestionarios (activos, inactivos, recientes o etiquetas).	Media
RF-42	Bonificación por tiempo	Algoritmo de cálculo de puntuación adicional ponderada según la rapidez de respuesta del participante.	Baja
RF-43	Sistema de rachas	Concesión de multiplicadores de puntuación por el encadenamiento consecutivo de respuestas correctas.	Baja
RF-44	Modo de puntuación	Selección de las reglas de asignación de puntos (estándar o adaptativa) en los parámetros iniciales de la sala.	Baja
RF-45	Visibilidad de ranking	Opción para conmutar la presencia de la pantalla intermedia de clasificación global entre preguntas.	Baja
RF-46	Formulario avanzado	Inclusión de imágenes de soporte y gestión de etiquetas avanzadas (tags) en la edición del cuestionario.	Baja
RF-47	Historial de salas	Acceso permanente al registro histórico de sesiones cerradas que hayan completado la fase de verificación presencial.	Alta

Tabla 5.7: Requisitos funcionales (del RF-38 al RF-47).

5.4.1. Criterios de aceptación y definición de hecho

Para garantizar que los requisitos funcionales especificados alcancen los estándares de calidad, seguridad y robustez técnica exigidos, el proceso de desarrollo se rige por una *Definition of Done (DoD)*. Este marco actúa como una lista de comprobación obligatoria y transversal que cualquier funcionalidad debe satisfacer antes de ser dada por finalizada y consolidada en el sistema.

De manera general, un requisito funcional se considerará formalmente aceptado cuando cumpla con las siguientes dimensiones de verificación:

■ **Comportamiento funcional:**

- **Flujo principal:** Se ejecuta correctamente en la interfaz del usuario sin producir bloqueos, errores ni excepciones no controladas.
- **Gestión de excepciones:** El sistema gestiona los flujos alternativos y de error (entradas inválidas, caídas de red o datos duplicados) devolviendo un *feedback* informativo e inmediato al usuario.
- **Restricciones operativas:** Se respetan los límites físicos, lógicos y de formato estructurados en las reglas de negocio y restricciones del

sistema.

■ **Calidad técnica y seguridad:**

- **Seguridad de datos:** El código fuente cumple con las directrices relativas a la persistencia íntegra de la información y al cifrado estricto de credenciales.
- **Protocolos de red:** Las comunicaciones síncronas o en tiempo real emplean de forma estricta los canales bidireccionales y protocolos establecidos en la arquitectura.
- **Integridad del modelo:** Los datos asociados se mapean de forma exacta en las estructuras definidas en los requisitos de información.

■ **Verificación y pruebas:**

- **Cobertura de pruebas:** La funcionalidad tiene asociado, al menos, un caso de prueba unitario, de integración o de sistema dentro del plan de verificación.
- **Validación de resultados:** El comportamiento real del software coincide con el esperado tras superar con éxito las pruebas funcionales, de carga o de seguridad.

■ **Documentación y gestión del ciclo de vida:**

- **Mantenibilidad:** Se ha redactado la documentación técnica correspondiente en los módulos afectados, incluyendo tipado estricto y comentarios según las buenas prácticas de desarrollo.
- **Trazabilidad en control de versiones:** El *issue* asignado en GitHub se ha actualizado con la solución implementada, asociando los *commits* pertinentes y procediendo a su cierre formal tras la aprobación del despliegue.

Estos criterios globales de aceptación aseguran que el paso del análisis de requisitos a la fase de pruebas finales sea completamente **trazable, transparente y verificable**, sirviendo de base para la posterior verificación del sistema en la sección dedicada a los **casos de prueba y su validación**.

5.5. Casos de uso

Los diagramas y especificaciones de casos de uso sirven para modelar el comportamiento del sistema desde el punto de vista de los actores involucrados, transformando los requisitos funcionales abstractos en interacciones lógicas concretas. Con el propósito de mantener la claridad de la memoria y evitar redundancias estructurales, las funcionalidades de la plataforma *Quizzie* se estructuran en paquetes funcionales según el área operativa donde actúan los usuarios principales (*Profesor y Alumno*).

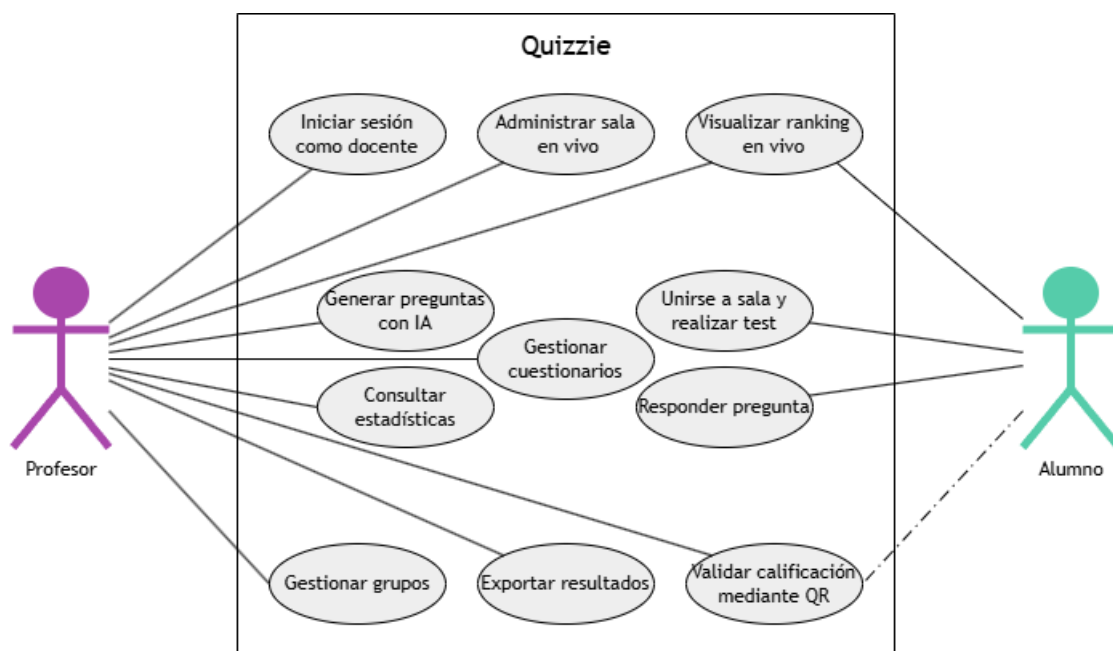


Figura 5.1: Diagrama general de casos de uso del sistema.

El conjunto de los casos de uso que componen el sistema es el siguiente:

- **CU-01: Iniciar sesión como docente.** Permite al profesor acceder a su panel privado, registrarse o cerrar sesión de forma segura.
- **CU-02: Gestionar cuestionarios.** El profesor puede crear, editar, listar y eliminar sus bancos de preguntas.
- **CU-03: Generar preguntas con IA.** Extensión del CU-02 donde el profesor solicita a la IA la creación de borradores de preguntas basados en un tema.
- **CU-04: Administrar sala en vivo.** El profesor instancia un cuestionario, genera un PIN de acceso y controla el flujo de la partida (inicio, paso de preguntas y cierre).
- **CU-05: Unirse a sala y realizar test.** El alumno accede mediante PIN, responde a las preguntas sincronizadas y recibe feedback inmediato sobre su rendimiento.
- **CU-06: Visualizar ranking en vivo.** Los alumnos y el profesor visualizan la progresión de los líderes de la sala tras cada pregunta.
- **CU-07: Validar calificación mediante QR.** El profesor utiliza su dispositivo para escanear el código QR del alumno, verificando su identidad y presencia física para oficializar la calificación.
- **CU-08: Consultar estadísticas.** El profesor analiza los puntos críticos y el rendimiento general de la clase tras finalizar la sesión.
- **CU-09: Exportar resultados.** El profesor descarga los datos de los alumnos verificados en formato CSV para su uso en plataformas externas.

- **CU-10: Responder pregunta.** El alumno puede responder durante el tiempo determinado a las preguntas del test obteniendo la puntuación pertinente.
- **CU-11: Gestionar grupos.** El profesor organiza el alumnado creando etiquetas de grupo para asociar y evaluar la evolución histórica de un mismo conjunto de estudiantes.

5.5.1. Especificación detallada de Casos de Uso

A continuación, se detallan las plantillas descriptivas de cada caso de uso del sistema, especificando sus actores, precondiciones, secuencias de pasos lógicos y la gestión de escenarios alternativos de error:

CU-01: Iniciar sesión como docente	
Actor principal	Profesor.
Precondición	El profesor debe estar registrado en la plataforma.
Flujo principal	1. El profesor introduce su correo electrónico y contraseña en el formulario de login. 2. El sistema cifra la contraseña introducida y la compara con el <i>hash</i> almacenado. 3. El sistema genera un token de sesión (JWT) y redirige al profesor a su panel principal.
Flujo alternativo	Si las credenciales son incorrectas, el sistema muestra un mensaje de error y permite reintentar el acceso.

Tabla 5.8: Especificación del caso de uso CU-01.

CU-02: Gestionar cuestionarios	
Actor principal	Profesor.
Precondición	El profesor ha iniciado sesión correctamente.
Flujo principal	1. El profesor accede a su biblioteca de contenidos. 2. Selecciona la opción de crear un nuevo cuestionario o editar uno existente. 3. El profesor define el título, descripción y añade preguntas con sus respectivas opciones. 4. El profesor marca la opción correcta para cada pregunta y asigna una puntuación. 5. El sistema guarda los cambios en la base de datos permanente.
Postcondición	El cuestionario queda disponible para ser utilizado en una sala.

Tabla 5.9: Especificación del caso de uso CU-02.

CU-03: Generar preguntas con IA	
Actores	Principal: Profesor. Secundario: Servicio de IA (API).
Precondición	El profesor se encuentra dentro del editor de cuestionarios.
Flujo principal	1. El profesor activa el asistente de IA e introduce un tema o pega un texto base. 2. El sistema envía la información a la API de IA externa. 3. El sistema recibe y presenta al profesor un listado de preguntas y respuestas sugeridas. 4. El profesor selecciona las preguntas que desea importar y las edita si es necesario. 5. El sistema integra las preguntas seleccionadas en el cuestionario actual.

Tabla 5.10: Especificación del caso de uso CU-03.

CU-04: Administrar sala en vivo	
Actor principal	Profesor.
Precondición	El profesor está autenticado y ha seleccionado un cuestionario de su biblioteca.
Flujo principal	1. El profesor solicita la creación de una nueva sala de juego. 2. El sistema genera un PIN único de 6 dígitos y abre el <i>socket</i> de comunicación. 3. El profesor controla en tiempo real la entrada de alumnos en la sala de espera. 4. El profesor inicia el test, enviando la señal de comienzo a todos los alumnos. 5. El profesor controla el paso de las preguntas o permite el avance automático. 6. El profesor finaliza la sesión manualmente o al terminar las preguntas.

Tabla 5.11: Especificación del caso de uso CU-04.

CU-05: Unirse a sala y realizar test	
Actor principal	Alumno.
Precondición	Existe una sala activa y el alumno posee el código PIN.
Flujo principal	1. El alumno introduce el PIN y un <i>nickname</i>. 2. El sistema valida el acceso y posiciona al alumno en la sala de espera. 3. Al comenzar el test, el alumno recibe las preguntas de forma sincronizada con el resto de la clase. 4. El alumno selecciona la opción deseada dentro del tiempo límite. 5. El sistema registra la respuesta y el tiempo empleado. 6. Al finalizar el <i>quiz</i>, el alumno recibe un código QR único de validación.
Flujo alternativo	Si el alumno sufre una desconexión, puede reingresar con el mismo PIN y <i>nickname</i> ; el sistema restaurará su sesión y lo situará en la pregunta activa.

Tabla 5.12: Especificación del caso de uso CU-05.

CU-06: Visualizar ranking en vivo	
Actores	Principal: Profesor / Alumno.
Precondición	Se ha completado al menos una pregunta en una sala activa.
Flujo principal	1. El sistema calcula la puntuación de cada alumno basándose en aciertos y velocidad de respuesta. 2. El sistema actualiza el ranking global de la sala. 3. El profesor visualiza el “Top 5” en la pantalla principal para motivar a la clase. 4. El alumno visualiza su posición relativa y sus puntos en su propio dispositivo.

Tabla 5.13: Especificación del caso de uso CU-06.

CU-07: Validar calificación mediante QR	
Actores	Principal: Profesor. Secundario: Alumno.
Precondición	La sala ha finalizado y el alumno muestra el QR generado por el sistema.
Flujo principal	1. El profesor selecciona la opción de “Validar Alumno” en su panel de sala. 2. El profesor escanea el código QR del dispositivo del alumno. 3. El sistema extrae el token cifrado del QR y lo valida contra el registro provisional en el backend. 4. Tras la validación positiva, el sistema cambia el estado del alumno a “Verificado”. 5. La nota se hace persistente y se vincula oficialmente al registro del alumno.

Tabla 5.14: Especificación del caso de uso CU-07.

CU-08: Consultar estadísticas	
Actor principal	Profesor.
Precondición	Existen salas finalizadas con respuestas registradas.
Flujo principal	1. El profesor accede al histórico de salas desde su panel. 2. Selecciona una sesión específica para analizar. 3. El sistema genera gráficas comparativas sobre el porcentaje de aciertos por pregunta. 4. El profesor identifica las áreas de conocimiento donde los alumnos han tenido más dificultades.

Tabla 5.15: Especificación del caso de uso CU-08.

CU-09: Exportar resultados	
Actor principal	Profesor.
Precondición	La sala ha finalizado y existen alumnos con estado “Verificado”.
Flujo principal	1. El profesor solicita la exportación de resultados desde el informe de la sala. 2. El sistema filtra los datos para incluir únicamente los resultados verificados. 3. El sistema genera un archivo en formato CSV o Excel. 4. El archivo se descarga automáticamente en el dispositivo del profesor.

Tabla 5.16: Especificación del caso de uso CU-09.

CU-10: Responder pregunta	
Actor principal	Alumno.
Precondición	El alumno está en una sala activa y el profesor ha lanzado una pregunta.
Flujo principal	1. El sistema muestra el enunciado y las opciones de respuesta en el dispositivo del alumno. 2. El sistema inicia el cronómetro de cuenta atrás definido para esa sala. 3. El alumno selecciona una de las opciones antes de que el tiempo se agote. 4. El sistema captura la opción elegida y el tiempo de la respuesta (<i>timestamp</i>). 5. Al agotarse el tiempo o responder todos los alumnos, el sistema cierra la recepción de datos. 6. El sistema mira si la respuesta es correcta y calcula los puntos obtenidos.
Flujo alternativo	Si el alumno no selecciona ninguna opción antes de que el cronómetro llegue a cero, el sistema registra la pregunta como “No contestada” con 0 puntos.
Postcondición	El resultado de la pregunta se almacena temporalmente para actualizar el ranking en vivo.

Tabla 5.17: Especificación del caso de uso CU-10.

CU-11: Gestionar grupos	
Actor principal	Profesor.
Precondición	El profesor ha iniciado sesión correctamente en el sistema.
Flujo principal	1. El profesor accede a la sección de gestión de grupos desde el panel de control. 2. Selecciona la opción de crear una nueva clase e introduce un nombre descriptivo o etiqueta de grupo. 3. El profesor asocia la lista de identificadores de alumnos válidos correspondientes a ese conjunto académico. 4. El sistema almacena la estructura del grupo vinculándola directamente al perfil del docente responsable. 5. El profesor consulta los reportes específicos para comprobar el progreso y la evolución histórica de las calificaciones de dicha clase.

Tabla 5.18: Especificación del caso de uso CU-11.

5.6. Requisitos no funcionales

Los requisitos no funcionales determinan los atributos de calidad, restricciones técnicas y criterios de rendimiento que debe satisfacer la plataforma *Quizzie*. Su categorización se articula atendiendo a las dimensiones del modelo de calidad del producto de software definido en la norma ISO/IEC 25010 [37]:

Dimensión	ID	Nombre	Descripción
Rendimiento	RNF-01	Sincronización	Los eventos en tiempo real deben procesarse de forma óptima en el backend para garantizar la coherencia síncrona en el aula.
	RNF-02	Concurrencia	Soporte mínimo de 100 alumnos interactuando de forma simultánea dentro de una misma sala sin degradación de los servicios básicos.
Seguridad	RNF-03	Integridad	El token de validación presencial embebido en el QR del alumno debe ser criptográficamente seguro, firmado por el servidor y no falsificable.
	RNF-04	Privacidad	Almacenamiento e inserción obligatoria de credenciales de acceso de profesores mediante algoritmos de <i>hashing</i> seguro y unidireccional.
	RNF-05	Cifrado	Uso estricto de canales de red protegidos bajo protocolos de comunicación seguros HTTPS y <i>WebSockets</i> seguros (WSS).
Usabilidad	RNF-06	Adaptabilidad	Interfaz de usuario adaptativa (<i>Responsive Design</i>) optimizada para una navegación fluida tanto en dispositivos móviles como en ordenadores.
	RNF-07	Eficiencia QR	Optimización del flujo óptico para que el tiempo de lectura, decodificación y validación de la firma del código QR por la cámara sea inmediato.
Fiabilidad	RNF-08	Reconexión	Capacidad de retener y recuperar el estado lógico y la puntuación transitoria de un participante ante caídas o pérdidas accidentales de conectividad.
Disponibilidad	RNF-09	Persistencia	Mecanismo de salvaguarda y respaldo de los resultados provisionales en el backend ante interrupciones críticas o fallos imprevistos del servidor.
Mantenibilidad	RNF-10	Modularidad	Separación clara de responsabilidades entre el servidor de base de datos, el backend y el cliente frontend para facilitar futuras actualizaciones.

Tabla 5.19: Requisitos no funcionales clasificados por dimensión.

5.7. Trazabilidad de requisitos

Para garantizar la integridad del desarrollo y demostrar el cumplimiento de las metas del proyecto, este apartado presenta la trazabilidad bidireccional del sistema. Este proceso permite certificar que cada decisión responde directamente a un fin práctico de la aplicación, evitando la existencia de elementos redundantes o injustificados. Las tres matrices que se muestran a continuación relacionan objetivos funcionales con requisitos de información, requisitos no funcionales y requisitos funcionales, respectivamente.

RI \ OBJ	01	02	03	04	05	06	07	08	09	10	11	12
01	×											
02		×							×			
03			×	×	×							
04					×	×	×	×				
05							×	×				×
06											×	×

Tabla 5.20: Matriz de trazabilidad RI-OBJ.

RNF \ OBJ	01	02	03	04	05	06	07	08	09	10	11	12
01					×							
02					×							
03						×	×					
04	×											
05	×				×							
06	×	×		×	×							
07										×		
08					×							
09			×		×		×					
10	×	×	×	×	×	×	×	×	×	×	×	×

Tabla 5.21: Matriz de trazabilidad RNF-OBJ.

RF \ OBJ	01	02	03	04	05	06	07	08	09	10	11	12
01	X											
02	X											
03	X											
04	X											
05	X											
06	X											
07		X										
08		X										
09								X				
10		X										
11			X									
12												X
13		X										
14		X										
15			X									
16				X								
17				X								
18			X									
19			X									
20					X							
21					X							
22			X									
23					X							
24					X							
25							X					
26						X						
27										X		
28										X		
29						X						
30							X					
31							X					
32								X				
33								X				
34									X			
35									X			
36									X			
37									X			
38											X	
39												X
40					X							
41		X										
42					X							
43					X							
44			X									
45			X									
46		X										
47							X					

Tabla 5.22: Matriz de trazabilidad RF-OBJ.

Por último, con el fin de unificar todo el ciclo de vida del desarrollo, se presenta la siguiente matriz de trazabilidad unificada. Este mapa conecta de manera jerárquica los módulos del sistema, los objetivos funcionales que persiguen, los casos de uso técnicos, y cómo estos se desglosan en las historias de usuario, compuestas por sus respectivos requisitos funcionales.

Módulo	OBJ	CU	HU	RF
Contenido	OBJ-02	CU-02	HU-PR-01	RF-07, RF-08, RF-10, RF-12
			HU-PR-02	RF-13, RF-14, RF-41
			HU-PR-13	RF-46
	OBJ-09	CU-03	HU-PR-09	RF-34, RF-35, RF-36, RF-37
Salas	OBJ-03	CU-04	HU-PR-03	RF-11, RF-15, RF-18, RF-22
			HU-PR-05	RF-19
			HU-PR-11	RF-42, RF-43, RF-44
			HU-PR-12	RF-45
	OBJ-04	CU-05	HU-AL-01	RF-16, RF-17, RF-40
	OBJ-05	CU-05	HU-AL-01	RF-16, RF-17, RF-40
		CU-10	HU-AL-02	RF-20, RF-21
			HU-AL-03	RF-23, RF-24, RF-42, RF-43
	OBJ-06	CU-07	HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
			HU-AL-04	RF-25, RF-26
	OBJ-07	CU-07	HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
			HU-AL-04	RF-25, RF-26
	OBJ-08	CU-06	HU-PR-04	RF-32
			HU-PR-12	RF-09
		CU-08	HU-PR-08	RF-33
			HU-PR-14	RF-47
	OBJ-10	CU-07	HU-PR-06	RF-27, RF-28, RF-29, RF-30, RF-31
			HU-AL-04	RF-25, RF-26
	OBJ-12	CU-09	HU-PR-10	RF-39
Usuarios	OBJ-01	CU-01	HU-PR-01	RF-01, RF-02, RF-03, RF-04, RF-05, RF-06
	OBJ-11	CU-11	HU-PR-07	RF-38

Tabla 5.23: Matriz de trazabilidad unificada del sistema.

6. Diseño y arquitectura

En este capítulo se describe la **estructura técnica y conceptual** sobre la que se asienta el sistema, detallando las decisiones arquitectónicas que garantizan su correcto **funcionamiento, mantenibilidad y rendimiento**. El diseño propuesto responde tanto a los requisitos funcionales del proyecto como a sus restricciones operativas: la ejecución de cuestionarios en tiempo real, la baja latencia en la transmisión de eventos a múltiples participantes simultáneos y una rigurosa separación de responsabilidades entre el cliente y el servidor.

6.1. Patrones arquitectónicos elegidos

Antes de seleccionar el *stack* y la estructura del código, se establecieron una serie de principios de diseño orientados a garantizar la integridad de las calificaciones, la seguridad del sistema y su capacidad de escalado:

- **Validación centralizada y control de roles (*anti-cheat*):** El cliente no toma decisiones sobre la corrección de respuestas, cálculo de puntuaciones ni temporización oficial. Existe una estricta división de responsabilidades en la que las acciones administrativas (como el control del avance de preguntas o el cierre de salas) quedan reservadas exclusivamente al profesor. El alumno se limita únicamente a unirse a la sala y enviar sus respuestas.
- **Desacoplamiento de la presentación y los datos:** El servidor se limita a enviar los datos y eventos de la partida, dejando que la aplicación del usuario se ocupe exclusivamente de cómo mostrar la información en pantalla, gestionar las animaciones y adaptarse a cada dispositivo.
- **Autenticación sin estado (*stateless*):** La validación de identidades mediante tokens integrados en las cabeceras de cada solicitud evita el almacenamiento de estado de sesión en la memoria del servidor, facilitando la distribución de la carga.
- **Gestión de la concurrencia y escalabilidad:** El sistema debe ser capaz de mantener decenas de conexiones bidireccionales simultáneas en un aula sin bloquear el procesamiento del servidor, distribuyendo el rendimiento de forma eficiente entre peticiones HTTP habituales y eventos en tiempo real.

Para dar respuesta a estos principios, se ha optado por una arquitectura híbrida que combina tres patrones complementarios: el modelo **cliente-servidor desacoplado**, formulado por Bass *et al.* [38] para la separación de responsabilidades; la **arquitectura en capas** (*layered architecture*), formalizada por Martin [4] para el aislamiento de las reglas de negocio; y la **arquitectura basada en eventos** (*event-driven architecture*), fundamentada en los patrones de comunicación asíncrona de Hohpe y Woolf [39].

6.1.1. Arquitectura cliente-servidor desacoplada

La aplicación adopta un esquema cliente-servidor desacoplado con una rigurosa delimitación de responsabilidades entre el cliente y el servidor:

- **Cliente (*frontend* - SPA):** Desarrollado como una (*SPA (single page application)*), se encarga de la representación visual, la experiencia de usuario y el control de temporizadores locales de apoyo. Opera sin acceso directo a la persistencia ni a las reglas de negocio.
- **Servidor (*backend* - API REST + WebSockets):** Actúa como la única fuente de verdad (*single source of truth*), gestionando la autenticación, la seguridad criptográfica, el ciclo de vida de los cuestionarios, la persistencia en base de datos y la orquestación síncrona de las salas.
- **Canales de comunicación:**
 - *HTTP/HTTPS (API REST):* Empleado para operaciones síncronas sin estado, tales como el inicio de sesión, la gestión de cuestionarios y la consulta de informes históricos.
 - *WebSockets:* Empleado para mantener un canal de comunicación persistente, bidireccional y de baja latencia durante la ejecución en vivo de los cuestionarios.

6.1.2. Arquitectura en capas

Tanto en el servidor como en el cliente, el sistema se estructura internamente en niveles independientes para aislar responsabilidades (*separation of concerns*).

En el *backend*, la lógica se organiza en cuatro capas conceptuales:

1. **Capa de presentación y controladores:** Expone los puntos de entrada HTTP (endpoints REST) y los canales *WebSocket*. Valida las peticiones de entrada, gestiona las respuestas de la API y coordina la autenticación.
2. **Capa de lógica de negocio y servicios:** Implementa las reglas del dominio: suma de puntuaciones por respuesta, transiciones de estado de las salas, opciones de cuestionario en vivo y lógica de verificación.
3. **Capa de acceso a datos y modelado (ORM):** Mapea las entidades de negocio del dominio con el modelo relacional, garantizando las restricciones de integridad y las relaciones entre entidades.
4. **Capa de persistencia:** Administra la conexión con el motor de base de datos relacional y gestiona las transacciones de lectura y escritura.

En el *frontend*, la arquitectura reproduce este aislamiento separando la **capa de vistas** (componentes de interfaz), la **capa de estado y contexto** (sesión de usuario y datos de partida), la **capa de servicios de red** (módulos REST y sockets) y la **capa de contratos de datos** (definición de tipos e interfaces).

6.1.3. Arquitectura basada en eventos

Para las sesiones en tiempo real, el sistema adopta una arquitectura basada en eventos que mantiene sincronizados a todos los participantes conectados a una sala.

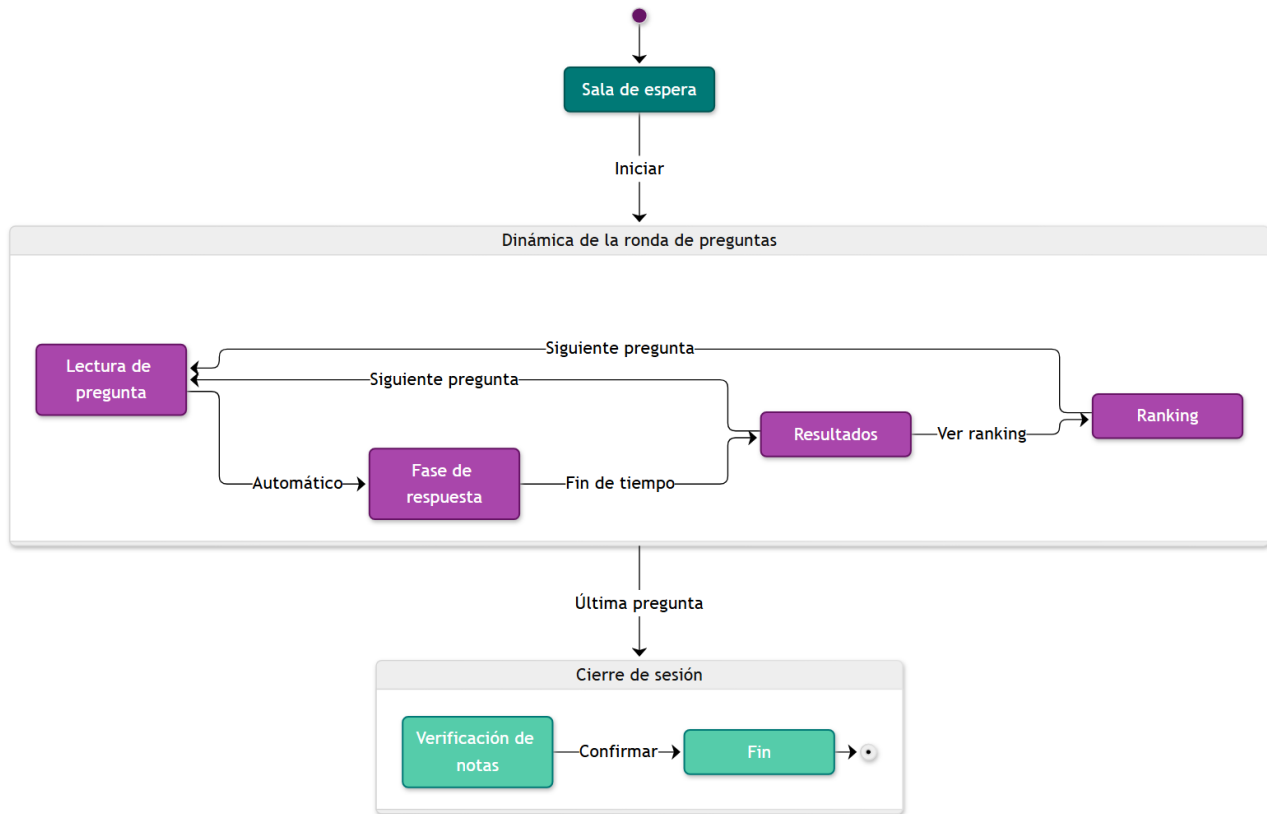


Figura 6.1: Estados de la sala de juego en tiempo real.

El diseño de este comportamiento se apoya en los siguientes componentes conceptuales:

- **Gestor centralizado de conexiones:** Mantiene el registro de las sesiones activas en memoria, administrando el ciclo de vida de los sockets y aplicando el patrón de difusión (*broadcasting*) para emitir instrucciones a todos los participantes de una sala de forma simultánea.
- **Bucle de sincronización temporal:** Proceso asíncrono en segundo plano que supervisa el transcurso del tiempo de respuesta en las salas activas, notifica el tiempo restante periódicamente y desencadena automáticamente la transición de fase al expirar el tiempo.
- **Ciclo de eventos y transmisión:** Los cambios de estado de la sala (fase de lectura, apertura de respuestas, muestra de resultados, actualización de temporizadores) se propagan como eventos estructurados JSON, permitiendo que la interfaz del cliente reaccione de forma inmediata.

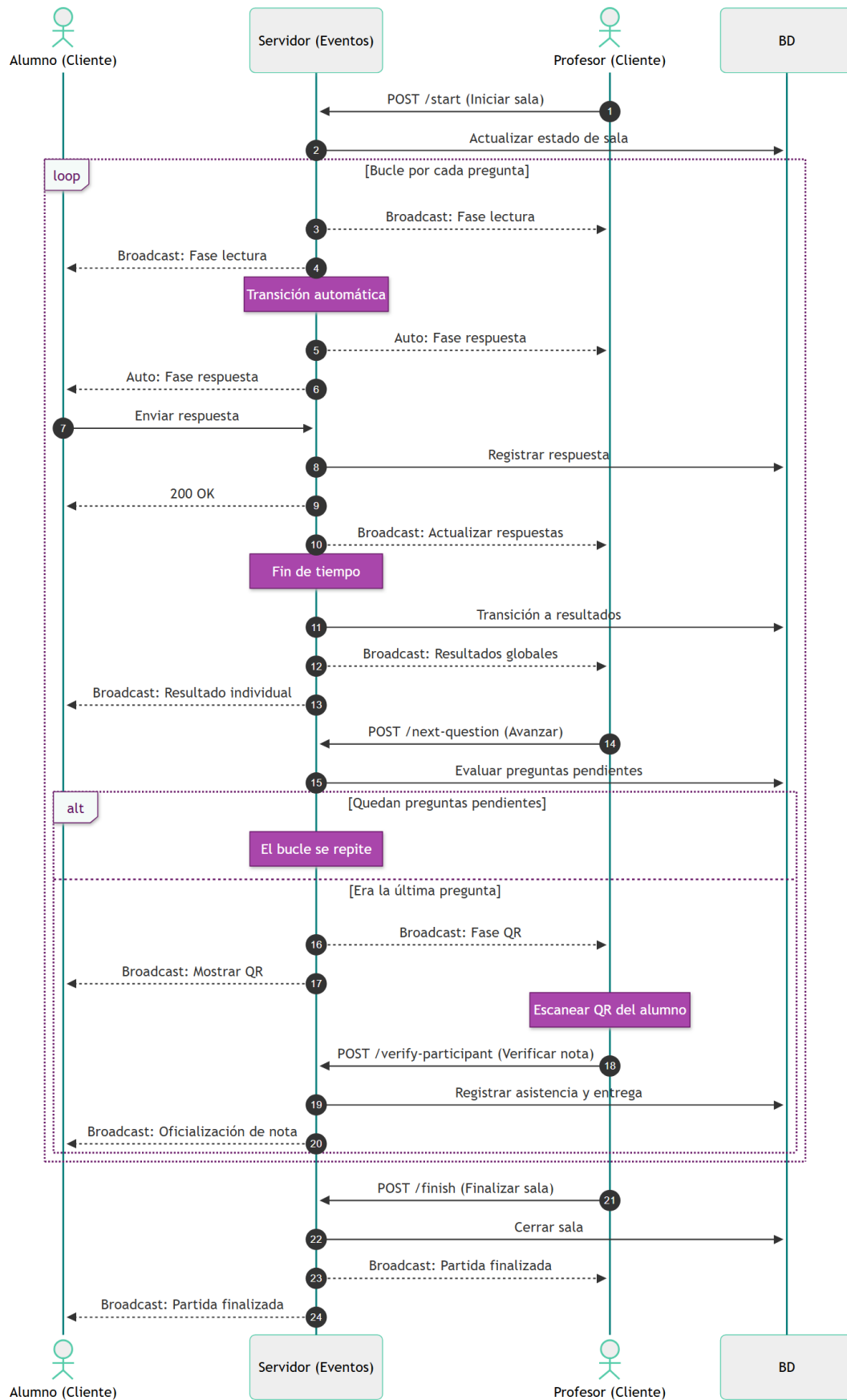


Figura 6.2: Diagrama de secuencia de la transmisión de eventos en una sala.

6.2. Arquitectura global del sistema

La combinación de los tres patrones previamente descritos se consolida en una estructura global unificada. En la comparativa de la arquitectura se presenta, a la izquierda, el marco conceptual abstracto de la arquitectura global y, a la derecha, su correspondencia directa con los componentes de software, módulos y tecnologías reales que ejecutan el sistema.

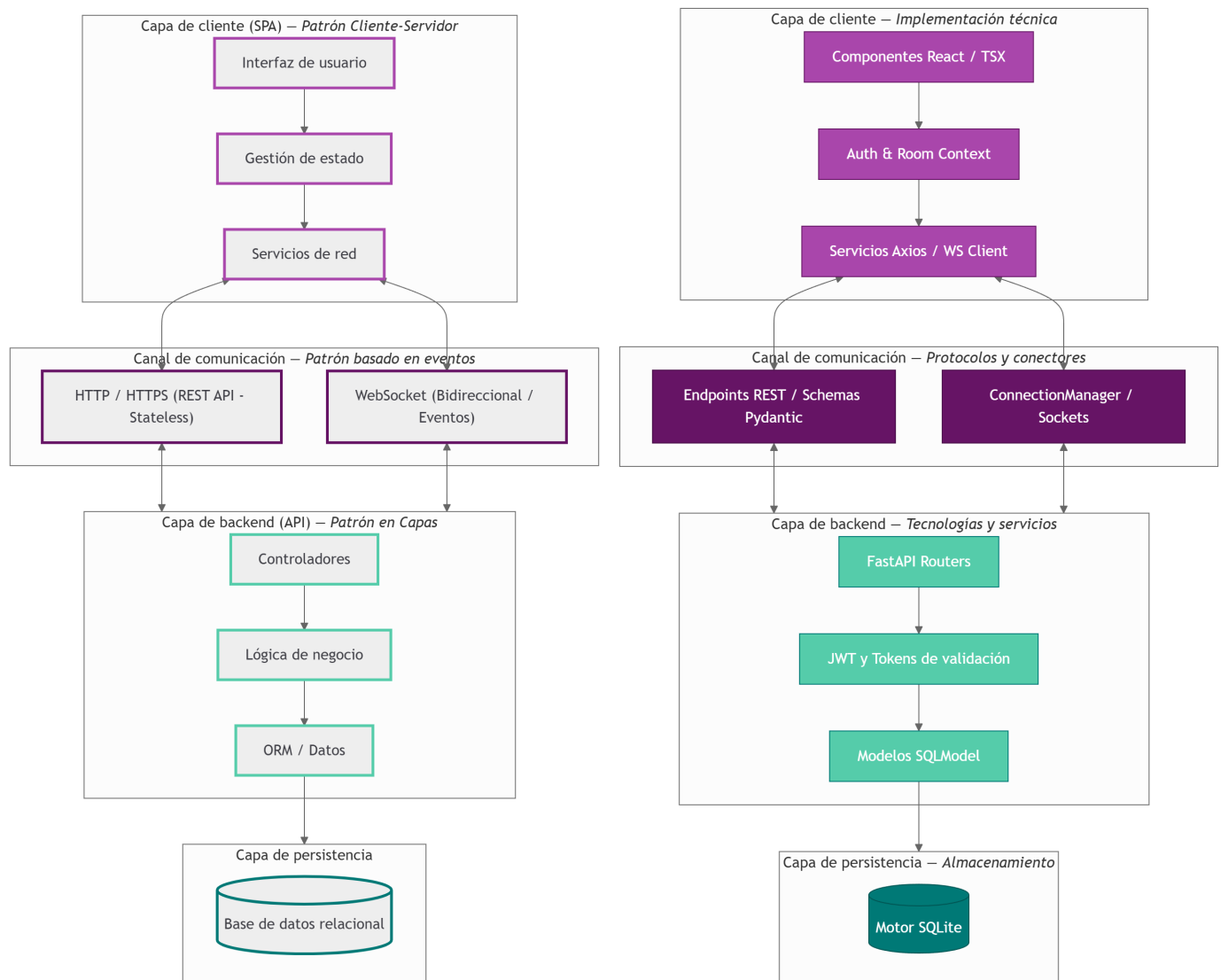


Figura 6.3: Arquitectura global del sistema y su implementación concreta.

Como se observa en la comparativa, cada capa conceptual se corresponde de forma directa con sus módulos de software equivalentes: la interfaz y el estado en el cliente React, los controladores y gestores de eventos en FastAPI, y el mapeo de datos mediante SQLAlchemy sobre el motor SQLite en la persistencia.

6.3. Justificación y decisiones de arquitectura

Una vez definidos el esquema conceptual y la estructura de componentes del sistema, resulta imprescindible formalizar las decisiones tecnológicas adoptadas y verificar su alineación con las necesidades del proyecto. En este apartado se aborda dicho proceso desde dos perspectivas complementarias: la documentación formal de las **decisiones de diseño clave** y la verificación explícita de su impacto sobre los **atributos de calidad** del proyecto.

6.3.1. Registros de decisiones de arquitectura (ADR)

Para documentar y formalizar las decisiones de diseño estructural más críticas del proyecto, se ha adoptado el patrón de registros de decisiones de arquitectura (*Architecture Decision Records*, **ADR**) propuesto por Nygard [40]. A continuación se detallan las decisiones arquitectónicas que fundamentan el diseño de *Quizzie*:

ADR-01: Adopción de FastAPI y WebSockets para la comunicación en tiempo real	
Estado	Aprobado.
Contexto	La app requiere gestionar la comunicación bidireccional y síncrona con hasta 100 alumnos por sala, ofreciendo baja latencia en la transmisión de eventos.
Alternativas	API REST mediante <i>polling</i> tradicional y <i>Server-Sent Events</i> (SSE).
Decisión	Utilizar FastAPI, motor asíncrono con protocolo <i>WebSockets</i> bidireccional nativo.
Consecuencias	Positivas: Se logra una latencia inferior a 50 ms en la propagación de eventos y un consumo mínimo de RAM por conexión activa.
	Negativas: Exige gestionar manualmente la reconexión y retención de estado de sockets en el cliente.

Tabla 6.1: Registro de decisiones de arquitectura - ADR-01.

ADR-02: Arquitectura basada en eventos para el manejo de salas	
Estado	Aprobado.
Contexto	El ciclo de vida de una sala exige informar inmediatamente a todos los clientes en tiempo real, ya que todos deben estar en la misma fase en todo momento.
Alternativas	Bucle síncrono <i>polling</i> desde el cliente o <i>sockets</i> puros sin tipado de eventos.
Decisión	Diseñar un gestor centralizado de eventos (<i>ConnectionManager</i>) que difunda mensajes JSON mediante un patrón de publicación/suscripción local.
Consecuencias	Positivas: Desacoplamiento total entre la lógica del juego y la capa de transporte; facilidad para testear eventos aislados.
	Negativas: Dificultad en serialización y deserialización de esquemas Pydantic.

Tabla 6.2: Registro de decisiones de arquitectura - ADR-02.

ADR-03: Selección del stack de persistencia SQLite + SQLAlchemy	
Estado	Aprobado.
Contexto	Se requiere un motor relacional liviano, de coste cero y portabilidad total, minimizando la carga operativa en el servidor sin duplicar esquemas de validación de datos.
Alternativas	PostgreSQL administrado en la nube o SQLAlchemy + Pydantic desacoplados.
Decisión	Adoptar SQLite como motor relacional en contenedor combinado con SQLAlchemy como ORM unificado.
Consecuencias	Positivas: Cumplimiento estricto del principio <i>DRY</i> (<i>Don't Repeat Yourself</i>); despliegue sin dependencias complejas de infraestructura.
	Negativas: Bloqueo a nivel de archivo en escrituras masivas concurrentes.

Tabla 6.3: Registro de decisiones de arquitectura - ADR-03.

ADR-04: Autenticación híbrida JWT + Código QR presencial	
Estado	Aprobado.
Contexto	Es necesario conciliar el acceso seguro del profesor (persistente) con la incorporación rápida del alumno (efímera y sin registro), garantizando que la nota solo se consolide si el estudiante está presente en el aula.
Alternativas	Cuentas obligatorias de usuario para alumnos o comprobación manual en lista por parte del docente.
Decisión	Implementar autenticación síncrona JWT para docentes y un flujo de tokens efímeros firmados criptográficamente presentados en formato código QR por el alumno al finalizar la prueba.
Consecuencias	Positivas: Eliminación del fraude por participación a distancia; fricción nula para el estudiante.
	Negativas: Requiere el uso de la cámara del dispositivo docente durante la fase de cierre de la sesión.

Tabla 6.4: Registro de decisiones de arquitectura - ADR-04.

6.3.2. Trazabilidad entre RNF y tácticas arquitectónicas

Para garantizar que el diseño propuesto satisface los atributos de calidad definidos en la especificación de requisitos, la siguiente tabla mapea cada requisito no funcional (RNF) con la táctica de arquitectura aplicada, el componente responsable de su ejecución y el método de verificación utilizado en el proyecto:

Requisito	Táctica arquitectónica	Responsable	Verificación
RNF-01	Canales bidireccionales y difusión asíncrona (<i>broadcasting</i>).	Controladores (routers/)	Test de latencia (< 50 ms)
RNF-02	Bucle de eventos no bloqueante e E/S asíncrona.	Motor FastAPI y Uvicorn	Carga simultánea (100 usuarios)
RNF-03	Tokens criptográficos firmados con HMAC de un solo uso.	routers/ y auth.py	Test de unicidad de firmas
RNF-04	Derivación y <i>hashing</i> adaptativo de credenciales.	auth.py	Inspección de almacenamiento
RNF-05	Forzado de canal seguro extremo a extremo (HTTPS / WSS).	Proxy inverso (TLS)	Auditoría de certificados TLS
RNF-06	Diseños fluidos reactivos con puntos de ruptura adaptativos.	Componentes React UI y layouts/	Verificación en multidispositivo
RNF-07	Escaneo optimizado del flujo de cámara en hilo secundario.	Módulo de partida (room-play/)	Medición de tiempo (< 1 s)
RNF-08	Persistencia de sesión local y reconexión mediante token/PIN.	Estado global (context/)	Simulación de caída de red
RNF-09	Transacciones relacionales ACID con sincronización en disco.	Motor SQLite y SQLAlchemy	Prueba de fallo de proceso
RNF-10	Separación estricta de responsabilidades en capas independientes.	Arquitectura global	Inspección de dependencias

Tabla 6.5: Matriz de trazabilidad entre RNF y tácticas arquitectónicas.

6.4. Estructura modular del código

Para trasladar los patrones arquitectónicos descritos a la **fase de diseño**, el sistema se organiza conceptualmente en torno a una estructura *monorepo* dividida en dos grandes módulos independientes (backend/ y frontend/). Esta organización delimita claramente las responsabilidades y facilita el desarrollo desacoplado de ambos componentes.

A continuación se presenta el **esquema preliminar** de la jerarquía de directorios y componentes principales del proyecto:

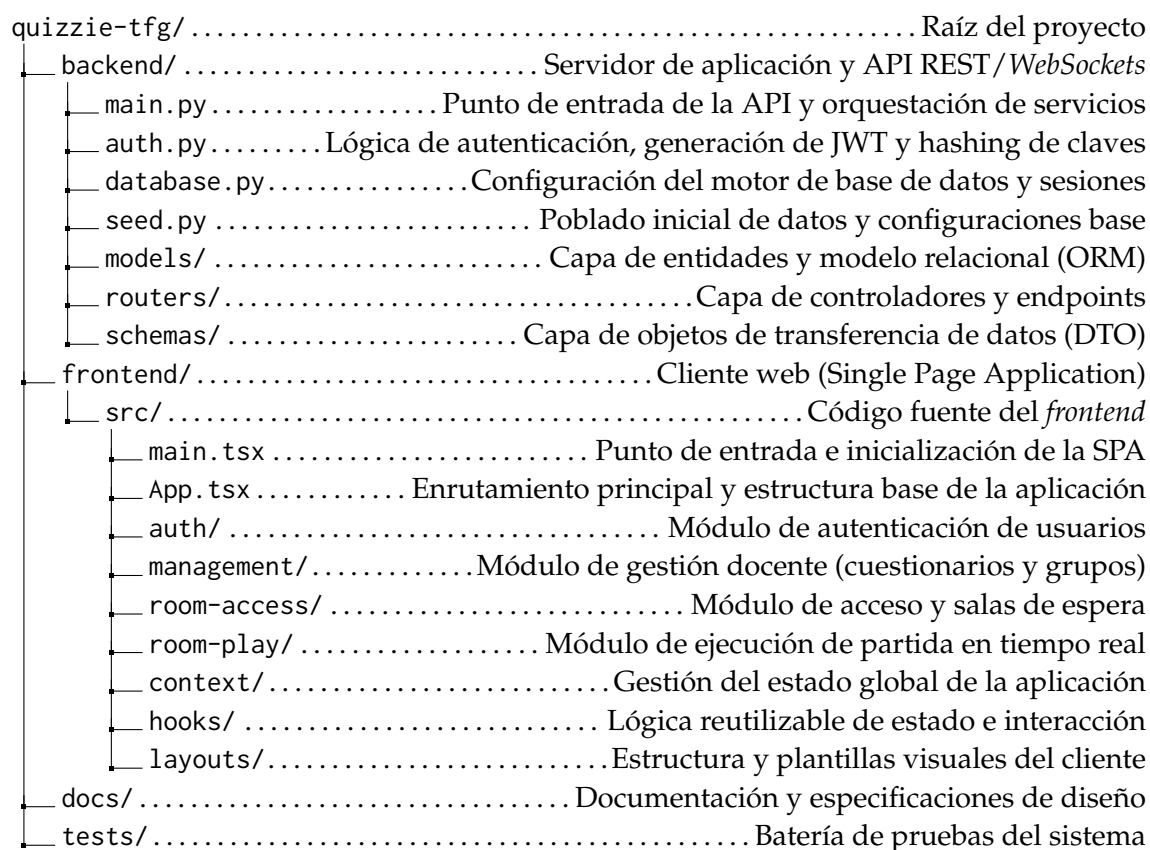


Figura 6.4: Estructura preliminar de módulos y componentes del proyecto.

6.4.1. Organización del backend

En el servidor, la estructura abstrae la lógica en tres archivos clave de inicialización y tres paquetes fundamentales que responden a la arquitectura en capas:

- **Arranque y soporte:** Los módulos `main.py`, `database.py` y `seed.py` se encargan del punto de entrada del servidor, la gestión del motor de base de datos y la siembra inicial de información necesaria para la ejecución.
- **Controladores (routers/):** Definen los puntos de entrada para las peticiones HTTP REST y gestionan las conexiones bidireccionales mediante *WebSockets*.
- **Modelos (models/):** Representan las entidades del dominio y sus relaciones en la base de datos relacional.
- **Esquemas y DTOs (schemas/):** Definen el manejo de datos para la entrada y salida de la API, garantizando el filtrado y la validación de la información.

6.4.2. Organización del frontend

En el cliente, el directorio `src/` estructura la aplicación por dominios funcionales y capas de soporte:

- **Entrada y enrutamiento:** `main.tsx` y `App.tsx` inicializan la aplicación cliente y gestionan las rutas de navegación.
- **Módulos de dominio:** Las carpetas `auth/`, `management/`, `room-access/` y `room-play/` encapsulan las vistas y los flujos de trabajo de la aplicación.
- **Soporte transversal:** Las carpetas `context/`, `hooks/` y `layouts/` proporcionan el estado global, la lógica reutilizable y la estructura visual compartida.

6.5. Modelo de datos

El diseño de la capa de persistencia se articula en torno a la definición de las entidades del dominio, sus relaciones y las restricciones de integridad necesarias para soportar la ejecución del sistema. En el diagrama de clases se ilustra la estructura general y la interacción entre las distintas entidades previstas para la aplicación.

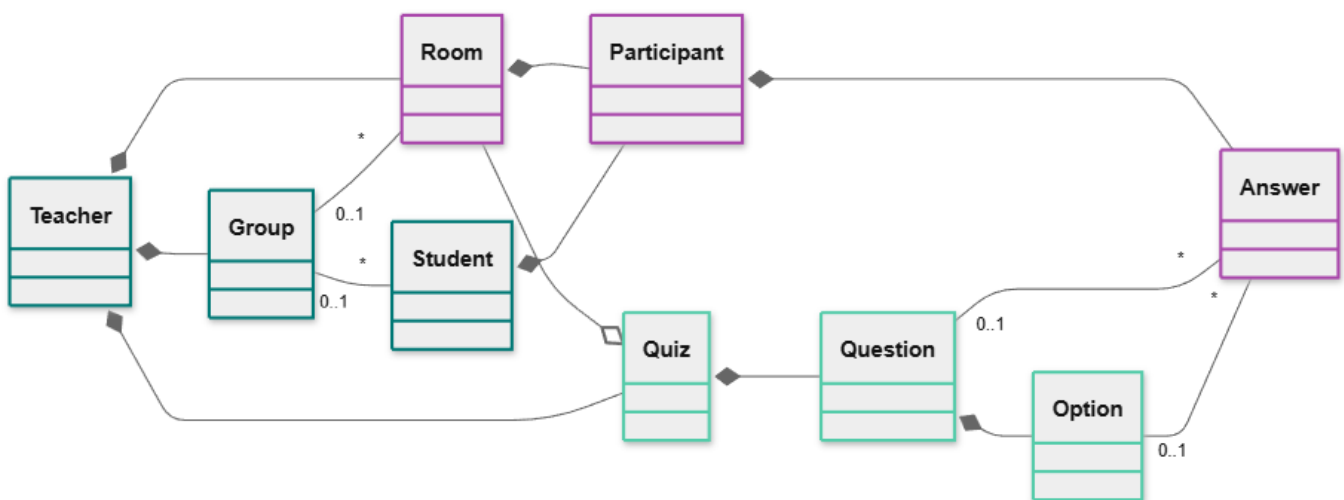


Figura 6.5: Diagrama de clases conceptual y relaciones globales del dominio.

Las entidades del sistema se organizan conceptualmente en tres módulos funcionales: **sesiones**, **usuarios** y **contenido**. A continuación se analiza cada uno de estos bloques.

6.5.1. Módulo de sesiones

Engloba la lógica persistente encargada de soportar las partidas síncronas en tiempo real y el registro de la actividad de los estudiantes:

- **Room:** Entidad central que gestiona la instancia activa de un cuestionario en directo, controlando el acceso mediante un código PIN único y su disponibilidad.

- **Participant:** Actúa como entidad intermedia para resolver la relación de muchos a muchos entre Student y Room, registrando la incorporación del alumno a la partida en una marca de tiempo específica.
- **Answer:** Almacena los eventos de respuesta enviados por los participantes, registrando la opción elegida, los puntos calculados y el instante exacto de recepción.

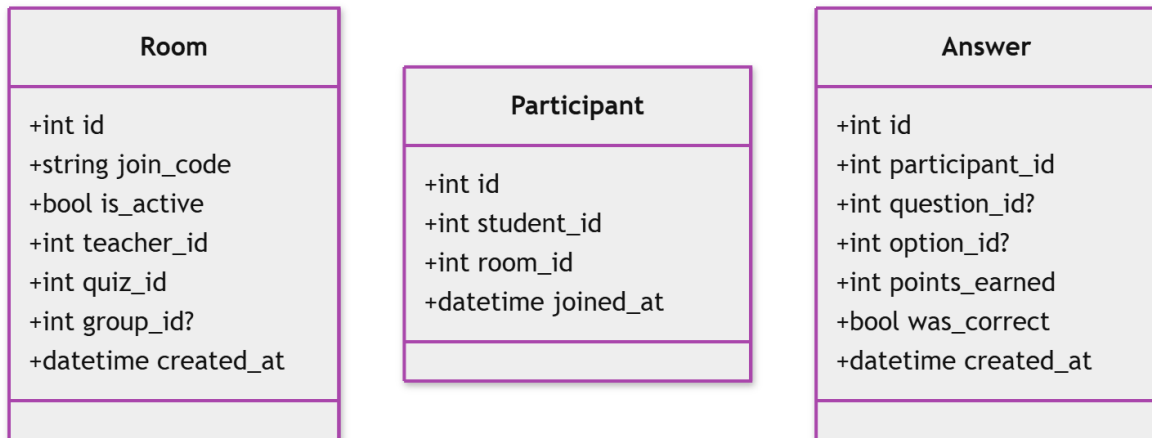


Figura 6.6: Entidades y atributos del módulo de sesiones.

6.5.2. Módulo de gestión de usuarios

Este módulo administra la identidad de los actores del sistema y la articulación del aula:

- **Teacher:** Modela al docente y sus credenciales de acceso, actuando como el propietario principal de los recursos creados (cuestionarios, salas y grupos).
- **Student:** Representa el registro persistente del alumno en la base de datos mediante su identificador o credencial de usuario (*nickname*/UVUS), el cual se vincula a las salas mediante la entidad intermedia de participación.
- **Group:** Articula a los estudiantes en conjuntos académicos para facilitar la asignación y seguimiento de partidas dirigidas.

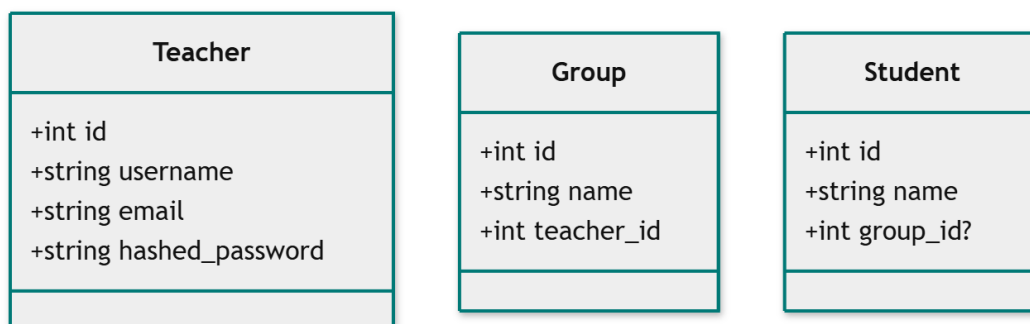


Figura 6.7: Entidades y atributos del módulo de usuarios.

6.5.3. Módulo de contenido

Comprende las entidades que estructuran el material de evaluación generado por el profesorado, vinculadas por relaciones de composición estricta:

- **Quiz:** Entidad raíz de la evaluación. Agrupa la información contextual del cuestionario e incluye metadatos visuales y de categorización.
- **Question:** Define cada ítem evaluable perteneciente a un cuestionario, determinando su enunciado y el peso en puntuación.
- **Option:** Contiene las alternativas de respuesta propuestas por ítem, marcando mediante un booleano cuál es la opción correcta para la validación automática.

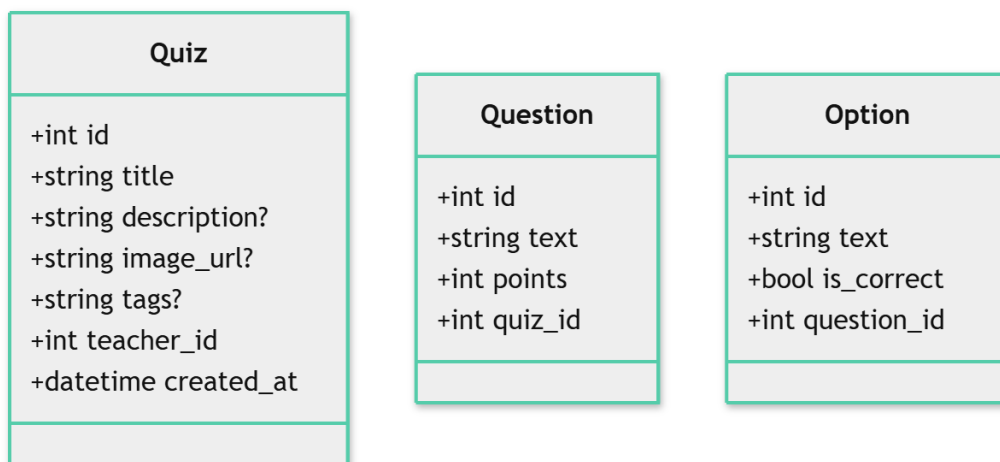


Figura 6.8: Entidades y atributos del módulo de contenido.

6.6. Diseño de interfaz de usuario

El diseño de las interfaces de usuario (UI) y la experiencia de usuario (UX) constituye una fase crítica dentro del desarrollo de la plataforma, ya que condiciona la adopción, la facilidad de uso y la fluidez interactiva durante las sesiones de *tests*.

Para satisfacer las necesidades del sistema, el diseño ha considerado de manera diferenciada dos perfiles de usuario principales: el **profesorado**, que requiere herramientas de gestión, control de salas y análisis de resultados; y el **alumnado**, para el cual prima una interfaz limpia, altamente intuitiva y libre de distracciones, orientada al envío rápido de respuestas en entornos móviles o de escritorio.

6.6.1. Principios de diseño y guía de estilo

Las interfaces han sido diseñadas siguiendo principios consolidados de usabilidad y diseño centrado en el usuario (*User-Centered Design*):

- **Consistencia e identidad visual:** Se ha mantenido una estructura coherente en el maquetado de formularios, botones de acción y distribución de contenedores. Para facilitar la identificación inmediata del rol durante el análisis de las pantallas del sistema, los prototipos incorporan una codificación por color en su barra lateral:
 - **Borde magenta:** Asociado a la interfaz del perfil del profesor.
 - **Borde cian:** Asociado a la interfaz del perfil del estudiante.
- **Simplicidad y claridad cognitiva:** Durante las partidas en vivo, se reduce al mínimo el texto explicativo y los elementos estéticos para enfocar la atención del estudiante exclusivamente en las preguntas, opciones y temporizadores.
- **Diseño adaptable y accesibilidad:** La interfaz del estudiante está optimizada para una interacción táctil ágil, empleando áreas de pulsación amplias y garantizando un diseño plenamente adaptativo (*responsive*).

6.6.2. Flujos funcionales y prototipos

La arquitectura de navegación y la interacción visual de la plataforma se han organizado en cuatro secuencias funcionales principales. Estos escenarios se han maquetado en prototipos de alta fidelidad (*mockups*) y se encuentran detallados en el **anexo de prototipos UI**.

Los cuatro flujos cubren el ciclo de vida completo de una sesión interactiva, estructurándose según el momento operativo de la actividad:

1. **Registro y autenticación de usuarios:** Módulo administrativo de control de acceso y alta del perfil docente mediante correo y contraseña.
2. **Creación de cuestionarios y gestión de salas:** Entorno de preparación previa donde el profesorado gestiona los contenidos, configura los parámetros de la sesión y habilita la sala de espera.
3. **Incorporación del estudiante a la sala:** Proceso simplificado de acceso en tiempo real para el alumnado mediante código PIN e identificación por UVUS o apodo, diseñado para minimizar la fricción de entrada.
4. **Ejecución de la partida en vivo:** Núcleo dinámico del sistema que coordina la experiencia síncrona entre ambos perfiles durante el desarrollo del cuestionario, abarcando la sincronización de tiempos, el envío de respuestas, la retroalimentación inmediata y la presentación de clasificaciones.

El diseño integrado de estos cuatro módulos responde a una lógica de interacción continua. Mientras que los dos primeros proporcionan la infraestructura de gestión previa a la clase, los dos últimos coordinan la sesión interactiva en directo.

6.7. Seguridad

La seguridad de la plataforma se ha diseñado atendiendo a la naturaleza diferenciada de sus dos perfiles de usuario: los **profesores** (usuarios registrados que requieren **acceso persistente** mediante credenciales) y los **alumnos** (usuarios efímeros que acceden a las partidas en vivo mediante un **PIN** y un **alias**, sin necesidad de registro previo).

6.7.1. Autenticación síncrona mediante JWT

Para la gestión de accesos del perfil docente se ha optado por un esquema sin estado (*stateless*) basado en *JSON Web Tokens (JWT)*. Este enfoque evita almacenar el estado de las sesiones en el servidor para la API REST, permitiendo un diseño **desacoplado**.

El token se genera tras verificar las credenciales del usuario (cuyas contraseñas se almacenan de forma segura mediante algoritmos de *hashing* y derivación de claves) y se adjunta en el encabezado estándar **Authorization: Bearer <token>** para validar las solicitudes posteriores. A continuación se ilustra la secuencia de emisión y validación del token.

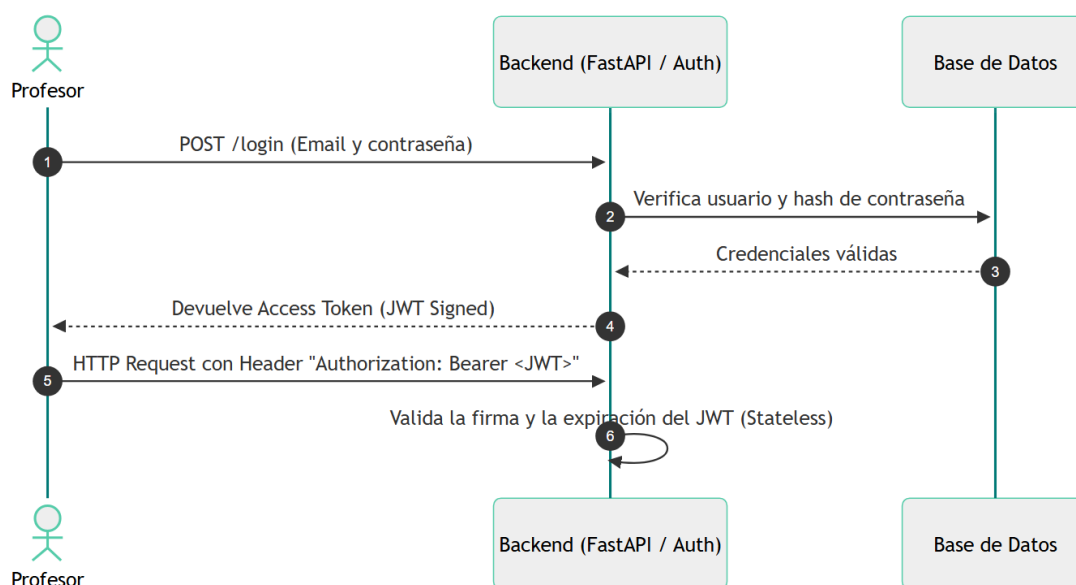


Figura 6.9: Flujo de autenticación del profesor mediante JWT.

6.7.2. Gestión de roles y control de acceso (RBAC)

El control de acceso se fundamenta en un modelo **RBAC** (*Role-Based Access Control*), definiendo dos niveles de autorización claramente delimitados:

- **Rol de profesor (Autenticado):** Protegido mediante la validación del JWT, otorga privilegios para la gestión integral de cuestionarios, la creación de salas en vivo, el control del flujo del juego y la consulta de informes.
- **Rol de participante (Efímero / No autenticado):** Asignado temporalmente a la sala activa sin requerir registro previo, restringe los permisos exclusivamente al envío de respuestas a las preguntas habilitadas en ese instante.

Bajo una arquitectura con **autoridad en el servidor** (*Server-Authoritative Architecture*), toda la lógica de validación, la temporización de la partida y la corrección de respuestas residen exclusivamente en el *backend*. El servidor actúa así como la **única fuente de verdad** (*Single Source of Truth, SSOT*)[41] del sistema, garantizando la **integridad** del proceso de evaluación y previniendo la manipulación de datos desde el cliente (*anti-cheat*).

6.7.3. Tokens de verificación para participantes

Para **consolidar** las calificaciones de forma presencial y **asegurar** que el resultado pertenece al estudiante correspondiente, la plataforma incluye un mecanismo de verificación mediante **códigos QR**. El docente escanea en el aula el código presentado por el alumno, el cual contiene su **alias** y su **token de verificación temporal** (*verification.token*).

Como se resume en el siguiente diagrama, al realizar el escaneo la aplicación solicita al *backend* la **verificación del participante**. El servidor valida la correspondencia del token con la sala activa, confirma la actualización del estado del participante inscribiendo la nota en la base de datos y difunde la actualización a la sala mediante la conexión en tiempo real por *WebSocket*.

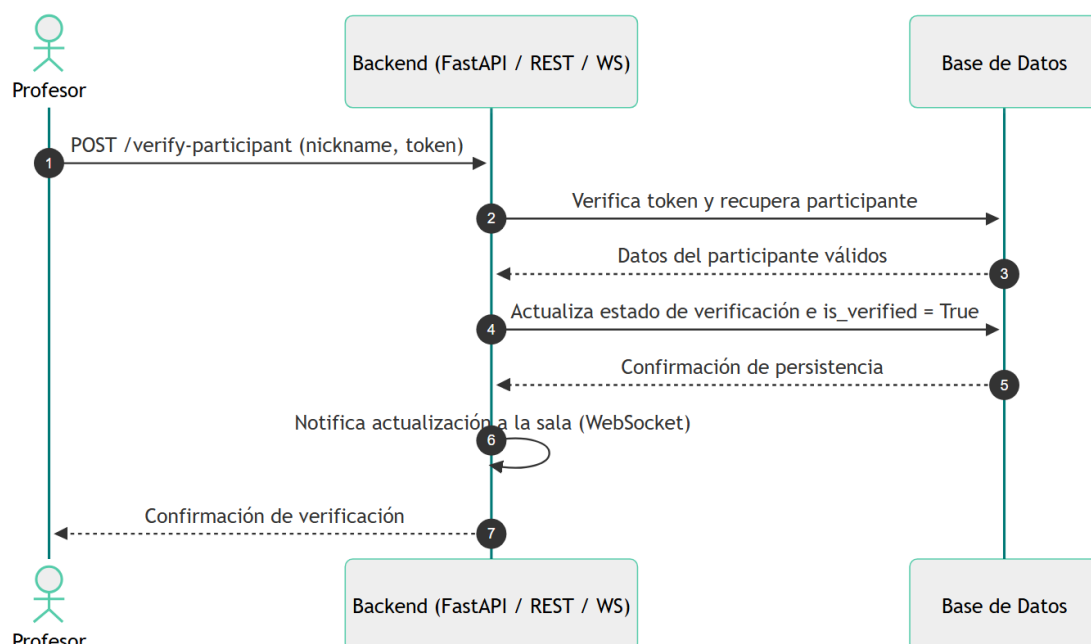


Figura 6.10: Diagrama del proceso de verificación del participante.

6.8. Documentación de la API

Para garantizar un desarrollo desacoplado entre el *frontend* y el *backend*, así como facilitar la mantenibilidad y evolución del sistema, la API de Quizzie se ha diseñado bajo el estándar OpenAPI.

6.8.1. Integración con OpenAPI y Swagger

La generación automatizada de la documentación técnica se fundamenta en el uso de modelos de datos declarativos mediante **Pydantic**. Al definir las cargas útiles de solicitud (*request bodies*), los parámetros de consulta y las respuestas esperadas mediante esquemas fuertemente tipados, el motor de **FastAPI** sintetiza e infiere automáticamente la especificación formal del sistema bajo el estándar **OpenAPI** (`openapi.json`).

Este enfoque de **documentación viva** elimina las incongruencias habituales entre el código en ejecución y su especificación escrita. De esta forma, el servidor expone de forma nativa dos interfaces web complementarias orientadas a cubrir diferentes necesidades del ciclo de vida del proyecto:

- **Swagger UI (/docs):** Proporciona una interfaz interactiva basada en el estándar visual de OpenAPI. Resulta clave durante la fase de desarrollo y depuración (*testing*), ya que permite autenticarse y ejecutar peticiones directas contra los *endpoints* del servidor en tiempo real desde el propio navegador.
- **ReDoc (/redoc):** Ofrece una presentación estructurada, jerárquica y de lectura limpia de la especificación técnica. Está pensada como documentación de referencia para la integración de clientes externos o para entornos de producción, optimizando la consulta de esquemas de datos complejos.

Como se ilustra en la siguiente figura, el flujo abarca desde la declaración formal de los esquemas en el código fuente hasta la exposición y renderizado de ambas herramientas de consulta.

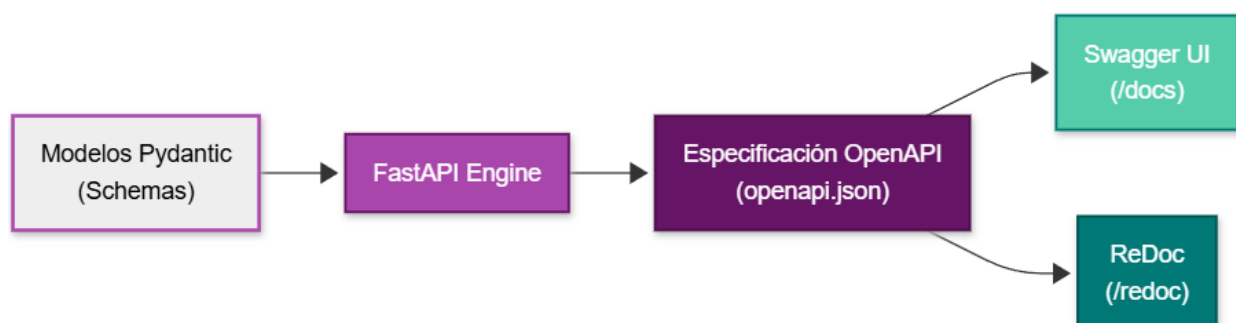


Figura 6.11: Flujo de generación de documentación OpenAPI.

6.8.2. Especificación de rutas fundamentales

La API traslada de forma directa los módulos conceptuales definidos en el **modelo de datos**. Esta separación de responsabilidades se materializa en el código fuente dividiendo los enrutadores (routers/) y los modelos de datos (models/) en tres bloques principales: users, content y stage.

A continuación se resumen las rutas proyectadas más representativas para cada uno de estos módulos:

Usuarios (routers/users.py)

- POST /users/login: Autentica las credenciales del docente y emite el token de acceso para autorizar sus peticiones posteriores.
- POST /users/register: Registra a un nuevo profesor en la plataforma e inicia el proceso de verificación por correo electrónico.

Contenido (routers/content.py)

- POST /content/quizzes: Permite al profesor crear un cuestionario completo con su conjunto de preguntas y opciones de respuesta.

Sesiones (routers/stage.py) Combina peticiones REST para la gestión del estado de la sala y un canal WebSocket para la comunicación en tiempo real:

- POST /stage/rooms: Crea e inicializa una nueva sala de juego asociada a un cuestionario y aplica su configuración inicial.
- POST /stage/participants: Permite el acceso efímero de los estudiantes a una sala activa mediante un PIN y un alias.
- POST /stage/answers: Almacena la respuesta enviada por un alumno a la pregunta activa.
- POST /stage/rooms/{room_id}/verify-participant: Permite al docente verificar presencialmente el resultado de un estudiante en el aula, según se describe en el **diagrama de verificación del token**.
- WS /stage/rooms/{room_id}/ws: Mantiene la conexión en tiempo real para sincronizar el avance del juego, los temporizadores y la clasificación entre todos los participantes.

Ciclo de vida y orquestación de la sesión en tiempo real

Para ilustrar la articulación práctica del canal WS junto a los *endpoints* REST, la siguiente figura describe la interacción y el flujo de control entre el profesor, el servidor y los estudiantes. El diagrama organiza el ciclo completo de la sala activa, desde la apertura de la sesión hasta la verificación de resultados.



7. Metodología técnica y de desarrollo

Una vez establecida la arquitectura y el diseño conceptual del sistema, resulta imprescindible definir un **marco operativo riguroso** para la fase de construcción e integración del código. En el desarrollo de software, la ausencia de políticas explícitas en el control de versiones incrementa paulatinamente la deuda técnica y dificulta la detección temprana de errores.

Este capítulo detalla la metodología técnica implantada en el proyecto *Quizzie*. En él se abordan las **políticas de trabajo en el repositorio**, la automatización del ciclo de integración continua y despliegue continuo (CI/CD), la gestión segura de **perfiles de configuración** y el **análisis estático** de la calidad del código.

7.1. Ecosistema base de desarrollo local

Concretando la selección tecnológica descrita en la subsección **soporte de desarrollo**, el trabajo local se sostiene sobre tres pilares operativos principales. La integración de este ecosistema busca asegurar la reproducibilidad homogénea del proyecto, garantizar el aislamiento de dependencias y simplificar la ejecución de herramientas de desarrollo:

- Gestión de entorno *backend* con **uv**: En lugar de recurrir a entornos virtuales creados manualmente (venv) o gestores tradicionales, el ciclo de vida del *backend* se articula íntegramente mediante el gestor de dependencias **uv**. Esta herramienta centraliza el aislamiento de la versión de Python, la resolución del árbol de dependencias expresado en `pyproject.toml` y la ejecución de comandos de soporte (linters, formateadores y motores de prueba) en un entorno reproducible mediante el prefijo `uv run`.
- Gestión de entorno *frontend* con **npm** y **Node.js**: Para la parte cliente, el ecosistema de trabajo se apoya en **npm** sobre el entorno de ejecución de **Node.js**. A través del fichero `package.json`, se estandarizan las tareas de desarrollo, la compilación y la verificación de tipos con TypeScript.
- Sistema de control de versiones distribuido con **Git**: Registra la evolución del código fuente, sirviendo de nexo entre el trabajo en la máquina local y las políticas de integración remota que se detallan a continuación.

Esta combinación permite que cualquier desarrollador pueda clonar el repositorio y levantar el entorno completo de trabajo de forma idéntica con un conjunto mínimo y estandarizado de comandos.

Los pasos detallados para la preparación inicial del entorno y el registro de dependencias se recogen en el **manual técnico**.

7.2. Políticas del repositorio y flujo de trabajo

Para asegurar un desarrollo **estructurado, auditable y mantenible**, se ha definido una **normativa estricta** sobre el repositorio centralizado en GitHub [42]. Esta normativa abarca desde el modelado de tareas hasta la consolidación de cambios en los entornos estables, siguiendo un flujo secuencial estandarizado compuesto por **seis etapas consecutivas**:

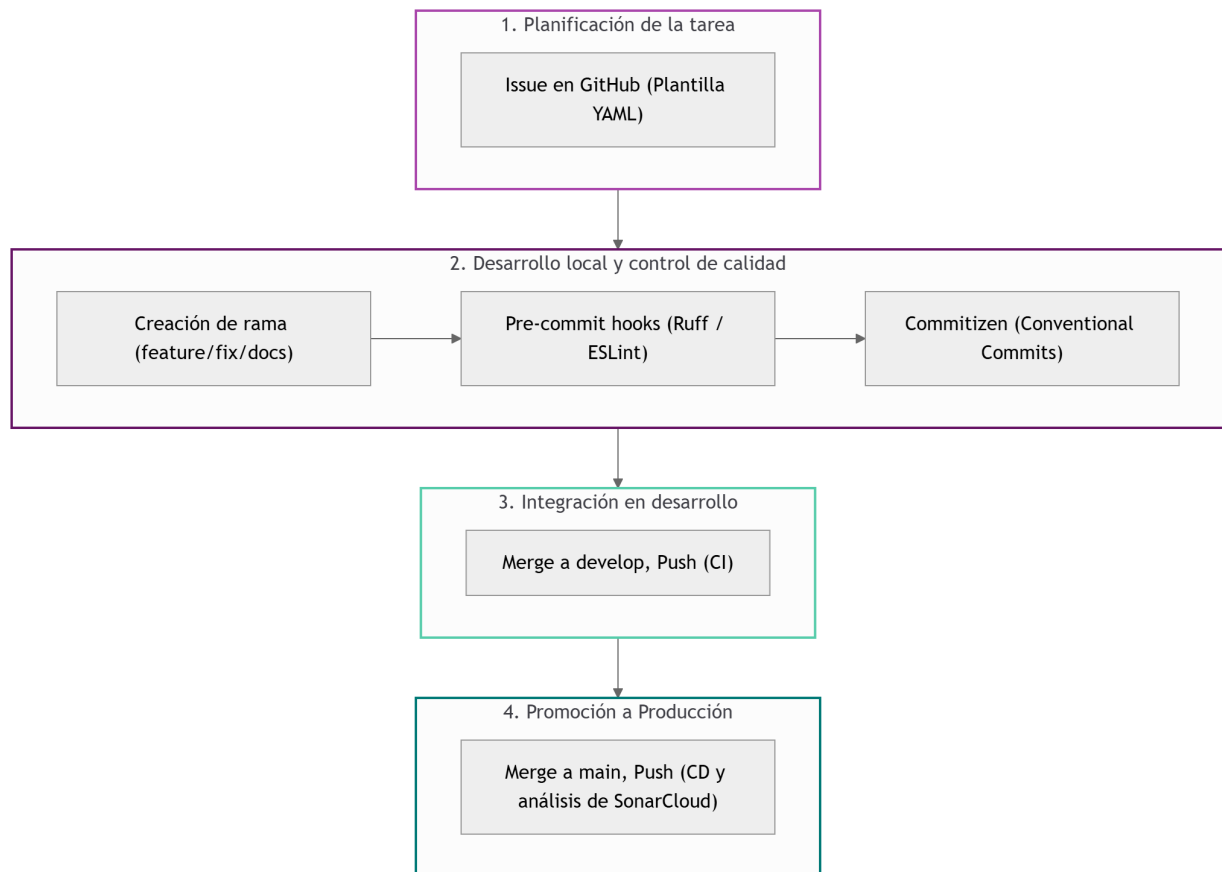


Figura 7.1: Ciclo de trabajo secuencial implementado en Quizzie.

7.2.1. Gestión de tareas mediante plantillas de *GitHub Issues*

Todo trabajo de desarrollo, refactorización, documentación o pruebas debe originarse mediante una incidencia (*Issue*) en GitHub redactada en inglés. Esto garantiza la trazabilidad bidireccional entre los requisitos funcionales (RF), los objetivos del proyecto y las modificaciones del código.

Para homogeneizar el registro de *Issues*, se han implementado plantillas declarativas en formato *YAML* ubicadas en el directorio `.github/ISSUE_TEMPLATE` del repositorio:

- **Requisitos funcionales** (`feature_request.yml`): Diseñada para definir la implementación de nuevos requisitos. Exige vincular explícitamente el

identificador del requisito (ej. RF-07), el objetivo asociado (OBJ-02), la descripción técnica de la solución y los criterios de aceptación.

- **Documentación** (docs.yml): Orientada a la redacción de documentación técnica o capítulos de la memoria en $\text{\LaTeX}$, indicando el documento afectado y los criterios de revisión.
- **Infraestructura y CI/CD** (devops.yml): Destinada a tareas de automatización, ajuste de canalizaciones de GitHub Actions, integración con SonarQube o configuración de infraestructura.
- **Baterías de pruebas** (test.yml): Específica para la creación o ampliación de pruebas (unitarias, integración, de componentes, E2E o pruebas de carga con Locust).

El uso de esta estructura estandarizada asegura que cualquier incidencia registrada contenga los datos necesarios para que el desarrollo posterior se adecúe a los **criterios de aceptación**.

7.2.2. Estrategia de ramas

El proyecto adopta una variante adaptada del modelo *Git Flow* orientada a entornos de desarrollo ágil. A diferencia del modelo tradicional que emplea solicitudes de extracción (*Pull Requests*) complejas para un único desarrollador, *Quizzie* utiliza **ramas de trabajo efímeras** e integración mediante **fusiones directas** (*merge*) locales.

Rama	Propósito	Origen
main	Código estable en producción. Solo recibe código probado y validado.	—
develop	Rama principal de integración y desarrollo continuo.	main
feature/*	Implementación de requisitos funcionales (RF).	develop
fix/*	Corrección de errores y mejoras rápidas.	develop
docs/*	Actualización de documentación o memoria del TFG.	develop
test/*	Incorporación o mejora de baterías de pruebas.	develop
refactor/*	Reestructuración interna de código sin cambio de comportamiento.	develop
build/*	Ajustes en CI/CD, dependencias o infraestructura.	develop

Tabla 7.1: Clasificación de ramas, nomenclatura y rama de origen.

El flujo de aislamiento y convergencia entre las ramas temporales, la rama de integración develop y la rama de producción main se esquematiza en el siguiente grafo:

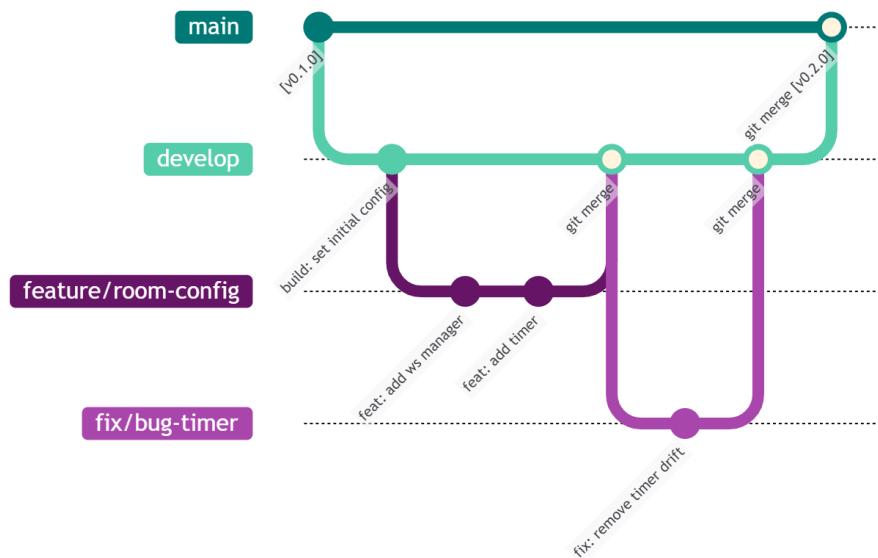


Figura 7.2: Diagrama del flujo de ramas en el repositorio.

7.2.3. Verificación local mediante *pre-commit*

Antes de que cualquier *commit* (modificación en el código) sea registrada definitivamente en el historial de Git, es fundamental asegurar que cumple con los **estándares de calidad y formato** fijados en el proyecto. Para automatizar este control, en la máquina local del desarrollador se utiliza la herramienta **pre-commit**, la cual evalúa el código de forma previa a la confirmación.

La configuración centralizada de estas verificaciones se define en el archivo `.pre-commit-config.yaml` ubicado en la raíz del repositorio. En dicho fichero se declaran los conectores (*hooks*) y herramientas que se deben ejecutar de forma transparente sobre los archivos modificados:

1. **Backend (Python):** Se integra la herramienta **Ruff** para realizar un **análisis estático** ultrarrápido, corregir automáticamente fallos de formato, eliminar importaciones sin usar y garantizar el cumplimiento del estándar **PEP 8** [43].
2. **Frontend (React / TypeScript):** Se ejecutan **ESLint** y **Prettier** para inspeccionar la sintaxis de los componentes JSX/TSX y unificar el **estilo visual** del código.
3. **Seguridad y limpieza:** Se añaden verificaciones generales para detectar y bloquear la subida accidental de **archivos de secretos** (`.env`), claves privadas o archivos con formato incorrecto.

Para forzar manualmente la inspección de la totalidad del código fuente sin necesidad de intentar un *commit*, se utiliza la instrucción:

```
> uv run pre-commit run --all-files
```

Si alguna de las comprobaciones detecta un fallo o una discrepancia de formato, *pre-commit* aborta la confirmación automáticamente, exigiendo corregir el problema antes de continuar.

7.2.4. Estandarización de commits

Para mantener un historial de control de versiones limpio, semántico y legible, se ha adoptado la especificación **Conventional Commits**. Esta convención asigna un significado explícito a cada confirmación, lo que permite auditar la evolución del código y automatizar la gestión de versiones mediante Versionado Semántico (*SemVer*) [44].

La construcción de los mensajes en la máquina local se canaliza a través de la herramienta **Commitizen**. Al ejecutar la instrucción interactiva correspondiente, se guía al desarrollador en la redacción del mensaje respetando la estructura obligatoria:

`<tipo>(<alcance>) : <descripción>`

Tipo	Descripción	Impacto <i>SemVer</i>
feat	Incorporación de una nueva funcionalidad o requisito.	MINOR (v0.X.0)
fix	Corrección de un fallo o error en el código.	PATCH (v0.0.X)
docs	Cambios exclusivamente en la documentación o memoria.	—
style	Ajustes de formato o estilo que no alteran la lógica.	—
refactor	Reestructuración de código sin cambio de comportamiento.	—
test	Añadido o corrección de baterías de pruebas.	—
build	Modificaciones en la configuración de build o dependencias.	—
ci	Ajustes en los flujos de integración continua (CI/CD).	—

Tabla 7.2: Tipos de *commits* según Conventional Commits.

Asimismo, se exige establecer la trazabilidad bidireccional entre el código registrado y las incidencias de GitHub. Para ello, en el pie del mensaje de confirmación se deben incluir las referencias correspondientes:

- Refs #83: Vincula la confirmación con la incidencia #83 en la interfaz de GitHub para facilitar su seguimiento.
- Closes #83 / Fixes #83: Asocia el cambio y provoca el cierre automático de la incidencia al fusionar la rama en la rama principal (main).

Para automatizar este proceso, se ha configurado previamente una plantilla en el proyecto que guía al desarrollador paso a paso. Al ejecutar la siguiente instrucción en la terminal, se despliega el asistente de Commitizen y se genera el *commit* de forma directa en el repositorio local una vez rellenados los campos:

```
> uv run cz commit
```

7.2.5. Consolidación y despliegue a producción

Una vez completado el desarrollo y comprobada la estabilidad de las funcionalidades en la rama de integración (*develop*), se procede a la transferencia del código hacia la rama principal (*main*). Este procedimiento traslada de forma controlada únicamente versiones consolidadas que han superado previamente todas las verificaciones locales y de integración continua. Dicho flujo de trabajo se encuentra oficialmente respaldado por las reglas de protección de ramas en GitHub (*Branch Protection Rules*), las cuales impiden subidas directas (*direct push*) y fuerzan el cumplimiento de las políticas de validación establecidas.

El procedimiento para llevar a cabo la fusión del código se compone de seis pasos clave. A continuación, se ilustran los pasos a seguir para integrar el código en la rama *main*, junto a las acciones y comandos de consola correspondientes.

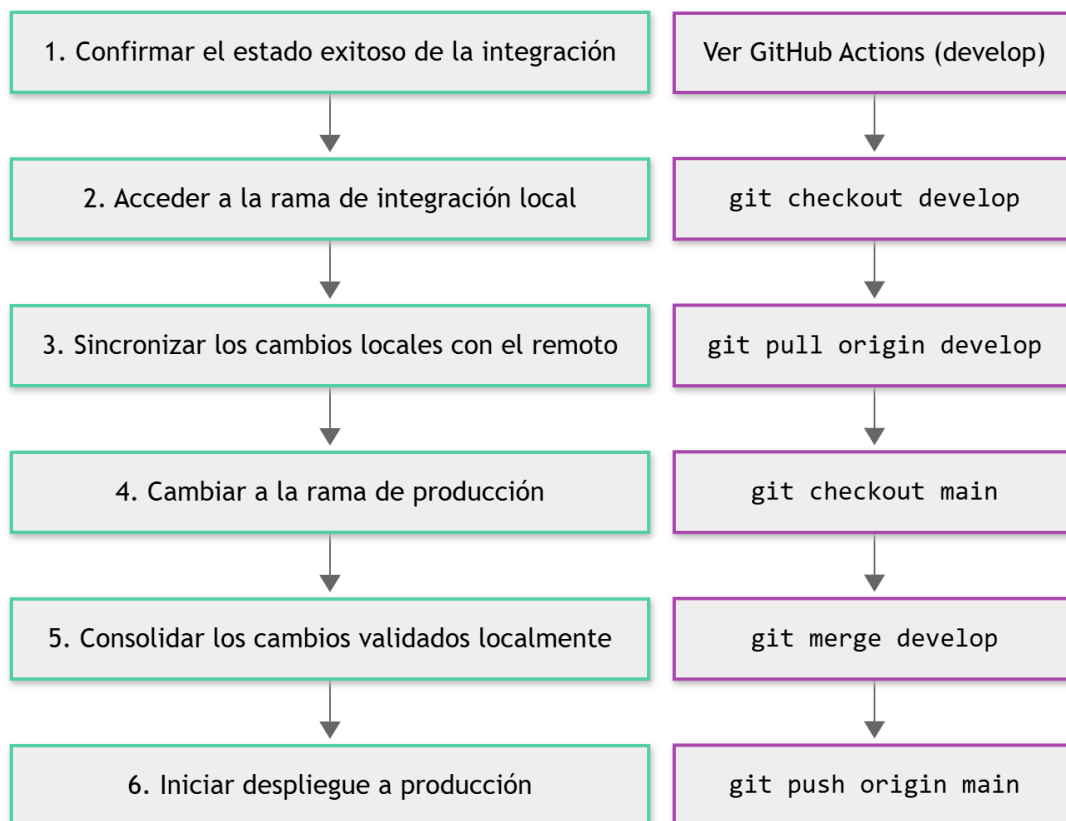


Figura 7.3: Procedimiento para integrar código en la rama *main*.

Para garantizar la estabilidad del entorno de producción, la rama *main* se reserva de forma exclusiva para el código listo para su despliegue. Las versiones del sistema (*Releases*) se formalizan al culminar las *issues* asociadas a cada una de las etapas de desarrollo, materializando formalmente los entregables de producto software definidos en la **EDT**. En dicho punto, se asigna la correspondiente etiqueta de versión semántica (*SemVer*) y se documentan las modificaciones directamente en la sección de *Releases* de GitHub a partir de las incidencias resueltas.

7.3. Métricas de calidad y análisis estático de código

Con el fin de asegurar la **mantenibilidad y robustez** del código consolidado en la rama principal (main), se utiliza **SonarQube** como auditor externo para el **análisis estático**. Esta herramienta evalúa de forma continuada la salud del proyecto, monitorizando la presencia de errores lógicos, brechas de seguridad, complejidad cognitiva y el porcentaje global de **cobertura de pruebas** tras cada integración en main, definiendo los criterios de calidad que posteriormente se automatizan en el flujo de CI/CD.

7.3.1. Justificación y métricas objetivo (*Quality Gate*)

Con el objetivo de evitar la acumulación de deuda técnica y asegurar la **mantenibilidad** a largo plazo, se ha configurado un **umbral de calidad (*Quality Gate*)** de obligado cumplimiento para cualquier nueva funcionalidad integrada en main.

La siguiente tabla resume los objetivos mínimos exigidos por el sistema y la justificación técnica de cada uno:

Métrica	Objetivo	Justificación técnica
Cobertura	> 80 %	Garantizar que la mayoría de los flujos críticos del sistema están probados.
Duplicación	< 3 %	Fomentar el principio DRY y facilitar el mantenimiento futuro evitando la repetición de código.
Fiabilidad	Calificación A	Probar la ausencia total de errores lógicos conocidos en el código nuevo.
Seguridad	Calificación A	Asegurar la inexistencia de vulnerabilidades y puntos de acceso no protegidos.
Mantenibilidad	Calificación A	Mantener una deuda técnica mínima y un nivel bajo de complejidad cognitiva.

Tabla 7.3: Métricas objetivo y umbrales del *Quality Gate* en SonarQube.

7.3.2. Configuración y exclusiones técnicas

Para que los indicadores del panel de control sean un reflejo fiel de la calidad del producto final, se han aplicado las siguientes configuraciones de calibración en la plataforma:

- **Alcance de la cobertura de pruebas:** La métrica de cobertura de código integrada en SonarQube se enfoca de forma exclusiva en el *backend*. Esto se

debe a que el reporte de ejecución de **Pytest** (`coverage.xml`) es el único que se exporta y vincula al análisis estático, mientras que las pruebas de componentes del *frontend* se gestionan y auditan de manera independiente en su respectivo entorno de integración continua

- **Exclusión de scripts de carga (seed.py):** Se ha excluido el archivo `backend/seed.py` del cálculo de cobertura y duplicación. Al tratarse de un script para la siembra masiva de datos con estructuras repetitivas, su inclusión penalizaba artificialmente la media de cobertura y distorsionaba la métrica de código duplicado.
- **Integración del reporte de cobertura:** Se ha configurado la propiedad `sonar.python.coverage.reportPaths=coverage.xml` para garantizar una lectura e interpretación precisa del informe XML generado por **Pytest** (`pytest-cov`).
- **Estandarización del entorno de análisis:** La ejecución del *job* de CI para SonarQube se realiza sobre la imagen virtual `ubuntu-latest` en GitHub Actions, asegurando la consistencia en el mapeo de rutas de archivos entre el reporte de cobertura y el código fuente.

7.4. Integración y despliegue continuos

Para garantizar la calidad del software, la detección temprana de errores, la seguridad de las dependencias y la entrega continua del servicio en *Quizzie*, se ha diseñado e implementado una arquitectura de integración y despliegue continuos (CI/CD) fundamentada en **GitHub Actions**. Este flujo automatizado actúa como un **control de calidad estricto** antes de consolidar cualquier versión del código hacia los entornos de integración y producción.

7.4.1. Ciclo de vida y arquitectura

El flujo de trabajo automatizado se estructura en **tres workflows independientes** en GitHub Actions, los cuales responden a eventos específicos en el repositorio y gestionan las distintas fases de validación técnica:

1. **Integración Continua (CI):** Se activa automáticamente ante eventos de *push* en la rama `develop`. Ejecuta la batería de análisis estático, seguridad, pruebas unitarias y de integración, de componentes, *End-to-End* y evaluación de rendimiento.
2. **Análisis de calidad (SQ):** Se dispara tras consolidar cambios en la rama `main`. Evalúa métricas avanzadas de mantenibilidad, duplicación de código y deuda técnica en la plataforma SonarQube.
3. **Despliegue Continuo (CD):** Se ejecuta de forma paralela al análisis de calidad tras una fusión exitosa en `main`, activando el despliegue automático en la infraestructura de producción.

La siguiente figura ilustra el funcionamiento general de los *workflows*, destacando las dependencias entre *jobs* y la separación de responsabilidades según la rama afectada.

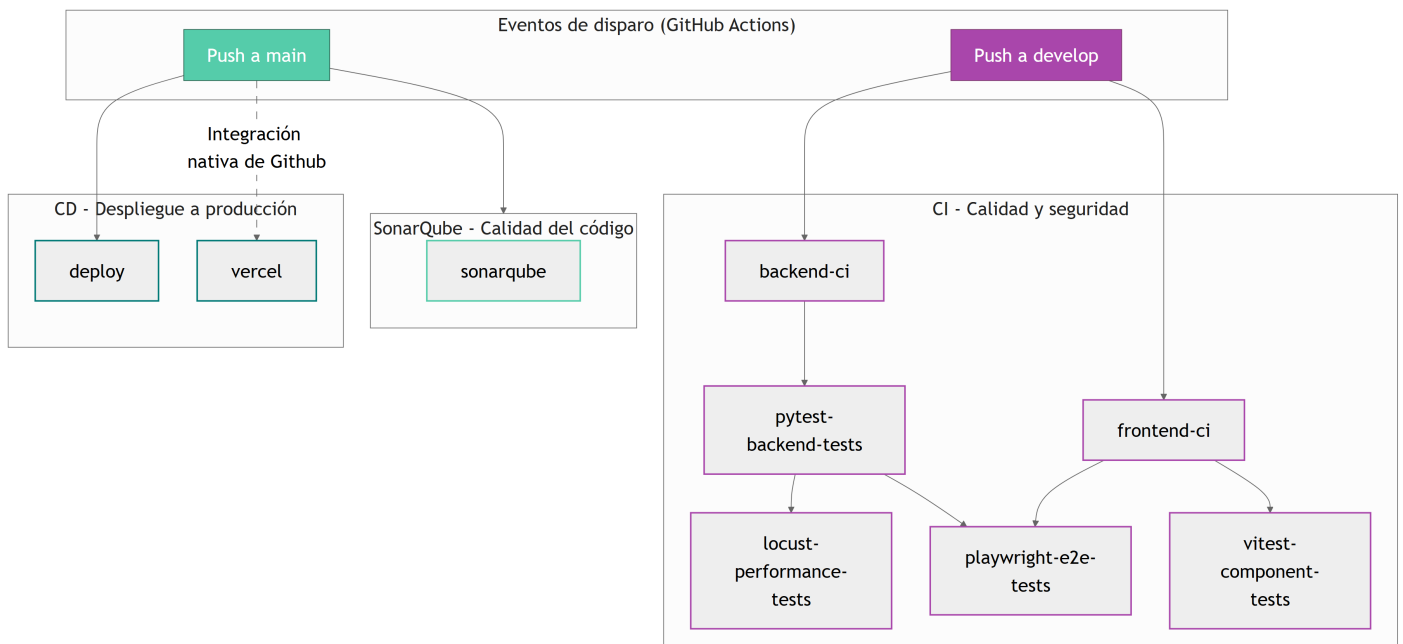


Figura 7.4: Estructura del *pipeline* de CI/CD

7.4.2. Flujo de Integración Continua (CI)

El proceso de validación de calidad y seguridad definido en `quality_and_security.yml` se compone de seis *jobs* interconectados distribuidos en distintas etapas secuenciales:

Análisis estático y verificación

- **backend-ci**: Inicializa el entorno Python 3.12 con `uv`, analiza el formato y estilo del código mediante **Ruff** y realiza la auditoría de seguridad de dependencias con `pip-audit`.
- **frontend-ci**: Prepara el entorno Node.js 20, ejecuta la verificación sintáctica mediante **ESLint** y valida el tipado de TypeScript junto con la compilación del proyecto.

Pruebas de código y componentes

- **pytest-backend-tests**: Requiere la superación previa de **backend-ci**. Ejecuta la suite de pruebas unitarias y de integración del *backend* mediante **Pytest** y exporta el informe de cobertura XML (`backend-coverage`) como artefacto.

- `vitest-component-tests`: Se ejecuta tras la consecución de `frontend-ci`. Realiza las pruebas sobre la interfaz de usuario utilizando **Vitest** y almacena su reporte de cobertura.

Pruebas de sistema y rendimiento

- `playwright-e2e-tests`: Depende del éxito conjunto de `pytest-backend-tests` y `frontend-ci`. Despliega el servidor backend de pruebas y ejecuta la batería *End-to-End* con **Playwright** simulando la navegación de usuario en navegadores *headless*.
- `locust-performance-tests`: Se inicia tras la superación de `pytest-backend-tests`. Inicia el servidor de prueba y ejecuta la simulación de carga concurrente con **Locust**, validando mediante `evaluate_thresholds.py` que las latencias y tasas de error se mantengan dentro de los límites establecidos en la **estrategia de testing**.

7.4.3. Análisis de calidad con SonarQube

Al consolidar cambios en la rama `main`, se activa el *workflow* `sonar.yml`. Este proceso reejecuta la medición de cobertura del *backend* y transmite los informes a SonarQube mediante la acción oficial `SonarSource/sonarqube-scan-action`, auditando el *Quality Gate* y controlando la deuda técnica acumulada.

7.4.4. Estrategia de Despliegue Continuo (CD)

Tras producirse un *push* en la rama `main`, el despliegue del entorno de producción se efectúa automáticamente de forma segregada:

- **Backend (Render)**: Se gestiona mediante el *workflow* `deploy.yml`, el cual notifica al servicio de Render tras cada *push* en `main` para activar la actualización del servidor de aplicaciones.
- **Frontend (Vercel)**: Se apoya en la integración nativa de Vercel con el repositorio de GitHub, la cual detecta los nuevos *commits* en la rama `main` e inicia automáticamente la compilación y publicación de la interfaz.

7.4.5. Resumen de la estructura de CI/CD

A continuación, se detalla la estructura del flujo de integración y despliegue continuos, desglosando las fases ejecutadas según la rama de destino:

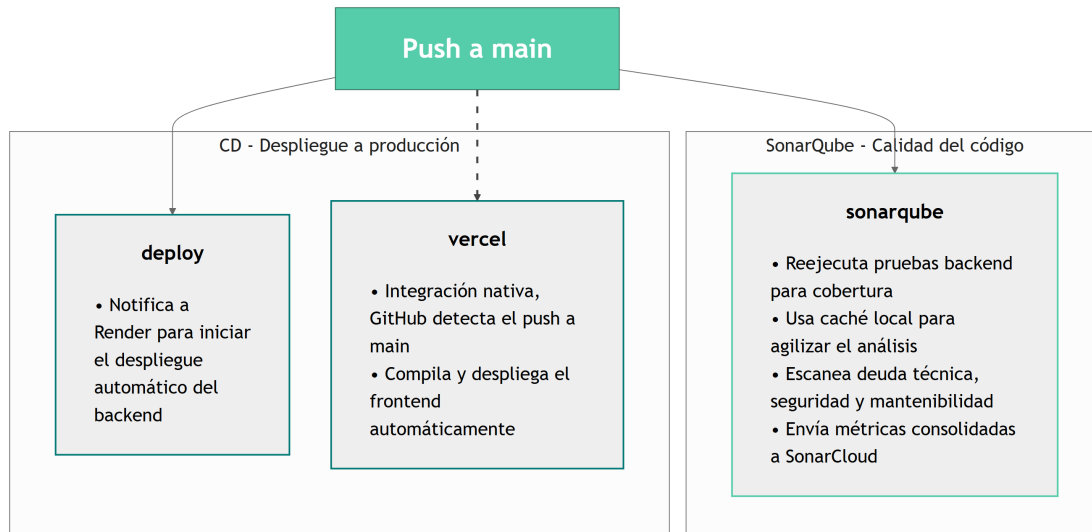


Figura 7.5: Resumen del flujo de CI/CD para la rama *main*.

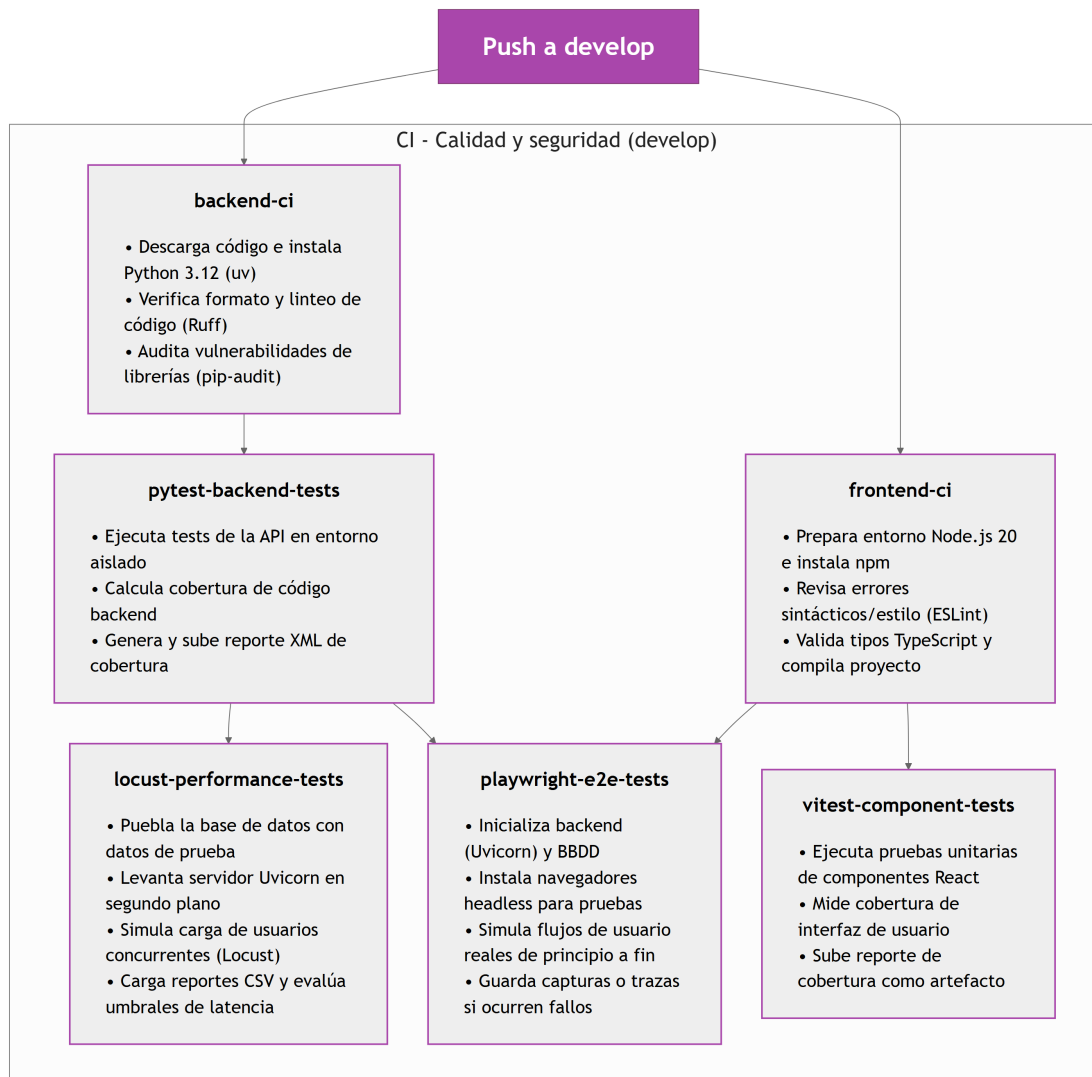


Figura 7.6: Resumen del flujo de CI para la rama *develop*.

7.5. Gestión de configuración y entorno

Para garantizar la seguridad, el cumplimiento de buenas prácticas y la portabilidad entre el desarrollo local y los entornos de producción, *Quizzie* desvincula completamente la configuración del código fuente. Los datos sensibles, credenciales de correo y claves de cifrado se gestionan de forma centralizada mediante variables de entorno seguras.

7.5.1. Categorización funcional de los secretos

Las variables de entorno del sistema se estructuran en cinco categorías funcionales según su cometido dentro de la arquitectura:

- **Autenticación y seguridad JWT:** Comprende la clave secreta de firma **SECRET_KEY**, el algoritmo de cifrado **ALGORITHM** y el tiempo de expiración de las sesiones **ACCESS_TOKEN_EXPIRE_MINUTES**. Garantizan la integridad y validez de los tokens de usuario.
- **Servicio de inteligencia artificial (Gemini API):** Agrupa las credenciales necesarias **GEMINI_API_KEY** para la autenticación y consumo de los modelos de lenguaje de Google Gemini dedicados a la generación automatizada de preguntas y cuestionarios.
- **Servicio de correo transaccional (SMTP):** Incluye los parámetros de conexión al servidor SMTP **SMTP_HOST**, **SMTP_PORT**, las credenciales de autenticación **SMTP_USERNAME**, **SMTP_PASSWORD** y la dirección del remitente **SMTP_FROM**.
- **Conectividad y cliente React:** Contiene la URL base del backend **VITE_API_URL**, requerida para orientar las peticiones HTTP del frontend tanto en desarrollo local como en producción.
- **Integración continua y calidad (CI/CD):** Agrupa los tokens de autenticación para servicios externos del pipeline, como el token de análisis en Sonar **SONAR_TOKEN** y la URL del disparador de despliegue en Render **RENDER_DEPLOY_HOOK_URL**.

7.5.2. Entorno de desarrollo local

Para facilitar la configuración durante el desarrollo sin comprometer la seguridad del repositorio, se proporciona la plantilla `.env.example`. Este archivo define la estructura y variables obligatorias del sistema definidas anteriormente.

A partir de dicha plantilla, el desarrollador genera su archivo local `.env`, ignorado por Git mediante `.gitignore` para evitar filtraciones. Este proceso se aborda en el [manual técnico](#).

7.5.3. Distribución de secretos por plataforma

En los entornos remotos se aplica de forma estricta el **principio de mínimo privilegio**, tal y como define el *National Institute of Standards and Technology* (NIST) en su informe especial SP 800-53 [45]. Bajo este criterio, se garantiza que cada plataforma o servicio contenga de forma aislada únicamente las variables estrictamente necesarias para cumplir su función técnica, reduciendo la superficie de ataque y minimizando el impacto ante eventuales vulnerabilidades.

La siguiente tabla resume la matriz de presencia de variables de entorno y secretos según la plataforma de despliegue:

Variable / Secreto	GitHub	Render	Vercel
SECRET_KEY	Sí	Sí	Sí
ALGORITHM	Sí	Sí	Sí
ACCESS_TOKEN_EXPIRE_MINUTES	—	Sí	Sí
VITE_API_URL	—	—	Sí
GEMINI_API_KEY	Sí	Sí	—
RENDER_DEPLOY_HOOK_URL	Sí	—	—
SONAR_TOKEN	Sí	—	—
PYTHON_VERSION / PYTHONPATH	—	Sí	—
SMTP_HOST / SMTP_PORT	—	Sí	—
SMTP_USERNAME / SMTP_PASSWORD	—	Sí	—
SMTP_FROM	—	Sí	—

Tabla 7.4: Variables de entorno y secretos por plataforma.

Justificación por entorno

- **GitHub (Actions):** Registra únicamente las credenciales requeridas para ejecutar las suites de prueba durante los *workflows*, la autenticación del escáner en Sonar y el disparador del despliegue en producción.
- **Render (Backend):** Concentra la configuración del servidor de aplicaciones, alojando la infraestructura de correo SMTP, la clave de acceso a la API de Gemini, los parámetros de runtime de Python y la gestión del ciclo de vida de los tokens JWT.
- **Vercel (Frontend):** Contiene exclusivamente el parámetro público VITE_API_URL para el enrutamiento de peticiones HTTP desde la SPA React hacia la API, además de los parámetros de control de sesión en el cliente.

8. Implementación y pruebas

Una vez establecida la base arquitectónica y el diseño del sistema, corresponde abordar la materialización técnica de la plataforma, transformando los requisitos y decisiones de diseño arquitectónico previamente definidos en componentes funcionales de software. Para ello, en primer lugar se detallan los **módulos críticos** de la implementación tanto en el *backend* como en el *frontend*, haciendo especial énfasis en la sincronización en **tiempo real** y la **seguridad**. Posteriormente, se expone la **estrategia integral de pruebas**, los criterios de calidad adoptados y los resultados de **cobertura** obtenidos para garantizar la robustez del sistema.

8.1. Componentes clave e implementación del sistema

En esta sección se exponen los aspectos más representativos del núcleo de *Quizzie*, analizando las decisiones de diseño y **patrones de software** adoptados para dar respuesta a los requisitos funcionales más complejos: la **comunicación bidireccional**, la **gestión del ciclo de vida de las salas** y la **seguridad del estado** en el *frontend*.

8.1.1. Comunicación en tiempo real mediante *WebSockets*

La interactividad en vivo entre el profesor y los alumnos requiere una arquitectura de baja latencia capaz de difundir eventos de forma simultánea a múltiples clientes mediante el protocolo WebSocket [9].

Gestor de conexiones en el *backend* (*ConnectionManager*)

En la ruta `backend/routers/stage.py`, el gestor de conexiones implementa una adaptación concurrente del patrón de diseño *Observer* [46] combinada con una arquitectura de publicación-suscripción (*Pub-Sub*) [47], permitiendo registrar, desacoplar y administrar dinámicamente el ciclo de vida de las instancias activas de WebSocket asociadas a cada sala mediante su identificador único (`room_id`).

```
class ConnectionManager:
    def __init__(self):
        self.active_connections: Dict[int, List[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, room_id: int):
        await websocket.accept()
        if room_id not in self.active_connections:
            self.active_connections[room_id] = []
        self.active_connections[room_id].append(websocket)
```

```

def disconnect(self, websocket: WebSocket, room_id: int):
    if room_id in self.active_connections:
        if websocket in self.active_connections[room_id]:
            self.active_connections[room_id].remove(websocket)

async def broadcast_to_room(self, room_id: int, message: dict):
    if room_id in self.active_connections:
        for connection in self.active_connections[room_id]:
            try:
                await connection.send_json(message)
            except Exception:
                self.active_connections[room_id].remove(connection)

```

Extracto de código 8.1: Gestor de conexiones en el *backend*.

Esta estructura permite emitir mensajes masivos en formato JSON mediante el método `broadcast_to_room`. Asimismo, incluye un mecanismo de tolerancia a fallos mediante un bloque `try-except` que elimina automáticamente las conexiones diferidas o corruptas sin interrumpir la emisión hacia el resto de los participantes.

Suscripción y recepción de eventos en el cliente React

En el componente principal de la sala (`frontend/src/room-play/LiveRoom.tsx`), el *frontend* se acopla a este flujo mediante un cliente `WebSocket`. Tal y como se ilustra en el código, la conexión se inicializa aportando el rol correspondiente (teacher o student) como parámetro de consulta en la fase de enlace (*handshake*).

```

const onMessage = useCallback(
  (event: MessageEvent) => {
    const { type, data } = JSON.parse(event.data);
    switch (type) {
      case "next_question":
      case "room_start":
      case "room_update":
        updateRoomStatus(data);
        break;
      case "show_results":
        setStatistics(data.statistics);
        setCorrectOptionId(data.correct_option_id);
        break;
      case "show_leaderboard":
        setLeaderboardData(data.leaderboard);
        break;
      case "timer_update":
        if (data.time_left !== undefined) setTimeLeft(data.time_left);
        if (data.is_paused !== undefined) setIsPaused(data.is_paused);
        break;
      // [...] Otros eventos: answer_submitted, room_finish, participant_verified, etc.
    }
  },
  [updateRoomStatus, setRoomData],
);

useEffect(() => {

```

```

const idToUse = Number(urlRoomId || roomId);
if (!idToUse || Number.isNaN(idToUse)) return;

// [...] Sincronización inicial del estado de la sala mediante API REST: syncRoom()

const ws = new WebSocket(
  `${WS_BASE_URL}/stage/rooms/${idToUse}/ws?role=${isHost ? "teacher" : "student"}`,
);
ws.onopen = () => setIsConnected(true);
ws.onclose = () => setIsConnected(false);
ws.onmessage = onMessage;
return () => ws.close();
}, [roomId, urlRoomId, isHost, updateRoomStatus, onMessage]);

```

Extracto de código 8.2: Conexión y enrutamiento de eventos WebSocket en React.

El *handler* `onmessage`, optimizado mediante el *hook* `useCallback`, actúa como un enrutador de eventos de la interfaz de usuario, mutando el estado local de React en función de la acción recibida del servidor. Asimismo, la función de retorno del *hook* `useEffect` (*cleanup function*) garantiza el cierre formal del socket al desmontar el componente, previniendo fugas de memoria y conexiones huérfanas en el cliente.

8.1.2. Gestión de sala, estado, tiempo y aleatoriedad

El correcto funcionamiento de las sesiones depende de mantener la sala sincronizada y lista para todos los usuarios. Para lograrlo, el servidor asume el control centralizado de los eventos y las decisiones críticas, gestionando el avance del estado de la sala y evitando depender del estado o el reloj individual de cada cliente.

Cálculo determinista del tiempo restante en UTC

En `backend/routers/stage.py`, la medición del tiempo no depende del reloj del cliente, sino de la diferencia estricta de marcas temporales en UTC.

```

def get_calculated_time_left(room: Room) -> int:
    if room.is_paused:
        return room.remaining_time_at_pause
    if not room.timer_started_at:
        return room.remaining_time_at_pause

    now = get_utc_now()
    started_at = room.timer_started_at
    if started_at.tzinfo is None:
        from datetime import timezone as tz
        started_at = started_at.replace(tzinfo=tz.utc)

    elapsed = (now - started_at).total_seconds()
    calculated_time = room.remaining_time_at_pause - int(elapsed)
    return max(0, min(room.answer_time, calculated_time))

```

Extracto de código 8.3: Cálculo del tiempo restante sincronizado en el *backend*.

Al restar el instante actual (`get_utc_now()`) del momento de inicio (`timer_started_at`), el sistema inmuniza la cuenta atrás frente a retrasos de red o pausas de ejecución en el cliente.

Pausa automática por desconexión del profesor

Para mitigar imprevistos técnicos durante una sesión presencial, el servidor detecta si la conexión WebSocket del docente cae inesperadamente. En la ruta `backend/routers/stage.py`, se implementa una rutina de contingencia encargada de preservar la integridad temporal de la sala activa.

```
async def handle_teacher_disconnect(room_id: int, db: Session):
    room = db.query(Room).filter(Room.id == room_id).first()
    if not room or room.is_paused or room.status != RoomStatus.IN_PROGRESS:
        return

    room.is_paused = True
    room.remaining_time_at_pause = get_calculated_time_left(room)
    db.commit()

    await manager.broadcast_to_room(
        room_id,
        {
            "type": "timer_update",
            "time_left": room.remaining_time_at_pause,
            "is_paused": True,
        },
    )
```

Extracto de código 8.4: Rutina de pausa automática ante la desconexión del docente.

La función congela la partida almacenando el tiempo restante exacto en `remaining_time_at_pause` mediante `get_calculated_time_left` y notifica inmediatamente a todos los estudiantes, pausando la interfaz hasta la reconexión del profesor.

Barajado determinista de opciones

Para mitigar que los estudiantes puedan copiar visualmente la alternativa seleccionada en la pantalla de un compañero contiguo durante sesiones presenciales, el *backend* altera la disposición de las opciones de respuesta. No obstante, delegar esta tarea en generadores de números pseudoaleatorios convencionales (como el módulo estándar `random` de Python) introduce dos problemas críticos: la pérdida de determinismo ante caídas de conexión y bloqueos en el análisis estático de código.

Herramientas de aseguramiento de calidad como SonarQube restringen el uso de generadores pseudoaleatorios estándar en flujos de datos sensibles, catalogándolos como debilidades de seguridad si no garantizan entropía criptográfica o cuando su estado mutable global puede generar condiciones de carrera entre hilos de FastAPI. Para cumplir con el *Quality Gate* y preservar una

arquitectura estrictamente sin estado (*stateless*), se implementó en el mismo módulo del enrutador (`backend/routers/stage.py`) un algoritmo de barajado determinista basado en dispersión criptográfica:

```
def deterministic_shuffle(items: List[dict], room_id: int, question_id: int) -> List[dict]:
    def sort_key(item):
        seed_str = f"{room_id}_{question_id}_{item['id']}"
        return hashlib.sha256(seed_str.encode("utf-8")).hexdigest()

    return sorted(items, key=sort_key)
```

Extracto de código 8.5: Algoritmo de barajado determinista anti-copia.

El algoritmo ordena los elementos calculando al vuelo una clave derivada del algoritmo *SHA-256*, resultante de concatenar el identificador de la sala (`room_id`), la pregunta en curso (`question_id`) y el identificador único de cada respuesta (`item['id']`). Esta solución aporta tres garantías técnicas fundamentales:

- **Cumplimiento estricto de calidad (SonarQube):** Elimina el uso de funciones no seguras o pseudoaleatorias basadas en semillas globales mutables, superando las reglas de análisis estático sin requerir excepciones manuales en la tubería de CI/CD.
- **Idempotencia y tolerancia a fallos:** Al ser una función pura y reproducible, si un alumno pierde la conexión WebSocket o refresca el navegador en mitad de la cuenta atrás, el reenvío del evento recalcula exactamente la misma ordenación sin discrepancias con el estado previo.
- **Eficiencia y desacoplamiento de base de datos:** Evita persistir campos de ordenación efímeros en PostgreSQL o en memoria de sesión de FastAPI, reduciendo el volumen de operaciones de escritura por cada ronda de preguntas.

Transición de preguntas y fase de verificación

El *endpoint* `PATCH /rooms/:room_id/next-question` en el módulo `backend/routers/stage.py` coordina el avance secuencial del cuestionario o la transición a la fase final de la partida. Su diseño garantiza la persistencia del orden de preguntas en la sesión, la aplicación del barajado determinista y la sincronización en tiempo real vía WebSocket.

```
@router.patch("/rooms/{room_id}/next-question")
async def next_question(room_id: int, db: Annotated[Session, Depends(get_session)]):
    # [...] Validación de existencia de sala y comprobación de estado LIVE

    q_ids = [int(x) for x in room.question_order.split(",") if x] if room.question_order
    else [
        q.id for q in db.exec(select(Question).where(Question.quiz_id == room.quiz_id).
        order_by(Question.id)).all()
    ]
    if not room.question_order:
        room.question_order = ",".join(map(str, q_ids))
```

```

        db.add(room)
        db.commit()

    next_index = room.current_question_index + 1

    if next_index <= len(q_ids):
        next_q = db.get(Question, q_ids[next_index - 1])
        room.current_question_index = next_index
        room.phase = RoomPhase.ANSWERING
        room.remaining_time_at_pause = room.answer_time
        room.timer_started_at = get_utc_now()
        room.is_paused = False
        db.commit()

        options_list = [{"id": opt.id, "text": opt.text} for opt in next_q.options]
        if room.shuffle_options:
            options_list = deterministic_shuffle(options_list, room.id, next_q.id)

        # [...] Construccion del payload de estado y emision via WebSocket (
        next_question)
        await manager.broadcast_to_room(room_id, {"type": "next_question", "data": data
        })
        return data
    else:
        room.status = RoomStatus.VERIFYING
        participants = db.exec(select(Participant).where(Participant.room_id == room_id)
        ).all()
        for p in participants:
            p.verification_token = secrets.token_hex(8)
            db.add(p)

        db.commit()
        await manager.broadcast_to_room(
            room_id,
            {"type": "room_verifying", "data": {"status": "VERIFYING", "total_questions"
: len(q_ids)}},
        )
        return {"status": "VERIFYING"}

```

Extracto de código 8.6: Transición entre preguntas y paso a verificación.

Si restan preguntas por responder, el controlador actualiza el índice y emite el evento correspondiente. Al agotar la batería de preguntas, la sala cambia al estado VERIFYING y genera tokens criptográficos efímeros mediante `secrets.token_hex(8)`.

Verificación presencial mediante código QR

Una vez concluida la partida, la calificación debe ser validada por el profesor escaneando un código QR generado en el dispositivo del alumno. En el módulo `backend/routers/stage.py`, el *endpoint* POST `/rooms/:room_id/verify-participant` gestiona la recepción de la carga útil del código QR para auditar y validar la calificación.

```

@router.post("/rooms/{room_id}/verify-participant")
async def verify_participant(
    room_id: int, req: VerificationRequest, db: Annotated[Session, Depends(get_session)]
):
    statement = (
        select(Participant)
        .join(Student)
        .where(Student.name == req.nickname, Participant.room_id == room_id)
    )
    participant = db.exec(statement).first()

    # [...] Comprobacion de existencia, no verificacion previa y token efimero
    if not participant:
        raise HTTPException(status_code=404, detail=PARTICIPANT_NOT_FOUND)
    if participant.verification_token != req.token:
        raise HTTPException(status_code=400, detail="Token de verificacion invalido")
    if participant.is_verified:
        raise HTTPException(status_code=409, detail="Resultado ya verificado")

    actual_score = sum(a.points_earned for a in participant.answers)
    if participant.score != actual_score:
        participant.score = actual_score

    participant.is_verified = True
    participant.verified_at = get_utc_now()
    db.add(participant)
    db.commit()

    # [...] Notificacion en tiempo real via WebSocket de la verificacion
    await manager.broadcast_to_room(
        room_id,
        {
            "type": "participant_verified",
            "data": {"nickname": req.nickname, "participant_id": participant.id},
        },
    )
    return {"status": "success", "nickname": req.nickname}

```

Extracto de código 8.7: Validación del token y consolidación de calificación.

El controlador valida el token efímero (`req.token`), recalcula la puntuación final sumando los puntos auditados, marca al participante como verificado y notifica la confirmación en tiempo real.

8.1.3. Seguridad, contexto y autenticación

Este bloque aborda el tratamiento del estado del jugador, el saneamiento de datos en la SPA y la protección de endpoints sensibles mediante tokens JWT.

Persistencia y saneamiento de estado en la SPA

En el módulo `frontend/src/context/RoomContext.tsx` se encapsula la gestión de la sesión del jugador mediante la *API Context* de React.

```

export function RoomProvider({ children }: Readonly<{ children: ReactNode }>) {
  // Inicializacion sincronica desde almacenamiento de sesion
  const [rawRoomCode, setRawRoomCode] = useState(sessionStorage.getItem("roomCode") || "")
  const [rawRoomId, setRawRoomId] = useState<number | null>(() => {
    const savedId = sessionStorage.getItem("roomId");
    return savedId ? Number(savedId) : null;
  });
  // [...] Estados analogos para rawUserNickname, rawParticipantId y roomData

  // Saneamiento de datos antes de persistir y actualizar el estado
  const setRoomCode = useCallback((code: string) => {
    const cleanCode = code.replace(/^[a-zA-Z0-9_-]/g, "");
    if (/^[a-zA-Z0-9_-]*$/i.test(cleanCode)) {
      setRawRoomCode(cleanCode);
      sessionStorage.setItem("roomCode", cleanCode);
    }
  }, []);

  const setUserNickname = useCallback((name: string | undefined) => {
    setRawUserNickname(name);
    if (name !== undefined) {
      const cleanNickname = name.replace(/<>'"/g, "");
      if (/^[^<>'"/]*$/i.test(cleanNickname)) {
        sessionStorage.setItem("userNickname", cleanNickname);
      }
    } else {
      sessionStorage.removeItem("userNickname");
    }
  }, []);

  // [...] Manejadores identicos de mutacion saneada para setRoomId y setParticipantId

  const contextValue = useMemo(
    () => ({ roomCode: rawRoomCode, setRoomCode, userNickname: rawUserNickname,
      setUserNickname /* [...] */ }),
    [rawRoomCode, setRoomCode, rawUserNickname, setUserNickname /* [...] */],
  );

  return <RoomContext.Provider value={contextValue}>{children}</RoomContext.Provider>;
}

```

Extracto de código 8.8: Persistencia y saneamiento frente a inyecciones XSS.

El componente recupera de forma síncrona los identificadores desde `sessionStorage` para garantizar la persistencia del estado frente a recargas accidentales del navegador. Asimismo, se aplica una función de saneamiento que purga caracteres sensibles (<, >, ', ", &) antes de su persistencia y renderizado, mitigando vectores de ataque por inyección de scripts en sitios cruzados o XSS (*Cross-Site Scripting*) [48, 49].

Autenticación y protección de endpoints con JWT

En backend/auth.py, se implementa un mecanismo de inyección de dependencias para autenticar las peticiones de los docentes mediante cabeceras HTTP.

```
security = HTTPBearer()

def get_current_teacher_id(
    credentials: Annotated[HTTPAuthorizationCredentials, Depends(security)],
) -> int:
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        raw_sub = payload.get("sub")
        if raw_sub is None:
            raise HTTPException(status_code=401, detail="Token sin campo 'sub'")
        return int(raw_sub)

    except jwt.ExpiredSignatureError:
        raise HTTPException(status_code=401, detail="El token ha expirado")
    except jwt.PyJWTError:
        raise HTTPException(status_code=401, detail="Token invalido. Inicia sesion de nuevo.")
    except Exception:
        raise HTTPException(status_code=401, detail="No autorizado")
```

Extracto de código 8.9: Extracción y verificación criptográfica de cabeceras JWT.

Esta función intercepta el *Bearer token* emitido en las peticiones HTTP, descifra la firma criptográfica del estándar JSON Web Token [50] utilizando la clave secreta del servidor (SECRET_KEY) y verifica su validez temporal antes de permitir el acceso a los controladores protegidos.

8.2. Estrategia de *testing*

Para garantizar la **estabilidad** del sistema, la **integridad** de los datos en tiempo real y una experiencia de usuario **fluida**, se ha diseñado un marco integral de verificación y pruebas automatizadas fundamentado en el modelo clásico de la **pirámide de pruebas** [51]. Este enfoque distribuye el esfuerzo analítico en distintos niveles de granularidad, equilibrando la velocidad de ejecución con el nivel de confianza obtenido sobre el comportamiento de la plataforma.

8.2.1. Niveles de prueba y selección de herramientas

El ecosistema de pruebas desacopla las tecnologías encargadas de la ejecución de casos de prueba de aquellas orientadas a la auditoría estricta de cobertura de código, tal como se resume a continuación:

Nivel de prueba	Objetivo principal	Herramientas
Unitarias (<i>Backend</i>)	Validar lógica de negocio aislada (cálculo de puntuaciones, <i>hashing</i> , validaciones Pydantic).	pytest
Integración (<i>Backend</i>)	Comprobar la interacción entre API REST, persistencia relacional y <i>WebSockets</i> .	pytest + TestClient (FastAPI)
Componentes (<i>Frontend</i>)	Verificar el renderizado y comportamiento reactivo de la interfaz en aislamiento.	Vitest + React Testing Library
Extremo a Extremo	Simular flujos completos de navegación y concurrencia entre roles (docente y alumno).	Playwright
Rendimiento y carga	Evaluar la estabilidad y latencia del servidor ante concurrencia masiva en tiempo real.	Locust

Tabla 8.1: Clasificación de niveles de prueba y herramientas del sistema.

8.2.2. Organización del plan de pruebas y matriz de trazabilidad

La *suite* de pruebas se estructura modularmente y se alinea con la evolución temporal del proyecto, estableciendo una correspondencia unívoca entre la ingeniería de requisitos previa y los artefactos de verificación construidos:

- **Trazabilidad con requisitos:**
 - **Pruebas unitarias** → **Requisitos Funcionales (RF):** Garantizan que cada regla de negocio o cálculo atómico cumpla estrictamente con las especificaciones funcionales unitarias.
 - **Pruebas de integración** → **Historias de Usuario (HU):** Verifican que la interacción coordinada entre capas satisfaga los criterios de aceptación desde la perspectiva del usuario.
 - **Pruebas E2E** → **Casos de Uso (CU):** Validan el recorrido holístico del sistema simulando navegadores reales para cada caso de uso del sistema.
- **Segmentación modular del sistema:** La batería de pruebas se divide según los dominios de la plataforma:
 - En el *backend*, las suites se organizan por áreas funcionales: contenido (cuestionarios y preguntas), sesiones (gestión de salas y sincronización en vivo) y usuarios (autenticación y perfiles).
 - En el *frontend*, las pruebas se delimitan en los cuatro módulos cliente principales: autenticación (auth), gestión (management), acceso a salas (room-access) y juego en vivo (room-play).
- **Evolución por fases de desarrollo:** Las pruebas se estructuraron formalmente en tres bloques, correspondientes a cada una de las fases del proyecto (*MVP*, *Core* y *QA*), facilitando la organización del trabajo y permitiendo abordar la verificación del sistema de manera progresiva y ordenada.

8.2.3. Auditoría y umbrales de cobertura de código

Para evaluar el alcance y la exhaustividad de la batería de pruebas, se establecen umbrales mínimos (*thresholds*) en las capas críticas del sistema. Estas métricas garantizan que la lógica de negocio del servidor y las vistas interactivas del cliente mantengan unos niveles de comprobación suficientes antes de considerar el código listo para producción:

Ámbito	Herramienta	Umbral mínimo exigido
Backend	pytest-cov	≥ 90 % en endpoints y rutas API
		≥ 80 % en código global del servidor
Frontend	@vitest/coverage-v8	≥ 80 % en líneas, funciones y sentencias
		≥ 75 % en ramas (<i>branches</i>)
Pruebas E2E	monocart-coverage-reports	≥ 80 % en líneas, funciones y sentencias
		≥ 75 % en ramas

Tabla 8.2: Métricas y umbrales mínimos de cobertura por ámbito de prueba.

En las pruebas E2E, los componentes modales emergentes (como `LogoutModal.tsx` o `ScannerModal.tsx`) quedan deliberadamente excluidos de la métrica de `monocart-coverage-reports` y se evalúan de forma exhaustiva mediante `Vitest`. De este modo se evitan distorsiones en la cobertura debidas a guardas de renderizado condicional cuando los diálogos permanecen cerrados.

8.2.4. Ejecución y automatización de pruebas

Las diferentes baterías de pruebas que componen el sistema pueden ejecutarse en el entorno de desarrollo local. Las instrucciones de configuración, los prerequisites del entorno y los comandos específicos para su lanzamiento se detallan en el [manual técnico](#).

Asimismo, tal y como se describió en la especificación del [flujo de integración continua](#), la ejecución de la batería de pruebas se encuentra automatizada mediante GitHub Actions, activándose con cada evento de *push* dirigido a la rama `develop`.

8.3. Casos de prueba y validación

Esta sección expone los casos de prueba implementados para verificar el cumplimiento de los criterios de aceptación y las reglas de negocio del sistema, estructurados según los niveles de la pirámide de pruebas definida previamente.

8.3.1. Pruebas de la capa *Backend*

Las pruebas unitarias y de integración se definen y ejecutan de forma conjunta mediante Pytest y el cliente de pruebas asíncrono de FastAPI. Esta batería cubre desde la lógica de dominio pura hasta la interacción coordinada entre la API REST, la persistencia en base de datos y la comunicación en tiempo real por *WebSockets*, estructurándose en tres módulos funcionales:

ID	Caso de prueba	Alcance y condiciones verificadas
PB-01	Control de acceso JWT	Intercepción mediante la dependencia <code>get_current_teacher_id</code> ante tokens expirados, firmas ilegítimas o ausencia de cabecera <code>Authorization</code> , asegurando el rechazo inmediato y la protección de endpoints privados.
PB-02	Ciclo de vida y verificación	Flujo de registro con derivación segura de claves mediante <code>bcrypt</code> , emisión de códigos temporales de 6 dígitos, prevención de accesos no autorizados a cuentas pendientes de validación y modo de contingencia del servicio de mensajería.
PB-03	Integridad relacional y datos	Cumplimiento estricto del borrado en cascada sobre cuestionarios, grupos y salas al tramitar la baja de una cuenta docente, control de concurrencia y colisiones por identificadores duplicados, y garantía de enmascaramiento irreversible de contraseñas en las respuestas serializadas por <code>Pydantic</code> .
PB-04	Gestión de estudiantes	Instanciación e identificación de alumnos, validación formal del identificador universitario (<i>uvus</i>) y detección de alias preexistente.

Tabla 8.3: Casos de prueba de la capa *backend*: Módulo de usuarios.

ID	Caso de prueba	Alcance y condiciones verificadas
PB-05	Persistencia transaccional	Creación y actualización atómica de cuestionarios con preguntas y opciones en una única transacción de base de datos, validación estricta de límites operativos y aseguramiento de que toda pregunta contenga obligatoriamente una opción marcada como correcta.
PB-06	Seguridad y aislamiento	Comprobación de que ningún usuario pueda consultar, mutar o eliminar cuestionarios ajenos, bloqueando cualquier intento de escalado horizontal de privilegios.
PB-07	Integridad referencial	Bloqueo de borrado (<i>hard delete</i>) si el cuestionario posee salas activas asociadas, con validación de la eliminación en cascada de preguntas y opciones solo cuando las salas vinculadas ya han sido finalizadas.
PB-08	Filtrado, historial y exportación	Auditoría de la lógica de segmentación de cuestionarios, consulta y agregación histórica de las sesiones finalizadas asociadas a cada cuestionario, y extracción estructurada de calificaciones y aciertos formateados para su descarga en CSV.

Tabla 8.4: Casos de prueba de la capa *backend*: Módulo de contenido.

ID	Caso de prueba	Alcance y condiciones verificadas
PB-09	Salas únicas y transición	Control estricto de concurrencia que impide abrir múltiples salas simultáneas para un mismo cuestionario y gestión del ciclo de vida y transiciones de los estados y fases de la sala.
PB-10	Sincronización y temporizador	Validación del bucle asíncrono <code>timer_sync_loop</code> en el servidor, garantizando la emisión periódica de marcas de tiempo UTC sin bloqueo del hilo principal, junto con el control docente de pausas, reanudaciones y cierres forzados.
PB-11	Mecanismos anti-fraude y QR	Comprobación del algoritmo de barajado pseudoaleatorio de opciones, procesamiento atómico de respuestas con rechazo de dobles envíos, validación mediante escaneo del código QR y purga de alumnos no validados al cerrar la sala.
PB-12	Resiliencia en <i>WebSockets</i>	Verificación de la resiliencia del <code>ConnectionManager</code> ante desconexiones abruptas por <i>WebSockets</i> , asegurando la expulsión ordenada de sockets corruptos sin interrumpir la difusión de eventos al resto del aula.

Tabla 8.5: Casos de prueba de la capa *backend*: Módulo de sesiones.

Para garantizar la cobertura sistemática de los requisitos del sistema, la siguiente tabla sintetiza la correspondencia entre los casos de prueba y las Historias de Usuario, junto con los Requisitos Funcionales que auditan de forma directa:

Módulo	ID	Historias de Usuario	Requisitos Funcionales
Usuarios	PB-01	HU-PR-01	RF-01, RF-02, RF-03
	PB-02	HU-PR-01	RF-04, RF-05, RF-06
	PB-03	HU-PR-01, HU-PR-07	RF-06, RF-38
	PB-04	HU-AL-01	RF-16, RF-17
Contenido	PB-05	HU-PR-02	RF-07, RF-08, RF-13, RF-14
	PB-06	HU-PR-02	RF-07, RF-10
	PB-07	HU-PR-02	RF-12, RF-41
	PB-08	HU-PR-10, HU-PR-13, HU-PR-14	RF-39, RF-46, RF-47
Sesiones	PB-09	HU-PR-03, HU-PR-05	RF-11, RF-15, RF-18, RF-19
	PB-10	HU-PR-03, HU-AL-03	RF-20, RF-22
	PB-11	HU-PR-04, HU-PR-06, HU-AL-04	RF-23, RF-24, RF-27, RF-28, RF-30, RF-31
	PB-12	HU-PR-05, HU-AL-02	RF-21, RF-40

Tabla 8.6: Matriz de trazabilidad de pruebas *backend* con HU y RF.

8.3.2. Pruebas de componentes de la interfaz (*Frontend*)

Las pruebas de componentes se ejecutan de manera aislada mediante Vitest y React Testing Library sobre un entorno DOM simulado (jsdom). Esta batería evalúa tanto los elementos visuales como las vistas completas de la interfaz, verificando el renderizado de componentes en pantalla, la gestión del estado reactivo ante eventos del usuario y la accesibilidad en las diferentes áreas del cliente web:

ID	Caso de prueba	Alcance y condiciones verificadas
PF-01	Interacción accesible en entradas de códigos	Verificación del componente de códigos (CodeInputGrid), evaluando el avance automático de foco entre celdas, el borrado secuencial y el soporte de pegado desde el portapapeles.
PF-02	Validación de credenciales y permisos	Pruebas sobre los formularios de acceso, registro y restablecimiento de contraseña, asegurando el manejo visual de errores y la persistencia segura de credenciales en almacenamiento local.
PF-03	Confirmación modal de acciones críticas	Comprobación del renderizado y comportamiento de los diálogos de cierre de sesión y baja definitiva de cuenta con validación previa de contraseña.

Tabla 8.7: Casos de prueba de la capa *frontend*: Módulo de autenticación y perfil.

ID	Caso de prueba	Alcance y condiciones verificadas
PF-04	Filtrado dinámico e indicadores de estado	Renderizado de la lista de cuestionarios con filtrado reactivo y opciones de reconexión para salas en curso.
PF-05	Formularios de edición y creación de cuestionarios	Creación y modificación de cuestionarios, límites en el número de preguntas y opciones, imposibilidad de guardar una pregunta sin opción correcta, atajos de teclado y gestos táctiles (<i>swipe</i>).
PF-06	Configuración de salas	Ajuste de temporizador y controles para el barajado de preguntas y la visibilidad del podio.

Tabla 8.8: Casos de prueba de la capa *frontend*: Módulo de gestión de contenidos.

ID	Caso de prueba	Alcance y condiciones verificadas
PF-07	Introducción del código de sala y verificación	Validación visual del campo numérico de acceso y gestión de mensajes de error ante códigos inexistentes o salas cerradas.
PF-08	Validación de formato de <i>uvus</i>	Comprobación del identificador mediante expresiones regulares previa a la llamada al servidor.
PF-09	Confirmación de nuevos alumnos	Gestión de diálogos emergentes de vinculación de identidad para participantes no registrados con anterioridad.

Tabla 8.9: Casos de prueba de la capa *frontend*: Módulo de acceso a salas.

ID	Caso de prueba	Alcance y condiciones verificadas
PF-10	Sincronización visual de sala de espera	Despliegue del código de sala, detalles de la sala y listado reactivo de participantes conectados.
PF-11	Experiencia interactiva del estudiante	Renderizado de alternativas barajadas, temporizador visual animado y bloqueo de botones tras la emisión del voto para evitar respuestas duplicadas.
PF-12	Control docente y visualización de resultados	Botones de finalización del temporizador, gráficos de barras con el recuento de opciones y podio animado con las mejores calificaciones.
PF-13	Generación de credenciales QR y escaneo presencial	Renderizado del código QR personal con token efímero en el dispositivo del alumno y modal con visor de cámara para la validación de notas por parte del profesor.

Tabla 8.10: Casos de prueba de la capa *frontend*: Módulo de sala en directo.

Con el fin de garantizar que la interfaz de usuario no solo sea funcionalmente correcta, sino que también cumpla con los estándares de rendimiento, accesibilidad, seguridad y diseño responsivo exigidos, se ha establecido una correspondencia explícita entre las pruebas de componentes y los requisitos no funcionales del sistema:

PF \ RNF	01	02	03	04	05	06	07	08	09	10
01						×				×
02				×	×	×				×
03				×		×				×
04						×		×		×
05						×				×
06						×				×
07						×		×		×
08						×				×
09						×				×
10	×	×			×	×		×		×
11	×	×			×	×		×	×	×
12	×				×	×			×	×
13			×		×	×	×			×

Tabla 8.11: Matriz de trazabilidad pruebas *frontend* con RNF.

8.3.3. Pruebas de extremo a extremo (E2E)

Las pruebas de extremo a extremo se implementan con Playwright sobre instancias reales de Chromium, automatizando los flujos de interacción completos sin recurrir a emulaciones intermedias. Esta *suite* reproduce fielmente la experiencia de uso del sistema mediante escenarios que validan de forma integrada el cliente web, la API REST, la base de datos y la comunicación en tiempo real. Los casos de prueba definidos para este nivel se recogen en la siguiente tabla:

ID	Caso de prueba	Alcance y condiciones verificadas
PE-01	Autenticación y perfil docente	Verificación de rutas protegidas (redirección a <i>login</i> si no existe sesión activa), registro docente con validación de formularios, introducción de códigos en la vista de verificación de correo, restablecimiento de contraseña y baja irreversible de usuario.
PE-02	Gestión de cuestionarios	Ciclo completo de creación de cuestionarios con preguntas de diversa ponderación, edición de enunciados, borrado lógico con opción de restauración y eliminación física definitiva con comprobación de avisos sobre salas históricas.
PE-03	Configuración y apertura de sala	Configuración de parámetros de sala, apertura de la sala de espera en estado WAITING y transición sincronizada hacia la cuenta atrás inicial tras la orden docente de arranque.
PE-04	Acceso a sala de espera	Acceso secuencial de participantes mediante código PIN, validación del formato UVUS, confirmación de identidad y recepción concurrente de la lista de espera por <i>WebSockets</i> .
PE-05	Podio y puntuaciones	Renderizado del podio con las mejores puntuaciones entre fases de preguntas, tratamiento de empates y visualización personalizada de la posición y puntuación individual en el navegador de cada alumno.
PE-06	Validación QR	Flujo de cierre de partida con muestra del código QR personal en el dispositivo del alumno, apertura del modal con cámara en el panel docente, escaneo del token criptográfico, rechazo de códigos ajenos a la partida y confirmación visual de calificación auditada.
PE-07	Historial de resultados	Revisión del histórico de sesiones pasadas y desglose porcentual de opciones seleccionadas por los alumnos al término de cada pregunta.
PE-08	Exportación de calificaciones	Descarga en el navegador del fichero CSV normalizado con las notas finales y número de aciertos de los alumnos tras finalizar la sala.
PE-09	Emisión de respuestas	Selección y emisión de votos por parte del alumno con desactivación inmediata de botones anti-duplicidad, gestión del temporizador (pausa y corte anticipado) y propagación de eventos <i>WebSocket</i> .

Tabla 8.12: Casos de prueba E2E.

Para asegurar una cobertura funcional completa y garantizar que los flujos críticos del sistema han sido evaluados en condiciones reales de ejecución, la siguiente matriz recoge la correspondencia entre los escenarios de prueba y los Casos de Uso del sistema.

PE \ CU	01	02	04	05	06	07	08	09	10
01	×								
02		×							
03			×						
04				×					
05					×				
06						×			
07							×		
08								×	
09									×

Tabla 8.13: Matriz de trazabilidad pruebas E2E con CU.

Cabe señalar que los casos de uso **CU-03** (asociado al **OBJ-09**) y **CU-11** (asociado al **OBJ-11**) no se encuentran reflejados en esta batería de pruebas, ya que no forman parte del alcance implementado actualmente. La justificación técnica, el análisis del estado de ambas funcionalidades y su planificación futura se detallan en la sección de **cumplimiento del alcance**.

La cuantificación del volumen total de pruebas ejecutadas, los porcentajes de cobertura alcanzados y las evidencias gráficas resultantes se analizan en detalle en el capítulo dedicado a los **resultados y evaluación** del sistema.

8.4. Pruebas de carga y rendimiento

Dada la exigencia de una comunicación bidireccional fluida en las sesiones en directo, se implementó un banco de pruebas de estrés mediante Locust. El escenario estándar evalúa el comportamiento del *backend* sometido a una carga simultánea de **200 usuarios** virtuales concurrentes, incorporados progresivamente a una tasa de 15 usuarios por segundo durante 1 minuto continuo.

El reparto del volumen total de usuarios simulados se distribuye como se muestra a continuación, reflejando la composición realista de la audiencia y los roles del sistema:

1. **TeacherManagerUser** (3 %, 6 usuarios): simula la navegación administrativa del profesor, la consulta del catálogo y la edición interactiva de cuestionarios.
2. **TeacherHostUser** (2 %, 4 usuarios): modela a los profesores dirigiendo salas activas en tiempo real, ordenando el tránsito entre fases de preguntas y forzando el despliegue de las clasificaciones.
3. **StudentUser** (95 %, 190 usuarios): representa al grueso del alumnado interactuando de forma concurrente, manteniendo conexiones persistentes

mediante *WebSockets*, emitiendo respuestas masivas al unísono y enviando señales periódicas de *ping*.

Para evaluar los tiempos de respuesta de forma rigurosa, se utiliza como métrica principal el **percentil 90 (P90)**. Este indicador asegura que al menos el 90 % de las peticiones analizadas se resuelven en un tiempo igual o inferior al límite establecido, garantizando una percepción de inmediatez para la inmensa mayoría de los usuarios concurrentes y evitando que los valores atípicos extremos distorsionen la evaluación del sistema.

Para superar la auditoría de rendimiento, el sistema debe satisfacer estrictamente los umbrales de latencia y estabilidad expuestos en esta tabla:

Ámbito	Métrica	Operación evaluada / Condición	Umbral objetivo
Global	Tasa de error	Régimen continuo con 200 usuarios concurrentes en carga sostenida	< 1,0 %
REST API	Latencia P90	Peticiones generales de lectura, unión a salas y emisión de votos	≤ 200 ms
Autenticación	Latencia P90	Inicio de sesión (POST /users/login con Bcrypt) y resto de rutas protegidas bajo token JWT	≤ 500 ms
<i>WebSockets</i>	Latencia P90	Negociación inicial (<i>handshake</i>), validación de token y apertura de canal	≤ 200 ms
Concurrencia	Latencia P90	Ráfagas masivas de respuestas concurrentes de alumnos (POST /stage/answers)	≤ 200 ms

Tabla 8.14: Umbrales y criterios de aceptación para las pruebas de rendimiento.

La verificación y el cumplimiento estricto de estos umbrales se auditan de forma automatizada tras cada ejecución mediante el *script* `evaluate_thresholds.py`. Esta herramienta procesa los ficheros de estadísticas consolidados por Locust, computa los percentiles reales obtenidos frente a las cotas fijadas y determina el éxito o fracaso de las pruebas de rendimiento.

9. Resultados y evaluación

Una vez finalizado el ciclo de desarrollo e implementadas las pruebas de verificación expuestas en el capítulo anterior, el presente capítulo aborda la **auditoría cuantitativa y cualitativa** del proyecto. A través de este análisis se evalúa el desempeño de la solución construida respecto a los compromisos definidos durante la fase de planificación, contrastando la **ejecución real frente a las líneas base** establecidas. Asimismo, se documenta la **gestión de cambios** articulada durante el desarrollo, la resolución efectiva de las pruebas y una retrospectiva técnica en torno a la infraestructura de integración continua y métricas de calidad de código.

9.1. Cumplimiento de la Línea Base del Alcance

La **Línea Base del Alcance** quedó formalizada en la **planificación del proyecto** a través de la **EDT** y su correspondiente **diccionario**, distribuyendo el esfuerzo en cuatro paquetes troncales: Gestión del proyecto, Diseño y arquitectura, Producto software y Documentación final.

A nivel global del proyecto, los entregables documentales, de gestión y de diseño preliminar se han completado al **100 %**, quedando consolidados a lo largo de los capítulos técnicos, anexos y código fuente de esta memoria. Por su parte, el paquete **Producto software** materializa con éxito el flujo íntegro del sistema a través de las tres *releases* planificadas; no obstante, al haberse pospuesto dos de los doce objetivos funcionales contemplados en el diseño inicial, el grado de consecución de dichos objetivos se sitúa en el **83,33 %**.

Para auditar las desviaciones respecto al alcance inicial, el siguiente cuadro detalla los objetivos funcionales pospuestos y su justificación técnica. Estas decisiones y su repercusión cronológica en los hitos del proyecto quedan documentadas formalmente en el **registro de cambios**.

ID	Objetivo funcional	Motivo del aplazamiento
OBJ-09	Asistente IA	Evitar la dependencia de cuotas de facturación por consumo de <i>tokens</i> , límites de tasa (<i>rate limits</i>) y sobrecostes imprevistos de servicios externos, priorizando un núcleo completamente autosuficiente.
OBJ-11	Gestión de grupos	Elevado coste de oportunidad: el gran esfuerzo temporal necesario para su implementación no justificaba el valor añadido, decidiendo concentrar ese tiempo en consolidar la concurrencia y la estabilidad de las salas en vivo.

Tabla 9.1: Objetivos funcionales pospuestos.

Gracias a las relaciones fijadas en la **matriz de trazabilidad unificada**, es posible conocer de forma exhaustiva el alcance afectado por la postergación de ambos objetivos: el aplazamiento de los casos de uso **CU-03** y **CU-11**, y de las historias de usuario **HU-PR-09** y **HU-PR-07**. En consecuencia, quedan pospuestos cinco requisitos funcionales, catalogados como trabajo futuro en el repositorio del proyecto:

Objetivo	ID	Requisito funcional	GitHub Issue
OBJ-09	RF-34	Generación preguntas	#95
	RF-35	Asistente de ayuda	#96
	RF-36	Navegación y tutorial por IA	#97
	RF-37	Revisión post IA	#98
OBJ-11	RF-38	Crear clase	#99

Tabla 9.2: Requisitos funcionales pospuestos asociados a OBJ-09 y OBJ-11.

Asimismo, dentro de los objetivos completados se identificaron siete requisitos funcionales secundarios que fueron catalogados como mejoras o extensiones secundarias para salvaguardar la estabilidad del núcleo y ajustarse al calendario. En la siguiente tabla se detallan estos requisitos de ampliación y las *issues* abiertas para su integración en versiones sucesivas:

ID	Requisito funcional	GitHub Issue
RF-12	Importación	#100
RF-32	Listado en vivo	#101
RF-33	Estadísticas	#102
RF-42	Bonificación por tiempo	#103
RF-43	Sistema de rachas	#104
RF-44	Configuración de puntuación	#105
RF-46	Creación/edición avanzada	#106

Tabla 9.3: Requisitos funcionales pospuestos como puntos de extensión.

En términos cuantitativos, este ajuste de alcance deliberado se traduce en que 9 de los 11 casos de uso diseñados (**81,82 %**) y 16 de las 18 historias de usuario (**88,89 %**) se encuentran plenamente operativos en sus flujos de valor principal. En el nivel de especificación atómica, 35 de los 47 requisitos funcionales (**74,47 %**) han sido implementados y verificados con éxito, garantizando la totalidad de la operativa requerida en el aula presencial y dejando acotadas las extensiones como **trabajo futuro**.

Como balance global del alcance, a continuación se recoge la retrospectiva técnica completa de los doce objetivos funcionales contemplados en el proyecto, sintetizando tanto los retos técnicos resueltos en los objetivos operativos como la resolución final adoptada en los aplazados:

ID	Objetivo	Retrospectiva técnica e impacto en el desarrollo
OBJ-01	Módulo de usuario y autenticación	Implementación sólida de JWT y Bcrypt. Para evitar bloqueos en el hilo principal de la API por la latencia del servidor SMTP de Gmail, el envío del código de verificación por correo se delegó a tareas en segundo plano.
OBJ-02	Gestión de cuestionarios	Se optó por borrado lógico (<i>soft delete</i>) en cascada para impedir que la edición o eliminación de preguntas corrompiera los registros de salas históricas.
OBJ-03	Creación de salas	Se implementó una máquina de estados en el servidor para controlar las fases de la partida, garantizando un flujo sincronizado y previniendo saltos de estado indebidos.
OBJ-04	Acceso rápido	Entrada ágil e inmediata sin necesidad de registro previo. Para evitar que un refresco accidental del navegador expulsara al estudiante de la partida, la sesión activa se conservó localmente mediante <code>sessionStorage</code> .
OBJ-05	Interacción en tiempo real	La recepción de respuestas se gestionó en memoria garantizando que cada participante emitiera un único voto por pregunta. Además, se centralizó el temporizador en el servidor para anular intentos de manipulación o ventajas por desajustes horarios entre los clientes.
OBJ-06	Verificación QR	Generación distribuida en el cliente para liberar de procesamiento al servidor mediante la librería <code>qrcode.react</code> . La carga útil del código QR estructura un objeto JSON con los datos clave de la evaluación (UVUS, sala y puntuación), acompañado de un token criptográfico de un solo uso.
OBJ-07	Persistencia validada	Se logró el desacoplamiento entre el estado efímero de la partida y el almacenamiento persistente: solo los resultados verificados se consolidan en la base de datos.
OBJ-08	Estadísticas y análisis	Agregación reactiva en memoria al cierre de cada ronda para computar la distribución de respuestas, aciertos globales y el podio en tiempo real, evitando sobrecargar la base de datos con consultas recurrentes.
OBJ-09	Asistente IA	Pospuesto conforme a lo descrito en el CR-02 .
OBJ-10	Escaneo de código QR	Lectura ágil mediante <code>html5-qrcode</code> con solicitud de permisos de cámara, validando el <i>token</i> criptográfico contra el servidor en un solo paso atómico.
OBJ-11	Gestión de grupos	Pospuesto conforme a lo descrito en el CR-03 .
OBJ-12	Exportación de datos	Se habilitó la descarga de resultados en formato CSV, garantizando la compatibilidad con hojas de cálculo.

Tabla 9.4: Retrospectiva técnica global de los objetivos funcionales.

9.2. Tiempo real frente a Línea Base del Cronograma

Para auditar el desempeño temporal del proyecto frente a la **Línea Base del Cronograma**, se ha consolidado el registro continuo de dedicación temporal a través de Clockify, cuyo informe detallado queda recogido íntegramente en el **anexo de dedicación temporal**. Las horas se han computado a nivel de tareas operativas individuales, asignando en la herramienta una etiqueta (*tag*) que las vincula directamente con su actividad homóloga más próxima de la planificación inicial. Este mecanismo permite clasificar el esfuerzo según el rol profesional desempeñado, trazabilidad que se empleará más adelante en la **tabla de dedicación por roles** para calcular las desviaciones en los gastos de contratación.

[TODO: Actualizar informe cuando estén todas las horas computadas]

El siguiente cuadro resume la comparativa entre la estimación inicial y la dedicación real registrada por fase de trabajo:

[TODO: Rellenar las horas]

ID	Fase	Horas previstas	Horas reales
F-01	Planificación	20 h	11 h
F-02	Documentación	20 h	27,14 h
F-03	Inicio	11 h	12,50 h
F-04	CI/CD	17 h	19,87 h
F-05	MVP	65 h	55,40 h
F-06	Testing	15 h	42,74 h
F-07	Core	49 h	45,24 h
F-08	QA	13 h	23,73 h
F-09	Memoria	70 h	
F-10	Defensa	20 h	
Total		300 h	

Tabla 9.5: Comparativa de horas previstas y reales por fase del proyecto.

[TODO: Redactar retrospectiva general de desviaciones temporales una vez cerrado el cómputo final de horas en Clockify]

Este incremento en el esfuerzo dedicado a la robustez técnica y las pruebas del sistema impactó de forma directa en los plazos globales. Si bien la planificación inicial contemplaba finalizar el proyecto para la convocatoria ordinaria de junio, mantener dicha fecha habría obligado a comprometer la calidad del producto o recortar el rigor de la memoria técnica. Por ello, se optó formalmente por trasladar la finalización y defensa a la convocatoria extraordinaria de octubre, quedando esta modificación justificada y registrada en la sección de **gestión de cambios**.

A continuación, se muestra una comparativa entre la Línea Base del Cronograma y la ejecución real, representada mediante un diagrama de Gantt:

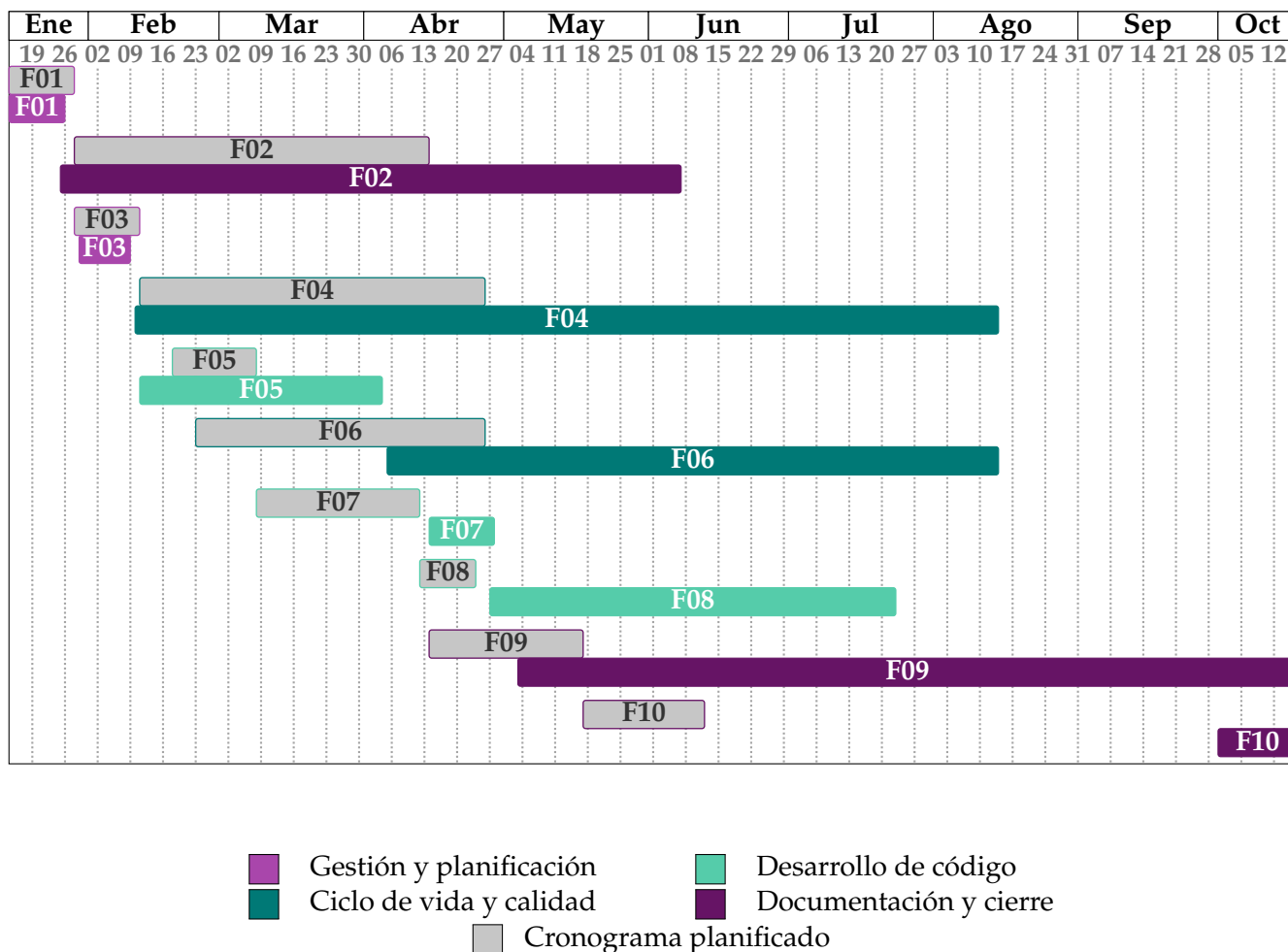


Figura 9.1: Diagrama de Gantt de seguimiento: cronograma planificado vs. real.

[TODO: Establecer fin de fase 9 y comienzo de fase 10]

9.3. Gastos incurridos frente a Línea Base de Costes

Para evaluar las diferencias entre la **Línea Base de Costes** y los gastos reales del proyecto, el análisis se centra exclusivamente en el CAPEX. El OPEX se mantiene idéntico a lo previsto, ya que los costes recurrentes de despliegue y consumo de servicios (incluida la API de IA para cuando entre en producción) se consideraron fijos para el proyecto y no han sufrido variaciones.

La variación del CAPEX se debe principalmente a dos factores derivados de las desviaciones temporales del proyecto: por un lado, el desvío en los costes laborales según el esfuerzo dedicado, y por otro, la extensión en el uso de las licencias de software. Al haberse trasladado el cierre del proyecto de junio a octubre, las suscripciones de GitHub Team y Microsoft Teams Essentials debieron mantenerse activas durante cuatro meses adicionales (pasando de 6 a 10 meses), mientras que la licencia comercial anual de Google AI Premium y el equipamiento no

experimentaron cambios.

En la siguiente tabla se recoge el balance consolidado del presupuesto de capital frente a los costes reales:

Concepto	Coste previsto	Coste real	Desviación
Costes laborales	11.552,65 €	[€]	[€]
Equipamiento informático	500,00 €	500,00 €	0,00 €
Licencia Google AI Premium (anual)	218,07 €	218,07 €	0,00 €
Licencia GitHub Team	21,82 € (6 m)	36,37 € (10 m)	14,55 €
Licencia Microsoft Teams Essentials	21,82 € (6 m)	36,37 € (10 m)	14,55 €
Reserva de contingencia	12.314,36 €	[€]	[€]
Total CAPEX	13.545,80 €	[€]	[€]

Tabla 9.6: Comparativa de Gastos de Capital (CAPEX) presupuestados vs. reales.

[TODO:Completar tabla]

Para comprender de dónde procede la desviación observada en la partida de costes laborales, es necesario descender al detalle del esfuerzo por rol profesional. La redistribución de horas entre tareas alteró la dedicación asignada a cada perfil. A continuación se desglosan las horas e importes correspondientes a partir de las tarifas por hora fijadas en la planificación inicial:

Rol	Precio/hora	Horas previstas	Horas reales	Coste previsto	Coste real
Project Manager	46,09 €	18 h	[h]	829,62 €	[€]
Analista	38,33 €	87 h	[h]	3.334,71 €	[€]
Diseñador UI	33,43 €	22 h	[h]	735,46 €	[€]
Programador Backend	23,85 €	79 h	[h]	1.884,15 €	[€]
Programador Frontend	23,85 €	47 h	[h]	1.120,95 €	[€]
Tester	23,68 €	15 h	[h]	355,20 €	[€]
Técnico	19,58 €	32 h	[h]	626,56 €	[€]
Total	–	300 h	[h]	8.886,65 €	[€]

Tabla 9.7: Comparativa de dedicación y costes de personal por rol profesional.

[TODO: Completar tabla y redactar pequeña retrospectiva final de desviaciones de los roles]

Como consecuencia de esta variación en el gasto de capital y al mantenerse inmutable el OPEX, el coste total de propiedad (TCO) del proyecto pasa de los 15.198,56 € estimados inicialmente a un valor definitivo de [€], cerrando la auditoría económica del desarrollo.

[TODO: Añadir tco final]

9.4. Desviaciones y gestión de cambios

A lo largo del ciclo de vida del proyecto surgieron contingencias técnicas, restricciones de tiempo y oportunidades de mejora que hicieron necesario activar mecanismos formales de control del cambio. Con el objetivo de proteger la calidad técnica del producto software y evitar desviaciones incontroladas de la planificación, cada alteración significativa respecto a la línea base se analizó y documentó mediante una ficha formal de solicitud de cambio (*Change Request*), evaluando su impacto directo sobre el alcance, el cronograma y los costes.

9.4.1. Registro de cambios

En este apartado se recoge el historial de las principales decisiones de cambio gestionadas durante el desarrollo, estructuradas como *Change Requests* (CR). Cada ficha documenta el estado de la solicitud, el contexto que la motivó, la decisión adoptada y las consecuencias derivadas de su resolución.

CR-01: Aplazamiento de la defensa a convocatoria de octubre	
Estado	Aprobado.
Contexto	El esfuerzo necesario para estabilizar las pruebas de integración en WebSockets y redactar la memoria con el rigor exigido impidió concluir en junio sin comprometer la calidad.
Decisión	Trasladar formalmente la entrega de la memoria y la defensa técnica de la convocatoria ordinaria de junio a la extraordinaria de octubre de 2026.
Impacto en alcance	Ninguno.
Impacto en tiempo	Ampliación de 4 meses en la ventana temporal global del proyecto.
Impacto en coste	Incremento de 29,10€ en licencias de desarrollo CAPEX por 4 meses adicionales de GitHub Team y Microsoft Teams Essentials.

Tabla 9.8: Registro de cambios - CR-01.

CR-02: Aplazamiento del asistente de generación con IA (OBJ-09)	
Estado	Aprobado.
Contexto	La latencia y los límites de consumo (<i>rate limits</i>) de la API de Gemini planteaban riesgos de bloqueo operativo en vivo, exigiendo un esfuerzo desproporcionado para asegurar un comportamiento determinista.
Decisión	Desacoplar la generación con IA del alcance actual, registrarla como línea de trabajo futuro en GitHub y priorizar la creación e importación manual de cuestionarios.
Impacto en alcance	Reducción del alcance funcional inmediato, preservando íntegro el flujo estándar de gestión de cuestionarios.
Impacto en tiempo	Liberación de horas de desarrollo <i>backend</i> y <i>frontend</i> , mitigando retrasos en el núcleo del sistema.
Impacto en coste	Derivado de la redistribución de horas entre roles profesionales.

Tabla 9.9: Registro de cambios - CR-02.

CR-03: Postergación de la creación de grupos de alumnos (OBJ-11)	
Estado	Aprobado.
Contexto	Se priorizó centrar el tiempo de desarrollo en la estabilidad de las partidas en tiempo real antes que en la gestión de grupos.
Decisión	Posponer la gestión de grupos de alumnos y dejarla registrada en GitHub como trabajo futuro.
Impacto en alcance	Se excluye la funcionalidad de grupos de alumnos en esta entrega para asegurar el funcionamiento del juego en vivo.
Impacto en tiempo	Se liberan horas de desarrollo para dedicarlas a la estabilidad y pruebas del sistema.
Impacto en coste	Derivado de la redistribución de horas entre roles profesionales.

Tabla 9.10: Registro de cambios - CR-03.

CR-04: Ampliación de la batería y estrategia de pruebas	
Estado	Aprobado.
Contexto	La estrategia de pruebas planificada inicialmente resultó insuficiente para asegurar la calidad global del sistema, haciendo necesario un enfoque de verificación más exhaustivo.
Decisión	Rediseñar y ampliar la estrategia de pruebas, incorporando una batería más completa de casos de prueba tanto a nivel unitario como de integración.
Impacto en alcance	Incremento sustancial en la cobertura de pruebas y en la robustez general de la aplicación.
Impacto en tiempo	Aumento notable de horas dedicadas a la fase de pruebas frente a las 15 horas presupuestadas originalmente.
Impacto en coste	Incremento en el coste imputado al rol de <i>Tester</i> dentro del balance contable del CAPEX.

Tabla 9.11: Registro de cambios - CR-04.

CR-05: Incorporación de Redis para la sincronización de la sala	
Estado	Desestimado.
Contexto	Se evaluó utilizar Redis para guardar el estado de las salas y permitir que varios servidores pudieran comunicarse entre sí ante un gran número de usuarios simultáneos.
Decisión	Desestimar el uso de Redis y mantener la gestión de las salas en la propia memoria del servidor actual.
Impacto en alcance	Ninguno; se mantiene una arquitectura más simple y directa, suficiente para las necesidades del proyecto.
Impacto en tiempo	Se evita un trabajo extra estimado entre 10 y 15 horas de desarrollo y configuración.
Impacto en coste	Ninguno; además, se evita el coste mensual extra de contratar un servicio adicional en la nube.

Tabla 9.12: Registro de cambios - CR-05.

CR-06: Despliegue de SonarQube y refuerzo de controles locales	
Estado	Aprobado.
Contexto	La activación en CI/CD del análisis con SonarQube se demoró; al procesar el código acumulado, el volumen de avisos obligó a atajar las incidencias antes de subirlas al repositorio.
Decisión	Habilitar formalmente el análisis continuo en GitHub Actions e implantar ganchos de <i>pre-commit</i> junto a formateadores locales para filtrar defectos de estilo y tipado en local.
Impacto en alcance	Incorporación de controles estáticos automatizados en el entorno local y en la integración continua.
Impacto en tiempo	Dedicación técnica imprevista en configurar las herramientas locales y solventar los avisos acumulados.
Impacto en coste	Ninguno en licencias (uso del plan gratuito de SonarQube); imputación adicional de horas al rol Técnico.

Tabla 9.13: Registro de cambios - CR-06.

CR-07: Reducción de deuda técnica en SonarQube y congelación del alcance	
Estado	Aprobado.
Contexto	La integración tardía del análisis de SonarQube generó un volumen acumulado de deuda técnica que exigió atención inmediata, poniendo en riesgo la fecha límite de entrega si se continuaba desarrollando nuevas funcionalidades.
Decisión	Congelar el desarrollo de nuevas características para priorizar la corrección de errores críticos en Sonar, posponiendo los requisitos de mejora no esenciales como tareas de trabajo futuro en GitHub, en consonancia con la retrospectiva del alcance.
Impacto en alcance	Se detiene la incorporación de requisitos de mejora secundarios, asegurando una base de código limpia y libre de fallos graves para el núcleo de la plataforma.
Impacto en tiempo	Permite cumplir con los plazos de entrega y dedicar el tiempo necesario a la redacción rigurosa de la memoria técnica.
Impacto en coste	Derivado de la redistribución de horas entre roles profesionales.

Tabla 9.14: Registro de cambios - CR-07.

Este conjunto de solicitudes de cambio pone de manifiesto que las desviaciones temporales y económicas registradas no fueron fruto de la improvisación, sino de decisiones conscientes de ingeniería orientadas a priorizar la estabilidad técnica, la verificación exhaustiva y la calidad del producto final frente a las restricciones temporales.

9.5. Resultados de las pruebas y validación

Una vez definida la estrategia de verificación en la fase de planificación y detallada la infraestructura técnica de ejecución en el capítulo de implementación, en este apartado se exponen exclusivamente los resultados obtenidos. El objetivo es contrastar las métricas reales de cobertura, robustez y rendimiento frente a los umbrales de aceptación fijados como criterio de calidad para el proyecto. Para facilitar la trazabilidad completa, el material probatorio complementario (reportes interactivos, registros íntegros y capturas) se encuentra recopilado en el anexo de [registro de pruebas de cobertura](#).

9.5.1. Pruebas unitarias y de integración del backend

La suite del *backend* cubre tanto la lógica de negocio aislada (modelos, servicios y esquemas) como la integración de los puntos de entrada de la API contra la base de datos y el motor de autenticación.

```
tests\backend\test_integration_content.py .... [ 5%]
tests\backend\test_integration_stage.py ..... [ 21%]
tests\backend\test_integration_users.py .... [ 26%]
tests\backend\test_unit_content.py ..... [ 45%]
tests\backend\test_unit_stage.py ..... [ 76%]
tests\backend\test_unit_users.py ..... [100%]
```

Name	Stmts	Miss	Cover

backend\routers\content.py	188	0	100 %
backend\routers\stage.py	454	9	98 %
backend\routers\users.py	211	0	100 %
Otros (auth, database, models, schemas...)	398	23	94 %

TOTAL	1251	32	97 %

```
===== 71 passed in 13.50s =====
```

Evidencia 9.1: Resumen de ejecución y cobertura en backend.

La batería completó la ejecución de 71 pruebas sin registrar ningún fallo en un tiempo de 13,50 segundos. A nivel de cobertura:

- Se alcanzó una cobertura agregada de sentencias del **97 %**, superando ampliamente el umbral mínimo del 80 % fijado en los [umbrales de cobertura](#).
- Los controladores de contenidos (content.py) y de usuarios (users.py) alcanzaron una cobertura plena del 100 %.

- La máquina de estados de las salas (`stage.py`) registró un **98 %**, garantizando la solidez de las dinámicas en tiempo real frente a condiciones imprevistas.

Estos resultados quedan respaldados en el anexo mediante la captura de la **salida de terminal de las pruebas de pytest** y el informe de **cobertura del backend** generado en HTML con el desglose detallado por archivos.

9.5.2. Pruebas de componentes del frontend

En la interfaz visual, las pruebas de componentes permiten verificar el renderizado, el ciclo de vida, la gestión de estados reactivos y la emisión de eventos de forma aislada.

Test Files **25 passed** (25)
Tests **193 passed** (193)

File	% Stmts	% Branch	% Funcs	% Lines
All files	90.13	80.29	88.56	92.68
auth	93.37	84.93	95.16	94.17
management	87.82	76.37	86.36	90.78
room-access	94.91	75.8	94.73	97.36
room-play	89.66	82.47	86.9	92.83

Evidencia 9.2: Resumen de pruebas y cobertura de componentes.

La ejecución completó los 193 casos de prueba a lo largo de los 25 ficheros evaluados. El análisis de las métricas arroja las siguientes conclusiones sobre la robustez visual:

- El umbral fijado del 80 % se superó de forma generalizada, alcanzando un **90,13 %** en sentencias y consolidando un **92,68 %** sobre el total de líneas de código fuente.
- La cobertura de ramas se situó en el **80,29 %**, certificando el correcto tratamiento de los flujos alternativos del cliente, tales como errores de red, validaciones de entrada o pantallas intermedias de espera.
- Los dominios de acceso e incorporación (`auth` y `room-access`) registraron los valores más altos (superando el 94 % de cobertura), blindando los puntos iniciales de entrada de la experiencia de usuario.

El detalle completo de esta batería se puede contrastar en el anexo consultando la **salida por consola de Vitest** junto con la perspectiva del **reporte HTML de cobertura de componentes**.

9.5.3. Pruebas de extremo a extremo (E2E)

Las pruebas de extremo a extremo verifican los flujos completos de usuario sobre instancias reales de navegador, asegurando la correcta integración entre la interfaz, los servicios de la API y los canales de comunicación bidireccional.

Test Files **10 passed** (10)
Tests **112 passed** (112)

Name	% Bytes	% Stms	% Branch	% Funcs	% Lines
auth	95.44	95.53	91.77	85.25	94.79
management	93.58	88.37	80.18	92.35	92.37
room-access	97.80	90.00	83.05	100.00	95.22
room-play	92.04	91.88	84.10	92.11	90.65
Summary	93.81	91.30	83.92	91.41	92.58

Evidencia 9.3: Resumen de pruebas y cobertura *End-to-End*.

La ejecución completó todos los escenarios sin registrar fallos sobre instancias reales de navegador. Más allá del volumen de pruebas aprobadas, la relevancia de este nivel reside en verificar la interoperabilidad entre la interfaz, los servicios de la API y los canales de comunicación en tiempo real. En este sentido, la cobertura global del 91,30 % en sentencias y del 83,92 % en *branches* valida la consistencia de los flujos críticos de la plataforma, desde la persistencia de sesión en auth hasta la sincronización reactiva durante la partida en room-play.

Destaca el comportamiento del módulo room-access, que alcanzó el 100 % en cobertura de funciones; esto garantiza que todos los métodos asociados a la validación de salas, control de estados y admisión de participantes se ejercitan tal y como lo experimenta el usuario final. La traza detallada de todos los recorridos y el análisis visual de cobertura se recogen en el anexo mediante el **reporte de ejecución de Playwright** y el **informe de cobertura de Monocart**.

9.5.4. Pruebas de carga y rendimiento

Las pruebas de rendimiento evalúan el comportamiento y los tiempos de respuesta del sistema bajo condiciones de concurrencia simulada. La batería se ejecutó de manera automatizada en modo desatendido, procesando las series temporales exportadas para contrastar las métricas obtenidas con los criterios de aceptación fijados.

=== INFORME DE EVALUACION DE UMBRALES DE RENDIMIENTO (P90) ===			
Metrica / Endpoint	Valor P90	Umbral	Estado
GET /content/quizzes/	20 ms	<= 500 ms	[OK]
POST /content/quizzes/	38 ms	<= 500 ms	[OK]
GET /content/quizzes/{id}	17 ms	<= 200 ms	[OK]
PUT /content/quizzes/{id}	22 ms	<= 200 ms	[OK]
POST /stage/participants	38 ms	<= 200 ms	[OK]
GET /stage/rooms	69 ms	<= 200 ms	[OK]
POST /stage/rooms/	84 ms	<= 200 ms	[OK]
GET /stage/rooms/verify/{join_code}	49 ms	<= 200 ms	[OK]
GET /stage/rooms/{id}	35 ms	<= 200 ms	[OK]
GET /stage/rooms/{id}/participants	12 ms	<= 200 ms	[OK]
POST /users/login	240 ms	<= 500 ms	[OK]
GET /users/me	12 ms	<= 200 ms	[OK]
WebSocket Connect	28 ms	<= 200 ms	[OK]
Tasa de Errores (Global)	0.00 %	< 1.0 %	[OK]
RESULTADO: TODAS LAS PRUEBAS DE RENDIMIENTO SUPERARON LOS UMBRALES [OK]			

Evidencia 9.4: Informe de evaluación de umbrales de latencia P_{90} y tasa de error.

A lo largo de la simulación no se registró ningún fallo en las peticiones ni interrupciones en los canales de comunicación, consolidando una tasa de error del 0,00 % sobre el total de operaciones. En términos de latencia, todos los puntos de entrada cumplieron holgadamente con sus umbrales de aceptación: los endpoints de consulta y sincronización mantuvieron tiempos de respuesta P_{90} inferiores a los 50 ms, e incluso operaciones de mayor coste computacional como la autenticación (/users/login, con 240 ms) se situaron muy por debajo del límite estricto de 500 ms fijado en los **umbrales de las pruebas de rendimiento**. Destaca el comportamiento del canal bidireccional (*WebSocket Connect*), cuyo P_{90} se consolidó en apenas 28 ms bajo ráfagas concurrentes de conexión.

La trazabilidad completa del procedimiento se respalda en el anexo: la ejecución de la batería queda reflejada en la **captura de terminal de las pruebas de carga**, cuyos resultados brutos se recopilaban en los registros CSV correspondientes y fueron evaluados posteriormente mediante la **ejecución del script de verificación de umbrales** de rendimiento.

9.5.5. Resumen de resultados de las pruebas

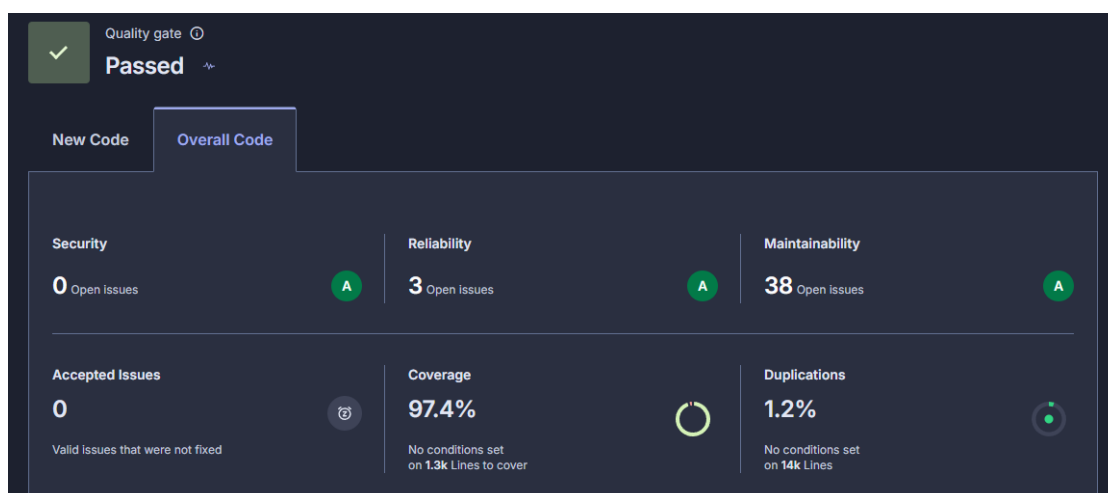
A modo de síntesis, la siguiente tabla resume los resultados obtenidos en las distintas *suites* de verificación frente a los umbrales de aceptación definidos en el plan de pruebas:

<i>Suite</i>	Métrica / Objetivo	Umbral	Resultado	Estado
Unitarias e integración	Cobertura global	$\geq 80 \%$	97,4 %	✓
	routers/content.py	$\geq 90 \%$	100 %	✓
	routers/stage.py	$\geq 90 \%$	98 %	✓
	routers/users.py	$\geq 90 \%$	100 %	✓
Componentes	Líneas	$\geq 80 \%$	92,68 %	✓
	Funciones	$\geq 80 \%$	88,56 %	✓
	Sentencias	$\geq 80 \%$	90,13 %	✓
	Ramas	$\geq 75 \%$	80,29 %	✓
Extremo a extremo (E2E)	Líneas	$\geq 80 \%$	92,58 %	✓
	Funciones	$\geq 80 \%$	91,41 %	✓
	Sentencias	$\geq 80 \%$	91,30 %	✓
	Ramas	$\geq 75 \%$	83,92 %	✓
Carga y rendimiento	Tasa global de errores	$< 1,0 \%$	0,00 %	✓
	Rutas <i>Auth</i>	$P90 \leq 500 \text{ ms}$	Media: 99 ms	✓
	Rutas generales	$P90 \leq 200 \text{ ms}$	Media: 38 ms	✓
	<i>WebSocket</i>	$P90 \leq 200 \text{ ms}$	28 ms	✓

Tabla 9.15: Resumen del cumplimiento de umbrales en las baterías de pruebas.

9.6. Análisis estático de código (SonarQube)

En paralelo a las pruebas funcionales y de rendimiento analizadas en los apartados anteriores, la salud estructural y la mantenibilidad del código se monitorizan de manera continua a través de análisis estático en la rama main. Con el fin de contrastar las métricas consolidadas frente a los umbrales definidos en las **métricas de calidad**, el escaneo en SonarQube certifica el cumplimiento íntegro de los criterios fijados, alcanzando el estado **Superado** (*Passed*) en su *Quality Gate*:



Evidencia 9.5: Estado del *Quality Gate* y métricas consolidadas en SonarQube.

La siguiente tabla compara los valores obtenidos sobre el código completo (*Overall Code*) con los **umbrales y métricas objetivo** de referencia:

Métrica	Objetivo	Valor obtenido	Estado
Cobertura	> 80 %	97,4 %	✓
Duplicación	< 3 %	1,2 %	✓
Seguridad	Calificación A	A	✓
Fiabilidad	Calificación A	A (3 issues)	✓
Mantenibilidad	Calificación A	A (38 code smells)	✓

Tabla 9.16: Comparativa de los umbrales del *Quality Gate* frente a resultados obtenidos.

A continuación, se presenta la evolución temporal del volumen de incidencias y de la calidad del código a lo largo de las distintas sesiones de trabajo, acompañada del desglose cuantitativo detallado por nivel de severidad y área técnica:

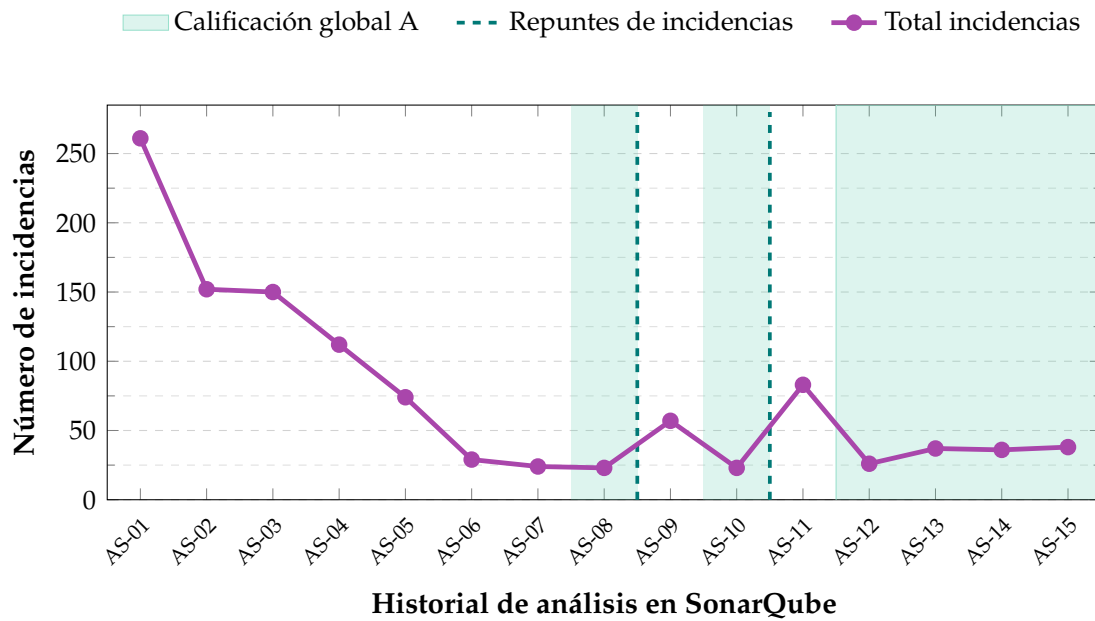


Figura 9.2: Evolución del volumen de incidencias en SonarQube.

Análisis	Issues	Severidad	Categoría	Calificación
		(Alta / Media / Baja)	(Seg / Fiab / Mant)	(Seg / Fiab / Mant)
AS-01	261	45 / 130 / 86	20 / 26 / 215	C C C
AS-02	152	24 / 74 / 54	27 / 48 / 109	C C B
AS-03	150	23 / 82 / 71	25 / 48 / 108	C C B
AS-04	112	17 / 65 / 43	25 / 24 / 79	C B B
AS-05	74	3 / 24 / 40	2 / 24 / 64	B B A
AS-06	29	3 / 13 / 14	1 / 2 / 28	B B A
AS-07	24	2 / 10 / 12	1 / 0 / 23	B A A
AS-08	23	1 / 9 / 13	0 / 0 / 23	A A A
AS-09	57	22 / 23 / 13	3 / 7 / 48	A A C
AS-10	23	5 / 7 / 11	0 / 0 / 23	A A A
AS-11	83	4 / 70 / 9	4 / 52 / 27	A C A
AS-12	26	3 / 14 / 9	0 / 0 / 26	A A A
AS-13	37	5 / 22 / 10	0 / 1 / 37	A A A
AS-14	36	4 / 22 / 10	0 / 1 / 35	A A A
AS-15	38	4 / 24 / 12	0 / 3 / 38	A A A

Tabla 9.17: Desglose de historial de *issues* registradas en SonarQube.

El análisis de estos datos refleja con claridad el ciclo de vida del proyecto y las decisiones técnicas adoptadas:

- **Adopción y saneamiento inicial (AS-01 a AS-08):** La integración del escaneo estático en la canalización de integración continua se produjo de forma tardía, una vez concluidas las fases **F-05 MVP** y **F-06 Core**. Esta demora provocó que el primer análisis registrado (**AS-01**) arrojase una deuda técnica de partida muy elevada, con un volumen inicial de 261 incidencias. Ante este escenario crítico, se adoptaron dos decisiones de gestión inmediatas:
 - Mediante la solicitud **CR-06**, se habilitó formalmente el análisis continuo en GitHub Actions y se implantaron *pre-commit hooks* junto a formateadores locales para atajar y filtrar defectos de estilo y tipado antes de subirlos al repositorio.
 - Mediante la solicitud **CR-07**, se congeló temporalmente el desarrollo de nuevas características para priorizar la corrección de errores en SonarQube, posponiendo los requisitos de mejora secundarios al *backlog* de **trabajo futuro**.

La dedicación técnica a este proceso intensivo de saneamiento permitió reducir drásticamente las *issues* de 261 a tan solo 23 en el **AS-08**, alcanzando por primera vez la máxima calificación global.

- **Primer repunte: Desarrollo del módulo de usuarios (AS-09 y AS-10):** Tras alcanzar por primera vez la máxima calificación global y superar los umbrales exigidos por SonarQube, la reanudación del desarrollo para completar las funcionalidades finales del módulo de autenticación y gestión de usuarios introdujo nuevo código que originó un repunte transitorio a 57 incidencias (**AS-09**). Al contar ya con el analizador estático plenamente integrado en el flujo de trabajo, estas desviaciones se atajaron con rapidez en la siguiente iteración, retornando de inmediato a la calificación **A A A** con 23 incidencias en **AS-10**.
- **Segundo repunte: Reglas de accesibilidad en la interfaz (AS-11 y AS-12):** El repunte registrado en **AS-11**, donde las incidencias subieron a 83 (con 52 defectos de fiabilidad que dejaron temporalmente la calificación en **A C A**), se debió a que, al seguir creciendo el proyecto y abarcar más vistas del frontend, SonarQube empezó a evaluar en profundidad las plantillas de React. En ese análisis comenzó a exigir de forma estricta que todas las etiquetas `<button>` incluyesen su atributo explícito, ya que por defecto HTML les asigna `type="submit"` y puede provocar comportamientos no deseados en la interfaz. Al no haberse contemplado este aviso antes, el fallo saltó en cascada en prácticamente todos los botones de la aplicación. Bastó con añadir `type="button"` a los elementos de navegación y acción para eliminar por completo esas 52 incidencias de fiabilidad, recuperando de inmediato la calificación **A A A** con 26 incidencias en **AS-12**.
- **Consolidación en la versión final y deuda residual (AS-13 a AS-15):** Con las buenas prácticas y los controles locales ya interiorizados durante la implementación de los módulos de contenido y sesiones de la etapa **F-08 QA**, el proyecto concluyó sin nuevos repuntes. La ligera variación observada entre **AS-12** y **AS-15** (pasando de 26 a 38 incidencias en total) responde únicamente

a pequeños *code smells* y ajustes menores de maquetación o tipado que no comprometen la lógica de la plataforma. De este modo, la base de código se mantuvo estable en la máxima calificación hasta el cierre definitivo del desarrollo.

Al cierre del desarrollo, la deuda técnica residual computada por SonarQube es de tan solo **3 horas y 46 minutos**. Con la máxima calificación afianzada y sin alertas de seguridad, se decidió no invertir más horas de desarrollo en estas incidencias menores para destinarlas a otros aspectos del proyecto. Así, su saneamiento queda pospuesto como línea de **trabajo futuro**. Por su parte, la valoración personal sobre la experiencia y el impacto de integrar Sonar se aborda en detalle en las **lecciones aprendidas**.

10. Conclusiones

Este capítulo contrasta los resultados alcanzados frente a las metas planteadas en el proyecto, evalúa el cumplimiento de cada objetivo y resume las lecciones aprendidas durante el desarrollo de *Quizzie*.

10.1. Grado de cumplimiento de los objetivos

A continuación se evalúa el nivel de consecución de las metas definidas en la sección de **objetivos del proyecto**, divididas en los tres ámbitos de trabajo.

10.1.1. Objetivos académicos

El bloque formativo y metodológico priorizó la calidad y verificación del código mediante pruebas rigurosas, asumiendo una replanificación del calendario hacia la tercera convocatoria para garantizar la madurez del sistema.

ID	Nombre	Meta fijada	Resultado	Retrospectiva
OA-01	Aplicación de competencias adquiridas	Cobertura global $\geq 80\%$.	✓ 97,4 % <i>backend</i> ✓ +90 % <i>frontend</i> .	Se consolidaron los conocimientos del grado logrando una suite de pruebas robusta y ampliamente superior al mínimo exigido.
OA-02	Metodologías ágiles	Desviación temporal $< 15\%$ en hitos.	✗ +50 % de desviación temporal.	El aplazamiento de la entrega de la primera a la tercera convocatoria alteró los plazos previstos, completándose a tiempo menos del 50 % de los hitos fijados.
OA-03	Buenas prácticas e ingeniería	100 % commits semánticos y reglas de estilo.	✓ 100 % commits semánticos ~ Fallos puntuales de lindeo y formato.	Aunque la política de commits semánticos se cumplió en todo el historial, surgieron fallos intermitentes de formateo y linting que requirieron correcciones manuales posteriores.

Tabla 10.1: Evaluación del cumplimiento de objetivos académicos.

10.1.2. Objetivos técnicos y profesionales

La arquitectura desacoplada y la justificación formal mediante registros de decisiones (ADR) guiaron la ingeniería del sistema, concentrándose las limitaciones en la incorporación tardía del flujo de automatización y la ausencia de auditorías de seguridad externas.

ID	Nombre	Meta fijada	Resultado	Retrospectiva
OT-01	Dominio tecnológico	Separación <i>backend</i> asíncrono y <i>frontend</i> reactivo.	✓ Arquitectura cliente-servidor desacoplada.	La separación entre el servidor asíncrono y la interfaz cliente facilitó el desarrollo, mantenimiento y pruebas independientes de cada subsistema.
OT-02	Ciclo de vida completo	Trazabilidad formal de decisiones.	✓ Fases del ciclo cubiertas y decisiones documentadas con ADRs.	La documentación estructurada y secuencial de las decisiones arquitectónicas justificó técnicamente la evolución del diseño del sistema.
OT-03	Seguridad y prevención de fraude	0 % fraude por suplantación remota en salas.	✓ Mitigación efectiva en pruebas controladas. ~ Sin auditoría de seguridad externa.	El esquema de tokens y códigos QR impidió la suplantación en los escenarios de prueba evaluados, si bien queda pendiente contrastar su resistencia en auditorías externas.
OT-04	Estrategia de pruebas y calidad	Cobertura global $\geq 80\%$ y control de calidad.	✓ Cobertura superada ✓ Análisis estático integrado.	Se superó el umbral fijado en los componentes críticos, evitando la acumulación de deuda técnica severa mediante la verificación continua de calidad.
OT-05	Automatización CI/CD	100 % de cambios se validan en CI.	✓ Flujo automatizado de CI/CD activo. ~ Configuración tardía del <i>pipeline</i> .	Aunque la canalización automatizada se configuró en una etapa avanzada del desarrollo, garantizó que solo versiones estables alcanzaran el entorno productivo.

Tabla 10.2: Evaluación del cumplimiento de objetivos técnicos y profesionales.

10.1.3. Objetivos de la plataforma

El producto resolvió eficazmente el acceso inmediato sin credenciales y la agilidad operativa en el aula, postergando la gestión avanzada de grupos a líneas futuras y requiriendo validación en despliegues con alumnado real.

ID	Nombre	Meta fijada	Resultado	Retrospectiva
OP-01	Evaluación docente simplificada	Puesta en marcha < 5 min y recogida automatizada.	✓ Configuración ágil y recolección automática.	La centralización de la gestión de cuestionarios y respuestas redujo significativamente la carga operativa previa a la sesión.
OP-02	Dinamismo e interactividad	Errores < 1,0 % en concurrencia (≥ 100 usuarios).	✓ Tolerancia a carga concurrente verificada. ~ Pendiente de prueba real en aula.	La comunicación bidireccional mantuvo la fluidez bajo pruebas simuladas, validar su comportamiento en un entorno real queda como trabajo futuro.
OP-03	Seguimiento continuo	Estadísticas en tiempo real e histórico por grupos.	✓ Métricas inmediatas y registro histórico. ✗ Gestión de grupos fuera del alcance de la entrega.	Se consolidó la analítica detallada y el volcado por sesión individual, posponiendo la gestión de grupos a una fase posterior.
OP-04	Validación presencial física	Verificación y firma por cámara < 2 s.	✓ Validación ágil <i>in situ</i> .	El flujo de escaneo y comprobación instantánea evitó cuellos de botella durante la validación QR.
OP-05	Accesibilidad e incorporación	Incorporación sin registro previo en < 5 s.	✓ Acceso directo sin registro previo.	Suprimir la necesidad de credenciales para los estudiantes permitió iniciar las actividades de inmediato sin fricción de acceso.
OP-06	Compatibilidad multidispositivo	Diseño responsivo sin pérdida funcional.	✓ Verificada compatibilidad multidispositivo. ~ Falta consistencia en navegadores menos habituales.	La interfaz responde de forma ágil y homogénea en los principales motores, aunque se detectaron ligeras disparidades visuales en navegadores minoritarios.
OP-07	Privacidad y cumplimiento normativo	Minimización de datos (RGPD) y credenciales seguras.	✓ Minimización aplicada y claves protegidas.	Se garantizó la privacidad prescindiendo de datos personales innecesarios y aplicando mecanismos seguros de autenticación docente.

Tabla 10.3: Evaluación del cumplimiento de los objetivos de la plataforma.

10.2. Lecciones aprendidas

El desarrollo del proyecto ha constituido un ejercicio práctico de ingeniería del que se extraen conclusiones clave sobre gestión, metodología y diseño técnico:

- **Gestión de tiempos y estimación del esfuerzo:** La planificación inicial resultó deficiente debido a la dificultad de compaginar el desarrollo con la carga lectiva de asignaturas pendientes, sumada a una clara subestimación del tiempo necesario para tareas críticas no ligadas a la codificación directa, como la redacción de la memoria, la elaboración de la documentación técnica y el diseño de las pruebas. Esta concatenación de factores forzó el traslado de la entrega a la tercera convocatoria, dejando como aprendizaje clave la necesidad de presupuestar márgenes realistas para las fases de documentación y validación desde el inicio.
- **Integración temprana de testing y automatización (CI/CD):** Aunque la *suite* de pruebas y la canalización de integración continua se articularon una vez consolidadas las fases MVP y *core* del sistema, postergar su adopción hasta ese punto evidenció los riesgos del desarrollo desasistido. Automatizar las pruebas junto al análisis en SonarQube mediante GitHub Actions debió priorizarse desde el inicio. Carecer de esta validación temprana provocó una deuda técnica y desajustes de código que exigieron refactorizaciones posteriores evitables.
- **Metodologías ágiles y trazabilidad documental:** El seguimiento estructurado mediante GitHub Projects, la gestión de *issues* con etiquetas estandarizadas y el registro formal de decisiones de cambio (*ADRs* y *CRs*) demostraron ser indispensables. Este marco organizativo permitió absorber las replanificaciones de manera ordenada, garantizando visibilidad total sobre el estado del desarrollo y evitando la pérdida de contexto entre iteraciones.
- **Madurez técnica y decisiones de diseño:** En el plano del desarrollo, la modularización del estado mediante contextos estructurados en el *frontend* previno el acoplamiento excesivo entre componentes. Asimismo, delegar la generación y lectura de códigos QR en librerías consolidadas y mantenidas por la comunidad resultó una decisión acertada, permitiendo centrar el esfuerzo en la lógica de negocio y la validación presencial sin reinventar mecanismos resueltos por la industria.

En retrospectiva, el proyecto alcanza un equilibrio sólido entre la ambición inicial y la viabilidad técnica real. Aunque la planificación temporal sufrió desviaciones que obligaron a postergar requisitos secundarios, la decisión de priorizar la robustez del núcleo en tiempo real, la seguridad en la gestión de salas y el desacoplamiento arquitectónico ha resultado acertada. *Quizzie* no concluye como una prueba de concepto frágil, sino como una base de *software* mantenible, trazable y con un nivel de calidad verificado.

10.3. Trabajos futuros y líneas de evolución

A partir de la base funcional y arquitectónica consolidada en *Quizzie*, y tomando en consideración las limitaciones identificadas a lo largo del desarrollo, se establecen las siguientes líneas de trabajo futuro:

- **Implementación de requisitos funcionales pospuestos:** Abordar el desarrollo de las funcionalidades relegadas al *backlog* del proyecto para la siguiente etapa, destacando la gestión avanzada de grupos de alumnos, y la generación automatizada de cuestionarios asistida por inteligencia artificial.
- **Reducción de deuda técnica y limpieza de código:** Continuar el proceso de saneamiento técnico a partir de las métricas y reportes generados por SonarQube, resolviendo los *code smells* pendientes, optimizando la complejidad ciclomática de determinados componentes y asegurando que las futuras modificaciones no reduzcan la cobertura alcanzada.
- **Optimización de diseño responsivo y compatibilidad multiplataforma:** Perfeccionar la experiencia de usuario y la adaptabilidad de la interfaz en una gama más amplia de resoluciones, factores de forma y pantallas móviles, además de verificar y corregir discrepancias visuales o de rendimiento en diversos navegadores web.
- **Mejoras en la infraestructura de despliegue en producción:** Mitigar el impacto de los tiempos de arranque en frío derivados del plan gratuito de Render (el cual hiberna la instancia de backend tras periodos de inactividad), evaluando la transición a un plan de computación persistente o implementando mecanismos de precalentamiento (*keep-alive*).
- **Auditoría externa de seguridad:** Someter la plataforma a revisiones y pruebas de penetración (*pentesting*) externas orientadas a detectar vectores de ataque en la gestión de tokens JWT, la exposición de variables de entorno y los mecanismos de validación presencial mediante códigos QR.
- **Integración con plataformas docentes e institucionales:** Diseñar conectores basados en estándares de interoperabilidad para integrar *Quizzie* en plataformas de gestión del aprendizaje, con especial foco en la plataforma de Enseñanza Virtual de la Universidad de Sevilla, permitiendo la sincronización de listas de alumnos y el volcado automático de calificaciones.
- **Validación y pruebas reales con alumnado:** Llevar a cabo experiencias piloto en aulas físicas reales con grupos de estudiantes y docentes, con el fin de evaluar la usabilidad, la latencia de las salas en tiempo real bajo concurrencia y la aceptación pedagógica de la herramienta.

A. Prototipos de interfaz de usuario

En este anexo se recopilan los prototipos de interfaz de usuario (*mockups*) correspondientes a las principales pantallas del sistema, organizados por flujos funcionales de navegación.

Para facilitar su interpretación visual y distinguir los roles de usuario en las capturas, se ha empleado la siguiente codificación de colores en el borde de las interfaces:

- **Borde magenta:** Identifica las pantallas pertenecientes a la vista del perfil docente (profesor).
- **Borde cian:** Identifica las pantallas pertenecientes a la vista del perfil estudiante (alumno).

A continuación, se presentan las distintas pantallas agrupadas según la secuencia lógica de cada flujo.

Flujo de registro y autenticación

Engloba las pantallas destinadas a la gestión de acceso de los profesores a la plataforma. Comprende el proceso de alta de nuevos usuarios, la verificación de credenciales a través del correo institucional con código de confirmación y el formulario de inicio de sesión estándar.

Iniciar Sesión
Bienvenido de nuevo, Scholar.

Email o Usuario
ejemplo@correo.com

Contraseña
•••••••

[¿Olvidaste tu contraseña?](#)

Entrar al Aula →

[¿No tienes una cuenta? Registrarse](#)

Figura A.1: Iniciar sesión.

Crear Cuenta

NOMBRE COMPLETO

 Ej. Juan Pérez

EMAIL

 usuario@ejemplo.com

CONTRASEÑA



CONFIRMAR CONTRASEÑA



Crear Cuenta →

¿Ya tienes una cuenta? [Iniciar Sesión](#)

Figura A.2: Crear cuenta.



Verifica tu cuenta

Hemos enviado un código de 6 dígitos a tu correo electrónico. Ingresalo para continuar.

Verificar Registro

¿No recibiste el código? [Reenviar](#)

Figura A.3: Verificar cuenta.

Flujo de creación de cuestionarios y gestión de salas

Comprende el conjunto de interfaces utilizadas por profesor para la preparación de las clases. Incluye creación de cuestionarios (preguntas, opciones y tiempos límite), la consulta del catálogo de cuestionarios creados, la pantalla de configuración de parámetros de la sala y el estado inicial de la sala (en espera).

The screenshot shows the 'Crear cuestionario' interface. At the top, there is a form with two fields: 'Título del Cuestionario' (Quiz Title) with the example 'Ej: Conocimientos Generales' and 'Descripción' (Description) with the placeholder 'Describe brevemente este cuestionario...'. To the right of these fields is a purple button labeled 'Crear Cuestionario'. Below the form is a preview of the quiz question. It is titled 'Pregunta 1' and has a value of '10 Puntos'. The question text is 'Escribe tu pregunta aquí...'. Below the question text are four multiple-choice options, each with a radio button: 'A Opción 1', 'B Opción 2', 'C Opción 3', and 'D Opción 4'. Navigation arrows are visible on the sides of the preview area.

Figura A.4: Crear cuestionario.

The screenshot shows the 'Mis Cuestionarios' interface. At the top, there is a header 'Mis Cuestionarios' with the subtitle 'Gestiona y juega tus cuestionarios creados.' and three tabs: 'Todos' (selected), 'Recientes', and 'Borradores'. Below the header is a list of three quizzes, each with a category icon, a title, a description, and a 'Play' button. The first quiz is 'Fundamentos de Programacion' (15 Preguntas, Hace 2 dias). The second is 'Geografia Mundial' (20 Preguntas, Ayer). The third is 'Química Orgánica II' (12 Preguntas, 1 semana). At the bottom, there is a dashed box with a plus icon and the text 'Crear un nuevo cuestionario' (Create a new quiz), followed by the subtitle 'Comparte tus conocimientos con el mundo creando un set de preguntas personalizado.'

Figura A.5: Listado de cuestionarios.

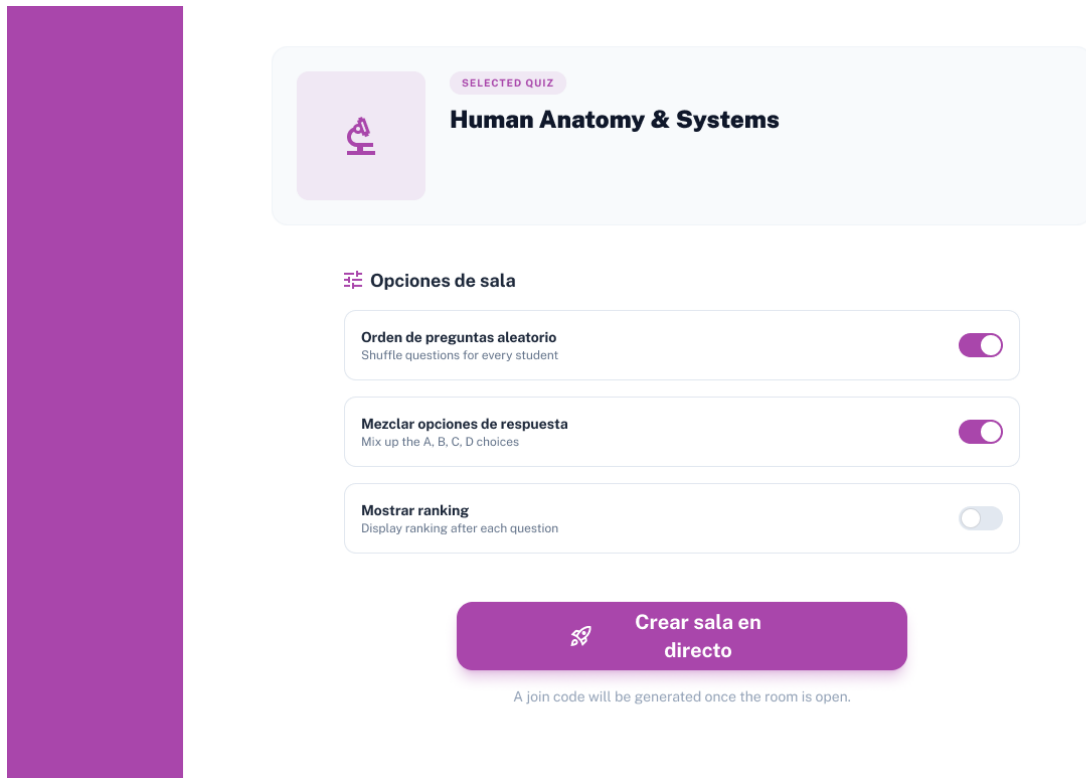


Figura A.6: Configuración de la sala.

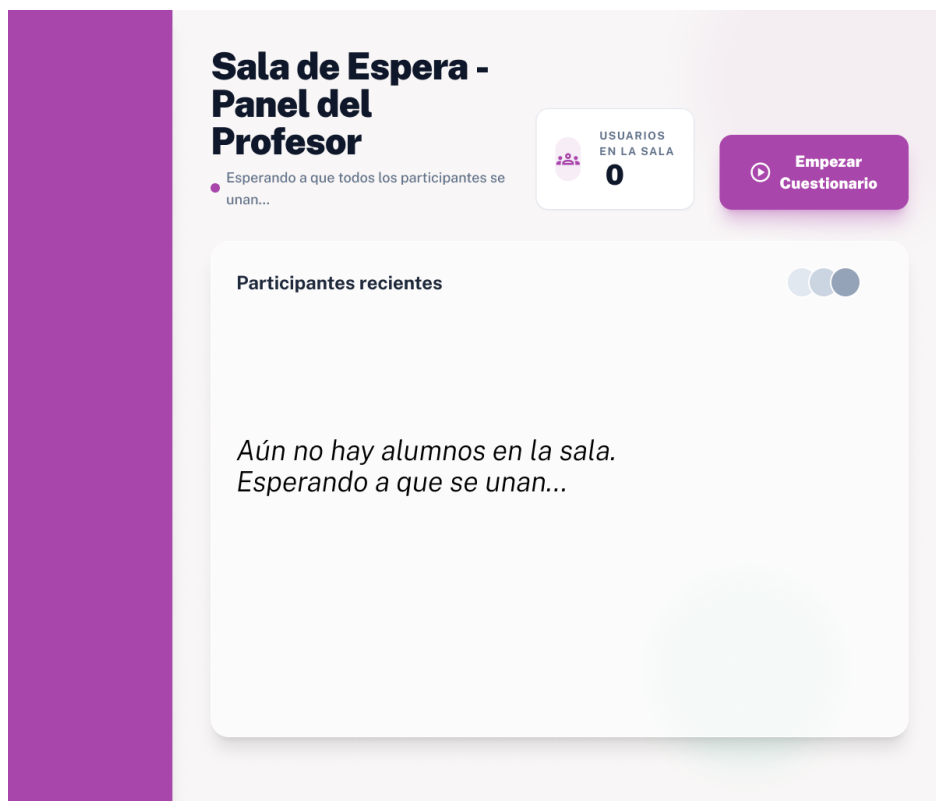


Figura A.7: Sala de espera de la sala recién creada.

Flujo de incorporación del estudiante a la sala

Muestra la secuencia que debe seguir un alumno para unirse a una sesión. Inicia con la introducción del código de sala, y continúa con la asignación de su identificador (UVUS o nickname), con su posterior aviso en caso de que el identificador no esté registrado. Por último, muestra la sala de espera hasta el inicio de la partida.

¿Listo para el desafío?

Ingresa el código de acceso para comenzar tu aventura de aprendizaje.

PIN DEL JUEGO

4 2

Entrar >

⚡ RÁPIDO ☆ DIVERTIDO 🎓 EDUCATIVO

"La educación es el arma más poderosa para cambiar el mundo"

Figura A.8: Ingresar código de sala.

¡Casi listo!

¡Elige tu nombre de batalla para entrar en la sala y comenzar el desafío!

Tu Nickname

@ Escribe tu nombre aquí...

¡Listo para jugar! 🚀

Al unirte, aceptas las reglas de convivencia del QuizMaster.

SALA #8234 12 JUGADORES ONLINE

Figura A.9: Ingresar UVUS o nickname.

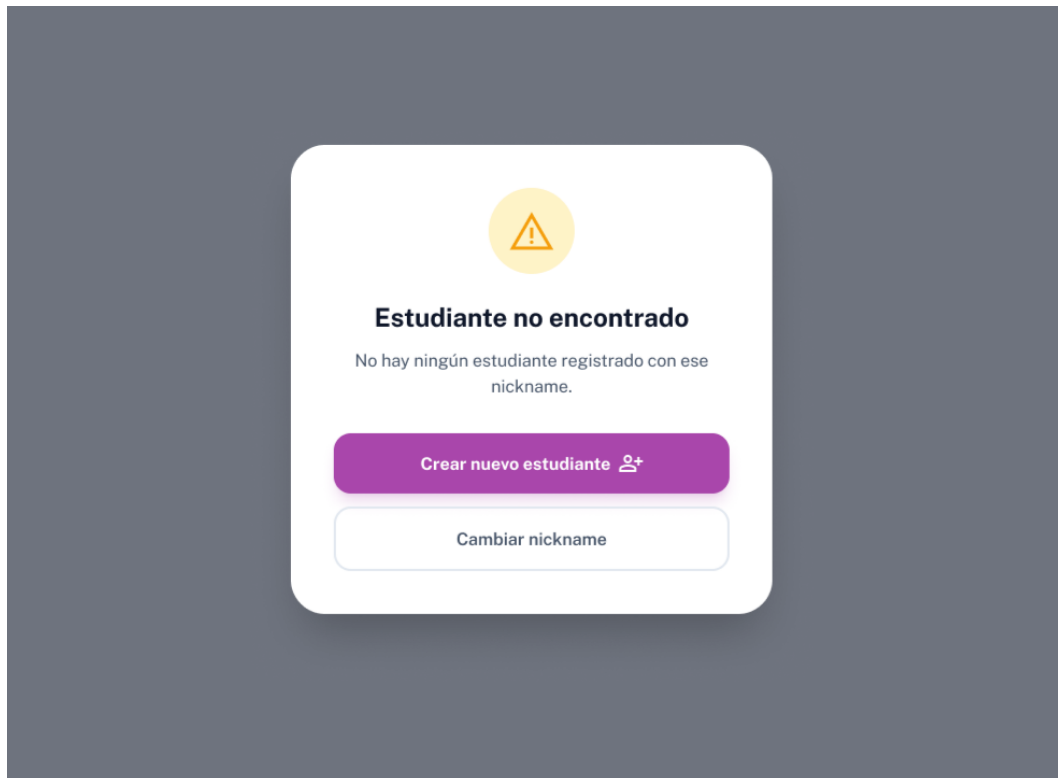


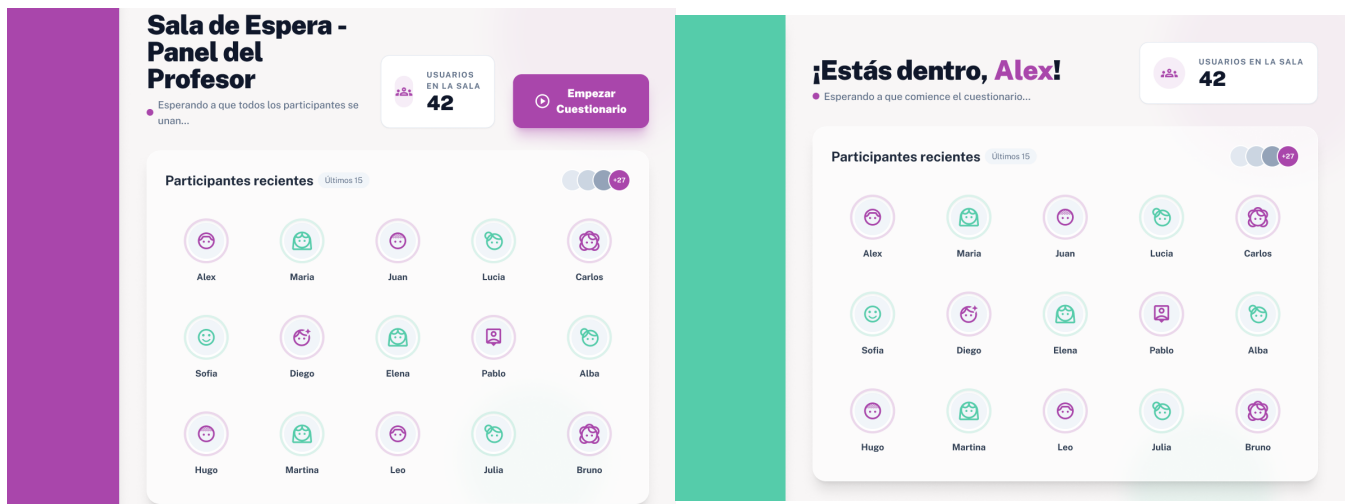
Figura A.10: Aviso para registrar nuevo UVUS o nickname.



Figura A.11: Sala de espera del alumno recién ingresado.

Flujo de ejecución de la partida en vivo

Describe la interacción en tiempo real durante el desarrollo de una sala. Para comparar la experiencia del profesor y del alumno, se presentan en paralelo las interfaces que muestran divergencias funcionales o de diseño. El flujo abarca las fases de sala de espera activa, lectura de pregunta, envío de respuestas, retroalimentación inmediata, tabla de clasificación y verificación final.



Vista Profesor

Vista Alumno

Figura A.12: Sala de espera.



Figura A.13: Cuenta atrás.



• Question 5 of 15

¿Cuál de los siguientes principios se refiere a la distribución visual del peso de los elementos en una interfaz?

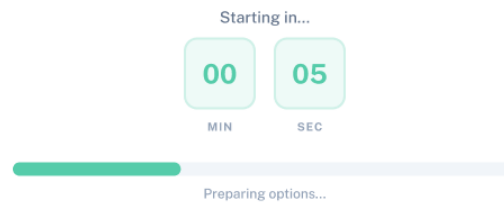
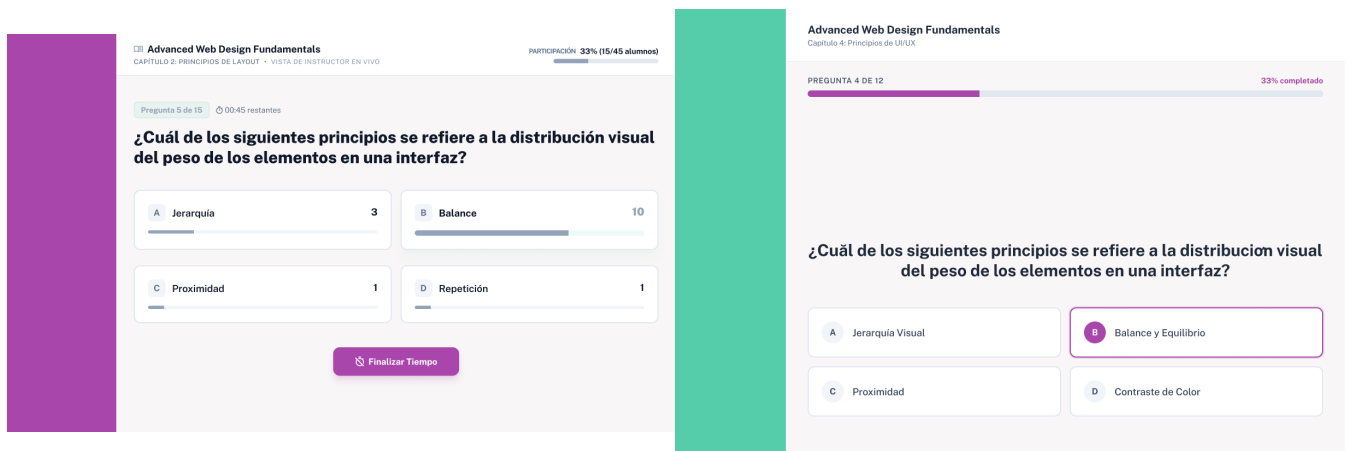


Figura A.14: Tiempo de lectura de pregunta.



(a) Vista Profesor

(b) Vista Alumno

Figura A.15: Pregunta en curso y envío de respuesta.



Figura A.16: Resultados de la pregunta.



Figura A.17: Podio de puntuaciones.

Resultados Finales del Cuestionario

Resumen detallado del desempeño del grupo en la sesión de hoy: **Introducción a la Biología Celular**.

PARTICIPANTES

32 ↑5%

PROMEDIO GENERAL

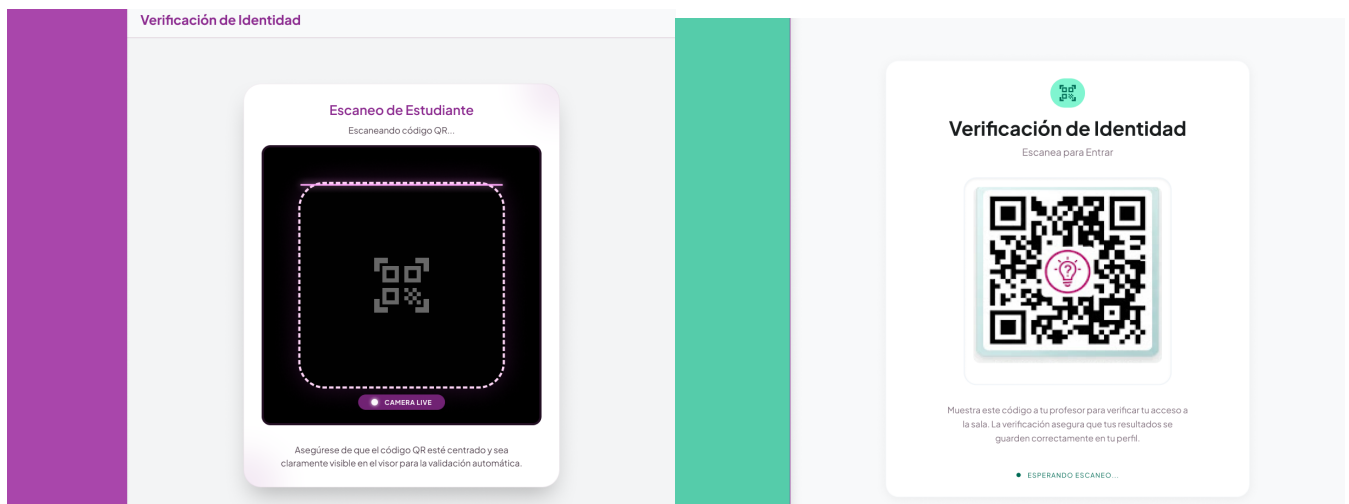
78% ↓2%

TIEMPO PROMEDIO

15:40 ↑1:20



Figura A.18: Distribución global de respuestas.



(a) Vista Profesor

(b) Vista Alumno

Figura A.19: Verificación final.

B. Manual de usuario

El presente manual de usuario tiene como objetivo proporcionar una guía detallada y visual para la utilización de la plataforma *Quizzie*. A diferencia de un catálogo estático de pantallas, este documento estructura la guía siguiendo la experiencia de usuario cronológica real: desde el punto de entrada a la aplicación web y el acceso diferenciado por roles, hasta la navegación por el panel principal, la preparación de contenidos y el desarrollo completo de una sesión interactiva en tiempo real.

Punto de entrada a la aplicación y roles de usuario

Al navegar a la dirección raíz del sitio web (/), la plataforma ofrece una interfaz unificada adaptada a los dos perfiles principales:

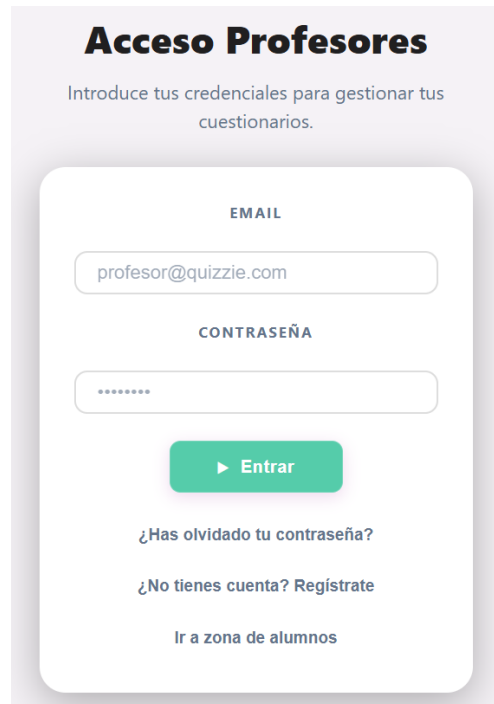
- **Alumno:** Accede directamente desde la pantalla de bienvenida introduciendo el código PIN proyectado en el aula, sin necesidad de contraseñas ni registros previos.
- **Profesor:** Dispone de accesos dedicados para iniciar sesión (/login) o registrarse (/register). Tras autenticarse, accede a un panel de control privado con una barra de navegación lateral (*sidebar*) para gestionar sus cuestionarios y salas.

Registro, autenticación y gestión del perfil docente

El ciclo de vida del usuario docente comienza desde la pantalla principal de la plataforma, guiando al profesor a través de un proceso de registro con correo electrónico, la administración de su cuenta y los mecanismos de recuperación de acceso.

1. Acceso inicial a la plataforma

Al acceder a la aplicación web, el profesor visualiza la pantalla de inicio, la cual permite iniciar sesión o registrarse, o bien saltar a la zona de estudiantes.



Acceso Profesores

Introduce tus credenciales para gestionar tus cuestionarios.

EMAIL

profesor@quizzie.com

CONTRASEÑA

▶ Entrar

¿Has olvidado tu contraseña?

¿No tienes cuenta? Regístrate

Ir a zona de alumnos

Figura B.1: Punto de entrada a la plataforma.

2. Formulario de registro docente

Al pulsar sobre la opción de registro, el sistema presenta un formulario donde el profesor debe ingresar su nombre, dirección de correo electrónico y una contraseña.



Registro de Profesores

Crea tu cuenta de profesor para empezar a gestionar tus cuestionarios.

NOMBRE DE USUARIO

profesor

EMAIL

quizzie.tfg@gmail.com

CONTRASEÑA

CONFIRMAR CONTRASEÑA

▶ Registrarse

¿Ya tienes cuenta? Inicia sesión

Ir a zona de alumnos

Figura B.2: Formulario de registro para profesores.

3. Notificación de verificación de cuenta

Una vez completado y enviado el formulario, la plataforma envía un correo electrónico a la dirección facilitada con un código numérico temporal de un solo uso para verificar la cuenta.



Figura B.3: Correo electrónico con el código de verificación de cuenta.

4. Confirmación del código y activación

El profesor es redirigido a una vista específica donde debe introducir el código recibido en el correo. Al pulsar sobre el botón de confirmación, la cuenta queda validada en la base de datos y habilitada para su uso.

The image shows a mobile app screen titled 'Verifica tu Cuenta'. Below the title, it says 'Introduce el código de 6 dígitos que hemos enviado a tu email.' There is a text input field containing 'quizzie.tfg@gmail.com'. Below that, it says 'CÓDIGO DE VERIFICACIÓN (6 DÍGITOS)'. There are six input boxes for the code, containing the digits '4', '0', '3', '4', '8', and '2'. The last box with '2' is highlighted with a green border. Below the code boxes is a green button with a right arrow and the text 'Activar Cuenta'. At the bottom, there is a link '¿No has recibido el código? Reenviar' and a link 'Volver a inicio de sesión'.

Figura B.4: Pantalla de introducción del código de verificación de cuenta.

5. Panel y opciones del perfil

Tras iniciar sesión, el docente puede acceder a la sección de su perfil personal. Desde esta vista se muestran los datos del usuario y se facilitan las acciones de actualización de datos, cambio de contraseña o borrado de la cuenta.

The image shows a user profile management interface with three distinct sections, each with a dashed border and a colored header.

- Datos personales** (purple header):
 - Subtitle: Gestiona tu nombre de usuario y tu dirección de correo electrónico.
 - Field 1: NOMBRE DE USUARIO, valor: profesor (with a user icon).
 - Field 2: CORREO ELECTRÓNICO, valor: quizzie.tfg@gmail.com (with an email icon).
 - Button: Editar Perfil (purple).
- Seguridad** (teal header):
 - Text: ¿Quieres restablecer tu contraseña? Te enviaremos un código de recuperación a tu correo electrónico para que puedas crear una nueva clave.
 - Button: Restablecer mi contraseña (teal).
- Zona de peligro** (red header):
 - Text: Al eliminar tu cuenta se borrarán permanentemente todos tus cuestionarios, salas, grupos, estudiantes y datos de juego. Esta acción no se puede deshacer.
 - Button: Eliminar mi cuenta (red).

Figura B.5: Vista de gestión del perfil.

6. Solicitud y correo de restablecimiento de contraseña

Si el profesor solicita una modificación o recuperación de contraseña, el sistema envía un nuevo correo electrónico que contiene el código de autorización requerido para efectuar dicho cambio.

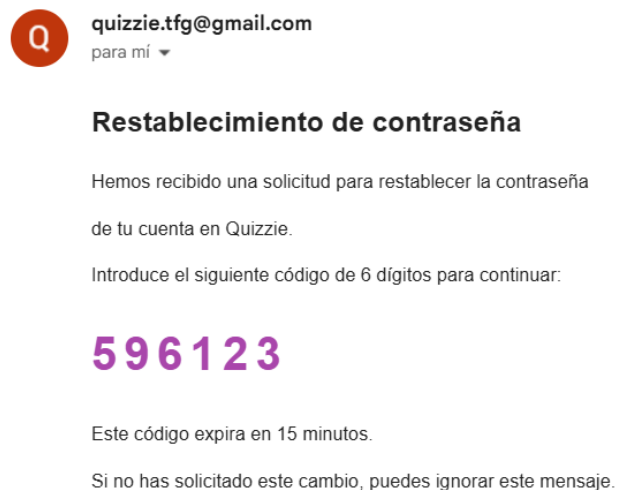


Figura B.6: Correo electrónico para el restablecimiento de contraseña.

7. Introducción de nueva contraseña

En la interfaz, el usuario introduce el código de un solo uso recibido por correo junto con su nueva clave de acceso, efectuándose así la actualización de la contraseña.

This image shows a web form titled 'Restablece tu Contraseña'. Below the title, it says 'Introduce tu correo, el código de 6 dígitos recibido y tu nueva contraseña.' The form has three main sections: 1. 'CORREO ELECTRÓNICO' with a text input field containing 'quizzie.tfg@gmail.com'. 2. 'CÓDIGO DE RECUPERACIÓN' with six individual input boxes containing the digits '5', '9', '6', '1', '2', and '3'. 3. 'NUEVA CONTRASEÑA' with a text input field containing six dots. Below this is a 'CONFIRMAR NUEVA CONTRASEÑA' section with another text input field containing six dots. At the bottom is a green button with a right arrow and the text 'Cambiar contraseña'. Below the button is a link that says 'Volver a inicio de sesión'.

Figura B.7: Formulario para establecer la nueva contraseña.

8. Eliminación definitiva de la cuenta

Finalmente, la plataforma contempla el derecho al borrado de datos. El docente puede solicitar la supresión completa de su perfil y datos asociados mediante un diálogo de confirmación que advierte sobre la irreversibilidad de la acción.

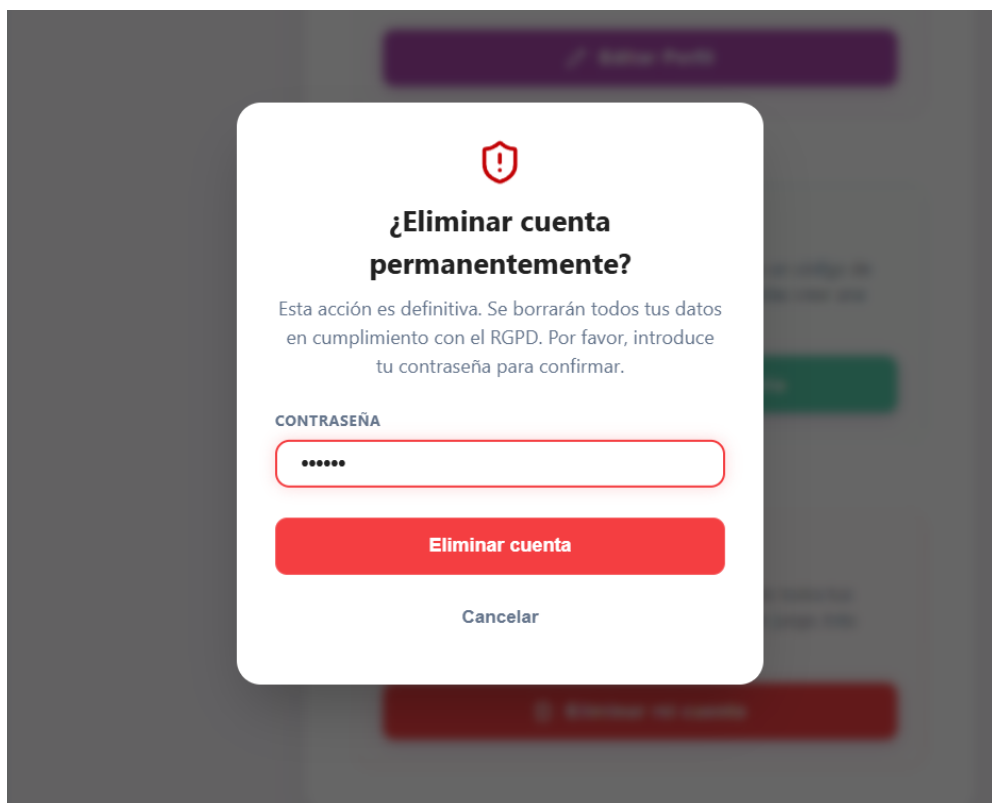


Figura B.8: Confirmación de eliminación de la cuenta de usuario.

Gestión del catálogo de cuestionarios e historial docente

Una vez autenticado en la plataforma, el docente dispone de un entorno centralizado para la administración de sus contenidos académicos. Desde este espacio es posible consultar el catálogo de pruebas creadas, editar sus preguntas y parámetros, examinar las métricas de sesiones anteriores, exportar los resultados y crear nuevos cuestionarios.

1. Inicio de sesión del profesor

El docente introduce su correo electrónico y contraseña en el formulario de acceso para ingresar a su panel privado de control.

Acceso Profesores

Introduce tus credenciales para gestionar tus cuestionarios.

EMAIL

cervantes@quizzie.com

CONTRASEÑA

.....

▶ Entrar

¿Has olvidado tu contraseña?

¿No tienes cuenta? Regístrate

Ir a zona de alumnos

Figura B.9: Formulario de inicio de sesión.

2. Catálogo de cuestionarios disponibles

Al acceder a la sección de cuestionarios, el profesor visualiza la lista con todas sus pruebas creadas. Cada entrada dispone de tres acciones rápidas: el icono del lápiz para modificar el cuestionario, el ojo para consultar el historial de salas asociadas y la papelera para su eliminación.

quizzie

Listar cuestionarios

Crear cuestionario

Mi perfil

Cerrar sesión

Mis Cuestionarios

Gestiona y lanza tus sesiones en tiempo real.

Todos Nuevos Inactivos

Lengua Española Básica EN LOBBY

Ortografía y gramática

22/9/2026

Reconectar Finalizar

Lengua Española Avanzada

Literatura y análisis sintáctico

22/9/2026

▶ Crear sala

Literatura Universal

Obras y autores clásicos

22/9/2026

▶ Crear sala

Figura B.10: Listado general de cuestionarios del profesor.

3. Edición de cuestionario

Pulsando sobre el botón de edición, se abre el formulario donde es posible cambiar el título, modificar su descripción o ajustar el contenido, opciones y puntajes de las preguntas existentes.

The screenshot shows the 'Lengua Española Básica' questionnaire editing interface. On the left is a green sidebar with the 'quizzie' logo and navigation links: 'Listar cuestionarios', 'Crear cuestionario', 'Mi perfil', and 'Cerrar sesión'. The main area has a title 'Lengua Española Básica' and a green 'Confirmar' button. Below the title is a text box for 'Descripción editada'. A modal window titled 'PREGUNTA 1 de 2' is open, showing a question '¿Cuál de estas es un sustantivo?' with a 'PUNTOS' field set to '5'. The question has four options: 'Perro' (selected with a checkmark), 'Correr', 'Rápido', and 'Saltar'. Each option has a delete 'x' button. Navigation arrows are visible on the sides of the modal.

Figura B.11: Vista de edición de un cuestionario.

4. Historial de salas jugadas

Al seleccionar la opción del ojo, el sistema presenta el registro histórico de las salas que se han disputado con dicho cuestionario. Al hacer clic sobre una sala concreta del listado, se accede al desglose pormenorizado de sus resultados.

The screenshot shows the 'Historial de salas' modal window. It has a title 'Historial de salas' and a subtitle 'Lengua Española Básica'. The modal lists two entries, each with a date and time '22/09/2026, 16:21' and a 'Ver resultados' button. The first entry shows '10 alumnos' and the second shows '0 alumnos'. A 'Cerrar' button is at the bottom of the modal. The background shows a blurred view of the 'Mis Cuestionarios' page.

Figura B.12: Historial de salas de un cuestionario.

5. Detalle de resultados históricos y exportación

En esta vista se muestran las puntuaciones registradas durante la sala seleccionada. Mediante el botón de descarga, el profesor puede exportar todo el conjunto de resultados a un archivo en formato CSV para su posterior tratamiento analítico o calificación externa.

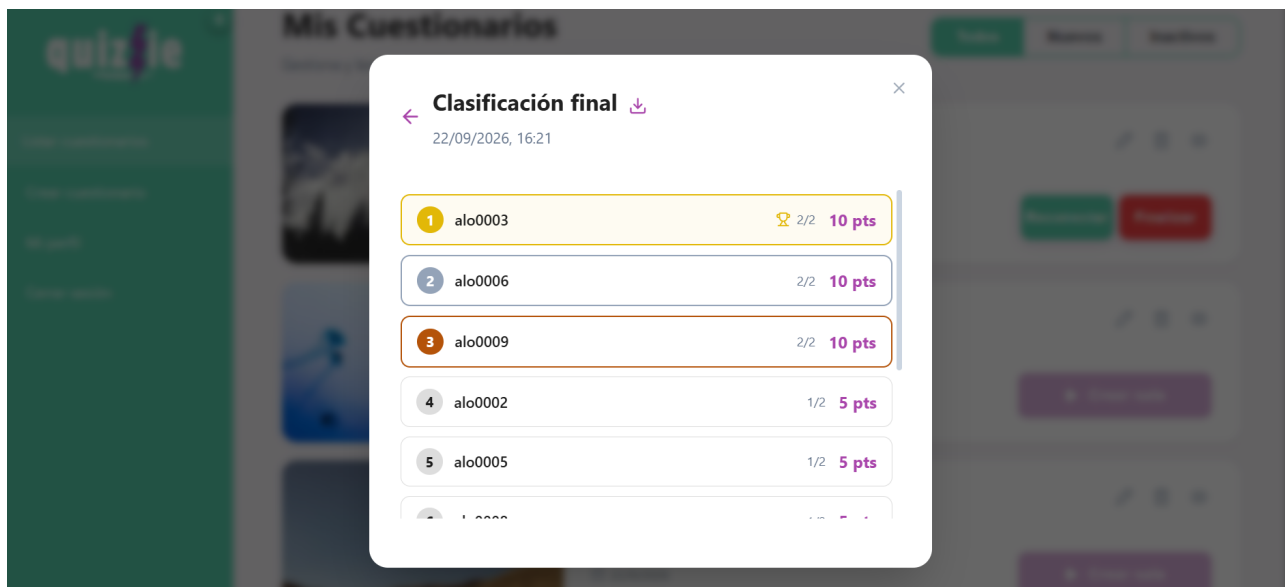


Figura B.13: Resultados de la sala y opción de exportación a CSV.

6. Eliminación de un cuestionario

Al pulsar sobre el icono de la papelera, la plataforma solicita una confirmación previa para evitar pérdidas accidentales de datos antes de suprimir definitivamente el cuestionario.

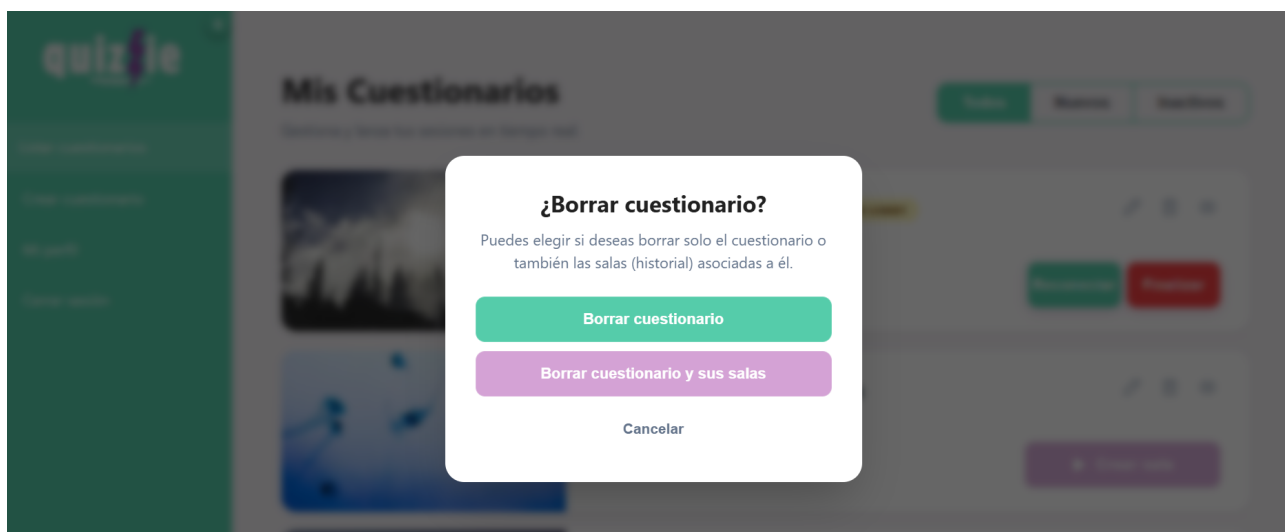


Figura B.14: Diálogo de confirmación para eliminar un cuestionario.

7. Creación de un nuevo cuestionario

A través del botón de creación, el profesor accede a la interfaz de diseño de nuevos cuestionarios, donde define el título, las preguntas, las opciones de respuesta marcando la correcta y la puntuación máxima permitida.

The screenshot shows the 'Nuevo cuestionario' (New Quiz) screen. On the left is a green sidebar with the 'quizzie' logo and navigation links: 'Listar cuestionarios', 'Crear cuestionario' (highlighted), 'Mi perfil', and 'Cerrar sesión'. The main area has a light purple background. At the top right is a purple button labeled 'Crear cuestionario'. Below it is a white box with the text '!Creando un nuevo cuestionario!'. The central part features a white rounded rectangle representing the quiz question editor. It shows 'PREGUNTA 1 de 1' and 'PUNTOS 5'. The question is '¿2+2?'. There are two answer options: '4' (selected with a checkmark) and '5' (unselected). Below the options is a text input field labeled 'Añadir opción...'. Navigation arrows are visible on the sides of the question editor.

Figura B.15: Formulario para la creación de un nuevo cuestionario.

8. Visualización del catálogo filtrado

Tras guardar los cambios, la nueva prueba queda registrada en el sistema y pasa a mostrarse dentro de la sección correspondiente en el catálogo de cuestionarios del usuario, lista para ser lanzada en una sala en directo.

The screenshot shows the 'Mis Cuestionarios' (My Quizzes) screen. The left sidebar is the same as in Figure B.15. The main area has a light purple background. At the top right are three tabs: 'Todos', 'Nuevos' (highlighted), and 'Inactivos'. Below the tabs is the text 'Gestiona y lanza tus sesiones en tiempo real.' The main content area shows a card for a 'Nuevo cuestionario' (New Quiz). The card has a dark image on the left and text on the right: 'Nuevo cuestionario', '!Creando un nuevo cuestionario!', and a date '22/9/2026'. There are icons for edit, delete, and share. A purple button labeled '▶ Crear sala' is at the bottom right of the card. Below the card is a large dashed box with a plus sign and the text 'Crear un nuevo cuestionario' and 'Diseña un set de preguntas personalizado para tus alumnos.'

Figura B.16: Apartado de cuestionarios nuevos.

Configuración, control de sala y verificación presencial

Esta sección documenta el ciclo de vida completo de una sesión desde la perspectiva del profesor: desde el lanzamiento y parametrización de la sala, pasando por la orquestación de la partida en tiempo real, hasta la fase de verificación presencial mediante escaneo de códigos QR.

1. Selección de cuestionario y apertura de sala

Desde la lista de cuestionarios, el profesor escoge la prueba que desea comenzar y pulsa sobre el botón para crear una nueva sala.

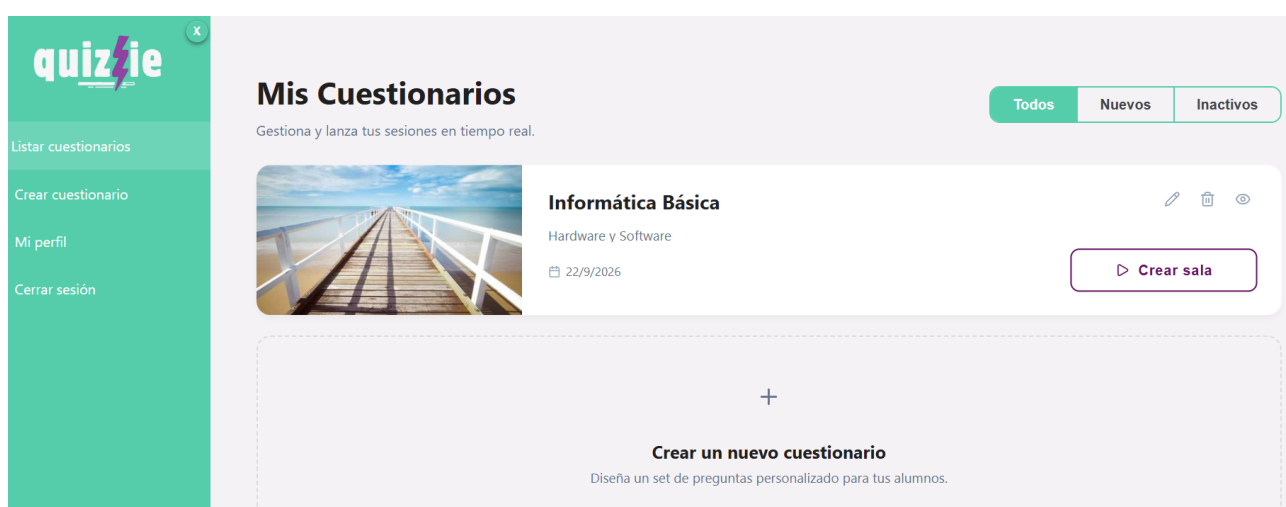


Figura B.17: Selección de un cuestionario para inicializar una sala.

2. Configuración de parámetros de la partida

El profesor configura la dinámica de la partida a través de diversas opciones: orden aleatorio de preguntas, permutación aleatoria de opciones de respuesta para prevenir la copia entre estudiantes, visualización o no de rankings parciales tras cada pregunta (el ranking final siempre estará visible) y el tiempo límite concedido por pregunta. Al confirmar los parámetros, el sistema genera un código de acceso único y da paso a la sala de espera.

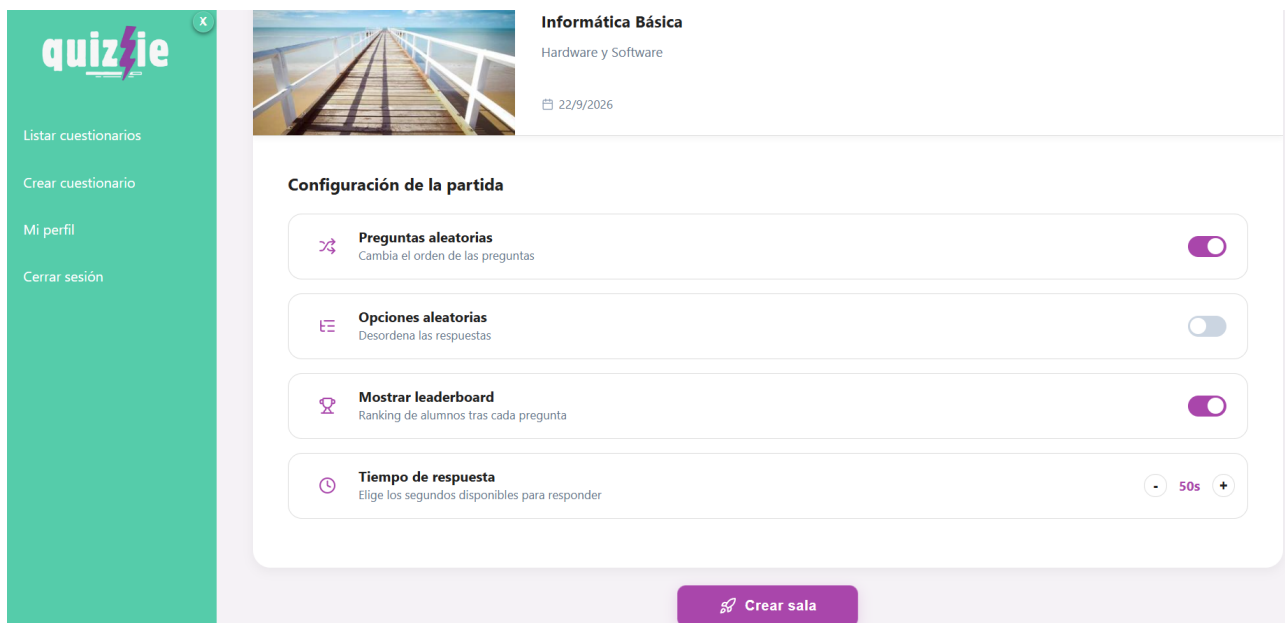


Figura B.18: Configuración de parámetros de la sala.

3. Sala de espera inicial

La interfaz proyecta la sala de espera en su estado inicial vacío, exponiendo el código numérico PIN que el alumnado debe ingresar en sus dispositivos para vincularse a la sesión lectiva.

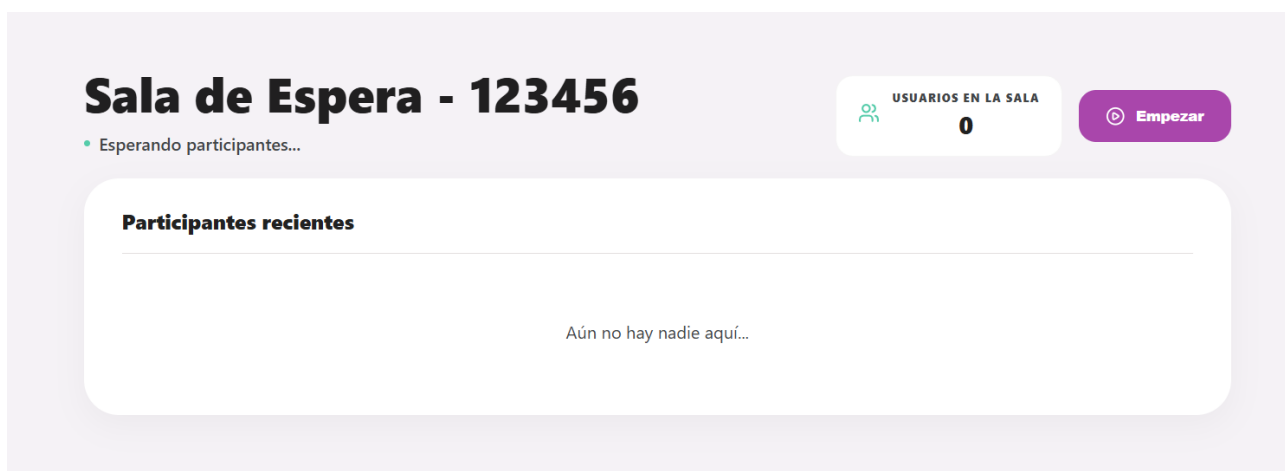


Figura B.19: Sala de espera inicial a la espera de participantes.

4. Incorporación de estudiantes y lanzamiento de la partida

A medida que los estudiantes acceden a la sala, sus nombres o identificadores aparecen reflejados en pantalla. Cuando todos los asistentes se han incorporado, el profesor pulsa el botón para comenzar, dando paso a una breve cuenta atrás de sincronización previa al inicio de las preguntas.

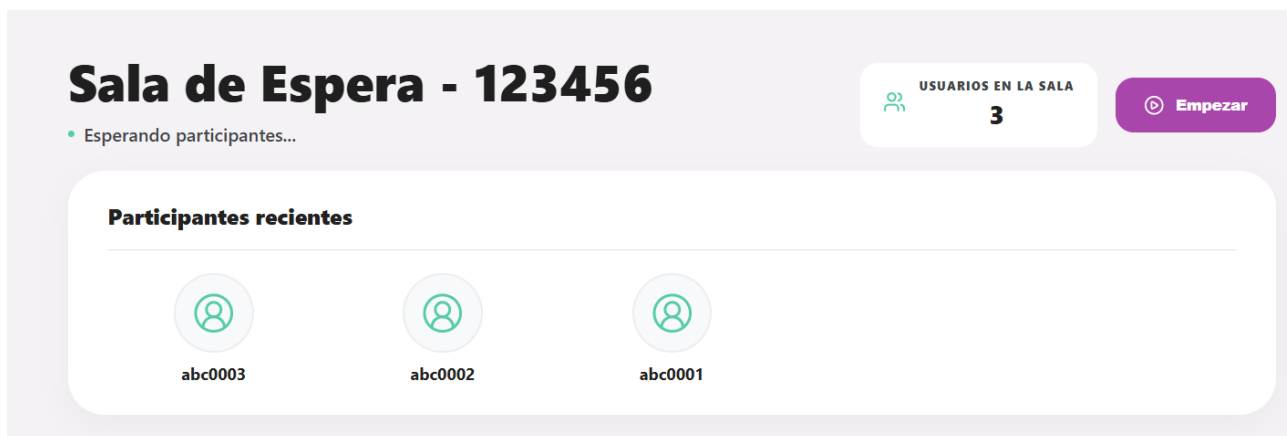


Figura B.20: Sala de espera con estudiantes incorporados.

5. Fase de pregunta activa y recepción de respuestas

Concluida la cuenta atrás, la plataforma proyecta el enunciado de la pregunta y el temporizador en cuenta regresiva según lo configurado. Los estudiantes solo pueden emitir respuestas mientras dicho reloj permanezca activo; si el temporizador se detiene o llega a cero, el sistema bloquea inmediatamente nuevos envíos.

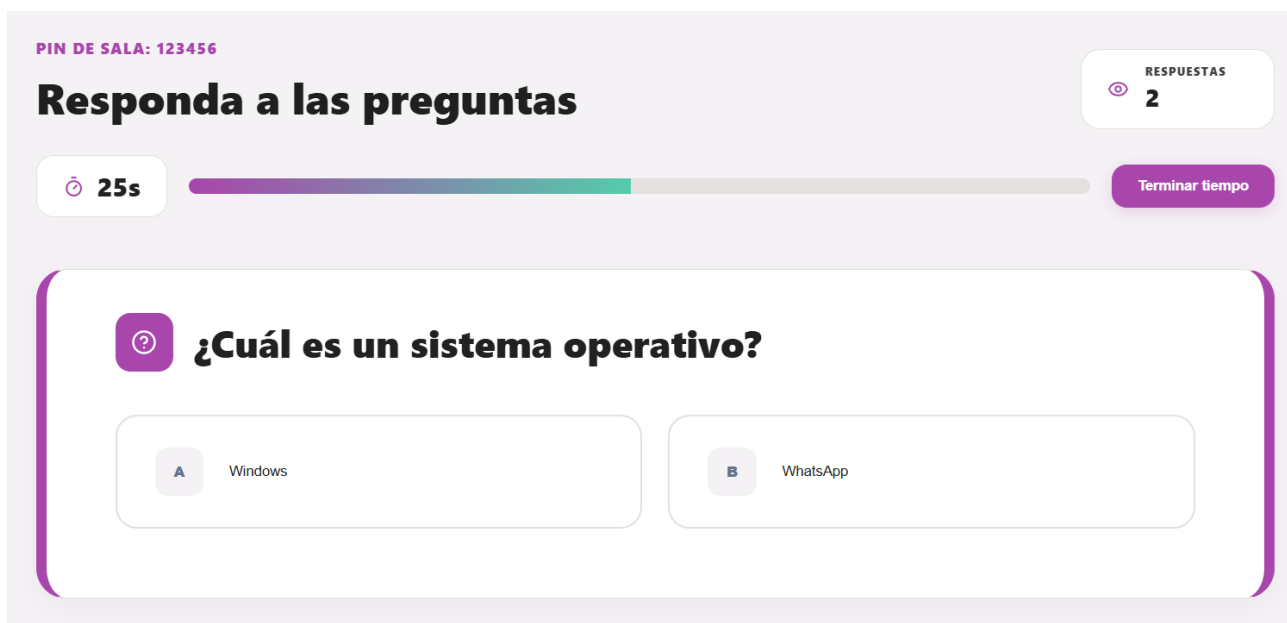


Figura B.21: Pregunta proyectada en curso y temporizador activo.

6. Tolerancia a fallos y reconexión del docente

En caso de que el profesor sufra una pérdida de conexión de red o cierre accidentalmente la ventana, el sistema suspende de inmediato el temporizador para proteger la equidad de la prueba. Al volver a acceder a la plataforma, se

ofrece la opción de reconexión directa a la sala activa, reanudándose el reloj y la interacción únicamente tras su regreso.



Figura B.22: Detección y recuperación de sesión activa tras desconexión.

7. Distribución analítica de respuestas

Una vez agotado el tiempo de respuesta y al avanzar a la siguiente pantalla, el sistema proyecta una gráfica de barras con la distribución de las opciones elegidas por los alumnos. A partir de esta vista, y según las opciones configuradas inicialmente, se habilita la transición al ranking parcial o directamente a la siguiente pregunta o al final del cuestionario.



Figura B.23: Gráfica de respuestas seleccionadas por el alumnado.

8. Conclusión de la prueba y podio final

Tras recorrer las rondas restantes de la evaluación, el sistema presenta la pantalla de podio final con las mejores calificaciones del aula. Al presionar el botón de finalización, la sala concluye su fase de preguntas y da paso al mecanismo de verificación presencial.



Figura B.24: Podio de calificaciones globales al término de las preguntas.

9. Panel de verificación presencial

En esta fase, el profesor dispone de un panel con el listado de participantes para validar su asistencia física en el aula. Desde esta vista se habilita la acción para desplegar la interfaz de escaneo.



Figura B.25: Pantalla de fase de verificación.

10. Escáner y concesión de permisos de cámara

Al solicitar la activación del lector de códigos, la aplicación despliega un diálogo modal que requiere la autorización del navegador para acceder a la cámara del dispositivo. Una vez concedido el permiso, el profesor procede al escaneo secuencial de los códigos QR mostrados en los dispositivos de los estudiantes para validar de forma fehaciente su presencialidad y asentar su nota.

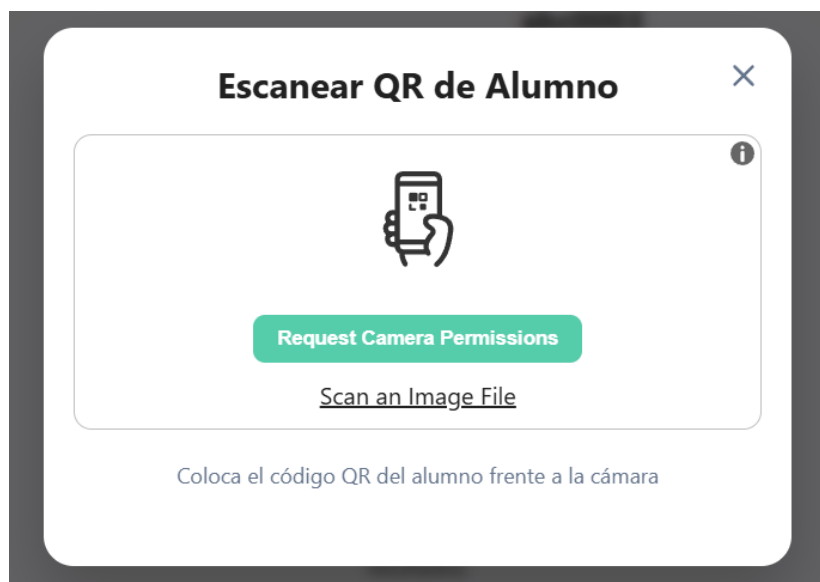


Figura B.26: Modal de captura y solicitud de permisos de cámara.

Participación y experiencia del estudiante

Esta sección documenta el flujo completo que experimenta el alumnado al unirse a una sesión: desde la introducción del código de sala y la identificación sin registro tradicional mediante un identificador, pasando por la interacción en tiempo real con las preguntas, hasta la obtención del código QR final para la verificación física de su presencia en el aula.

1. Ingreso del código de acceso a la sala

Al acceder a la aplicación web, el estudiante visualiza un diálogo modal principal donde debe introducir el código numérico PIN proyectado por el docente para unirse a la sala.

¿Listo para el desafío?

Ingresa el código de sala para empezar.

CÓDIGO DE SALA

1 2 3 4 5 6

▶ Entrar

Figura B.27: Introducción del código de la sala.

2. Identificación del alumno mediante UVUS

Tras validar el código de la sesión, el alumno introduce su identificador único (*uvus*) o *nickname* para vincular sus respuestas y calificación. La interfaz informa si la sala se encuentra en estado de espera o si ya ha comenzado, permitiendo la incorporación de estudiantes incluso con la sesión en curso.

Sala 123456

Introduce tu uvus para unirte.

¿CÓMO TE LLAMAS?

@ abc0001

Siguiente

Volver atrás

• EN ESPERA

Figura B.28: Entrada de *uvus* o apodo.

3. Aviso de registro de nuevo identificador

En caso de que el identificador ingresado no conste previamente en la base de datos de la plataforma, el sistema despliega un aviso preventivo para confirmar si se desea dar de alta dicho identificador o corregir un posible error tipográfico antes de acceder.

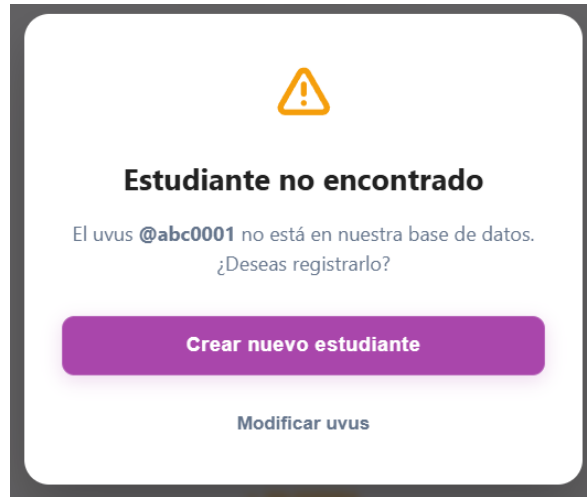


Figura B.29: Confirmación para un nuevo *uvus* o apodo.

4. Sala de espera del estudiante

Una vez confirmada la identidad, el alumno pasa a la sala de espera, donde aguarda a que el profesor inicie el cuestionario.

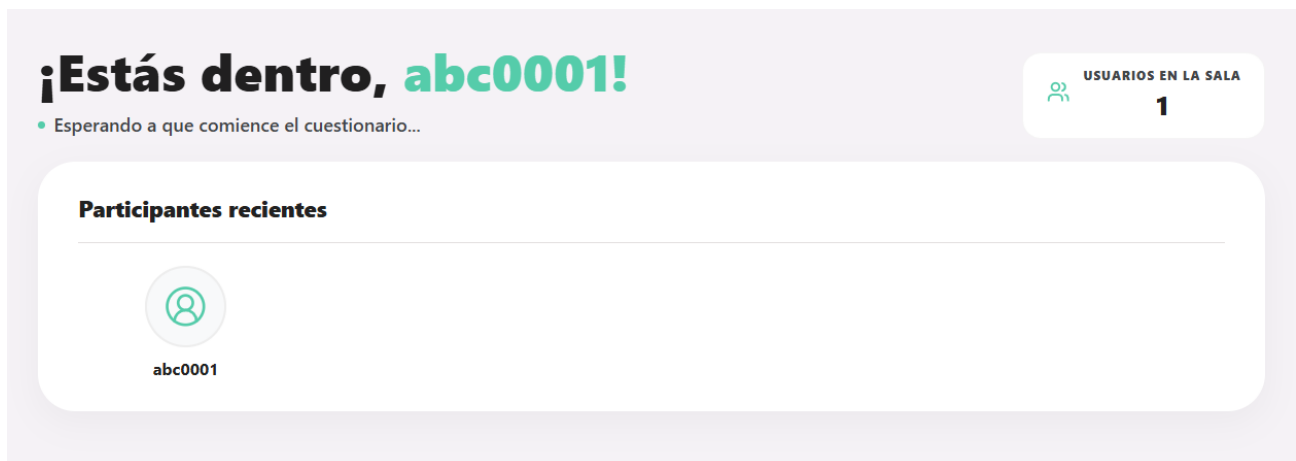


Figura B.30: Sala de espera del alumno.

5. Fase de pregunta activa y selección de respuesta

Al comenzar la ronda y proyectarse la pregunta, el alumno dispone de los botones de selección en su dispositivo. El envío de respuestas solo está permitido mientras el temporizador se encuentre en marcha.

PIN DE SALA: 123456

Responde a las preguntas

RESPUESTAS 1

27s

2 ¿Cuál es un sistema operativo?

A Windows

B WhatsApp

✓ Respuesta enviada

Figura B.31: Interfaz de selección y envío de respuesta.

6. Retroalimentación de respuesta correcta

Al concluir el tiempo de la pregunta, la interfaz revela de forma instantánea la solución y notifica al estudiante si su elección ha sido correcta, sumando la puntuación correspondiente.

2

1

A Windows

B WhatsApp

PARTICIPACIÓN 3 alumnos

TU RESULTADO ¡Correcto!

ÉXITO GLOBAL 67%

Figura B.32: Confirmación inmediata de respuesta correcta.

7. Retroalimentación de respuesta incorrecta

De igual modo, si el alumno falló en su elección, la aplicación le indica que la opción remitida no era la adecuada, mostrando la alternativa válida para afianzar el aprendizaje formativo.

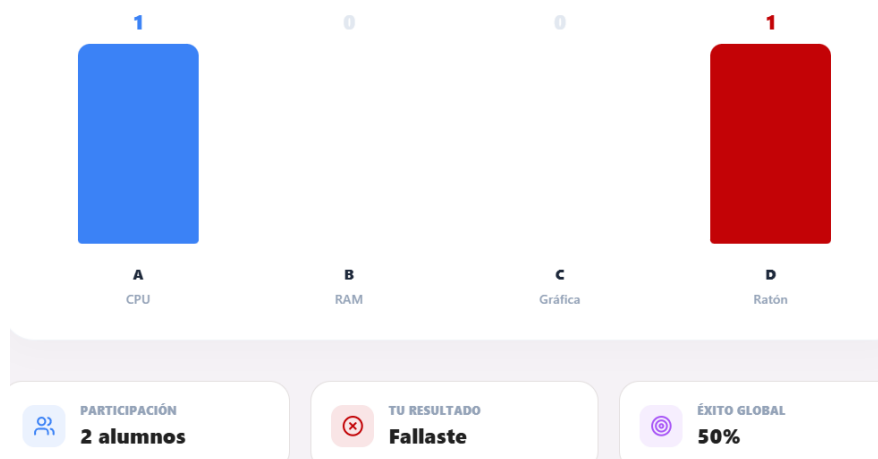


Figura B.33: Indicación de respuesta incorrecta y retroalimentación.

8. Suspensión de la interacción por pausa o desconexión

Si el profesor experimenta una caída de red o abandona temporalmente la sala, el sistema entra automáticamente en estado de pausa: el reloj se detiene y la interfaz del estudiante queda bloqueada, impidiendo el envío de respuestas fuera de tiempo hasta que el docente se reincorpore.

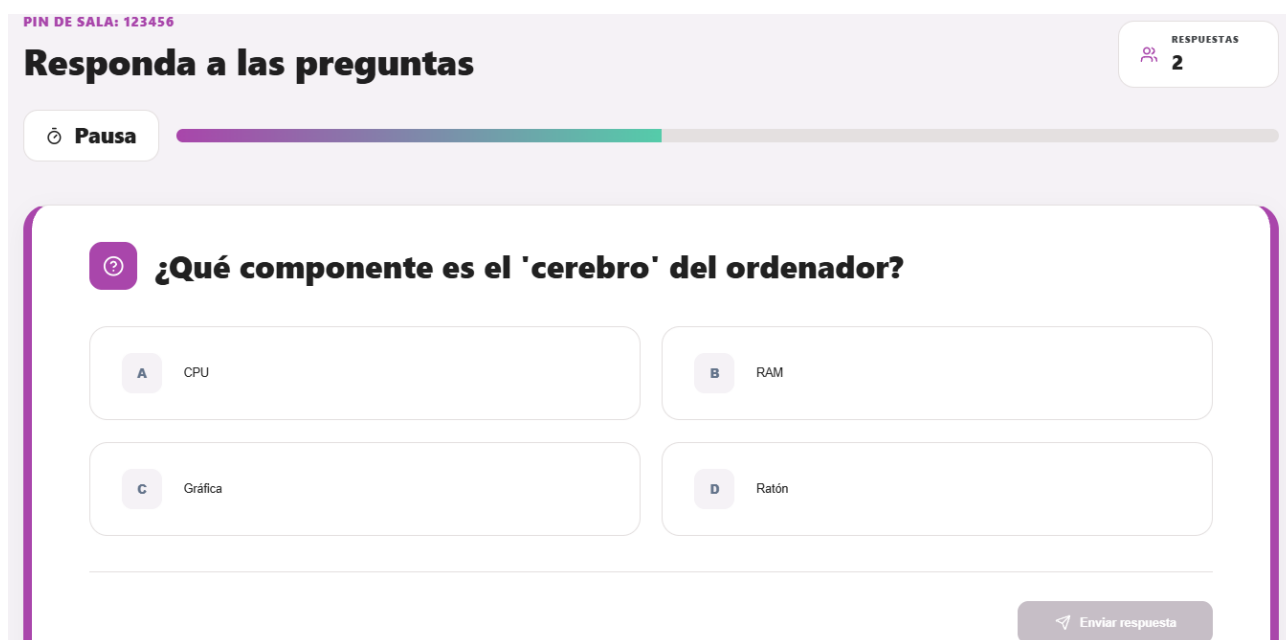


Figura B.34: Bloqueo temporal de respuestas por pausa de la sesión.

9. Agotamiento del tiempo de respuesta

Cuando la cuenta atrás del temporizador alcanza el valor cero sin que se haya emitido una respuesta previa, el sistema deshabilita las opciones de selección, registrando la ausencia de respuesta para dicha ronda.

PIN DE SALA: 123456

Responda a las preguntas

RESPUESTAS 2

0s

¿Qué componente es el 'cerebro' del ordenador?

A CPU

B RAM

C Gráfica

D Ratón

Enviar respuesta

Figura B.35: Bloqueo de opciones al expirar el tiempo reglamentario.

10. Generación de código QR para verificación presencial

Al finalizar la totalidad de las preguntas y proyectarse el podio general, la pantalla del alumno genera un código QR efímero que codifica su identidad y los resultados alcanzados. El estudiante presenta esta pantalla al docente para su escaneo físico; una vez verificado por la cámara del profesor, la sesión queda formalmente consolidada y finalizada.



Figura B.36: Código QR de verificación del alumno.

C. Manual técnico

El presente manual técnico detalla el procedimiento para la clonación, configuración de dependencias, puesta en marcha del entorno de desarrollo local y reproducción íntegra de la batería de pruebas y análisis de rendimiento del sistema.

Requisitos previos del sistema

Para garantizar la correcta ejecución de la plataforma en un entorno local, la máquina anfitriona debe disponer de las herramientas base instaladas y registradas en las variables de entorno del sistema (PATH):

- **Git**[52].
- **Node.js**[53].
- **uv**[12] (puede instalarse nativamente o mediante pip si el sistema ya cuenta con un intérprete de Python).

Preparación y configuración del repositorio

Descarga del código fuente

Clonar el repositorio del proyecto e ingresar en el directorio raíz:

```
> git clone https://github.com/frangago71/quizzie-TFG.git
> cd quizzie-TFG
```

Configuración del entorno del backend

Instalar el gestor uv en caso de no disponer de él previamente y sincronizar todas las dependencias del proyecto:

```
> pip install uv
> uv sync --all-groups
```

La sincronización anterior genera un entorno virtual aislado (.venv) con todas las librerías necesarias. Para la ejecución de órdenes en dicho entorno se puede emplear el prefijo `uv run` en cada invocación, o bien activar explícitamente el entorno en la sesión de terminal para omitir dicho prefijo y ejecutar los comandos directamente:

```
> & .\venv\Scripts\Activate.ps1
```

Para poblar la base de datos con los registros iniciales, acceder al directorio del backend y ejecutar el script de sembrado:

```
> cd backend  
> uv run python -m seed # Ejecutar el sembrado inicial de datos
```

Configuración del entorno del frontend

Acceder al directorio de la interfaz de usuario e instalar los módulos de dependencias:

```
> cd ../frontend  
> npm install # Instalar dependencias de Node.js
```

Configuración de variables de entorno

Copiar la plantilla de configuración situada en la raíz del proyecto para generar el archivo local `.env`:

```
> cp .env.example .env
```

A continuación, editar el archivo `.env` completando las siguientes secciones:

- **VITE_API_URL**: Dirección base donde escucha la API de FastAPI. En local se establece en `http://localhost:8000`; en producción apuntará a la URL del backend desplegado.
- **SECRET_KEY**: Clave simétrica utilizada para firmar criptográficamente los tokens de sesión. Para garantizar la seguridad, se recomienda generarla de manera aleatoria, por ejemplo ejecutando el siguiente comando en la terminal:

```
> openssl rand -hex 32
```
- **ALGORITHM**: Algoritmo criptográfico de firma de tokens (por defecto, HS256).
- **ACCESS_TOKEN_EXPIRE_MINUTES**: Tiempo de expiración del token de sesión expresado en minutos (fijado por defecto en 30).
- **SMTP_HOST**: Nombre de dominio del servidor de correo saliente (por defecto, `smtp.gmail.com`).
- **SMTP_PORT**: Puerto del servicio de transporte. Los valores habituales son 465 (cifrado implícito SSL) o 587 (cifrado explícito TLS/STARTTLS).
- **SMTP_USERNAME**: Cuenta de correo electrónico empleada para la autenticación ante el servidor SMTP.

- **SMTP_PASSWORD:** Clave de acceso al servicio. Al emplear Gmail, es indispensable utilizar una **Contraseña de aplicación** de Google generada desde la cuenta de seguridad, en lugar de la contraseña principal de usuario.
- **SMTP_FROM:** Dirección visible en la cabecera del remitente. Si se omite, toma automáticamente el valor de SMTP_USERNAME. En servidores como Gmail, este campo debe coincidir con la cuenta autenticada para evitar que el servidor rechace el envío por sospecha de suplantación.
- **GEMINI_API_KEY:** Clave de acceso a la API de Google Gemini para la generación asistida de cuestionarios. Corresponde a una funcionalidad planificada para la siguiente etapa y que queda fuera del alcance de la presente entrega del TFG.

El archivo `.env` resultante se encuentra excluido mediante el archivo `.gitignore`, garantizando que ninguna credencial privada alcance el repositorio remoto.

Ejecución del proyecto en el entorno local

La plataforma requiere la ejecución simultánea de dos procesos independientes. Para ello, se recomienda disponer de dos terminales de trabajo activas de manera concurrente.

Terminal 1: Servicio del backend (FastAPI)

Iniciar el servidor de desarrollo del *backend*:

```
> cd backend
> uv run fastapi dev
```

Una vez levantado el servicio, los accesos locales quedan disponibles en:

- `http://127.0.0.1:8000`: Raíz del servidor y servicio de la API.
- `http://127.0.0.1:8000/docs`: Documentación - **Swagger UI**.
- `http://127.0.0.1:8000/redoc`: Documentación - **ReDoc**.

Terminal 2: Cliente del frontend (Vite)

Acceder al directorio de la interfaz y levantar el servidor de desarrollo local:

```
> cd frontend
> npm run dev
```

La interfaz web quedará accesible desde el navegador en `http://localhost:5173`.

Procedimiento de ejecución de pruebas y cobertura

Pruebas unitarias y de integración del *backend*

Ejecutar la suite completa de pruebas unitarias y de integración del *backend* mediante pytest:

```
> uv run pytest
```

Para auditar la cobertura de código y generar reportes adicionales, se pueden combinar los siguientes modificadores:

- `--cov=backend`: Delimita el análisis exclusivamente al código fuente del *backend*.
- `--cov-report=term-missing`: Imprime en la consola la tabla desglosada por módulo señalando las líneas exactas sin cobertura.
- `--cov-report=html`: Genera un informe visual interactivo estructurado dentro del directorio local `htmlcov/`.

Al combinar estas tres opciones, se imprime en la terminal la tabla de cobertura del *backend* y se genera un informe interactivo en `htmlcov/index.html`:

```
> uv run pytest --cov=backend --cov-report=term-missing --cov-report=html
```

```
PS C:\WS-TFG\quizzie-tfg> uv run pytest --cov=backend --cov-report=term-missing --cov-report=html
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\WS-TFG\quizzie-tfg
configfile: pyproject.toml
testpaths: tests/backend
plugins: anyio-4.12.1, locust-2.46.3, cov-7.1.0
collected 71 items

tests\backend\test_integration_content.py .... [ 5%]
tests\backend\test_integration_stage.py ..... [ 21%]
tests\backend\test_integration_users.py .... [ 26%]
tests\backend\test_unit_content.py ..... [ 45%]
tests\backend\test_unit_stage.py ..... [ 76%]
tests\backend\test_unit_users.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.10-final-0 -----
Name                               Stmts   Miss  Cover   Missing
-----
backend\auth.py                     39      4    90%    55, 60, 65-66
backend\database.py                 18      3    83%    28, 32-33
backend\email_service.py            45      0   100%
backend\main.py                     27      1    96%    53
backend\models\__init__.py          10      0   100%
backend\models\content.py           47      4    91%    8-9, 37, 58
backend\models\stage.py             69      2    97%    9-10
backend\models\users.py             44      3    93%    8-9, 13
backend\routers\content.py          188      0   100%
backend\routers\stage.py            454      9    98%    55, 57, 83-84, 99, 153, 190, 222, 225
backend\routers\users.py            211      0   100%
backend\schemas\content.py         78      6    92%    25, 27, 29, 89, 91, 93
backend\schemas\users.py           21      0   100%
-----
TOTAL                             1251     32    97%
Coverage HTML written to dir htmlcov
===== 71 passed in 11.94s =====
```

Salida por terminal al ejecutar las pruebas y la cobertura del *backend*.

Para abrir el reporte web directamente en el navegador predeterminado desde la consola según el sistema operativo:

```
> start htmlcov/index.html      # Windows
> xdg-open htmlcov/index.html  # Linux
> open htmlcov/index.html      # macOS
```

De manera alternativa, el informe puede consultarse abriendo el archivo a través del explorador del sistema operativo.

Pruebas de componentes del *frontend*

Acceder al directorio de la interfaz y ejecutar la suite completa de pruebas unitarias y de componentes:

```
> cd frontend
> npm run test
```

Ejecutar la suite de pruebas evaluando la cobertura de código:

```
> npm run test:frontend:coverage
```

Al ejecutar este comando, se imprime por terminal el desglose cuantitativo de cobertura acotado a los módulos clave (auth, management, room-access y room-play) y se genera el directorio de cobertura frontend/coverage/:

```
PS C:\WS-TFG\quizzie-tfg\frontend> npm run test:frontend:coverage
```

Test Files	25 passed (25)
Tests	193 passed (193)
Start at	15:45:28
Duration	20.20s (transform 7.18s, setup 20.83s, import 17.63s, tests 46.06s, environment 86.99s)


```
% Coverage report from v8
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.13	80.29	88.56	92.68	
auth	93.37	84.93	95.16	94.17	
CodeInputGrid.tsx	100	100	100	100	
ForgotPassword.tsx	90.9	75	100	90.9	16-17
Login.tsx	100	91.66	100	100	33
LogoutModal.tsx	100	100	100	100	
Profile.tsx	95.5	77.77	92.85	96.59	119-120,154
Register.tsx	97.36	87.5	100	97.36	22
ResetPassword.tsx	84.48	84.61	88.88	85.96	50-51,54-55,89-91,122
VerifyEmail.tsx	90.16	85.36	87.5	91.66	49-50,53-54,130
authService.ts	100	100	100	100	
management	87.82	76.37	86.36	90.78	
CreateQuiz.tsx	87.93	72.63	89.58	92.85	84-90,101,103,124,199-200
DeleteQuizModal.tsx	100	92.85	100	100	36
EditQuiz.tsx	86.14	72.41	90	89.93	87,96-98,115,127-130,133-136,150-153,307,479-484
ListQuizzes.tsx	84.44	76.31	80	85.24	29-46,88,192-193,201,315,323,339-341,399,481
RoomHistory.tsx	98.48	85.71	100	100	91,149,280-296
SetupRoom.tsx	85.91	78	68.18	87.87	44,56-57,118,164,192,218,298
room-access	94.91	75.8	94.73	97.36	
NewNickname.tsx	100	75	100	100	40
NicknameEntry.tsx	95.23	78.37	87.5	98.33	137
RoomCode.tsx	92.5	71.42	100	94.87	36-37
room-play	89.66	82.47	86.9	92.83	
AnsweringPhase.tsx	100	100	100	100	
FinalScreen.tsx	96.22	95.45	84.61	96.15	314-323
LeaderboardPhase.tsx	100	93.93	100	100	111-114
LiveRoom.tsx	84.39	70.65	85.29	87.77	65,113-118,174,191,225-226,236-237,247-248,266,278,291,302-304,312
Lobby.tsx	91.17	87.27	83.33	96.87	40,64
ResultsPhase.tsx	97.56	75.86	100	100	45-51,114-115,133-139,204,213-226,252
ScannerModal.tsx	83.33	100	71.42	90.9	14

Salida por terminal al ejecutar las pruebas y la cobertura de componentes.

Para visualizar en el navegador el informe web interactivo generado, iniciar el servidor local de visualización:

```
> npm run cov-report:frontend
```

El reporte web queda desplegado en <http://localhost:4173>. Asimismo, es posible visualizarlo de forma estática cargando `frontend/coverage/index.html` en el navegador.

Pruebas de extremo a extremo y cobertura E2E

Ejecutar la suite de pruebas *End-to-End* sobre la compilación de producción del cliente web:

```
> cd frontend
> npm run test:e2e
```

Al finalizar la ejecución, la consola muestra el resumen de los escenarios evaluados y se generan los informes de trazas de ejecución de casos de uso y de cobertura:

```
PS C:\WS-TFG\quizzie-tfg\frontend> npm run test:e2e
[MCR] Playwright E2E Coverage Report
```

Name	Coverage %	Covered	Uncovered	Total
Bytes	93.81 %	160.691	10.610	171.301
Statements	91.30 %	609	58	667
Branches	83.92 %	814	156	970
Functions	91.41 %	298	28	326
Lines	92.58 %	4664	374	5038

```
112 passed (1.1m)
```

Salida por terminal al ejecutar las pruebas de extremo a extremo.

Para inspeccionar las trazas detalladas, capturas de pantalla y pasos de ejecución mediante el servidor integrado de Playwright:

```
> npx playwright show-report
```

El reporte queda desplegado en <http://localhost:9323>, permaneciendo almacenado como archivo estático en `frontend/playwright-report/index.html`.

Para previsualizar el informe de cobertura interactivo generado por **Monocart**, levantar el servidor local:

```
> npm run cov-report:e2e
```

El reporte queda accesible en <http://localhost:4174>, o bien abriendo directamente el archivo `frontend/coverage-e2e/index.html`.

Pruebas de carga y rendimiento

Para ejecutar estas pruebas es necesario disponer de dos terminales abiertas: una manteniendo el servicio del *backend* en ejecución en `http://127.0.0.1:8000` y otra situada en la raíz del proyecto para lanzar las pruebas.

El flujo de trabajo se compone de dos fases: en primer lugar, se inyecta la carga en el servidor mediante Locust, lo que genera automáticamente los registros de métricas en formato CSV dentro del directorio `tests/locust/`. Posteriormente, el script `tests/locust/evaluate_thresholds.py` procesa dichos ficheros para verificar si los tiempos de respuesta y las tasas de error se mantienen dentro de los umbrales de aceptación definidos.

Para llevar a cabo la inyección de carga se dispone de dos alternativas válidas:

Opción 1: Modo interactivo con interfaz web

Iniciar el panel gráfico de control de Locust para configurar los parámetros manualmente y monitorizar la prueba en tiempo real en `http://localhost:8089`:

```
> uv run locust -f tests/locust/locustfile.py --host http://127.0.0.1:8000
```

Opción 2: Ejecución por consola

Ejecutar las pruebas directamente desde la terminal utilizando el script automatizado con la configuración base (200 usuarios, tasa de 15 u/s y 1 minuto de duración):

```
> uv run python tests/locust/run_performance_tests.py
```

Para personalizar las condiciones de estrés, es posible sobrescribir estos valores mediante los siguientes argumentos:

```
> uv run python tests/locust/run_performance_tests.py -u 300 -r 25 -t 2m
```

- `-u` (`--users`): Número de usuarios virtuales simultáneos.
- `-r` (`--spawn-rate`): Tasa de incorporación de nuevos usuarios por segundo.
- `-t` (`--run-time`): Duración total de la prueba.
- `--host`: Dirección base del servicio *backend*.

Una vez concluida la prueba mediante cualquiera de las dos alternativas y generados los archivos CSV, ejecutar el script de auditoría para verificar cuantitativamente el cumplimiento de los umbrales de aceptación:

```
> uv run python tests/locust/evaluate_thresholds.py
```

Al procesar los registros, la herramienta contrasta las latencias y tasas de error frente a los límites admisibles:

```
PS C:\WS-TFG\quizzie-tfg> uv run python tests/locust/evaluate_thresholds.py

=====
=== INFORME DE EVALUACION DE UMBRALES DE RENDIMIENTO (P90) ===
=====

```

Metrica / Endpoint	Valor P90	Umbral	Estado
GET /content/quizzes/	20 ms	<= 500 ms	[OK]
POST /content/quizzes/	38 ms	<= 500 ms	[OK]
GET /content/quizzes/{id}	17 ms	<= 200 ms	[OK]
PUT /content/quizzes/{id}	22 ms	<= 200 ms	[OK]
POST /stage/participants	38 ms	<= 200 ms	[OK]
GET /stage/rooms	69 ms	<= 200 ms	[OK]
POST /stage/rooms/	84 ms	<= 200 ms	[OK]
GET /stage/rooms/verify/{join_code}	49 ms	<= 200 ms	[OK]
GET /stage/rooms/{id}	35 ms	<= 200 ms	[OK]
GET /stage/rooms/{id}/participants	12 ms	<= 200 ms	[OK]
POST /users/login	240 ms	<= 500 ms	[OK]
GET /users/me	12 ms	<= 200 ms	[OK]
WebSocket Connect	28 ms	<= 200 ms	[OK]
Tasa de Errores (Global)	0.00%	< 1.0%	[OK]

```

=====
RESULTADO FINAL: TODAS LAS PRUEBAS DE RENDIMIENTO SUPERARON LOS UMBRALES [OK]
=====
```

Salida por terminal del *script* de cumplimiento de umbrales.

D. Registro de pruebas y cobertura

En este anexo se recopilan los resultados de las pruebas, registros de coberturas e informes gráficos generados por las distintas baterías de pruebas automatizadas del proyecto.

Pruebas unitarias y de integración del backend

```
PS C:\WS-TFG\quizzie-tfg> uv run pytest --cov=backend --cov-report=term-missing --cov-report=html
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\WS-TFG\quizzie-tfg
configfile: pyproject.toml
testpaths: tests/backend
plugins: anyio-4.12.1, locust-2.46.3, cov-7.1.0
collected 71 items

tests\backend\test_integration_content.py .... [ 5%]
tests\backend\test_integration_stage.py ..... [ 21%]
tests\backend\test_integration_users.py .... [ 26%]
tests\backend\test_unit_content.py ..... [ 45%]
tests\backend\test_unit_stage.py ..... [ 76%]
tests\backend\test_unit_users.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.12.10-final-0 -----
```

Name	Stmts	Miss	Cover	Missing
backend\auth.py	39	4	90%	55, 60, 65-66
backend\database.py	18	3	83%	28, 32-33
backend\email_service.py	45	0	100%	
backend\main.py	27	1	96%	53
backend\models__init__.py	10	0	100%	
backend\models\content.py	47	4	91%	8-9, 37, 58
backend\models\stage.py	69	2	97%	9-10
backend\models\users.py	44	3	93%	8-9, 13
backend\routers\content.py	188	0	100%	
backend\routers\stage.py	454	9	98%	55, 57, 83-84, 99, 153, 190, 222, 225
backend\routers\users.py	211	0	100%	
backend\schemas\content.py	78	6	92%	25, 27, 29, 89, 91, 93
backend\schemas\users.py	21	0	100%	
TOTAL	1251	32	97%	

```
Coverage HTML written to dir htmlcov
===== 71 passed in 11.94s =====
```

Evidencia D.1: Captura de la terminal al ejecutar las pruebas del *backend*.

Coverage report: 97%				
filter...				
Files Functions Classes				
☐ hide covered				
coverage.py v7.13.5, created at 2026-09-15 13:27 +0200				
File ▲	statements	missing	excluded	coverage
backend \ auth.py	39	4	0	90%
backend \ database.py	18	3	0	83%
backend \ email_service.py	45	0	0	100%
backend \ main.py	27	1	0	96%
backend \ models \ __init__.py	10	0	0	100%
backend \ models \ content.py	47	4	0	91%
backend \ models \ stage.py	69	2	0	97%
backend \ models \ users.py	44	3	0	93%
backend \ routers \ content.py	188	0	0	100%
backend \ routers \ stage.py	454	9	0	98%
backend \ routers \ users.py	211	0	0	100%
backend \ schemas \ content.py	78	6	0	92%
backend \ schemas \ users.py	21	0	0	100%
Total	1251	32	0	97%

Evidencia D.2: Reporte HTML de cobertura del *backend*.

Pruebas de componentes del frontend

```
PS C:\WS-TFG\quizzie-tfg\frontend> npm run test:frontend:coverage
```

Test Files **25 passed** (25)
 Tests **193 passed** (193)
 Start at 15:45:28
 Duration 20.20s (transform 7.18s, setup 20.83s, import 17.63s, tests 46.06s, environment 86.99s)

% Coverage report from v8

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	90.13	80.29	88.56	92.68	
auth	93.37	84.93	95.16	94.17	
CodeInputGrid.tsx	100	100	100	100	
ForgotPassword.tsx	90.9	75	100	90.9	16-17
Login.tsx	100	91.66	100	100	33
LogoutModal.tsx	100	100	100	100	
Profile.tsx	95.5	77.77	92.85	96.59	119-120,154
Register.tsx	97.36	87.5	100	97.36	22
ResetPassword.tsx	84.48	84.61	88.88	85.96	50-51,54-55,89-91,122
VerifyEmail.tsx	90.16	85.36	87.5	91.66	49-50,53-54,130
authService.ts	100	100	100	100	
management	87.82	76.37	86.36	90.78	
CreateQuiz.tsx	87.93	72.63	89.58	92.85	84-90,101,103,124,199-200
DeleteQuizModal.tsx	100	92.85	100	100	36
EditQuiz.tsx	86.14	72.41	90	89.93	87,96-98,115,127-130,133-136,150-153,307,479-484
ListQuizzes.tsx	84.44	76.31	80	85.24	29-46,88,192-193,201,315,323,339-341,399,481
RoomHistory.tsx	98.48	85.71	100	100	91,149,280-296
SetupRoom.tsx	85.91	78	68.18	87.87	44,56-57,118,164,192,218,298
room-access	94.91	75.8	94.73	97.36	
NewNickname.tsx	100	75	100	100	40
NicknameEntry.tsx	95.23	78.37	87.5	98.33	137
RoomCode.tsx	92.5	71.42	100	94.87	36-37
room-play	89.66	82.47	86.9	92.83	
AnsweringPhase.tsx	100	100	100	100	
FinalScreen.tsx	96.22	95.45	84.61	96.15	314-323
LeaderboardPhase.tsx	100	93.93	100	100	111-114
LiveRoom.tsx	84.39	70.65	85.29	87.77	65,113-118,174,191,225-226,236-237,247-248,266,278,291,302-304,312
Lobby.tsx	91.17	87.27	83.33	96.87	40,64
ResultsPhase.tsx	97.56	75.86	100	100	45-51,114-115,133-139,204,213-226,252
ScannerModal.tsx	83.33	100	71.42	90.9	14

Evidencia D.3: Captura de la terminal al ejecutar las pruebas de componentes.

All files

90.13% Statements 1343/1490 80.29% Branches 819/1020 88.56% Functions 302/341 92.68% Lines 1280/1381

Press n or j to go to the next uncovered block, b, p or k for the previous block.

Filter:

File	Statements	Branches	Functions	Lines
auth	93.37% 310/332	84.93% 141/166	95.16% 59/62	94.17% 307/326
management	87.82% 548/624	76.37% 278/364	86.36% 152/176	90.78% 512/564
room-access	94.91% 112/118	75.8% 47/62	94.73% 18/19	97.36% 111/114
room-play	89.66% 373/416	82.47% 353/428	86.9% 73/84	92.83% 350/377

Evidencia D.4: Reporte HTML de cobertura de componentes.

Pruebas End-to-End (E2E)

```
PS C:\WS-TFG\quizzie-tfg\frontend> npm run test:e2e
```

[MCR] Playwright E2E Coverage Report

Name	Coverage %	Covered	Uncovered	Total
Bytes	93.81 %	160.691	10.610	171.301
Statements	91.30 %	609	58	667
Branches	83.92 %	814	156	970
Functions	91.41 %	298	28	326
Lines	92.58 %	4664	374	5038

112 passed (1.1m)

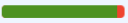
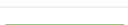
Evidencia D.5: Captura de la terminal al ejecutar las pruebas E2E.

Q Search tests	All 112	✓ Passed 112	Failed 0	Flaky 0	Skipped 0	🕒 ⚙️
Project: chromium 15/9/2026, 16:06:06 Total time: 1.1m						
> cu-01-autenticacion.spec.ts						
> cu-02-gestionar-cuestionarios.spec.ts						
> cu-04-administrar-sala.spec.ts						
> cu-05-unirse-a-sala.spec.ts						
> cu-06-ver-ranking-en-vivo.spec.ts						
> cu-07-validar-nota-qr.spec.ts						
> cu-08-consultar-estadisticas.spec.ts						
> cu-09-exportar-resultados.spec.ts						
✓ cu-10-responder-preguntas.spec.ts						
✓ CU-10: Responder Preguntas > 1a. Respuestas - Alumno selecciona opcion y envia respuesta durante la fase de juego 3.3s						
chromium						
cu-10-responder-preguntas.spec.ts:4						
✓ CU-10: Responder Preguntas > 1b. Respuestas - manejo de errores al enviar respuesta y al cargar titulo de cuestionario 2.8s						
chromium						

Evidencia D.6: Reporte de ejecución de pruebas E2E.

Playwright E2E Coverage Report

Q Group

▼ Name	Type	▼ Bytes					► Statement	► Branch	► Function	► Lines
			%	Covered	Uncovered	Total	%	%	%	%
Summary			93.85%	160.762	10.537	171.299	91.75%	84.23%	91.41%	92.60%
▼ auth			95.51%	37.789	1776	39.565	95.53%	92.41%	85.25%	94.96%
1 CodeInputGrid.tsx	tsx		100.00%	1133	0	1133	100.00%	100.00%	100.00%	100.00%
2 ForgotPassword.tsx	tsx		95.75%	2569	114	2683	83.33%	87.50%	75.00%	93.98%
3 Login.tsx	tsx		96.36%	3917	148	4065	93.75%	83.33%	83.33%	95.16%
4 Profile.tsx	tsx		99.36%	12.674	82	12.756	95.45%	94.44%	100.00%	98.89%
5 Register.tsx	tsx		97.44%	5138	135	5273	100.00%	87.50%	71.43%	96.71%
6 ResetPassword.tsx	tsx		91.07%	5616	551	6167	96.88%	94.59%	88.89%	91.53%
7 VerifyEmail.tsx	tsx		88.93%	5011	624	5635	97.22%	94.87%	87.50%	89.09%
8 authService.ts	ts		93.42%	1731	122	1853	94.12%	87.50%	62.50%	90.48%
▼ management			93.77%	64.382	4280	68.662	89.92%	80.77%	92.94%	92.52%
9 CreateQuiz.tsx	tsx		85.91%	13.399	2197	15.596	87.01%	70.33%	87.50%	86.13%
10 EditQuiz.tsx	tsx		93.54%	15.714	1085	16.799	86.96%	80.49%	92.00%	91.38%
11 ListQuizzes.tsx	tsx		97.95%	14.518	304	14.822	96.61%	88.16%	97.14%	96.70%
12 RoomHistory.tsx	tsx		94.40%	10.073	597	10.670	90.00%	76.19%	93.33%	91.91%
13 SetupRoom.tsx	tsx		99.10%	10.678	97	10.775	91.30%	93.62%	100.00%	98.31%
▼ room-access			97.80%	10.546	237	10.783	90.00%	83.05%	100.00%	95.22%
14 NewNickname.tsx	tsx		100.00%	2400	0	2400	100.00%	100.00%	100.00%	100.00%
15 NicknameEntry.tsx	tsx		96.92%	5158	164	5322	87.50%	78.38%	100.00%	92.86%
16 RoomCode.tsx	tsx		97.62%	2988	73	3061	89.47%	88.89%	100.00%	95.40%
▼ room-play			91.88%	48.045	4244	52.289	91.25%	84.10%	90.79%	90.40%
17 AnsweringPhase.tsx	tsx		99.70%	7772	23	7795	100.00%	97.96%	100.00%	99.20%
18 FinalScreen.tsx	tsx		82.63%	10.772	2264	13.036	96.15%	84.85%	76.92%	83.05%
19 LeaderboardPhase.tsx	tsx		97.97%	3806	79	3885	100.00%	93.94%	100.00%	97.35%
20 LiveRoom.tsx	tsx		90.76%	11.912	1213	13.125	87.64%	83.75%	94.12%	87.63%
21 Lobby.tsx	tsx		95.23%	6053	303	6356	93.75%	86.54%	83.33%	91.72%
22 ResultsPhase.tsx	tsx		95.53%	7730	362	8092	100.00%	63.64%	100.00%	93.63%

Evidencia D.7: Informe interactivo de cobertura E2E.

Pruebas de rendimiento y carga

```
PS C:\WS-TFG\quizzie-tfg> uv run locust -f tests/locust/locustfile.py --host http://127.0.0.1:8000
=====
=== Suite de pruebas de carga y estrés con distribución realista ===
- 3% TeacherManagerUser (Gestión de cuestionarios)
- 2% TeacherHostUser (Profesores con salas de juego activas)
- 95% StudentUser (Alumnos conectados en vivo vía WebSockets)
=====
```

Evidencia D.8: Salida por terminal al iniciar el panel de control de Locust.

LOCUST	Host	Status	RPS	Failures	
	http://127.0.0.1:8000	READY	0	0%	

Start new load test

Number of users (peak concurrency) *

150

Ramp up (users started/second) *

3

Host

http://127.0.0.1:8000

Advanced options 

START

Evidencia D.9: Vista de la interfaz web de Locust.

Type	Name	# Requests	# Fails	Median (ms)	95%ile (ms)	99%ile (ms)	Average (ms)	Min (ms)	Max (ms)	Average size (bytes)	Current RPS	Current Failures/s
GET	/content/quizzes/	17	0	17	94	94	22.28	9	94	40354.06	0.4	0
POST	/content/quizzes/	7	0	19	20	20	18.79	18	20	50	0.3	0
GET	/content/quizzes/{id}	9	0	10	13	13	10.22	8	13	356	0.3	0
PUT	/content/quizzes/{id}	4	0	19	22	22	19.61	17	22	41	0.1	0
POST	/stage/participants	142	0	7	11	14	7.18	3	15	52	0.5	0
GET	/stage/rooms	145	0	9	15	20	10.23	3	24	2188	0.5	0
POST	/stage/rooms/	3	0	10	11	11	9.2	6	11	53	0	0
GET	/stage/rooms/verify/{join_code}	2	0	8.19	9	9	8.81	8	9	57	0	0
GET	/stage/rooms/{id}	183	0	8	13	15	8.65	3	17	546.3	1.5	0
GET	/stage/rooms/{id}/participants	16	0	7	8	8	6.82	3	8	727.88	0.5	0
POST	/users/login	8	0	100	110	110	97.51	65	106	160	0.1	0
GET	/users/me	7	0	8	10	10	8.59	7	10	117	0.3	0
WebSocket	WS Connect	142	0	6	16	19	7.21	1	19	0	0.5	0
WebSocket	WS Ping	1508	0	0	1	1	0.07	0	10	0	39.8	0

Evidencia D.10: Vista de los resultados de la prueba en la interfaz web de Locust.

```

PS C:\WS-TFG\quizzie-tfg> uv run python tests/locust/run_performance_tests.py

=====
EJECUTANDO BENCHMARKING DE RENDIMIENTO Y ESTRÉS CON LOCUST
Servidor Objetivo: http://127.0.0.1:8000
Usuarios Virtuales: 200 | Tasa de Creación: 15/s | Duración: 1m
=====

```

Evidencia D.11: Captura de la ejecución de pruebas de carga y rendimiento.

A continuación se detallan las métricas consolidadas a partir de los ficheros CSV generados durante la prueba de carga, estructuradas según el comportamiento por *endpoint* y la distribución de latencias.

Recuento	Mensaje	Traza	Proceso
—	—	—	—

Evidencia D.12: Registro de excepciones detectadas durante la prueba de carga.

t (s)	Usuarios	Pct/s	Fallos/s	P50 (ms)	P66 (ms)	P75 (ms)	P80 (ms)	P90 (ms)	P95 (ms)	P98 (ms)	P99 (ms)	99.9 (ms)	99.99 (ms)	P100 (ms)	Peticiones	Fallos	Mediana	Media	Mínimo	Máximo	Tamaño(B)
0	0	0,00	0,0	-	-	-	-	-	-	-	-	-	-	-	0	0	0	0,00	0	0,00	0,00
1	15	0,00	0,0	10	14	16	18	21	25	37	40	40	40	40	75	0	10	10,90	0	39,51	557,04
2	30	0,00	0,0	16	22	26	32	50	69	86	130	140	140	140	147	0	16	21,11	0	135,77	793,82
3	45	75,00	0,0	15	24	27	31	43	54	78	86	140	140	140	229	0	15	19,30	0	135,77	694,50
4	60	65,50	0,0	13	20	24	27	38	53	76	86	140	140	140	308	0	13	17,32	0	135,77	643,52
5	75	71,67	0,0	12	18	22	25	33	48	76	92	140	140	140	395	0	12	16,05	0	135,77	698,54
6	105	74,75	0,0	13	20	26	31	42	69	140	160	240	240	240	485	0	13	20,30	0	235,30	651,68
7	120	75,20	0,0	12	19	24	27	40	54	120	150	240	240	240	585	0	12	18,59	0	235,30	674,72
8	135	71,33	0,0	10	17	21	25	38	50	92	150	240	240	240	682	0	10	16,75	0	235,30	636,35
9	150	79,14	0,0	9	14	19	23	35	47	86	140	240	240	240	788	0	9	15,08	0	235,30	650,32
10	165	83,00	0,0	8	13	17	21	33	44	83	140	240	240	240	898	0	8	13,88	0	235,30	615,52
11	180	84,78	0,0	8	13	18	21	32	44	78	130	170	240	240	1004	0	8	13,70	0	235,30	593,63
12	195	86,80	0,0	8	14	20	24	37	50	78	120	170	240	240	1127	0	8	14,61	0	235,30	598,67
13	200	88,70	0,0	8	14	20	25	37	49	76	110	170	240	240	1261	0	8	14,45	0	235,30	567,12
14	200	93,90	0,0	8	13	20	24	36	48	76	100	170	240	240	1330	0	8	13,89	0	235,30	548,99
15	200	97,40	0,0	7	13	19	23	35	46	72	100	170	240	240	1384	0	7	13,38	0	235,30	554,64
16	200	98,70	0,0	6	12	18	22	34	46	72	94	170	240	240	1448	0	6	12,83	0	235,30	531,43
17	200	96,30	0,0	6	11	17	21	33	46	71	94	170	240	240	1496	0	5	12,42	0	235,30	514,38
18	200	97,40	0,0	5	11	17	21	32	45	69	92	170	240	240	1558	0	5	11,97	0	235,30	519,13
19	200	90,40	0,0	4	10	16	20	32	44	69	92	170	240	240	1616	0	4	11,54	0	235,30	500,50
20	200	84,80	0,0	2	10	16	20	32	44	69	92	170	240	240	1660	0	2	11,24	0	235,30	488,05
21	200	80,60	0,0	0	9	14	19	31	44	69	86	170	240	240	1744	0	0	10,74	0	235,30	465,83
22	200	75,80	0,0	0	9	14	18	31	43	66	85	170	240	240	1803	0	0	10,41	0	235,30	451,11
23	200	71,90	0,0	0	9	13	18	30	43	66	85	170	240	240	1860	0	0	10,11	0	235,30	438,20
24	200	68,70	0,0	0	8	13	17	30	42	65	84	170	240	240	1915	0	0	9,85	0	235,30	445,22
25	200	62,60	0,0	0	8	12	17	29	41	63	84	170	240	240	1972	0	0	9,59	0	235,30	433,10
26	200	58,50	0,0	0	8	12	16	29	41	62	83	170	240	240	2032	0	0	9,34	0	235,30	456,89
27	200	58,60	0,0	0	8	11	16	28	41	61	83	170	240	240	2085	0	0	9,11	0	235,30	445,54
28	200	58,30	0,0	0	7	11	15	27	40	57	80	170	240	240	2150	0	0	8,88	0	235,30	450,07
29	200	58,80	0,0	0	7	11	14	27	39	56	79	170	240	240	2215	0	0	8,63	0	235,30	437,38
30	200	58,30	0,0	0	6	10	14	27	38	56	79	170	240	240	2270	0	0	8,44	0	235,30	427,79
31	200	59,10	0,0	0	6	10	14	26	38	55	78	170	240	240	2316	0	0	8,27	0	235,30	419,29
32	200	60,00	0,0	0	5	10	13	26	38	55	78	170	240	240	2384	0	0	8,08	0	235,30	423,58
33	200	59,70	0,0	0	5	10	13	25	38	54	78	170	240	240	2445	0	0	7,89	0	235,30	413,54
34	200	57,10	0,0	0	5	9	13	25	37	54	78	170	240	240	2491	0	0	7,76	0	235,30	406,57
35	200	58,20	0,0	0	4	9	12	24	37	53	76	170	240	240	2558	0	0	7,58	0	235,30	410,79
36	200	58,10	0,0	0	3	9	12	24	36	53	76	170	240	240	2624	0	0	7,41	0	235,30	429,03
37	200	57,20	0,0	0	1	9	12	24	36	52	76	170	240	240	2666	0	0	7,30	0	235,30	422,77
38	200	57,80	0,0	0	1	8	11	23	35	52	72	170	240	240	2726	0	0	7,15	0	235,30	413,96
39	200	58,50	0,0	0	1	8	11	23	35	52	72	170	240	240	2790	0	0	7,01	0	235,30	418,25
40	200	57,50	0,0	0	1	8	11	23	34	51	72	170	240	240	2851	0	0	6,87	0	235,30	409,95
41	200	57,20	0,0	0	0	8	10	22	34	51	71	170	240	240	2910	0	0	6,75	0	235,30	401,85
42	200	57,00	0,0	0	0	8	10	22	34	51	71	170	240	240	2977	0	0	6,61	0	235,30	405,92
43	200	57,70	0,0	0	0	8	10	21	33	50	69	170	240	240	3022	0	0	6,52	0	235,30	400,18
44	200	58,50	0,0	0	0	8	10	21	33	50	69	170	240	240	3090	0	0	6,39	0	235,30	392,09
45	200	58,10	0,0	0	0	7	10	21	32	50	69	170	240	240	3150	0	0	6,28	0	235,30	385,28
46	200	57,10	0,0	0	0	7	9	20	32	49	69	170	240	240	3198	0	0	6,20	0	235,30	379,55
47	200	59,50	0,0	0	0	7	9	20	32	49	69	170	240	240	3255	0	0	6,10	0	235,30	373,45
48	200	58,90	0,0	0	0	7	9	20	32	48	69	170	240	240	3311	0	0	6,01	0	235,30	378,64
49	200	57,00	0,0	0	0	6	9	20	31	47	69	170	240	240	3376	0	0	5,91	0	235,30	371,88
50	200	58,50	0,0	0	0	6	9	19	31	47	69	170	240	240	3428	0	0	5,83	0	235,30	366,60
51	200	58,00	0,0	0	0	6	9	19	31	46	69	170	240	240	3488	0	0	5,75	0	235,30	371,65
52	200	58,00	0,0	0	0	5	8	19	31	46	69	170	240	240	3538	0	0	5,67	0	235,30	366,39
53	200	57,30	0,0	0	0	5	8	19	31	46	66	170	240	240	3604	0	0	5,58	0	235,30	360,31
54	200	56,00	0,0	0	0	5	8	18	30	46	66	170	240	240	3665	0	0	5,50	0	235,30	354,85
55	200	57,80	0,0	0	0	4	8	18	30	46	66	170	240	240	3720	0	0	5,43	0	235,30	360,03
56	200	57,10	0,0	0	0	4	8	18	30	46	66	170	240	240	3787	0	0	5,36	0	235,30	364,03
57	200	56,60	0,0	0	0	4	8	18	30	45	65	170	240	240	3840	0	0	5,29	0	235,30	359,45
58	200	58,00	0,0	0	0	2	8	17	30	45	65	170	240	240	3898	0	0	5,22	0	235,30	354,38

Evidencia D.13: Historial cronológico de la prueba de carga segundo a segundo.

Método	Nombre	Error	Número	Primer registro	Último registro
-	-	-	-	-	-

Evidencia D.14: Registro de fallos detectados durante la prueba de carga.

Tipo	Operación							Tamaño Medio (B)													
		Peticiones Fallos	Mediana (ms)	Media (ms)	Mínima (ms)	Máxima (ms)	Peticiones/s		Fallos/s	P50 (ms)	P66 (ms)	P75 (ms)	P80 (ms)	P90 (ms)	P95 (ms)	P98 (ms)	P99 (ms)	P99,9 (ms)	P99,99 (ms)	P100 (ms)	
GET	/content/quizzes/	21	0	15	19,82	8,26	86,49	36581,05	0,36	0,0	15	16	16	17	20	46	86	86	86	86	86
POST	/content/quizzes/	11	0	19	25,08	10,01	78,32	50,00	0,19	0,0	19	21	24	24	38	78	78	78	78	78	78
GET	/content/quizzes/:id	23	0	11	11,62	9,50	19,10	359,43	0,39	0,0	11	11	12	12	17	18	19	19	19	19	19
PUT	/content/quizzes/:id	11	0	20	19,74	17,04	25,04	41,00	0,19	0,0	20	21	21	21	22	25	25	25	25	25	25
POST	/stage/participants	190	0	15	18,53	3,24	53,91	52,00	3,25	0,0	16	22	28	30	38	43	44	51	54	54	54
GET	/stage/rooms	194	0	21	31,99	3,90	173,95	2188,00	3,32	0,0	22	33	45	49	69	79	150	170	170	170	170
POST	/stage/rooms/	4	0	11	40,45	9,53	84,28	53,00	0,07	0,0	57	57	84	84	84	84	84	84	84	84	84
GET	/stage/rooms/verify/:code	8	0	8	17,97	6,66	48,85	57,00	0,14	0,0	8	13	44	44	49	49	49	49	49	49	49
GET	/stage/rooms/:id	268	0	12	17,62	3,47	162,76	546,13	4,59	0,0	12	18	23	26	35	44	52	54	160	160	160
GET	/stage/rooms/:id/participants	26	0	8	8,13	3,26	13,29	723,69	0,44	0,0	8	8	8	9	12	13	13	13	13	13	13
POST	/users/login	10	0	94	113,52	68,73	235,30	160,00	0,17	0,0	100	110	130	140	240	240	240	240	240	240	240
GET	/users/me	18	0	9	9,20	5,40	12,92	117,00	0,31	0,0	9	9	10	11	12	13	13	13	13	13	13
WS	WS Connect	190	0	11	14,66	2,00	149,00	0,00	3,25	0,0	11	16	21	23	28	34	37	39	150	150	150
WS	WS Ping	2924	0	0	0,04	0,00	2,00	0,00	50,04	0,0	0	0	0	0	0	0	1	1	1	2	2
Total		3898	0	0	5,22	0,00	235,30	354,38	66,71	0,0	0	0	2	8	17	30	45	65	170	240	240

Evidencia D.15: Métricas y percentiles de latencia en la prueba de carga.

```

PS C:\WS-TFG\quizzie-tfg> uv run python tests/locust/evaluate_thresholds.py

=====
=== INFORME DE EVALUACION DE UMBRALES DE RENDIMIENTO (P90) ===
=====
Métrica / Endpoint | Valor P90 | Umbral | Estado
-----
GET /content/quizzes/ | 20 ms | <= 500 ms | [OK]
POST /content/quizzes/ | 38 ms | <= 500 ms | [OK]
GET /content/quizzes/{id} | 17 ms | <= 200 ms | [OK]
PUT /content/quizzes/{id} | 22 ms | <= 200 ms | [OK]
POST /stage/participants | 38 ms | <= 200 ms | [OK]
GET /stage/rooms | 69 ms | <= 200 ms | [OK]
POST /stage/rooms/ | 84 ms | <= 200 ms | [OK]
GET /stage/rooms/verify/{join_code} | 49 ms | <= 200 ms | [OK]
GET /stage/rooms/{id} | 35 ms | <= 200 ms | [OK]
GET /stage/rooms/{id}/participants | 12 ms | <= 200 ms | [OK]
POST /users/login | 240 ms | <= 500 ms | [OK]
GET /users/me | 12 ms | <= 200 ms | [OK]
WebSocket Connect | 28 ms | <= 200 ms | [OK]
Tasa de Errores (global) | 0.00% | < 1.0% | [OK]
=====
RESULTADO FINAL: TODAS LAS PRUEBAS DE RENDIMIENTO SUPERARON LOS UMBRALES [OK]
=====

```

Evidencia D.16: Captura de ejecución de *script* de verificación de umbrales.

E. Informe de dedicación temporal

En este anexo se consolida y procesa el registro del seguimiento temporal extraído de la plataforma Clockify. Para facilitar la auditoría y verificación del esfuerzo invertido frente a la planificación original, las imputaciones horarias se presentan estructuradas en tres niveles progresivos de agregación.

El primer reporte ofrece una visión general de la dedicación temporal, desglosando las horas acumuladas y el peso porcentual de cada etapa del ciclo de vida del proyecto:

ID	Fase	Duración (h)	Dedicación (%)
F-01	Planificación	11,00 h	3,34 %
F-02	Documentación	26,80 h	8,15 %
F-03	Inicio	12,50 h	3,80 %
F-04	CI/CD	18,75 h	5,70 %
F-05	MVP	49,70 h	15,11 %
F-06	Testing	41,11 h	12,50 %
F-07	Core	40,21 h	12,22 %
F-08	QA	21,42 h	6,51 %
F-09	Memoria	87,43 h	26,58 %
F-10	Defensa	20,00 h	6,09 %
Total		328,94 h	100,00 %

Tabla E.1: Distribución global del tiempo invertido por fases del proyecto.

Descendiendo al nivel de actividad, el siguiente balance detalla el total de horas registradas a través de las 29 actividades definidas en el proyecto, vinculando cada tarea a su fase correspondiente para garantizar la trazabilidad:

Fase	Actividad	Descripción de la actividad	Horas
F-01	ACT-01	Redacción del acta de constitución	4,83 h
	ACT-02	Definición de hitos y estimación del cronograma inicial	6,17 h
F-02	ACT-03	Elección de estilos, paleta de colores y componentes básicos	0,72 h
	ACT-04	Diseño de prototipos de las pantallas de gestión de profesor	4,06 h
	ACT-05	Diseño de la interfaz de respuesta para pantallas de alumnos	0,37 h
	ACT-06	Elaboración del manual funcional como guía para usuarios	0,82 h
	ACT-07	Redacción del manual de despliegue técnico de infraestructura	4,09 h
	ACT-08	Elicitación y redacción de requisitos funcionales y no funcionales	3,41 h
	ACT-09	Modelado de diagramas de casos de uso y flujos esenciales	1,50 h
	ACT-10	Diseño del modelo conceptual y lógico de datos	4,14 h
	ACT-11	Modelado del diagrama de entidades relacional	3,80 h
	ACT-12	Generación del <i>script</i> de inicialización de tablas y poblado	3,90 h
F-03	ACT-13	Inicialización del repositorio en GitHub y estructura base	12,50 h
F-04	ACT-14	Diseño de plantillas para <i>issues</i> y automatización base	4,69 h
	ACT-15	Desarrollo de <i>scripts</i> de despliegue e integración continuos	14,06 h
F-05	ACT-16	Diseño y desarrollo de la persistencia base para cuestionarios	16,31 h
	ACT-17	Implementación de la interfaz de creación de test y panel base	15,56 h
	ACT-18	Lógica de control y validación para la ejecución asíncrona	17,83 h
F-06	ACT-19	Desarrollo y ejecución de test de integración de servicios	17,49 h
	ACT-20	Pruebas de componentes, de carga y <i>end-to-end</i>	23,62 h
F-07	ACT-21	Desarrollo de la infraestructura síncrona y canales <i>WebSocket</i>	24,27 h
	ACT-22	Implementación en <i>frontend</i> de la conexión reactiva a salas	9,46 h
	ACT-23	Programación del motor de validación y control de integridad	6,48 h
F-08	ACT-24	Implementación del módulo de autenticación y panel avanzado	9,28 h
	ACT-25	Desarrollo de vistas de historial y exportación de datos	3,75 h
	ACT-26	Puesta a punto, empaquetado, corrección de errores y despliegue	8,40 h
F-09	ACT-27	Redacción y estructuración técnica de la memoria en LaTeX	87,43 h
F-10	ACT-28	Diseño, desarrollo y maquetación de diapositivas de defensa	10,00 h
	ACT-29	Ensayos de oratoria y preparación de la defensa pública	10,00 h
Total acumulado			328,94 h

Tabla E.2: Desglose detallado del cómputo temporal por fase y actividad.

Finalmente, se agrupan las horas registradas por rol profesional, sirviendo de base directa para el cálculo de los costes de personal en el balance económico:

Recurso (Rol)	Actividad	Horas
Project Manager	ACT-01	4,83 h
	ACT-02	6,17 h
	ACT-29	10,00 h
	Subtotal	21,00 h
Analista	ACT-06	0,82 h
	ACT-08	3,41 h
	ACT-09	1,50 h
	ACT-10	4,14 h
	ACT-11	3,80 h
	ACT-27	87,43 h
	Subtotal	101,10 h
Diseñador UI	ACT-03	0,72 h
	ACT-04	4,06 h
	ACT-05	0,37 h
	ACT-28	10,00 h
	Subtotal	15,15 h
Programador Backend	ACT-12	3,90 h
	ACT-16	16,31 h
	ACT-18	17,83 h
	ACT-21	24,27 h
	ACT-23	6,48 h
	ACT-24	9,28 h
	Subtotal	78,07 h
Programador Frontend	ACT-17	15,56 h
	ACT-22	9,46 h
	ACT-25	3,75 h
	Subtotal	28,77 h
Tester	ACT-19	17,49 h
	ACT-20	23,62 h
	Subtotal	41,11 h
Técnico	ACT-07	4,09 h
	ACT-13	12,50 h
	ACT-14	4,69 h
	ACT-15	14,06 h
	ACT-26	8,40 h
	Subtotal	43,74 h
TOTAL		328,94 h

Tabla E.3: Horas reales computadas agrupadas por perfil profesional.

Bibliografía

- [1] Roger S. Pressman. *Ingeniería del software: Un enfoque práctico*. McGraw-Hill, 7^o edition, 2010. ISBN 9786071503145.
- [2] Ian Sommerville. *Ingeniería de Software*. Pearson, 9^o edition, 2011. ISBN 9786073206037.
- [3] Robert C. Martin and Michael C. Feathers. *Clean code: a handbook of agile software craftsmanship*. Prentice Hall, 2009. ISBN 9780132350884.
- [4] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017. ISBN 9780134494166.
- [5] Dave Farley. *Continuous Delivery Pipelines: How To Build Better Software Faster*. Independently published, 2021. ISBN 979-8517979551.
- [6] Parlamento Europeo y Consejo de la Unión Europea. Reglamento (ue) 2016/679 relativo a la protección de las personas físicas en lo que respecta al tratamiento de datos personales y a la libre circulación de estos datos (rgpd), 2016. URL <https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX:32016R0679>.
- [7] Python Software Foundation. Python programming language official website, 2026. URL <https://www.python.org/>.
- [8] Sebastián Ramírez. Fastapi framework documentation, 2026. URL <https://fastapi.tiangolo.com/>.
- [9] MDN Web Docs. Websockets api - communication protocol, 2026. URL https://developer.mozilla.org/es/docs/Web/API/WebSockets_API.
- [10] Sebastián Ramírez. Sqlmodel library documentation, 2026. URL <https://sqlmodel.tiangolo.com/>.
- [11] SQLite Development Team. Sqlite database engine documentation, 2026. URL <https://www.sqlite.org/>.
- [12] Astral Software. uv: An ultra-fast python package installer and resolver, 2026. URL <https://docs.astral.sh/uv/>.
- [13] Meta Open Source. React - a javascript library for building user interfaces, 2026. URL <https://react.dev/>.
- [14] Vite Team. Vite - next generation frontend tooling, 2026. URL <https://vite.dev/>.
- [15] npm, Inc. npm documentation and package registry. <https://www.npmjs.com/>, 2024. Accedido el 15 de mayo de 2024.

- [16] Microsoft Corporation. Typescript - javascript with syntax for types, 2026. URL <https://www.typescriptlang.org/>.
- [17] Remix Software. React router dom documentation, 2026. URL <https://reactrouter.com/>.
- [18] Axios Project. Axios - promise based http client for the browser and node.js, 2026. URL <https://axios-http.com/>.
- [19] Andrew M. St. Laurent. *Understanding Open Source and Free Software Licensing*. O'Reilly Media, 2008.
- [20] Project Management Institute. *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*. Project Management Institute, 7th edition, 2021.
- [21] Kent Beck, Mike Beedle, Arie van Bennekum, Alistair Cockburn, Ward Cunningham, Martin Fowler, James Grenning, Jim Highsmith, Andrew Hunt, and Dave Thomas. Manifesto for agile software development, 2001. URL <https://agilemanifesto.org/>.
- [22] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2nd edition, 2014.
- [23] International Organization for Standardization / IEEE. ISO/IEC/IEEE 21839:2019 Systems and software engineering – Systems and software Work Breakdown Structure (WBS). Technical report, International Organization for Standardization / IEEE, 2019.
- [24] James M. Wilson. Gantt charts: A review of the project management tool. *International Journal of Project Management*, 21(7):521–528, 2003.
- [25] Dan Remenyi, Frank Bannister, and Arthur Money. *The Effective Measurement and Management of IT Costs and Benefits*. Butterworth-Heinemann, 2007.
- [26] Junta de Andalucía. Informe final sobre la consulta preliminar del mercado: “perfiles profesionales Ámbito informático”. Informe técnico, Consejería de Agricultura, Pesca y Desarrollo Rural. Secretaría General Técnica. Servicio de Informática, Sevilla, España, 2018. Firmado por D. Juan Luis Ceada Ramos. Publicado en el Perfil del Contratante.
- [27] Lisa M. Ellram. Total cost of ownership: elements and implementation. *International Journal of Purchasing and Materials Management*, 29(4):2–11, 1993.
- [28] IEEE Computer Society. IEEE Std 828-2012 IEEE Standard for Configuration Management in Systems and Software Engineering. Technical report, IEEE Computer Society, 2012.
- [29] Armando Fox and David Patterson. *Engineering Software as a Service: An Agile Approach Using Cloud Computing*. Strawberry Canyon LLC, 2014. ISBN 9780984881246.
- [30] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002. ISBN 9780321127426.

- [31] Eve Porcello and Alex Banks. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, 2nd edition, 2020. ISBN 9781492051725.
- [32] Greg MacDonald. *Practical UI Patterns for Design Systems: Fast-Track Interaction Design for a Seamless User Experience*. Apress, 2019. ISBN 9781484249380.
- [33] Vincent Driessen. A successful git branching model, 2010. URL <https://nvie.com/posts/a-successful-git-branching-model/>.
- [34] Emma Jane Hogbin Westby. *Git for Teams: A User-Centered Approach to Creating Efficient Workflows in Git*. O'Reilly Media, 2015.
- [35] Ian Sommerville. *Engineering Software Products: An Introduction to Modern Software Engineering*. Pearson, 2021. ISBN 9781292376356.
- [36] Jason Gregory. *Game Engine Architecture*. CRC Press, 2nd edition, 2014. ISBN 9781466560017.
- [37] International Organization for Standardization. Iso/iec 25010:2011 systems and software engineering – systems and software quality requirements and evaluation (square) – system and software quality models, 2011.
- [38] Len Bass, Paul Clements, and Rick Kazman. *Software Architecture in Practice*. Addison-Wesley Professional, 3rd edition, 2012. ISBN 9780321815736.
- [39] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley Professional, 2004. ISBN 9780321200686.
- [40] Michael Nygard. Documenting architecture decisions. *Cognitect Blog*, 2011. URL <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>.
- [41] Andrew Hunt and David Thomas. *The Pragmatic Programmer: From Journeyman to Master*. Addison-Wesley Professional, 1999. ISBN 978-0201616224.
- [42] Francisco Gago Vázquez. Repositorio oficial de quizzie, 2026. URL <https://github.com/frangago71/quizzie-tfg>.
- [43] Guido van Rossum, Barry Warsaw, and Nick Coghlan. PEP 8 – Style Guide for Python Code, 2001. URL <https://peps.python.org/pep-0008/>. Python Enhancement Proposals.
- [44] Tom Preston-Werner. Semantic Versioning 2.0.0, 2013. URL <https://semver.org/>.
- [45] National Institute of Standards and Technology. Security and privacy controls for information systems and organizations. Technical Report NIST Special Publication 800-53, Revision 5, U.S. Department of Commerce, 2020.
- [46] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley Professional, Boston, MA, USA, 1994.

- [47] Patrick Th. Eugster, Pascal A. Felber, Rachid Guerraoui, and Anne-Marie Kermarrec. The many faces of publish/subscribe. *ACM Computing Surveys (CSUR)*, 35(2):114–131, 2003. doi: 10.1145/857076.857078.
- [48] OWASP Foundation. OWASP Top 10:2021 – The Ten Most Critical Web Application Security Risks. <https://owasp.org/Top10/>, 2021. Accedido en 2026.
- [49] Michal Zalewski. *The Tangled Web: A Guide to Securing Modern Web Applications*. No Starch Press, San Francisco, CA, USA, 2011.
- [50] Michael Jones, John Bradley, and Nat Sakimura. JSON Web Token (JWT). RFC 7519, IETF, 2015.
- [51] Mike Cohn. *Succeeding with Agile: Software Development Using Scrum*. Addison-Wesley Professional, Boston, MA, USA, 2009.
- [52] Git Development Team. Git - fast, scalable, distributed revision control system, 2026. URL <https://git-scm.com/>.
- [53] OpenJS Foundation. Node.js - javascript runtime environment, 2026. URL <https://nodejs.org/>.